1ST EDITION

Databricks ML in Action

Learn how Databricks supports the entire ML lifecycle end to end from data ingestion to the model deployment

STEPHANIE RIVERA
ANASTASIA PROKAIEVA | AMANDA BAKER | HAYLEY HORN

Databricks ML in Action

Learn how Databricks supports the entire ML lifecycle end to end from data ingestion to the model deployment

Stephanie Rivera

Anastasia Prokaieva

Amanda Baker

Hayley Horn

‹packt›

Databricks ML in Action

Copyright © 2024 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Ali Abidi
Publishing Product Manager: Sanjana Gupta
Content Development Editor: Priyanka Soam
Technical Editor: Kavyashree K S
Copy Editor: Safis Editing
Project Coordinator: Shambhavi Mishra
Proofreader: Priyanka Soam
Indexer: Rekha Nair
Production Designer: Jyoti Kadam
Marketing Coordinator: Nivedita Singh

First published: May 2024

Production reference: 2170424

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK.

ISBN 978-1-80056-489-3

www.packtpub.com

To the strong women who have come before me, for their sacrifices and for exemplifying the power of determination. To the memory of my grandmother, Hazel Adolph, for being my best friend and cheerleader.

– Stephanie Rivera

To females who pursued a STEM career and did not give up no matter what obstacles occurred on their way. Some would say science it's not for girls, well, prove them wrong.

- Anastasia Prokaieva

To my mother, Mary Baker. Thank you for showing me what true strength is, for being both a voice of reason and unbridled support, and for believing in me no matter what.

– Amanda Baker

This is dedicated to the women who inspired me to lead by example, to my mom, Susan Charba, who reminded me that I could be the one who inspires, and to the women still working their way up. I'll send the elevator back down. There is plenty of room for us all.

– Hayley Horn

Contributors

About the authors

Stephanie Rivera has worked in big data and machine learning since 2011. She collaborates with teams and companies as they design their data intelligence platform as a senior solutions architect for Databricks.

Previously, Stephanie was the VP of data intelligence for a global company, ingesting 20+ terabytes of data daily. She led the data science, data engineering, and business intelligence teams.

Her data career has also included contributing to and leading a team creating software that teaches people how to explore fictional planets using data science algorithms. Stephanie authored numerous sections of Booz Allen Hamilton's publication *The Field Guide to Data Science*.

I want to thank my loving partner, Rami Alba Lucio, Databricks coworkers, family, and friends for their unwavering support.

Anastasia Prokaieva began her career 9 years ago as a research scientist at CEA (France), focusing on large data analysis and satellite data assimilation, treating terabytes of data. She has been working within the big data analysis and machine learning domain since then. In 2021, she joined Databricks and became the regional AI subject matter expert.

On a daily basis, Anastasia is a consultant Databricks users on best practices for implementing AI projects end to end. She also delivers training and workshops to democratize AI. Anastasia holds two MSc degrees in theoretical physics and energy science.

I would like to thank my partner, Julien, and my family for their tremendous support. My gratitude to my talented teammates all around the globe, as you inspire me every day!

Amanda Baker began her career in data 8 years ago. She loves leveraging her skills as a data scientist to orchestrate transformative journeys for companies across diverse industries as a solutions architect for Databricks. Her experiences have brought her from large corporations to small start-ups and everything in between. Amanda is a graduate of Carnegie Mellon University and the University of Washington.

Thank you to my partner, Emmanuel, my parents, sisters, and friends for their enduring love and support.

Hayley Horn started her data career 15 years ago as a data quality consultant on enterprise data integration projects. As a data scientist, she specialized in customer insights and strategy. Hayley has presented at data science and AI conferences in the US and Europe. She is currently a senior solutions architect for Databricks, with expertise in data science and technology modernization.

A graduate of the MS data science program at Southern Methodist University in Dallas, Texas, USA, she is now a capstone advisor to students in their final semesters of the program.

I'd like to thank my husband, Kevin, and my sons, Dyson and Dalton, for their encouragement and enthusiastic support.

About the reviewers

Jeanne Choo studied plant biology and zoology at college, teaching herself how to code while doing so, to understand better how genes evolve over time. She was then hired by AI Singapore, a Singaporean government entity, as their first AI engineer. At AI Singapore, she was key in building the technical consulting practice, apprenticeship program, and technical stack from nothing. She set up the AI Apprenticeship Program, which won the Talent Accelerator Award Asia-Pacific IDC Digital Transformation Awards.

Most of Jeanne's work has focused on challenges specific to the Asia-Pacific region. Some past projects include building ML pipelines for Japanese search query understanding, training speech recognition models for Singlish, and building NLP models and corpora for Indonesian. At Databricks, she advises customers on best practices related to all things MLOps and generative AI.

Benjamin Faircloth completed a Master of Science degree in Data Science at Northwestern University. Ben is a key figure in AI and machine learning within the public sector. As a Delivery Solutions Architect and Embedded Product Lead at Databricks, he has driven significant enhancements in service delivery and innovation across top accounts. Ben's contributions to developing Databricks' delivery methodologies have streamlined processes and improved customer engagement. He led pivotal LLM hackathon initiatives with the Department of Defense, advancing AI capabilities crucial for national security. Throughout his career, Ben has witnessed the transformation from conceptual data architectures to the practical implementation of the lakehouse paradigm. At a significant moment in his career, as this book goes to press, his daughter Lydia is also entering the world, marking a personal milestone alongside his professional achievements. Committed to the mission, Ben strives to drive further innovation in applying advanced technologies to meet critical objectives.

Clever Anjos is a principal solutions architect at Qlik, a data analytics and data integration software company.

He has been working for Qlik since 2018 but has been around the Qlik ecosystem as a partner and customer since 2009. He is a business discovery professional with several years of experience working with Qlik, AWS, Google Cloud, Databricks, and other BI technologies.

He is a highly active member of Qlik Community, with over 8,000 posts and 4,500 page views.

In May 2022, he was named Qlik Community's featured member.

Amreth Chandrasehar is an engineering leader in the cloud, AI/ML engineering, observability, and SRE. Over the last few years, Amreth has played a key role in cloud migration, generative AI, AIOps, observability, and ML adoption at various organizations. Amreth is also co-creator of the Conducktor Platform, serving T-Mobile's over 100 million customers, and a tech/customer advisory board member at various companies on observability. Amreth has also co-created and open sourced Kardio.io, a service health dashboard tool. Amreth has been invited to and spoken at several key conferences and has won several awards.

I would like to thank my wife, Ashwinya, and my son, Athvik, for their patience and support provided during my review of this book.

Databricks subject matter experts, we want to extend our heartfelt gratitude to each of you who took the time to review the text and code. The speed at which Databricks evolves with technology makes it an incredible feat to be current. Your guidance and support throughout this journey has been invaluable. Your contributions have not only enhanced the technical accuracy and depth of our book but have also provided invaluable context and perspective rooted in your firsthand experiences at Databricks.

Table of Contents

Preface xiii

Part 1: Overview of the Databricks Unified Data Intelligence Platform

1

Getting Started and Lakehouse Concepts 3

Components of the Data Intelligence Platform 3
Advantages of the Databricks Platform 4
Open source features 5
Databricks AutoML 7
Reusability and reproducibility 7
Open file formats give you flexibility 7
Applying our learning 10
Technical requirements 10
Getting to know your data 11
Project: Streaming Transactions 11
Project: Favorita Sales Forecasting 13
Project: Multilabel Image Classification 14
Project: Retrieval Augmented Generation Chatbot 15
Summary 16
Questions 16
Answers 16
Further reading 17

2

Designing Databricks: Day One 19

Planning your platform 19
Defining a workspace 21
Selecting the metastore 21
Defining where the data lives, and cloud object storage 23
Discussing source control 24
Discussing data preparation 25
Planning to create features 27
Modeling in Databricks 28

Monitoring data and models 30
Applying our learning 31
Technical requirements 31
Setting up your workspace 32
Kaggle setup 32
Starting the projects 40
Project: Favorita Store Sales – Time Series Forecasting 40
Project: Streaming Transactions 41
Project: Retrieval-Augmented Generation Chatbot 44
Project: Multilabel Image Classification 45
Summary 46
Questions 46
Answers 46
Further reading 46

3

Building the Bronze Layer 49

Revisiting the Medallion architecture pattern 49
Transforming data to Delta with Auto Loader 50
Schema Evolution 51
Delta Live Tables for the Bronze layer 51
Delta Live Tables benefits and features 52
Bronze data with DLT 53
Maintaining and optimizing Delta Tables 53
Vacuum 54
Liquid Clustering 54
Optimize 55
Predictive optimization 55
Applying our learning 55
Technical requirements 55
Project: Streaming Transactions 56
Project: Favorita Store Sales - Time Series Forecasting 64
Project: Retrieval Augmented Generation Chatbot 65
Project: Multilabel Image Classification 71
Summary 74
Questions 74
Answers 75
Further reading 75

Part 2: Heavily Project Focused

4

Getting to Know Your Data 79

Improving data integrity with DLT 80
Monitoring data quality with Databricks Lakehouse Monitoring 82
Mechanics of Lakehouse Monitoring 82
Visualization and alerting 82
Creating a monitor 83

Exploring data with Databricks Assistant 84
Generating data profiles with AutoML 88
Using embeddings to understand unstructured data 91
Enhancing data retrieval with Databricks Vector Search 92
Flexibility in embedding model support 93
Setting up a vector search 93
Applying our learning 94
Technical requirements 94
Project: Favorita Store Sales – Time Series Forecasting 94
Project: Streaming Transactions 101
Project: RAG Chatbot 101
Project: Multilabel Image Classification 105
Summary 107
Questions 108
Answers 108
Further reading 109

5

Feature Engineering on Databricks 111

Databricks Feature Engineering in Unity Catalog 112
Feature engineering on a stream 114
Employing point-in-time lookups for time series feature tables 115
Computing on-demand features 116
Applying our learning 117
Technical requirements 117
Project: Streaming Transactions 117
Project: Favorita Store Sales – Time Series Forecasting 132
Summary 138
Questions 138
Answers 138
Further reading 139

6

Tools for Model Training and Experimenting 141

Building a training set with feature tables 142
Baselining with AutoML 143
Tracking experiments with MLflow 144
Integrating external data and models 146
Integrating data via the Marketplace 147
Databricks LLM Playground 148
Applying our learning 151
Technical requirements 151
Project: Favorita Store Sales - Time Series Forecasting 151
Project: Multilabel Image Classification 164
Summary 172
Questions 172
Answers 173
Further reading 173

7

Productionizing ML on Databricks 175

Deploying the MLOps inner loop 176
Registering a model 176
Collaborative Development 176
Deploying the MLOps Outer Loop 177
Workflows 178
Databricks Asset Bundles 179
Rest API 180
Deploying your model 180
Model Inference 180
Model Serving 181
Applying our learning 182
Technical requirements 183
Project: Favorita Sales Forecasting 183
Project: Streaming Transactions 187
Project: Multilabel Image Classification 197
Project: Retrieval Augmented Generation Chatbot 204
Setting up the evaluation workflow 206
Summary 214
Questions 214
Answers 215
Further reading 215

8

Monitoring, Evaluating, and More 217

Monitoring your models 218
Building gold layer visualizations 219
Leveraging Lakeview dashboards 219
Visualizing big data with Databricks SQL dashboards 221
Python UDFs 224
Connecting your applications 231
Incorporating LLMs for analysts with SQL AI Functions 231
Applying our learning 235
Technical requirements 235
Project: Favorita Store Sales 236
Project: Streaming Transactions 240
Project: Retrieval-Augmented Generation Chatbot 241
Summary 247
Questions 247
Answers 248
Further reading 248

Index 251

Other Books You May Enjoy 258

Preface

In this book, you will discover what makes the Databricks Data Intelligence Platform the go-to choice for top-tier machine learning solutions. *Databricks ML in Action* presents cloud-agnostic, end-to-end examples with hands-on illustrations of executing data science, machine learning, and generative AI projects on the Databricks Platform. You'll develop expertise in Databricks' managed MLflow, Vector Search, AutoML, Unity Catalog, and Model Serving as you learn to apply them practically in everyday workflows. This Databricks book not only offers detailed code explanations but also facilitates seamless code importation for practical use. You'll discover how to leverage the open source Databricks platform to enhance your learning, boost your skills, and elevate your productivity with supplemental resources. By the end of this book, you'll have mastered the use of Databricks for data science, machine learning, and generative AI, enabling you to deliver outstanding data products.

Who this book is for

This book is for machine learning engineers, data scientists, and technical managers seeking hands-on expertise in implementing and leveraging the Databricks Data Intelligence Platform and its lakehouse architecture to create data products.

What this book covers

Chapter 1, *Getting Started and Lakehouse Concepts*, covers the different techniques and methods for data engineering and machine learning. The goal is not to unveil insights into data never seen before. If that were the case, this would be an academic paper. Instead, the goal of this chapter is to use open and free data to demonstrate advanced technology and best practices. You will list and describe each dataset present in the book.

Chapter 2, *Designing Databricks: Day One*, covers workspace design, model life cycle practices, naming conventions, what not to put in DBFS, and other preparatory topics. The Databricks platform is simple to use. However, there are many options available to cater to the different needs of different organizations. During my years as a contractor and my time at Databricks, I have seen teams succeed and fail. I will share with you the successful dynamics as well as any configurations that accompany those insights in this chapter.

Chapter 3, *Building the Bronze Layer*, begins your data journey in the Databricks DI Platform by exploring the fundamentals of the Bronze layer of the Medallion architecture. The Bronze layer is the first step in transforming your data for downstream projects, and this chapter will focus on the Databricks features and techniques you have available for the necessary transformations. We will start by introducing you to Auto Loader, a tool to automate data ingestion, which you can implement with or without **Delta Live Tables** (**DLT**) to insert and transform your data.

Chapter 4, *Getting to Know Your Data*, explores the features within the Databricks DI Platform that help improve and monitor data quality and facilitate data exploration. There are numerous approaches to getting to know your data better with Databricks. First, we cover how to oversee data quality with DLT to catch quality issues early and prevent the contamination of entire pipelines. We will take our first close look at Lakehouse Monitoring, which helps us analyze data changes over time and can alert us to changes that concern us.

Chapter 5, *Feature Engineering on Databricks*, progresses from *Chapter 4*, where we harnessed the power of Databricks to explore and refine our datasets, to delve into the components of Databricks that enable the next step – feature engineering. We will start by covering **Databricks Feature Engineering** (**DFE**) in Unity Catalog to show you how you can efficiently manage engineered features using Unity Catalog. Understanding how to leverage DFE in UC is crucial for creating reusable and consistent features across training and inference. Then, you will learn how to leverage Structured Streaming to calculate features on a stream, which allows you to create stateful features needed for models to make quick decisions.

Chapter 6, *Tools for Model Training and Experimenting*, examines how to use data science to search for a signal hidden in the noise of data. We will leverage the features we created within the Databricks platform during the previous chapter. We will start by using AutoML in a basic modeling approach, providing auto-generated code and quickly enabling data scientists to establish a baseline model to beat. When searching for a signal, we experiment with different features, hyperparameters, and models. Historically, tracking these configurations and their corresponding evaluation metrics is a time-consuming project in and of itself. A low-overhead tracking mechanism, such as the tracking provided by MLflow, an open source platform for managing data science projects and supporting MLOps, will reduce the burden of manually capturing configurations. More specifically, we'll introduce MLflow Tracking, an MLflow component that significantly improves tracking each permutation's many outputs. However, that is only the beginning.

Chapter 7, *Productionizing ML on Databricks*, explores productionizing a machine learning model using Databricks products, which makes the journey more straightforward and cohesive by incorporating functionality such as the Unity Catalog Registry, Databricks Workflows, Databricks Asset Bundles, and Model Serving capabilities. This chapter will cover the tools and practices to take your models from development to production.

Chapter 8, *Monitoring, Evaluating, and More*, covers how to create visualizations for dashboards in both the new Lakeview dashboards and the standard DBSQL dashboards. Deployed models can be shared via a web application. Therefore, we will not only introduce Hugging Face Spaces but also deploy the RAG chatbot using a Gradio app to apply what we have learned.

To get the most out of this book

Software/hardware covered in the book	Operating system requirements
Databricks	Windows, macOS, or Linux
Python and its associated libraries	Windows, macOS, or Linux

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

This book contains a few long screenshots which have been captured to show the overview of workflows and also the UI. Due to this, the content in this images may appear small at 100% zoom. Please check out the PDF copy provided with the book to zoom in for clearer images.

Download the example code files

You can download the example code files for this book from GitHub at `https://github.com/PacktPublishing/Databricks-ML-In-Action`. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

`Code in text`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: " For example, you could select the `ml_in_action.favorita_forecasting.train_set` table."

A block of code is set as follow:

```
import opendatasets as od

od.download("https://www.kaggle.com/competitions/store-sales-time-
series-forecasting/data",raw_data_path)

dbutils.fs.ls(raw_data_path + "/store-sales-time-series-forecasting/")
```

Any command line input or output is written as follow:

```
/usr/local/bin/databricks configure
```

Bold: Indicates a new term, an important word, or words that you see on screen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: " Once you have a dataset, return to the **Canvas** tab."

> **Tips or important notes**
> Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at `customercare@packtpub.com` and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit `www.packtpub.com/support/errata` and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packtpub.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

Share Your Thoughts

Once you've read *Databricks ML in Action*, we'd love to hear your thoughts! Please `click here to go straight to the Amazon review page` for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below

`https://packt.link/free-ebook/978-1-80056-489-3`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1: Overview of the Databricks Unified Data Intelligence Platform

The goal of this part is not to unveil insights into data never seen before. If that were the case, this would be an academic paper. Instead, the goal is to use open and free data to demonstrate advanced technology and best practices. This part will list and describe each dataset present in the book. It also introduces you to the successful dynamics as well as any configurations that accompany the insights in this part. This part covers workspace design, model life cycle practices, naming conventions, what not to put in DBFS, and other preparatory topics.

This part has the following chapters:

- *Chapter 1*, *Getting Started and Lakehouse Concepts*
- *Chapter 2*, *Designing Databricks: Day One*
- *Chapter 3*, *Building the Bronze Layer*

1

Getting Started and Lakehouse Concepts

"Give me six hours to chop down a tree, and I will spend the first four sharpening the axe."

– Abraham Lincoln

We will start with a basic overview of how **Databricks Data Intelligence Platform** (**DI**) is an open platform on a **lakehouse** architecture and the advantages of this in developing **machine learning** (**ML**) applications. For brevity, we will use terms such as *Data Intelligence Platform* and *Databricks* interchangeably throughout the book. This chapter will introduce the different projects and associated datasets we'll use throughout the book. Each project intentionally highlights a function or component of the DI Platform. Use the example projects as hands-on lessons for each platform element we cover. We progress through these projects in the last section of each chapter – namely, applying our learning.

Here is what you will learn in this chapter:

- Components of the Data Intelligence Platform
- Advantages of the Databricks Platform
- Applying our learning

Components of the Data Intelligence Platform

The Data Intelligence Platform allows your entire organization to leverage your data and AI. It's built on a lakehouse architecture to provide an open, unified foundation for all data and governance layers. It is powered by a **Data Intelligence Engine**, which understands the context of your data. For practical purposes, let's talk about the components of the Databricks Data Intelligence Platform:

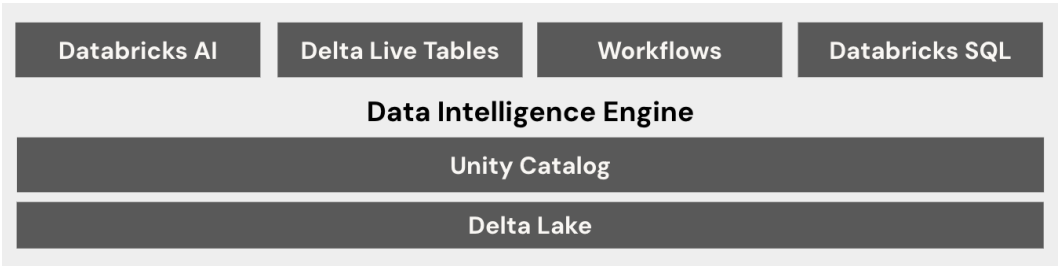


Figure 1.1 – The components of the Databricks Data Intelligence Platform

Let's check out the following list with the descriptions of the items in the figure:

- **Delta Lake**: The data layout within the Data Intelligence Platform is automatically optimized based on common data usage patterns
- **Unity Catalog**: A unified governance model to secure, manage, and share your data assets
- **Data Intelligence Engine**: This uses AI to enhance the platform's capabilities
- **Databricks AI**: ML tools to support end-to-end ML solutions and **generative AI** capabilities, including creating, tuning, and serving LLMs
- **Delta live tables**: Enables automated data ingestion and **data quality**
- **Workflows**: A fully integrated orchestration service to automate, manage, and monitor multi-task workloads, queries, and pipelines
- **Databricks SQL (DBSQL)**: An SQL-first interface, similar to how you would interact with a data warehouse, and with functionality such as text-to-SQL, which lets you use natural language to generate queries

Now that we have our elements defined, let's discuss how they help us achieve our ML goals.

Advantages of the Databricks Platform

Databricks' implementation of a lakehouse architecture is unique. Databricks' foundation is built on a Delta-formatted data lake that Unity Catalog governs. Therefore, it combines a data lake's scalability and cost-effectiveness with a data warehouse's governance. This means not only are table-level permissions managed through **access control lists** (**ACLs**) but file and object-level access are also regulated. This change in architecture from a data lake and/or a data warehouse to a unified platform is ideal – a lakehouse facilitates a wide range of new use cases for analytics, business intelligence, and data science projects across an organization. See the *Introduction to Data Lakes* blog post in the *Further reading* section for more information on lakehouse benefits.

This section will discuss the importance of open source frameworks and two critical advantages they provide – transparency and flexibility.

Open source features

How open source features relate to the Data Intelligence Platform is unique. This uniqueness lies in the concepts of openness and transparency, often referred to as the "glass box" approach by Databricks. It means that when you use the platform to create assets, there's no inscrutable black box that forces you to depend on a specific vendor for usage, understanding, or storage. A genuinely open lakehouse architecture uses open data file formats to make accessing, sharing, and removing your data simple. Databricks has optimized the managed version of Apache Spark to leverage the open data format Delta (which we'll cover in more detail shortly). This is one of the reasons why the Delta format is ideal for most use cases. However, nothing stops you from using something such as the CSV or Parquet format. Furthermore, Databricks introduced **Delta Lake Universal Format** (**Delta Lake UniForm**) to easily integrate with other file formats such as Iceberg or Hudi. For more details, check out the *Further reading* section at the end of this chapter.

Figure 1.2 illustrates the coming together of data formats with UniForm.

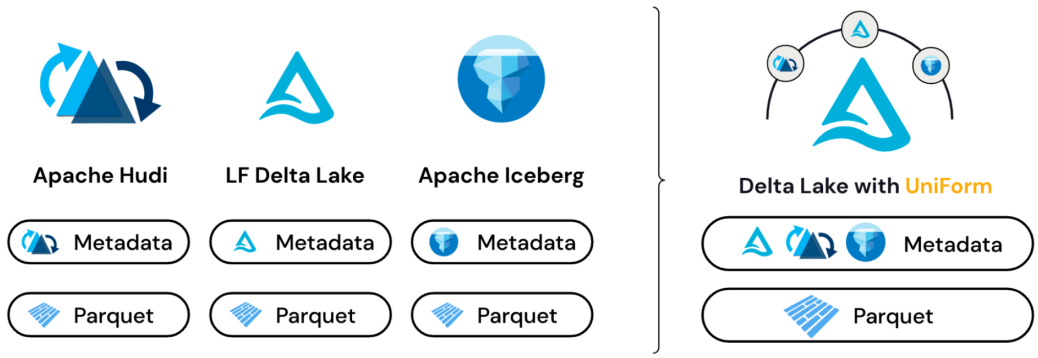


Figure 1.2 – Delta Lake UniForm makes consuming Hudi and Iceberg file formats as easy as consuming Delta

The ability to use third-party and open source software fuels rapid innovation. New advances in data processing and ML can be quickly tested and integrated into your workflow. In contrast, proprietary systems often have longer wait times for vendors to incorporate updates. Waiting for a vendor to capitalize on open source innovation may seem rare, but it is the rule rather than the exception. This is especially true for data science. The speed of software and algorithmic advances is incredible. Evidence of this frantic pace of innovation can be seen daily on the Hugging Face community website. Developers share libraries and models on Hugging Face; hundreds of libraries are updated daily on the site alone.

Delta, Spark, the Pandas API on Spark (see *Figure 1.3*), and MLflow are notable examples of consistent innovation, largely driven by their transparency as open source projects. We mention these specifically because they were all initially created by either the founders of Databricks or company members following its formation.

ML developers benefit significantly from this transparency, as it provides them with unparalleled flexibility, easy integration, and robust support from the open source community – all without the overhead of maintaining an open source full stack.

Starting development as a contractor using Databricks is super-fast compared to when companies require a fresh development environment to be set up. Some companies require a service request to install Python libraries. This can be a productivity killer for data scientists. In Databricks, many of your favorite libraries are pre-installed and ready to use, and of course, you can easily install your own libraries as well.

Additionally, there is a large and vibrant community of Databricks users. The Databricks community website is an excellent resource to ask and answer questions about anything related to Databricks. We've included a link in the *Further reading* section at the end of this chapter.

```
import pandas as pd

df = pd.read_csv(r'/folder/single.csv', sep=',', decimal='.')
df.columns = ['a', 'b', 'c']
df['a2'] = df.a * df.a
spark.createDataFrame(df).write.saveAsTable("pandas_as_table")
```

```
import pyspark.pandas as ps

df = ps.read_csv(r'/folder-of-csv-files/', sep=',', decimal='.')
df.columns = ['a','b','c']
df['a2'] = df.a * df.a
df.to_table("pyspark_pandas_as_table")
```

Figure 1.3 – The pandas API on Spark

The pandas API on Spark is nearly identical syntax to standard pandas, making distributed computing with Spark easier to learn for those who have written pandas code in Python

While continuing with a focus on transparency, let's move on to Databricks **AutoML**.

Databricks AutoML

Databricks refers to its AutoML solution as a **glass box**. This terminology highlights the fact that there is nothing hidden from the user. This feature in the Data Intelligence Platform leverages an open source library, Hyperopt, in conjunction with Spark for hyperparameter tuning. It intelligently explores different model types in addition to optimizing the parameters in a distributed fashion. The use of Hyperopt allows each run within the AutoML experiment to inform the next run, reducing the overall number of runs needed to reach an optimal solution compared to a grid search. Each run in the experiment has an associated notebook with the code for the model. This method increases productivity, reduces unnecessary computing, and lets scientists perform experiments instead of writing boilerplate code. Once AutoML has converged on the algorithmically optimal solution, there is a "best notebook" for the best scoring model. We'll expand on AutoML in several chapters throughout this book.

Reusability and reproducibility

As data scientists, transparency is especially important. We do not trust black box models. How do you use them without understanding them? A model is only as good as the data going in. In addition to not trusting the models, black boxes create concerns about our research's reproducibility and model drivers' explainability.

When we create a model, who does it belong to? Can we get access to it? Can we tweak, test, and, most importantly, reuse it? The amount of time put into the model's creation is not negligible. Databricks AutoML gives you everything to explain, reproduce, and reuse the models it creates. In fact, you can take the model code or model object and run it on a laptop or wherever. This open source, glass-box, reproducible, and reusable methodology is our kind of open.

Open file formats give you flexibility

Flexibility is also an essential aspect of the Databricks platform, so let's dive into the file format Delta, an open source project that makes it easy to adapt to many different use cases. For those familiar with Parquet, you can think of Delta as Parquet-plus – Delta files are Parquet files with a transaction log. The transaction log is a game changer. The increased reliability and optimizations make Delta the foundation of Databricks' lakehouse architecture. The data lake side of the lakehouse is vital to data science, streaming, and unstructured and semi-structured data formats. Delta has also made the warehouse side possible. There are entire books on Delta; see the *Further reading* section for some examples. We are focusing on the fact that it is an open file format with key features that support building data products.

Integration and control

Having an open file format is essential to maintain ownership of your data. Not only do you want to be able to read, alter, and open your data files, but you also want to keep them in your cloud tenant. Maintaining control over your data is possible in the Databricks Data Intelligence Platform. There is no need to put the data files into a proprietary format or lock them away in a vendor's cloud. Take a look at *Figure 1.4* () to see how Delta is part of the larger ecosystem.

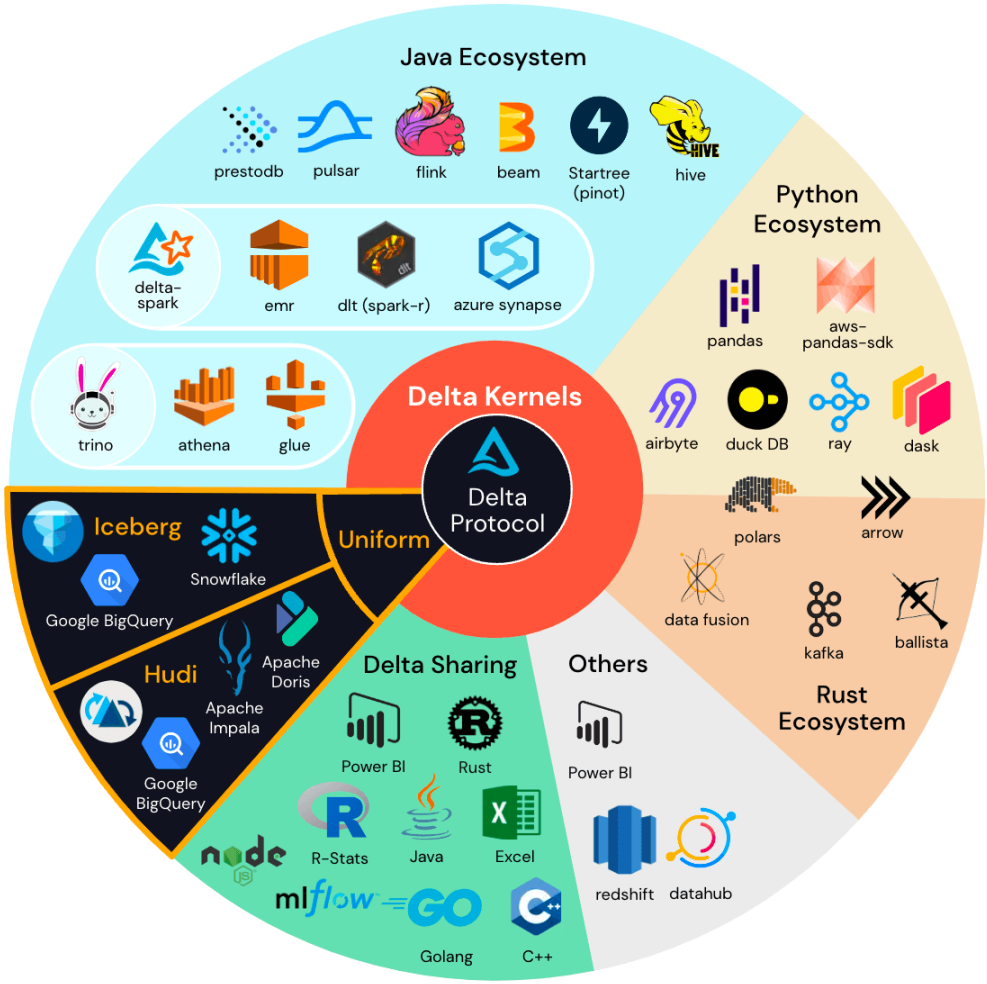


Figure 1.4 – The Delta Kernel connection ecosystem

The Delta Kernel introduces a fresh approach, offering streamlined, focused, and reliable APIs that abstract away the intricacies of the Delta protocol. By simply updating the Kernel version, connector developers can seamlessly access the latest Delta features without needing to modify any code.

Time-travel versioning in Delta

The freedom and flexibility of open file formats make it possible to integrate with new and existing external tooling. Delta Lake, in particular, offers unique support to create data products thanks to features such as time-travel versioning, exceptional speed, and the ability to update and merge changes. Time travel, in this context, refers to the capability of querying different versions of your data table, allowing you to revisit the state of the table before your most recent changes or transformations (see *Figure 1.5*). The more obvious use is to back up after making a mistake rather than writing out multiple copies of the table as a safety measure. A possibly less obvious use for time travel is reproducible research. You can access the data your model was trained on in the previous week without creating an additional copy of the data. Throughout the book, we will detail features of the Data Intelligence Platform you can use to facilitate reproducible research. The following figure shows you how the previous version of a table, relative to a timestamp or a version number, can be queried.

```
-- view table as it was by timestamp
SELECT * FROM ml_in_action.favorita_forecasting.favorita_transactions TIMESTAMP AS OF '2023-09-29 22:57:25';

-- view table as it was by version
SELECT * FROM lakehouse_in_action.favorita_forecasting.favorita_transactions VERSION AS OF 1;
```

Figure 1.5 – A code example of the querying techniques to view previous versions of a table

The speed of Databricks' optimized combination

Next, let us discuss the speed of Databricks' lakehouse architecture. In November 2021, Databricks set a new world record for the gold standard performance benchmark for data warehousing. The Barcelona Computing Group shared their research supporting this finding. This record-breaking speed resulted from the Databricks' engines (Spark and Photon) paired with Delta (see the *Databricks Sets Official Data Warehousing Performance Record* link in the *Further reading* section).

The additional benefits of Delta

Delta's impressive features include **change data feed** (**CDF**), **change data capture** (**CDC**), and **schema evolution**. Each plays a specific role in data transformation in support of ML.

Starting with Delta's CDF capability, it is exactly what it sounds like – a feed of the changed data. Let's say you have a model looking for fraud, and that model needs to know how many transaction requests have occurred in the last 10 minutes. It is not feasible to rewrite the entire table each time a value for an account needs to be updated. The feature value, or in this case, the number of transactions that occurred in the last 10 minutes, needs to be updated only when the value has changed. The use of CDF in this example enables updates to be passed to an **online feature store**; see *Chapters 5* and *6* for more details.

Finally, let's talk about change data capture, a game-changer in the world of data management. Unlike traditional filesystems, CDC in Delta has been purposefully designed to handle data updates efficiently. Let's take a closer look at CDC and explore its capabilities through two practical scenarios:

- **Scenario 1 – effortless record updates**: Picture a scenario involving Rami, one of your customers. He initially made a purchase in Wisconsin but later relocated to Colorado, where he continued to make purchases. In your records, it's essential to reflect Rami's new address in Colorado. Here's where Delta's CDC shines. It effortlessly updates Rami's customer record without treating him as a new customer. CDC excels at capturing and applying updates seamlessly, ensuring data integrity without any hassles.
- **Scenario 2 – adapting to evolving data sources**: Now, consider a situation where your data source experiences unexpected changes, resulting in adding a new column containing information about your customers. Let's say this new column provides insights into the colors of items purchased by customers. This is valuable data that you wouldn't want to lose. Delta's CDC, combined with its schema evolution feature, comes to the rescue.

Schema evolution, explored in depth in *Chapter 3*, enables Delta to gracefully adapt to schema changes without causing any disruptions. When dealing with a new data column, Delta smoothly incorporates this information, ensuring your data remains up to date while retaining its full historical context. This ensures that you can leverage valuable insights for both present and future analyses.

Applying our learning

This book is heavily project-based. Each chapter starts with an overview of the important concepts and Data Intelligence Platform features that will prepare you for the main event – the *Applying our learning* sections. Every *Applying our learning* section has a *Technical requirements* section so that you know what technical resources you will need, in addition to your Databricks workspace and GitHub repository, to complete the project work in the respective chapter.

Technical requirements

Here are the technical requirements needed to get started with the hands-on examples used throughout this book:

- We use Kaggle for two of our datasets. If you do not already have an account, you will need to create one.
- Throughout the book, we will refer to code in GitHub. Create an account if you do not already have one.

Getting to know your data

There are four main projects that progress sequentially throughout the book. In each subsequent chapter, the code will expand upon the code in previous chapters. We chose these projects to highlight a variety of Data Intelligence Platform features across different ML projects. Specifically, we include streaming data into your lakehouse architecture, forecasting sales, building a **deep learning** (**DL**) model for computer vision, and building a chatbot using **Retrieval Augmented Generation** (**RAG**) techniques. Read through the descriptions of each project to get an idea of what it will cover. If some of the concepts and features are unfamiliar, don't worry! We'll explain them in the following chapters.

Project: Streaming Transactions

This first project is a data solution for the streaming transactions dataset we will generate. The transaction data will include information such as the customer ID and transaction time, which we'll use to simulate transactions streaming in real time; see the sample data in *Figure 1.6*.

Amount	CustomerID	Label	Product	TransactionTimestamp
10.0	1240	0	Product A	2023-12-12T23:36:16.284Z
250.01	1253	0	Product A	2023-12-12T23:36:16.284Z
10.0	1254	0	Product A	2023-12-12T23:36:16.284Z
250.01	1253	0	Product A	2023-12-12T23:36:16.284Z
69.99	1244	0	Product A	2023-12-12T23:36:16.284Z
10.0	1247	0	Product A	2023-12-12T23:36:16.284Z
250.01	1242	0	Product A	2023-12-12T23:36:16.284Z

Figure 1.6 – A sample of the synthetic streaming transactions data

Our goal with this project is to demonstrate how flexible the Data Intelligence Platform is compared to proprietary data warehouses of the past, which were more rigid for data ingestion. Additionally, we want to highlight important Databricks capabilities such as **Spark Structured Streaming**, **Auto Loader**, schema evolution, Delta Live Tables, and **Lakehouse Monitoring**.

When we generate the transactions, we also generate a label based on statistical distributions (note that the label is random and only used for learning purposes). This is the label we will be predicting. Our journey includes generating transaction records as multiline JSON files, formatting the files to a Delta table, creating a streaming feature for our ML model, wrapping a **Light Gradient-Boosting Machine** (**LightGBM** or **LGBM**) model in a Python function (`pyfunc`) with the preprocessing steps, and deploying the model wrapper via a workflow. Take a look through the project pipeline in *Figure 1.7* to understand how we'll progress through this project.

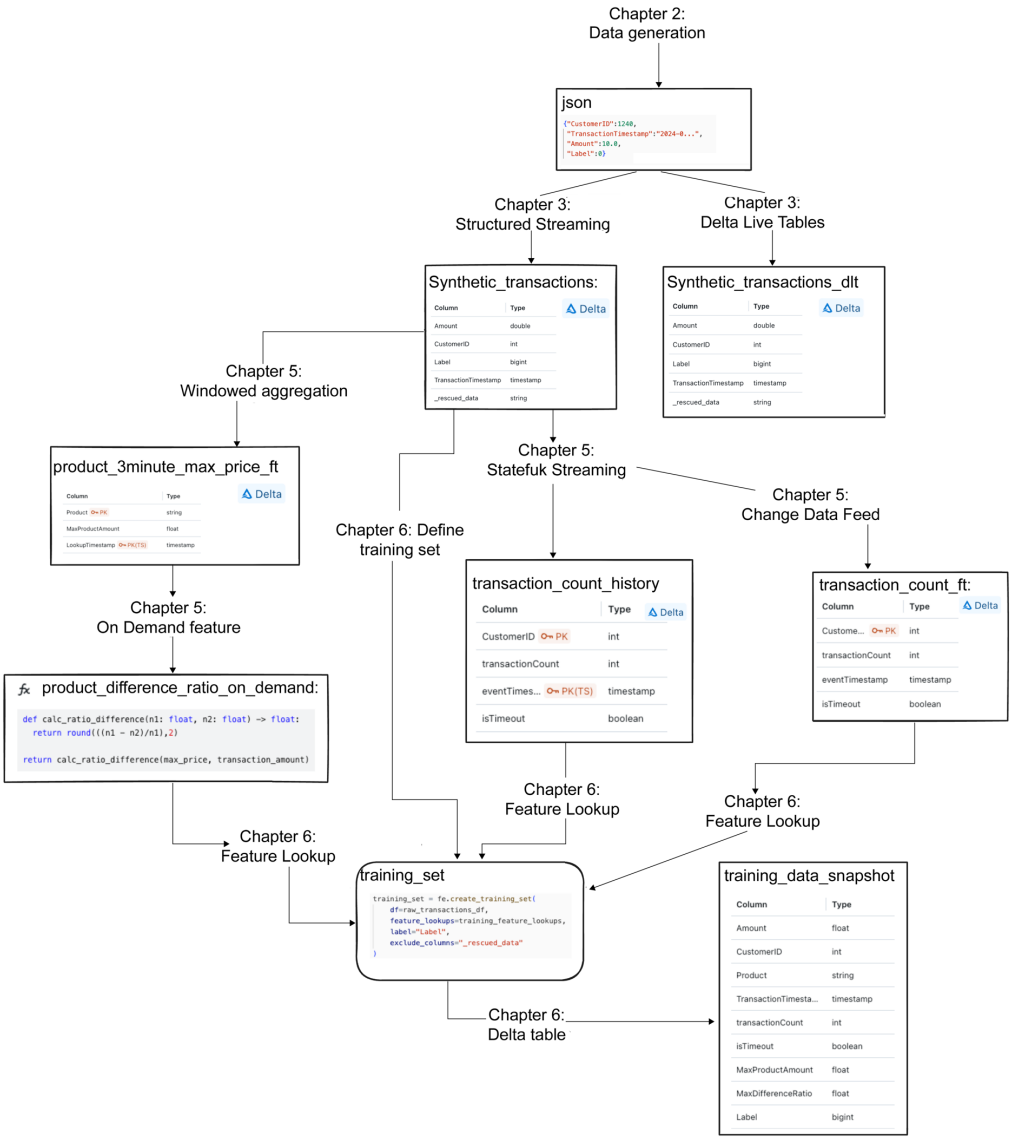


Figure 1.7 – The project pipeline for the streaming transactions project

That concludes the streaming transaction project explanation. Next, we will look at the forecasting project.

Project: Favorita Sales Forecasting

This is a typical forecasting project. Our dataset is hosted on the Kaggle website (see the *Further reading* section). We will use the data to build a model to predict the total sales amount for a family of goods at a specific Favorita store in Ecuador. The data includes train, test, and supplementary data. This project will use Databricks' AutoML for data exploration and to create a baseline model. Take a look through the project pipeline in *Figure 1.8* to understand how we'll progress through this project. The store sales dataset is a rich time-series dataset, and we encourage you to build on the project framework we provide using your favorite time-series library.

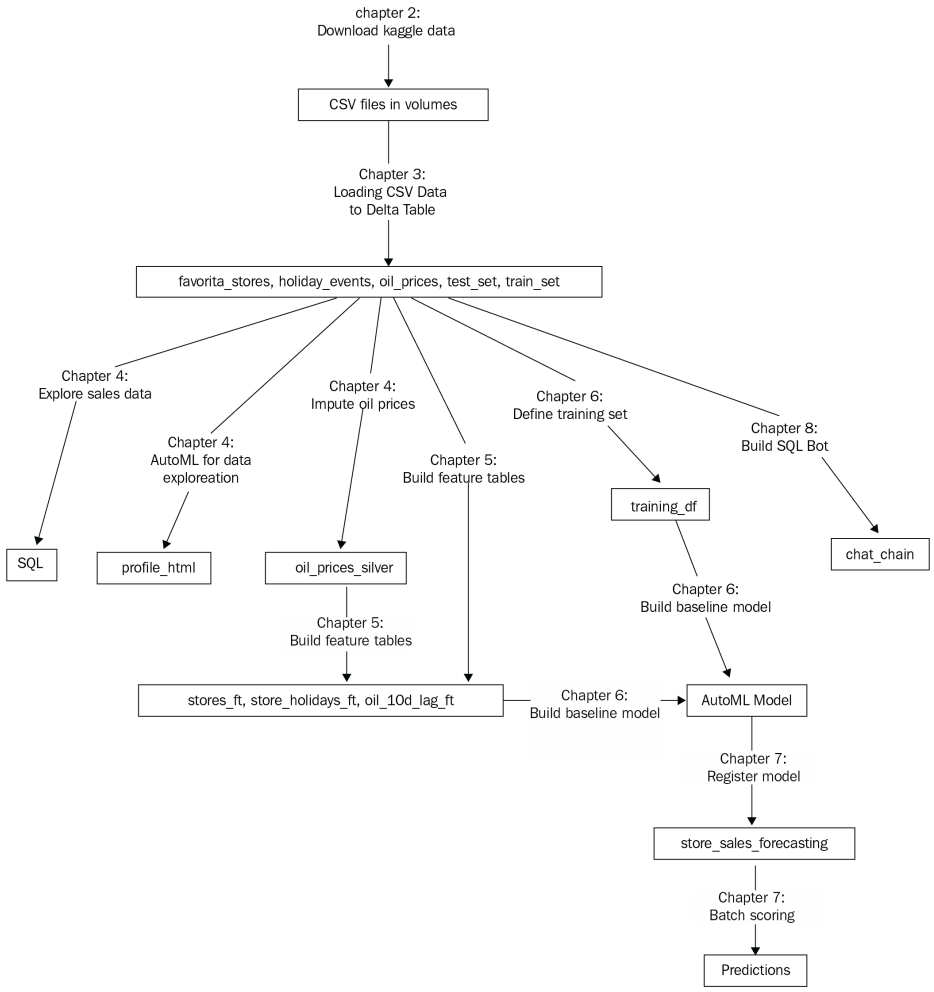


Figure 1.8 – The project pipeline for the Favorita store sales project

That concludes the forecasting project explanation. Next, we will look at the DL project.

Project: Multilabel Image Classification

This project is a DL data solution that uses another Kaggle dataset. We will use these datasets images to fine-tune a deep-learning model, using PyTorch and Lightning to predict a corresponding label. We will implement MLflow code for experiment and model tracking, Spark for fast training and **inference**, and Delta for data version control. We will deploy the model as we would for a real-time scenario by creating a model wrapper, similar to the wrapper we use for the streaming transactions project. Take a look through the project pipeline in *Figure 1.9* to understand how we'll progress through this project.

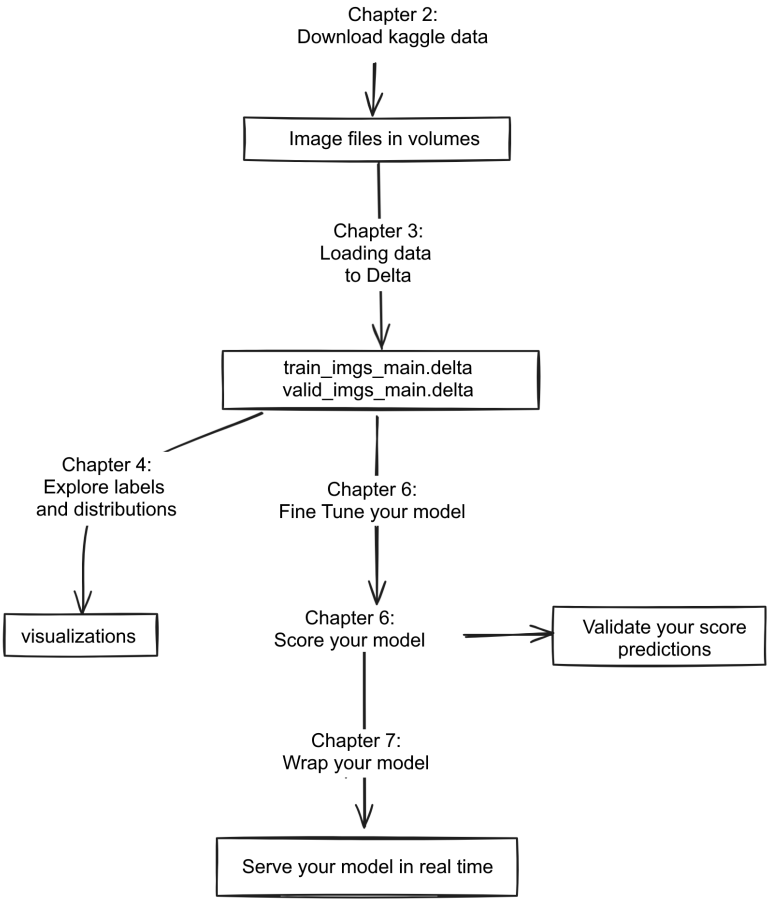


Figure 1.9 – The project pipeline for the multilabel image classification project

That concludes the image classification project explanation. Next, we will look at the chatbot project.

Project: Retrieval Augmented Generation Chatbot

This project is a RAG chatbot. The dataset we use comes from the *arXiv* website. We have selected a few research articles about the impact of generative AI on humans and labor. We will download and store them in a volume in *Chapter 2*. After we download the PDF documents, we will extract and prepare the text through a process of chunking and tokenization, creating embeddings of the documents to be referenced in the chatbot. We will use Databricks Vector Search to store the embeddings. Then, we will use the new Foundation Model API to generate answers when text is retrieved. The final bot will be deployed as an application using **Databricks Model Serving**. This example allows you to build a chatbot from start to finish! Take a look through the project pipeline in *Figure 1.10* to understand how we'll progress through this project.

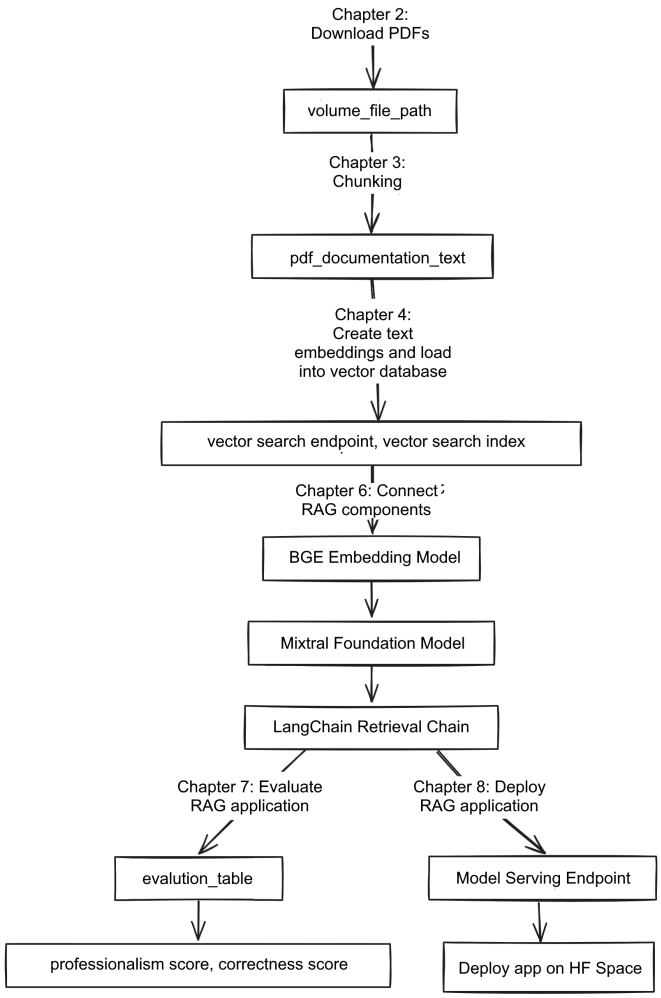


Figure 1.10 – The project pipeline for the chatbot project

The four projects are hands-on examples that can be implemented on Databricks. *Databricks ML in Action* provides best practices and recommendations from an ML perspective based on our experiences and supplements online documentation. All the code and solutions presented in this book have been developed and tested on the full version of Databricks. However, we understand that accessibility matters. There is also a free community version of the Databricks Data Intelligence Platform available, enabling everyone to follow along with the examples to a certain point before considering an upgrade.

Summary

In this chapter, we introduced you to *Databricks ML in Action*. We emphasized that the Databricks Data Intelligence Platform is designed with openness, flexibility, and tooling freedom in mind, which greatly accelerates productivity. Additionally, we've given you a sneak peek at the projects and the associated datasets that will be central to this book.

Now that you've gained a foundational understanding of the Data Intelligence Platform, it's time to take the next step. In the upcoming chapter, we'll guide you through setting up your environment and provide instructions on downloading the project data. This will prepare you for the practical, hands-on ML experiences that lie ahead in this journey.

Questions

Let's test ourselves on what we've learned by going through the following questions:

1. How will you use this book? Do you plan to go cover to cover or pick certain sections out? Have you chosen sections of interest?
2. We covered why transparency in modeling is critical to success. How does Databricks' glass-box approach to AutoML support this?
3. Databricks has developed a new way of uniting the open data formats, called UniForm. Which data formats does UniForm unite?
4. Delta is the foundation of the lakehouse architecture. What is one of the benefits of using Delta?
5. What is the main advantage of using the Delta file format for large-scale data processing over simple Parquet files in Databricks?

Answers

After putting thought into the questions, compare your answers to ours:

1. We cannot answer this question, but we hope you learn something you can use in your career soon!
2. The glass-box approach supports transparency by providing the code run for each run in the experiment and the best run, thus enabling reusability and reproducibility.

3. Apache Iceberg, Apache Hudi, and Linux Foundation Delta Lake (an open source/unmanaged version of Delta).
4. There are several. Here are a few:
    - Open protocol (no vendor lock-in)
    - Speed
    - Change data capture
    - Time travel
5. While Parquet also provides columnar storage and has efficient read/write operations, its lack of ACID transaction capabilities distinguishes Delta Lake.

Further reading

In this chapter, we introduced vital technologies. Look at these resources to go deeper into the areas that interest you most:

- YouTube video – *Introduction to Databricks Data Intelligence Platform*: `https://youtu.be/E885Ld3N2As?si=1NPg85phVH8RhayO`
- *Introduction to Data Lakes*: `https://www.databricks.com/discover/data-lakes`
- *5 Steps to a Successful Data Lakehouse* by Bill Inmon, father of the data warehouse: `https://www.databricks.com/resources/ebook/building-the-data-lakehouse`
- *Delta Lake: Up & Running* by O'Reilly: `https://www.databricks.com/resources/ebook/delta-lake-running-oreilly`
- *Delta Lake: The Definitive Guide*: `https://www.oreilly.com/library/view/delta-lake-the/9781098151935/`
- *Comparing Apache Spark and Databricks*: `https://www.databricks.com/spark/comparing-databricks-to-apache-spark`
- *Databricks MLflow*: `https://www.databricks.com/product/managed-mlflow`
- *Databricks Community Edition FAQ*: `https://www.databricks.com/product/faq/community-edition#:~:text=What%20is%20the%20difference%20between,ODBC%20integrations%20for%20BI%20analysis`
- *Delta 2.0 - The Foundation of your Data Lakehouse is Open*: `https://delta.io/blog/2022-08-02-delta-2-0-the-foundation-of-your-data-lake-is-open/`
- *Delta Lake Integrations*: `https://delta.io/integrations/`

- *Delta vs Iceberg*: https://databeans-blogs.medium.com/delta-vs-iceberg-performance-as-a-decisive-criteria-add7bcdde03d
- *UniForm*: https://www.databricks.com/blog/delta-uniform-universal-format-lakehouse-interoperability
- *Delta Kernel: Simplifying Building Connectors for Delta*: https://www.databricks.com/dataaisummit/session/delta-kernel-simplifying-building-connectors-delta/
- *Databricks Community Website*: https://community.cloud.databricks.com
- *Podcast: Delta Lake Discussions with Denny Lee*: https://open.spotify.com/show/6YvPDkILtWfnJNTzJ9HsmW?si=214eb7d808d84aa4
- *Databricks Sets Official Data Warehousing Performance Record*: https://www.databricks.com/blog/2021/11/02/databricks-sets-official-data-warehousing-performance-record.html
- *LightGBM*: https://github.com/microsoft/LightGBM https://lightgbm.readthedocs.io/en/latest/
- *Kaggle | Store Sales*: https://www.kaggle.com/competitions/store-sales-time-series-forecasting/overview
- *Kaggle | Multi Label Image Classification*: https://www.kaggle.com/datasets/meherunnesashraboni/multi-label-image-classification-dataset
- *Store Sales - Time Series Forecasting*: Kaggle.com/competitions/store-sales-time-series-forecasting/overview
- *arXiv website*: https://arxiv.org

2

Designing Databricks: Day One

"Design is not just what it looks like and feels like. Design is how it works."

- Steve Jobs

This chapter will introduce concepts and topics that engineers, data scientists, and people in similar roles should know to set themselves up for success in the Databricks Data Intelligence Platform. When setting up your data and AI platform, in our case, Databricks, there are always best practices to follow. We share those in this chapter to give you a better understanding of the setup options and their impacts; these can be strategic decisions that impact the entire data product workflow, as well as simply matters of preference. We start by explaining Databrick's general architecture and key terminology, then cover the most important decisions to be made during platform setup, and conclude with code examples and configurations to download the data for our example projects. We also introduce a variety of platform features and components throughout the chapter, which we will cover in more detail throughout the rest of this book. Here is what you will learn as part of this chapter:

Here is what you will learn about as part of this chapter:

- Planning your platform
- Defining a workspace
- Selecting the metastore
- Discussing data preparation
- Planning to create features
- Modeling in Databricks
- Applying learning

Planning your platform

This section covers topics for discussion before and during the DI Platform setup process. The role of the data team often determines the platform setup. One of Databricks' ideal attributes is that the

technology stack is unified, making the setup and collaboration between teams more straightforward. The data team reporting structure frequently determines the border where one role ends and another begins, rather than the actual data product workflow. Luckily, we do not have to worry because the DI Platform serves data engineers, scientists, and analysts alike.

In *Figure 2.1*, you can see an end-to-end lakehouse architecture and the components in Databricks.

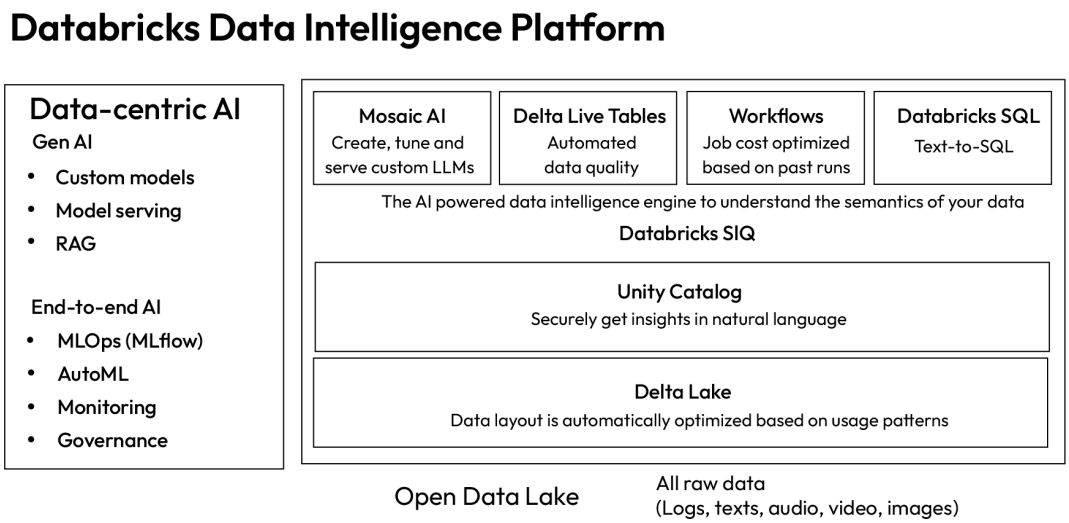


Figure 2.1 – Overview of a lakehouse architecture and how Databricks DI Platform fits this paradigm

The DI Platform consists of one or more Databricks accounts. Most of the time, companies only have one. However, there are situations where companies require extra environment isolation, and having separate accounts for development, staging, and production is an option. Discussion about multiple accounts for levels of isolation is outside of this book's scope, but if you have questions or want to know more, please check out the resources in *Further reading*.

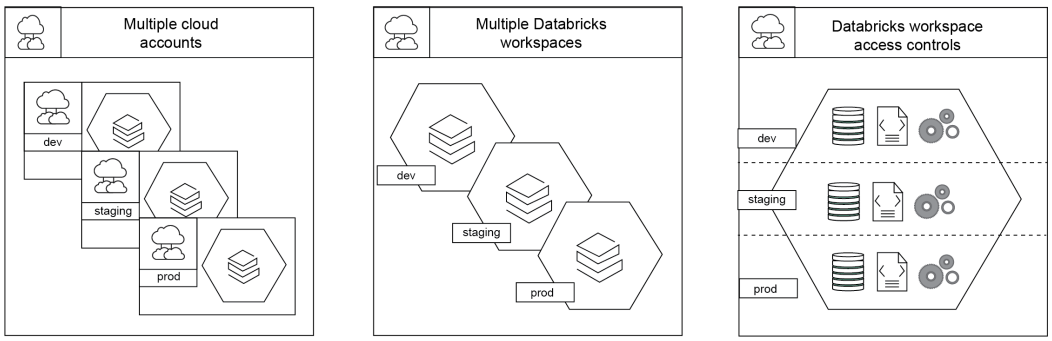


Figure 2.2 – Visual representation of the environment isolation options

We separate our environments using different catalogs. Most of this book's project work occurs using the `ml_in_action` catalog. For the production version of some models, we use the `ml_in_prod` catalog. Setting up multiple workspaces is another way to separate environments. We recommend using documentation and your company policies to guide your isolation setup. Let's move on to what precisely a workspace is in the context of Databricks.

Defining a workspace

It's important to know that Databricks uses the word **workspace** to refer to two distinct components: an instance of Databricks (meaning your hosted Databricks deployment that you access via your unique URL address) and the folder environment for accessing your work products, like notebooks, queries, and dashboards.

Let's go through the two components:

- **Workspace as an instance**: A Databricks account can have multiple workspaces attached to it, meaning instances of the DI Platform are deployed and often accessible from a browser, as mentioned previously, but are also accessible via an SDK or a REST API.
- **Workspace as a folder**: Workspace also refers to the folder that contains your user's home folder, repositories, projects, and a shared folder that is visible to all users on the workspace instance. Often, users set their main system path to their `Workspace` folder to store their MLFlow experiments or Terraform states for pipeline deployment. You can also create and store notebooks outside source control in your home and project folders.

We now have a clearer understanding of a workspace. Now let's discuss why we choose **Unity Catalog** (**UC**) as our preferred metastore.

Selecting the metastore

A metastore is a system that stores metadata for a data platform and can be thought of as the top-level container of objects. It registers a variety of information about databases, tables, views, **User-Defined Functions** (**UDFs**), and other data assets. Metadata includes details such as storage location and the permissions that govern access to each asset.

Two types of metastores are natively available in the DI Platform: **Unity Catalog** (**UC**) and the **Hive Metastore** (**HMS**). UC has a three-level namespace consisting of a catalog, a database (also called a schema), and a table name. In contrast, the HMS only uses a two-level namespace containing just a database and table name. A metastore is required for your Databricks Workspace instance, as this is the component that organizes and governs data access. Deciding on the right metastore is an early decision in your DI Platform journey, and we recommend Unity Catalog. Let's talk about why.

Notice in *Figure 2.3* that you can have multiple workspaces assigned to the same metastore. **Access Controls** and **User Management** are scoped to the account level, as shown in the figure. A UC **Metastore**, a group of catalogs, is scoped to a region with precisely one metastore per region. Within the region, you can easily share **Data**, **Features**, **Volumes** access, **Functions**, and **Models**.

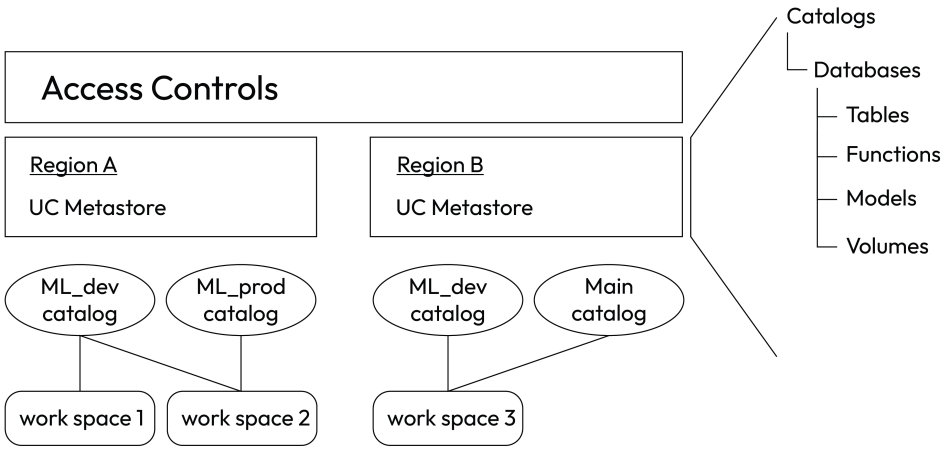


Figure 2.3 – The Design of Unity Catalog with multiple workspaces

Unity Catalog is more than a group of data assets. UC also tracks who has accessed assets, which makes auditing a simple exercise. Using UC allows companies to administer privileges and secure data and objects easily while being able to share them between various workspaces. Securely sharing between environments is one of the reasons why we recommend using the UC metastore.

The HMS design is less centralized than that of UC. For example, historically, workspaces have been created as data and code isolation, meaning there is a separate workspace for separate isolation levels. This design often required a centralized model registry workspace in addition to development, staging, and production workspaces. If not using UC, each workspace requires its own HMS and user and group management. In contrast, UC governs all assets at the account level rather than the individual workspace level; see *Figure 2.3*, *Figure 2.4*, and *Further reading*. The centralized governance model provides the ability to integrate multiple workspaces more seamlessly.

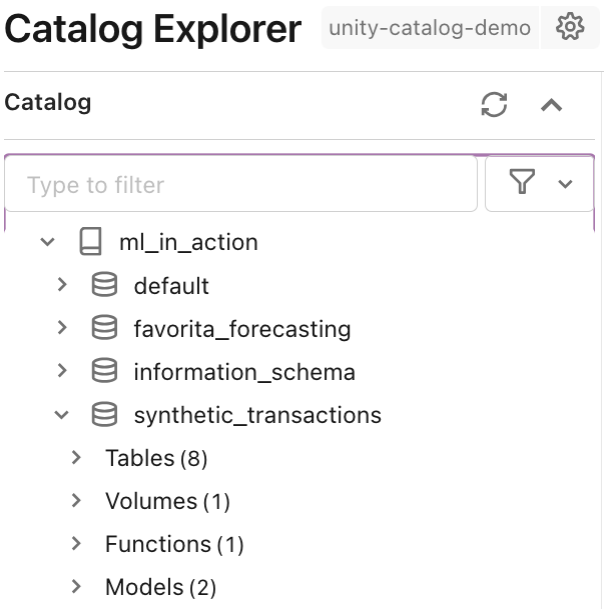


Figure 2.4 – UC governs all assets under catalogs, including databases, tables, volumes, functions, and models

Deciding on your metastore does not have to be a permanent choice. However, migrating later could become a headache. UC is continually improving and integrating with new Databricks features. The list of reasons to choose UC over HMS continues to grow, and our recommendation is to begin with and stick with UC.

To determine whether Unity Catalog is the right choice for you and your company, you can check out the shrinking list of limitations for choosing UC in *Further reading*. As UC continues to expand in capability, it is the path of the future. Specifically, for machine learning, there is an integration with the Feature Engineering client and the new Model Registry. Using the UC Model Registry for model sharing and governing is simpler. We will cover more about the Model Registry in Unity Catalog and Databricks Feature Engineering Client in *Chapters 5, 6*, and *7*, but if you're curious and eager to learn more now, you can check out Manage model lifecycle in Unity Catalog in the Further reading section. Given the ever-growing number of reasons to use UC, all project code in this book will use UC.

Defining where the data lives, and cloud object storage

All data products start with data, and so how we make the data accessible for data teams is another important early design choice. Databricks is a cloud-based platform that connects to cloud object storage – **Azure Data Lake Storage** (**ADLS**), Amazon **Simple Storage Service** (**S3**), or **Google Cloud Storage** (**GCS**). There is a separation of compute and storage. Databricks orchestrates compute; the data is in cloud object storage.

Originally, data had to be in cloud object storage before being utilized on the DI Platform. Now, **Query Federation** allows customers to query their data no matter where it resides (see the documentation for any possible limitations) without first worrying about ingestion and data engineering from that remote system. However, that is data in memory, not persistent data. You can land your data in cloud storage in various ways. There are many documentation sources and external tools for the actual landing of data in cloud storage. These may depend on your cloud service provider of choice. Despite best practices of storing data in your cloud storage, using the **Databricks File System** (**DBFS**) to store the data for this book's example projects is also possible.

DBFS is a shared filesystem provided by Databricks that all users of a given workspace can access. Any data stored in DBFS is potentially accessible to all users, regardless of their group, role, or permissions. Therefore, only non-sensitive and non-production data you are willing to share openly across your organization should be in DBFS. An example of non-sensitive data would be the publicly available *Kaggle* datasets. This lack of governance is why we recommend storing data in Databricks volumes, where you can apply governance. We will cover more on volumes in the last section of this chapter.

When it comes to cloud storage, the optimal format for structured data is almost always Delta, which we talked about in detail in *Chapter 1*. When a table is stored in the Delta format, we refer to it as a Delta table. You can choose tables to be "managed" or "external" tables. We use both types of tables in this book (the choice is justified when required). Please see the resources in *Further reading* for more information on the two types of tables.

Discussing source control

Whether or not to use a source control is usually not the question. The question is how should someone use it? Databricks has a few features that can aid in source control.

The first is the version history that is built into notebooks. The history of changes for each notebook is tracked even before submitting it to a remote repository using Git. Version tracking is beneficial as we are not always ready to make a Git commit but still want to track progress and collaborate. It's also a game changer if you accidentally pull someone's code into your working remote branch and forget to push your code before it. The notebook's history will keep your edited copy so you can simply roll back in time and restore all your work!

The second feature is the ease of connecting notebooks and files in your workspace to a remote Git repository. Historically, saving Jupyter notebooks to remote repositories was a technical nightmare for code reviews, sharing, and diffs. The Databricks code repository integration allows Databricks notebooks to contain multiple languages (Python, Scala, SQL) and track them as nicely as a typical Python file. This ability to track notebooks in source as a standard file is an improvement for data engineers and scientists wanting to review notebooks compared to previously converting files to Python and losing all output and images. The days of setting up hooks to automatically save your notebook as a regular Python file every time you save your notebook are over.

Of course, you can store standard file formats such as markdown, delimiter separated, JSON, or YML for a whole reproducibility approach. Note that we do not recommend keeping data under repos unless it's a data sample for testing.

Within a repository, how a team defines the expected folder structure for each project is generally less important than the consistent use of that structure. However, defining your project structure is still important. We recommend reading through *Big Book of MLOps* (Joseph Bradley, Rafi Kurlansik, Matthew Thomson, and Niall Turbitt, 2023, *Big Book of MLOps, second edition*, `https://www.databricks.com/resources/ebook/the-big-book-of-mlops`) to determine the best structure for your team or organization. As we will see in future chapters, MLflow and repositories are essential for **reproducible research**. In the world of data science and machine learning specifically, we want to ensure the reproduction of models and experiments.

Discussing data preparation

Generally, the first step for any data science project is to explore and prepare the data. We will refer to this process as moving the data from "Bronze" to "Silver" layers in reference to the Medallion architecture methodology. You might think of this type of data transformation exclusively as a data engineering task, but it's also essential for data science and machine learning.

If you aren't familiar with this architecture terminology, the **Medallion architecture** is a data design pattern used to organize data logically in a warehouse. This architecture is also commonly called "multi-hop" architecture. It aims to incrementally and progressively improve the structure and quality of data as it flows through each layer. The Medallion architecture has three layers: Bronze, Silver, and Gold, listed as follows:

- The **Bronze** layer is the raw data layer. It contains all the raw, unprocessed data ingested from the source systems. This data still needs to be cleaned or transformed.
- The **Silver** layer is the validated data layer. It contains data that has been cleaned and is subject to various validation and transformation steps. This data is ready to be used for analysis and modeling.
- The **Gold** layer is the enriched data layer. It is the highest level and contains data enriched with additional information, such as business intelligence metrics and key performance indicators, to meet the requirements of the business users.

The Medallion architecture is a flexible and customizable architecture that can meet the specific needs of each organization. The Medallion architecture is compatible with the concept of Data Mesh. Data Mesh is an architectural and organizational paradigm to ensure value from data. Lakehouse and Data Mesh are complementary, paradigms. See *Further reading* for blog posts on leveraging a data mesh with the DI Platform. This distributed data architecture enables organizations to unlock the value of their data by making it accessible and usable by everyone in the organization.

Communication and collaboration are vital for the data preparation process. This step involves identifying and correcting errors, filling in missing values, and resolving inconsistencies in the data. The actions you take should be discussed as a team and documented. This is especially important when working collaboratively across data teams because data engineers and data scientists often have different perspectives on how data should be prepared. For example, we have seen situations where an engineer imputed all the missing values in a column with a zero. The rationalization made sense; many zeros were already in the column, making the KPIs' values come out correctly. However, from a modeling perspective, missing data differs from zeros, especially if there are already zeros in the dataset. The approach of replacing nulls with zeros is not necessarily incorrect; it simply needs to be discussed with the downstream consumers of the data. One helpful communication tool is the column tagging functionality from the Databricks Catalog UI. See *Figure 2.5* for an example:

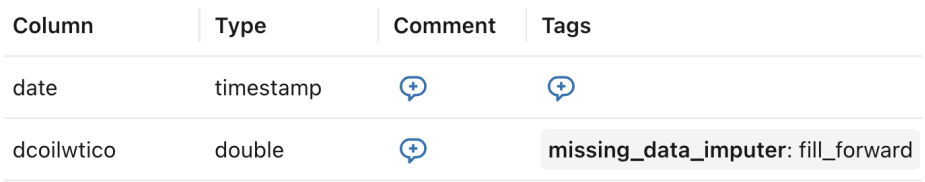


Figure 2.5 – Example of how to use tagging in a catalog to communicate the transformation performed on a column to all table users

This implementation of an incorrect imputation method also serves as an example of wanting to go back and reprocess history. Luckily, using the Medallion architecture methodology is a saving grace. In the situation mentioned previously, the chosen imputation would only be present in the Silver and Gold data layers. Meanwhile, the Bronze layer still contains the original raw data, so the source of truth is not lost, and reprocessing is possible.

One of the ways that the Databricks Platform boosts productivity and collaboration is the feature of real-time collaboration support for notebooks. This feature allows two or more people to simultaneously see and edit a notebook. The ability to pair-program virtually during the pandemic was a lifesaver for many. We're big fans of people who have worked remotely for much of our careers. Collaborative editing of a notebook is much easier than sharing code via video call. While there are many options for reviewing code, historically, reviewing code in notebooks has been difficult, particularly when committing notebooks to source control.

After completing transformations, documentation is easy using markdown in notebooks. Even if the resulting notebook is not not itself designated for production ETL, documenting the how and why of your data transformations is important for all downstream users. To read more about Databricks notebooks, see the documentation in *Further reading*.

Planning to create features

A data engineer may build Gold tables from Silver tables for consumption by the business. At the same time, a data scientist is building features from the same Silver tables for models. If we aren't careful, two people working separately without communication can create different versions of the same metrics. When architecting your unified DI Platform, be sure to think about reusability and maintainability. For this reason, with features specifically, the features-as-code approach is our recommendation. Features-as-code refers to the software development practice *everything is code*, with a focus on creating a repository of reusable code to define features rather than features stored in tables.

You can implement features-as-code in various ways. Initially, we mainly focus on function reusability. You can place functions you execute in multiple notebooks or scripts in a folder within the repository root directory. In the *Applying our learning* section, you will see this is where we store functions even when not calculating a feature per se. We call these the utils. You will be referencing the `mlia_utils` notebook throughout the example projects.

You can find the `mlia_utils` functions in the root folder of the GitHub repository (`https://github.com/PacktPublishing/Databricks-Lakehouse-ML-In-Action`). We walk through pulling the GitHub repository into Databricks in the *Applying our learning* section. In it, you will find Python files containing useful functions we will use in the projects. It is best practice to save, share, and track functions so that the metrics and features we calculate are consistent. Note the empty `__init__.py` file is also in the `utils` folder. Having an `__init__.py` file is required. With this structure, we can use all functions as imports, for example, from `mlia_utils.rag_funcs` import `extract_doc_text`.

Features-as-code is not only a way to reduce duplicative work by reusing functions. It is also a great way to ensure consistent business metrics. For example, online advertising often has complex calculations for the different types of revenue. Therefore, if other people or teams calculate business-critical metrics differently, it will be hard to establish the true metric value. Instead, you can often avoid this confusion by providing executive-approved functions for use. We will talk about this again in *Chapters 5* and *6*. In addition to features being centrally located and thus easier to find, Databricks offers easy ways to document your data assets.

Creating a business process requiring the team to document tables and functions with descriptions makes current and previous efforts more discoverable. In the Databricks Catalog UI, you should see *AI-generated* suggestions to fill in your table and column descriptions, so you don't have to start from scratch. Another great way to document transformations performed on a data table would be using tags. Tags can help with documentation and communication. Recall the example of missing data being imputed (*Figure 2.5*).

Focusing on ML features more specifically, you will learn how to store and serve feature functions to simplify your final deployment process. You will create an on-demand feature function and use it in your model. We also will show you how to leverage saved feature tables to create training sets. If you want to jump ahead right away, see *Chapter 5*, where we cover topics such as on-demand feature functions, feature lookups, syncing to the online store, and the Databricks Feature Engineering client.

Modeling in Databricks

After features have been created and stored as feature tables, we create training sets and focus on model training. We will cover modeling in terms of leveraging Databricks to facilitate the model lifecycle in *Chapter 6*. In *Chapter 7*, we'll discuss the Unity Catalog Registry and how to use it to track an enormous amount of information from the associated experiments, in addition to details such as model lineage. You can register multiple versions of a model at every stage and can give these different versions aliases, such as **champion** and **challenger**, or a more specific alias referring to versions A and B in an A/B test. See aliasing in *Figure 2.6*.

Figure 2.6 – A user can alias a model with specific names for A/B testing or multi-armed bandits

In *Chapter 7*, we demonstrate how to trigger a testing script to test every model before having a human review it. Testing models is an efficient practice to reduce the time to production when used consistently and with intention. We suggest defining the criteria for successfully transitioning models/code through isolation environments (from development to stage to production). Clearly defined environments are one of the practices that enable you to create clear and consistent model quality expectations across all models. Be sure to consult *The Big Book of MLOps* on best practices for isolation and model promotion. No matter where your environment is, it is beneficial to incorporate logging into model experimentation.

We discuss logging in the ML context rather than the software development sense. Logging for ML is focused on reproducible research and is also known as experiment tracking. It is common practice to track the following:

- The input data used to train a model
- The parameters used to train a model
- The accuracy and speed performance of a model during training and inference
- The errors that occur during training and inference
- The runtime environment of a model

When using MLflow, you have access to a powerful feature called automatic logging, or autologging. Autologging is excellent because it makes it easy to track the parameters, metrics, and artifacts of your machine learning experiments without explicit instructions.

> **Note**
>
> Auto logging only tracks flavors supported by MLflow. Custom *pyfunc* models are not supported. For more information, check *Further reading*.

MLflow auto logging logs parameter values and models for each run in a single experiment. Every time you train and evaluate your model, MLflow logs your standard metrics and parameters. If you have custom metrics to track with your models, you can also easily add them. We demonstrate tracking custom metrics in *Chapter 6*, when we log parameters for the streaming transactions model.

An admin can enable auto logging for all notebooks attached to interactive clusters at the workspace level. At the cluster level, you can add `spark.databricks.mlflow.autologging.enabled=true` to the advanced section of your cluster configuration to turn on auto logging with a cluster scope. It is less common but possible to enable auto logging within a notebook scope by adding `mlflow.autolog()` to a Python cell in a notebook. Be sure to check the list of modeling flavors supported by autolog.

By default, MLflow saves the tracked items in the managed folder in DBFS (which will be in UC in the future). You can also set `artifact_location` to point to a volume path, which is what we do in the example projects. You also have the option to set the location to another cloud storage location, although doing so eliminates the ability to see your experiments in the MLflow UI.

The MLflow UI makes it incredibly easy to compare each trail; see *Figures 2.7* and *2.8*.

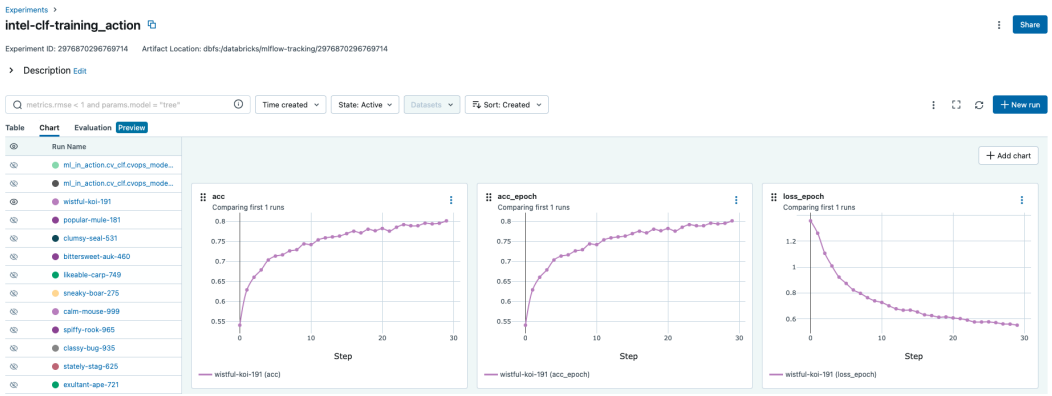


Figure 2.7 – Comparison of experiment runs using the MLflow UI

There are numerous options for visualizing the results of experiments tracked using MLflow.

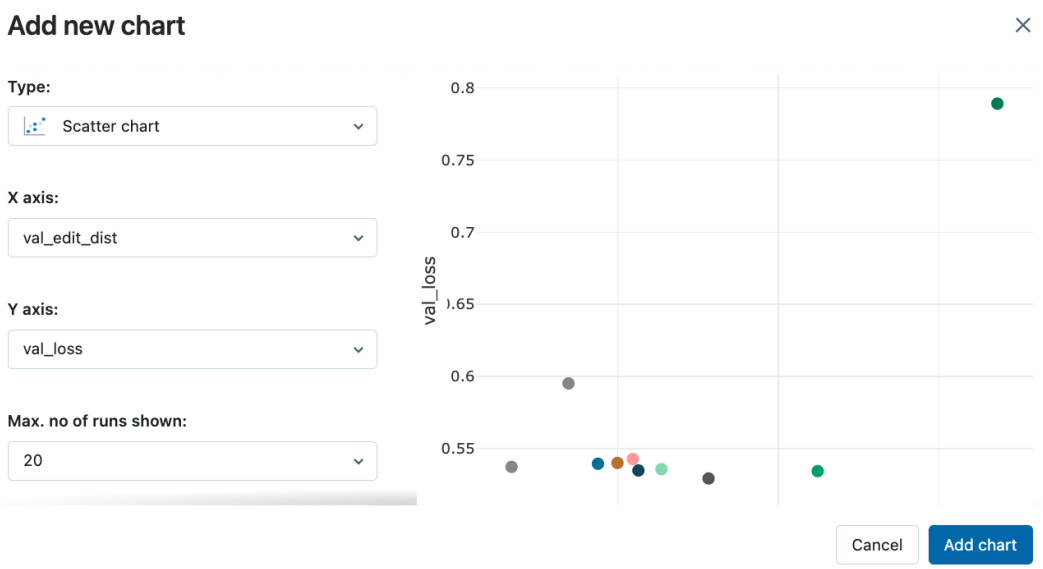


Figure 2.8 – Graphically comparing parameters and model performance in the MLflow UI

We've examined how to compare parameters and model performance in your experiment using the MLflow UI. Next, we'll look at how to use model monitoring (Lakehouse monitoring) to keep track of your model's performance over time.

Monitoring data and models

When we think about model monitoring and how to implement it, it becomes less about the actual model itself and more about the model's input and output. For this reason, Databricks Lakehouse Monitoring focuses on monitoring a model's input and output, which is simply data. The computation of table metrics occurs in the background using serverless compute, or as we like to call it, managed compute. Fully managed compute abstracts away the complexities and optimization so users focus on which tables to monitor, known as primary tables, rather than how. Lakehouse Monitoring is currently in public preview, meaning not all information is ready for release. For the latest on this feature, check out the Lakehouse Monitoring product page. We demonstrate how to use Lakehouse Monitoring in *Chapters 4 and 7*.

We've touched on a wide variety of topics so far, from the earliest design decisions when setting up your Databricks Data Intelligence Platform to the key topics we'll cover throughout the rest of this book. Now let's dive into the example projects. Get ready to follow along in your own Databricks workspace as you work through setting up your workspace.

Applying our learning

This chapter's *Applying our learning* section focuses on getting your Databricks workspace set up and ready for each project we'll be working through. We'll also go over getting set up in Kaggle so that you can download the datasets we will use throughout the rest of this book. Let's get started!

Technical requirements

Before we begin setting up a workspace, please review the technical requirements needed to complete the hands-on work in this chapter:

- We utilize a Python package, `opendatasets`, to download the data we need from the Kaggle API easily.
- We use the Databricks Labs Python library, `dbldatagen`, to generate synthetic data.
- To use the Kaggle API, you must download your credential file, `kaggle.json`.
- A GitHub account is beneficial for connecting Databricks and the code repository for the book (`https://github.com/PacktPublishing/Databricks-ML-In-Action`). In addition to a GitHub account, it is ideal to fork the book repository into your GitHub account. You will see that each chapter has a folder, and each project has a folder under the chapters. We will refer to the notebooks by name throughout the project work.
- We will use the Databricks Secrets API to save both Kaggle and OpenAI credentials. The Secrets API requires the Databricks CLI. We will walk through this setup. However, you will need to create a **personal access token** (**PAT**) on your own for the configuration step: `https://docs.databricks.com/en/dev-tools/auth/pat.html`
- The compute clusters we use are as follows (they vary slightly depending on your data cloud):
  - Single-node CPU configuration

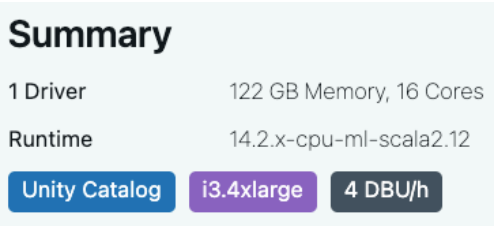


Figure 2.9 – Single-node CPU cluster configuration, DBR ML 14.2

This will work for most workloads in this book.

Setting up your workspace

As defined previously, the workspace discussed in this section refers to the deployment instance. Here, we will discuss workspace setup recommendations, project setup files, and download instructions for each dataset used throughout this book.

There is comprehensive documentation on deploying your workspace for the first time. If you do not already have a Databricks account and deployed workspace, then you have a couple of places to start from. One method is going into your cloud account and activating Databricks through the marketplaces. Another method is to begin on the Databricks website. For more advanced users, consider using Terraform. Given the amount of documentation and the ever-changing world of technology, we leave the exercise of activating a workspace up to you.

Once we have a workspace deployed, we can begin setting it up. Generally, we start with user groups and governance. The experience of setting up Unity Catalog is frequently updated for simplicity. Therefore, we recommend you watch the latest video documentation on how to do so (see *Further reading*). The process is the same, regardless of the data persona using the platform. Please be sure to complete metastore and governance setup before going forward.

Kaggle setup

You will need a Kaggle account to download the Kaggle datasets we'll be using, which require an API token for authentication. There is an official Kaggle API, but, there are numerous other ways to connect to Kaggle to download data and interact with the Kaggle site as well. All methods require downloading your API credentials file, `kaggle.json`, from the Kaggle website. Before downloading data, you need to make your credentials accessible. Here are three methods for accomplishing this:

- **Option 1**: Upload the `kaggle.json` file to your project folder. If you choose to do this, be aware that your credentials are viewable to others, even if only admins. Also, add `kaggle.json` to your `.gitignore` file to prevent committing your credentials to the repository and ensure you do not commit your credentials to a Git repository.
- **Option 2**: Paste your credentials into a notebook with the same concerns as option one. Keeping secret API keys in your Databricks notebook is far from best practice, but it is the simplest option and you can lock down your notebook access and add a configuration notebook to your `.gitignore` file to prevent committing your credentials to the repository. However, the ability to remove other users' access may not be in your control, depending on your role. Furthermore, in general, admins can see all files.

```
import os
os.environ['kaggle_username'] = 'YOUR KAGGLE USERNAME HERE'

os.environ['kaggle_key'] = 'YOUR KAGGLE KEY HERE'
```

Figure 2.10 – Passing user credentials to the notebook

- **Option 3**: Use Databricks secrets to store and retrieve your username and token, for optimal security, as shown in *Figure 2.11*. This is the method we use for downloading images.

```
import os

os.environ['kaggle_username'] = dbutils.secrets.get("lakehouse-in-action",
"kaggle_username")

os.environ['kaggle_key'] = dbutils.secrets.get("lakehouse-in-action", "kaggle_key")
```

Figure 2.11 – Using secrets to store and retrieve your username and token

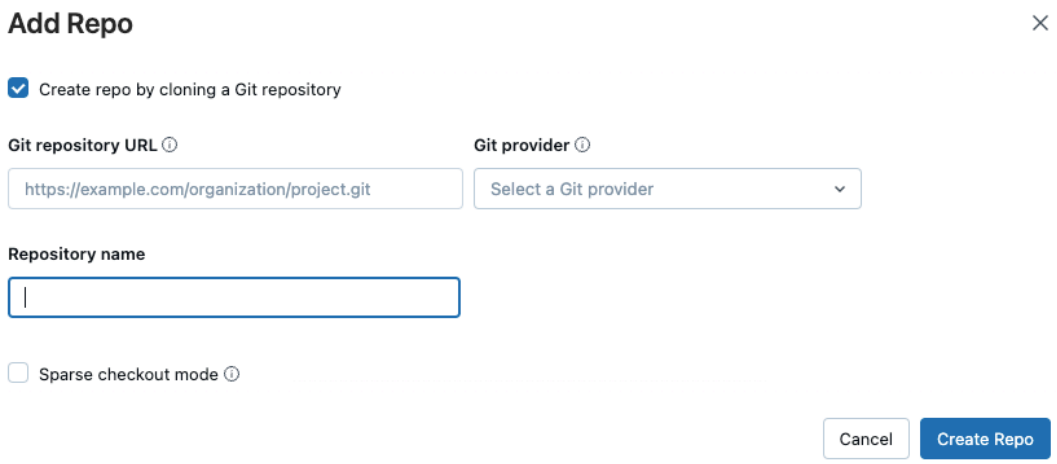
This code is in global_setup.py, but you could also put it in the notebook itself

- **Option 4**: Use the `opendatasets` library to paste your credentials in at the time of download. This is a safe way to download data, so we demonstrate this with the Favorita Sales data.

We'll will walk through `global-setup.py` later. The last section of the file is setting your Kaggle credentials. We recommend setting up a secret scope with your credentials. We will show you how to set this up once you have a cluster running, so there is no need to jump around. Simply download your Kaggle credentials for now.

Setting up our GitHub repository

The first thing is to pull the code you will work with throughout the book from the GitHub repository.



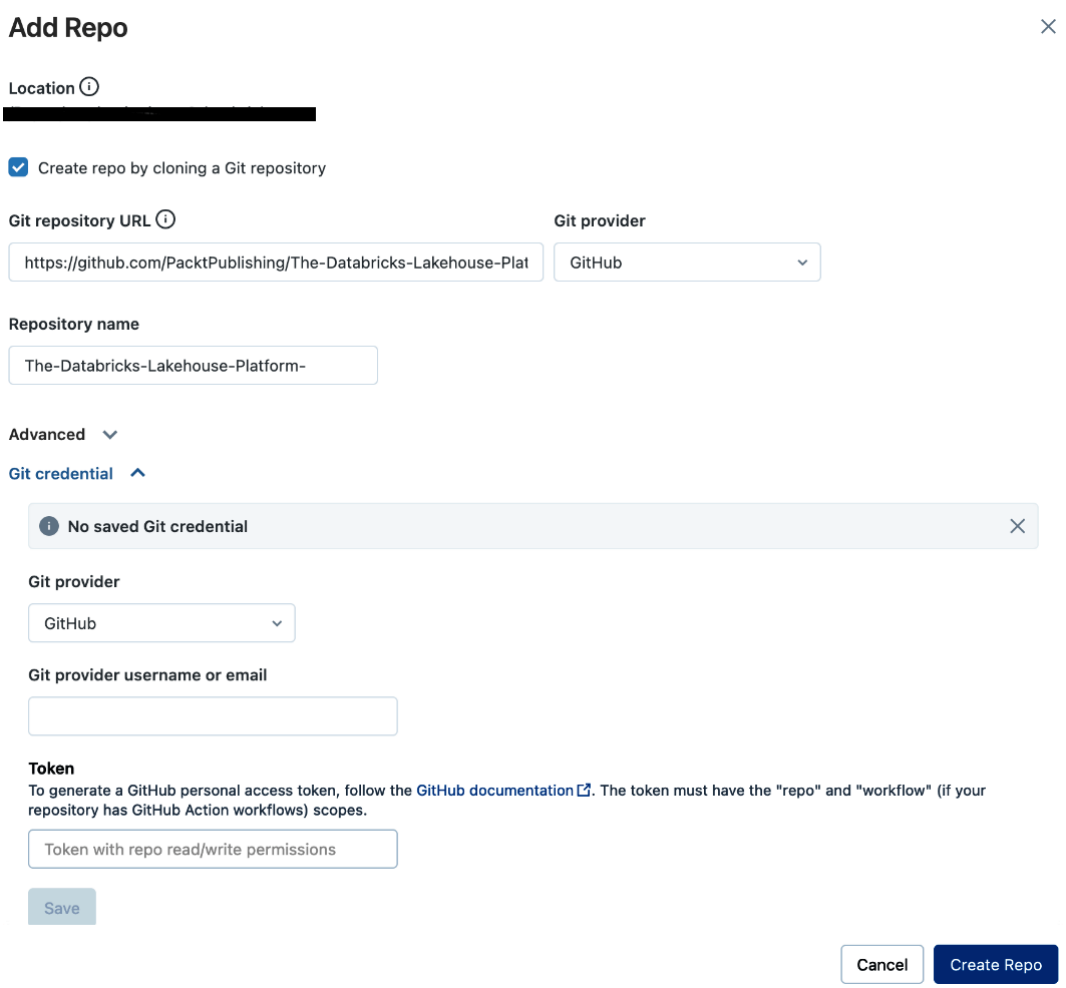


Figure 2.12 – Setting up a Git repository: Workspace > Repos > Home folder > Add > Repo

Navigate to your fork of the Book's GitHub repository, as mentioned in the *Technical requirements* section under the *Applying our learning* section. You can copy and paste the HTTPS link into the **Add Repo** screen's URL section, shown in *Figure 2.12*. Next, you will link your GitHub account. If you are unsure how to do this, follow the documentation linked in *Further reading* titled *About personal access tokens* and *Set up Databricks Repos*. Once your repository is ready, you can create a cluster if no one has done so yet.

Creating compute

We provide the cluster configurations we use for this project in the *Technical requirements* section. You can use the same configuration if you like. There are several options to choose from when creating

a new cluster configuration. It might seem complicated for new users, but stay calm when trying to choose the right one. We highly recommend the *Best Practices for Cluster Configuration* linked in *Further reading* for guidance, especially if you are responsible for setting up compute for one or more teams. Let's talk about some of the compute options as they relate to ML and this book:

- **Multi-node versus single-node**: Multi-node is excellent for distributed projects (think Spark). Single-node is suitable for projects or workloads that are performed on the driver (think scikit-learn or pandas).
- **Access mode**: Some cluster configurations support Unity Catalog, and some don't. Choose a cluster that supports UC for the projects in this book.
- **Databricks Runtime (DBR) and Python packages**: The DBR is part of the managed service of Databricks. A DBR is a group of commonly used packages bundled and pinned to a release version and then versioned as a DBR. There are two significant variations of a DBR: standard and ML. Data engineering workloads often utilize the standard DBR. The ML DBR, as the name implies, is for DS and ML workloads. Think of the greatest hits of ML Packages – sklearn, pandas, numpy, and seaborn, as well as the latest CUDA libraries for GPU clusters, and so on, these are included in the ML runtime, which means they are pre-installed and ready to be used in your code. DBRs provide the benefit of reproducibility as well as reducing the engineering development time of packaging and pinning versions. When adding additional libraries, you can add them to your notebook, cluster, or workspace.

  The best practice for production is to install cluster-scoped libraries rather than notebook-scoped libraries. However, notebook-scoped libraries are great for interactive use and development (please note that if you opt for notebook-scoped libraries, every notebook will require you to re-install non-default libraries each time, as one notebook represents a unique `pyenv`). You will install libraries both at the cluster level and in a couple of project notebooks. You can see the **Libraries** tab in *Figure 2.13*. Simply click **Install new** to install a new library.

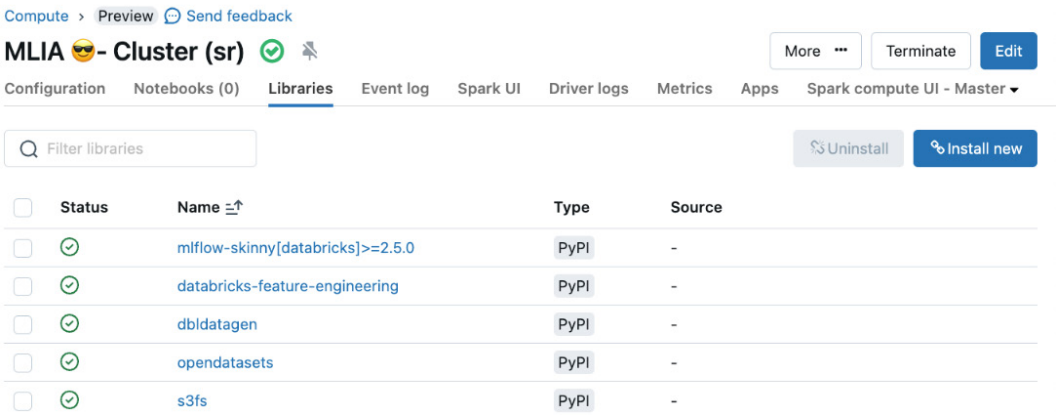


Figure 2.13 – Adding libraries to the Machine Learning in Action (MLIA) cluster configuration

This is the ideal time to install the libraries you will need. Install via `PyPI opendatasets`, `dbldatagen`, `databricks-feature-engineering`, and `mlflow-skinny[databricks]>=2.5.0`. These libraries are used across multiple notebooks throughout the book.

- **Photon acceleration**: Photon is an acceleration engine that speeds up ETL and SQL workloads. Photon currently is not advantageous for standard ML modeling.
- **VM types**: There are many VMs to pick from. You can choose VMs by family from the drop-down list. You can start with a VM in the *General Purpose* group if you need more clarification or are just starting.
- **Min and max workers**: The rule of thumb is to start with a few max workers and increase the number of workers as your workload increases. Keep in mind that your cluster will autoscale for you. However, we still recommend starting smaller and growing out for only the more compute-heavy examples, such as certain notebooks in the Multilabel Image Classification deep learning project.

You now have your development environment set up. You are ready to lock down your credentials for safe use in your code.

Setting up the Databricks CLI and secrets

The Databricks CLI is the command line interface for Databricks. We recommend using the web terminal from your cluster to install Databricks CLI as well as create Databricks secrets. As we mentioned earlier in the chapter, there are other options to get access to Kaggle datasets, but we walk you through the steps to set up secrets here. Please see the documentation in *Further reading* for more details on the CLI, installation, usage, and secrets.

1. Go to the **Apps** tab of the compute cluster you set up in the last section. You can refer to *Figure 2.13* to see the location of the **Apps** tab. Apps are only available while the cluster is, so it may be greyed out initially. You will have to start your cluster to proceed.
2. Select the web terminal.
3. Install the latest version of the CLI. `curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sudo sh`
4. Check your Databricks version to be sure it's greater than `0.2`. We had to point to the updated version in the location installed by curl. `/usr/local/bin/databricks-v`.
5. Next, you need to configure the connection. You need your PAT for this:

```
/usr/local/bin/databricks configure
```

6. Create a secret scope for storing credentials related to this book:

```
/usr/local/bin/databricks secrets create-scope "machine-
learning-in-action"
```

7. Create a secret for storing your Kaggle username:

```
/usr/local/bin/databricks secrets put-secret --json '{"scope":
"machine-learning-in-action",
"key": "kaggle_username",
"string_value": "readers-username"
}'
```

8. Create a secret for storing your Kaggle API key:

```
/usr/local/bin/databricks secrets put-secret --json '{
"scope": "machine-learning-in-action",
"key": "kaggle_key",
"string_value": "readers-api-key"
}'
```

9. Last, list your secrets to make sure everything works as expected:

```
/usr/local/bin/databricks secrets list-secrets "machine-
learning-in-action"
```

In *Chapter 8*, we'll create another scope to hold an OpenAI API key, but for now we just need the Kaggle credentials. Now that we have our secrets set up, let's get our codebase ready!

Setting up your code base

We use a setup file to help keep variables consistent across multiple project notebooks. You will run the setup file each time you run the project notebooks using a magic command, `%run`. This command brings everything into the memory of your notebook session. The `global-setup.py` file has numerous components to it. Let's walk through each section. Feel free to edit the file to fit your needs.

> **Note**
>
> It's possible you'll receive an error message: `py4j.security.Py4JSecurityException: Method public scala.collection.immutable.Map com.databricks.backend.common.rpc.CommandContext.tags() is not whitelisted on class class com.databricks.backend.common.rpc.CommandContext`
>
> This is because you are on a shared compute cluster. You can simply hardcode `current_user` to your username.

Passing variables via widgets

Widgets pass variables to notebooks similarly to how command-line arguments pass variables to Python scripts. The code block in *Figure 2.14* creates the widgets needed to pass variables from the `Run` command to the `global-setup.py` file using Databricks Utilities or `dbutils`. You can read more about the `dbutils` capabilities in the Databricks Utilities documentation in *Further reading*. These widgets create, pass, and access parameters. The arguments are the variable name, default value, and verbose name in respective order.

```
# RUN TIME ARGUMENTS
dbutils.widgets.text("env", "dev", "Environment")

#ignored if db is set (we force the databse to the given value in this case)
dbutils.widgets.text("project_name", "", "Project Name")

#Empty value will be set to a database scoped to the current user using project_name
dbutils.widgets.text("db", "", "Database")
```

Figure 2.14 – Creating widgets for accepting notebook-specific variables

You can pass each variable while running the file by adding a single line cell with appropriate parameters at the top of the notebook, as shown in *Figure 2.15*.

```
%run ../../global-setup $project_name=synthetic_transactions
```

Figure 2.15 – Running project-specific variables in our global setup file

Running `global-setup.py` saves all the variables defined in the script in memory for easy reference.

Checking for compatibility

Next, in `global-setup.py`, we run checks for compatibility between the code base and the cluster attached to the notebook.

The compatibility code block checks the following:

- A project name was submitted as a variable.
- The cluster is configured with an ML runtime and meets the minimum version. To be sure all features in the code are available in the runtime used, we set a minimum.

Once all checks pass, we assign a user and paths.

Setting a default catalog and project-specific database

This book provides code that uses the Unity Catalog catalog. Your default catalog is set based on your environment. If you do not set the environment or you set it to `dev`, then the catalog is named `ml_in_action`. When the environment is `prod`, the catalog is `ml_in_prod`. The default name for the database is always the project name. However, you can provide a different name if you desire by entering a project variable for the database name.

```
for i in range(10):
  try:
    spark.sql(f"""USE `{catalog}`.`{database_name}`""")
    break
  except Exception as e:
    time.sleep(1)
    if i >= 9:
      raise e
```

Figure 2.16 – Using the defined variables to set the default with retries

We want to be sure that the catalog and database are set to the notebooks' defaults. Occasionally, with parallel execution, this command can fail during initialization; therefore, we add a few retries to work around this issue, as shown in *Figure 2.16*.

Granting permissions

Now that we've set our catalog and database defaults, we can grant permissions.

```
# Granting UC permissions to account users - change if you want your data private
spark.sql(f"GRANT CREATE, SELECT, USAGE on SCHEMA {catalog}.{database_name} TO `account users`")
```

Figure 2.17 – Granting permissions to the catalog and database

We grant the group `account users` permission. If you do not want to make your assets available to others, remove this or comment it out.

> **Important note**
>
> Be sure to use tick marks around your group name or email address when granting permission. If you use single quotes instead, you will get an error message.

The catalog and database are ready for tables. However, not all data we use in machine learning goes into a table. For other data, files, and objects, we have volumes.

Setting up volumes

Volumes are views of cloud object storage. We create project-specific volumes. Use them for path-based access to structured or unstructured data. Volumes sit under a database in a catalog and are used to manage and provide access to data files. You can govern access to volumes using `GRANT` statements. Volumes provide scalable file-based storage without sacrificing governance. Often, we use unstructured, semi-structured, or non-tabular data in machine learning. Images are a good example of unstructured, non-tabular data that we will use for the Multilabel Image Classification project. To work with these images, the Multilabel Image Classification project uses volumes.

Starting the projects

We have planned our platform and set up our workspace environment. Next, let's work through each project. In GitHub, you will see that each chapter has a folder containing folders corresponding to each project. When we refer to the notebooks by name, we assume you are in the appropriate chapter and project folder. For example, this chapter has the first notebook:

```
Chapter 2: Designing Databricks: Day One/
  Project: Favorita Store Sales - TimeSeries Forecasting/
CH2-01-Downloading_Sales_Forecast_Data
```

We refer to the notebook by only the filename itself, `CH2-01-Downloading_Sales_Forecast_Data`. Let's jump into the first project.

Project: Favorita Store Sales – Time Series Forecasting

Recall from *Chapter 1* that we use a Kaggle-provided dataset to forecast sales. In this chapter, we download our data from the Kaggle website. To follow along in your workspace, please open the following notebook:

- `CH2-01-Downloading Sales Forecast Data`

In the notebook, and as well as the code here in *Figures 2.18 and 2.19*, we set our path and download our data from Kaggle.

First, we designate `raw_data_path` to store the files.

```
raw_data_path = volume_file_path + 'raw_data'

dbutils.fs.mkdirs(raw_data_path)
```

Figure 2.18 – Setting the path for our volume

In the following code block (*Figure 2.19*), we use the Python package `opendatasets`, a library specifically created to download data from the Kaggle API. You can find more information in the *Further reading* section.

```
import opendatasets as od

od.download("https://www.kaggle.com/competitions/
store-sales-time-series-forecasting/data",raw_data_path)
```

Figure 2.19 – Downloading Favorita data from opendatasets

That is all for the *Favorita Store Sales* project in this chapter! Now, we can focus on generating data for our `Streaming Transactions` project.

Project: Streaming Transactions

Your goal with the Streaming Transactions project is to build a model to classify transactions. The dataset consists of JSON-formatted transactions with `Transaction`, `timestamp`, `Label`, `Amount`, and `CustomerID`.

In later chapters, you will add a product column to demonstrate schema evolution. In this chapter, you'll create the first version of transaction data used throughout the rest of the book. To follow along in your workspace, please open the following notebook:

- `CH2-01-Generating Records Using DBKS Labs Datagen`

You can run each cell of the notebook as we work through them or run them all at once. After the setup commands, we set notebook variables to establish the number of rows generated per batch of transactions (`nRows`), the number of positively labeled rows per batch (`nPositiveRows`), the path to the volume where you will store the JSON dataset (`destination_path`), a temporary path (`temp_path`), and the number of seconds between each batch of data you generate (`sleepIntervalSeconds`).

The following code block accesses the value of the `Reset` widget. Any data already written to the volume will be deleted if the widget is set to `True` (its default value).

```
if bool(dbutils.widgets.get('Reset')):
  dbutils.fs.rm(temp_path, recurse=True)
  dbutils.fs.rm(destination_path, recurse=True)
  dbutils.fs.mkdirs(destination_path)
```

Figure 2.20 – Checking the Reset widget

Next, we set the parameter values used in the data generator to create the transactions. We set the minimum and maximum values for each `CustomerID`. We also create a dictionary of product types and set `min`, `max`, `mean`, `alpha`, and `beta` variables, which you use to generate random transaction amounts according to a distribution.

```
CustomerID_vars = {"min": 1234, "max": 1260}

Product_vars = {"None": {"min": 1000, "max": 25001, "mean": 15520, "alpha": 4, "beta": 10}}
```

Figure 2.21 – Dictionaries to hold variables for use within the define_specs function

With the variables set, we build out the functions to create the transaction data, starting with the `define_specs` function. The function accepts as input a product type (defined in the dictionary in *Figure 2.21*), a positive or negative label, and a timestamp; it returns a dollar amount for the transaction. *Figure 2.22* shows a portion of the code; the rest is in the accompanying notebook.

```
import dbldatagen as dg
from datetime import datetime
import dbldatagen.distributions as dist
from pyspark.sql.types import IntegerType, FloatType, StringType

def define_specs(Product, Label, currentTimestamp = datetime.now()):
  pVars = Product_vars[Product]
  if Product == "None":
    if Label:
      return (dg.DataGenerator(spark, name="syn_trans", rows=nRows, partitions=4)
        .withColumn("CustomerID", IntegerType(), nullable=False,
                    minValue=CustomerID_vars["min"], maxValue=CustomerID_vars["max"], random=True)
        .withColumn("TransactionTimestamp", "timestamp",
                    begin=currentTimestamp, end=currentTimestamp,nullable=False,
                   random=False)
        .withColumn("Amount", FloatType(),
                    minValue=pVars["min"],maxValue=pVars["max"],
                    distribution=dist.Beta(alpha=pVars["alpha"], beta=pVars["beta"]), random=True)
        .withColumn("Label", IntegerType(), minValue=1, maxValue=1)).build()
```

Figure 2.22 – Defining the define_specs function to generate transaction records

Next, we write a function to generate a single record by calling `define_specs` and including the current timestamp.

```
from pyspark.sql.functions import expr
from functools import reduce
import pyspark
import os

# Generate a record
def generateRecord(Product,Label):
  return (define_specs(Product=Product, Label=Label, currentTimestamp=datetime.now()))
```

Figure 2.23 – Defining a function to generate a single transaction record

We then build `generateRecordSet` to generate the `recordCount` number of records in each batch. Notice that in this notebook, we're using the `None` product type, so the records generated will only have four features: `CustomerID`, `TransactionTimestamp`, `Amount`, and `Label` (this will be important in the next chapter!).

```
# Generate a list of records
def generateRecordSet():
  Products = ["None"]
  Labels = [0,1]
  recordSet = []
  for Prod in Products:
    for Lab in Labels:
      recordSet.append(generateRecord(Prod, Lab))
  return reduce(pyspark.sql.dataframe.DataFrame.unionByName, recordSet)
```

Figure 2.24 – The generateRecordSet function creates a record for each product and each label. Each record contains nRows transactions

Finally, we write a function to generate a set of data, convert the data to a DataFrame, and write it out as one JSON file to a temporary path. Then, we move that file to the final volume destination.

```
def writeJsonFile(destination_path):
  recordDF = generateRecordSet()
  recordDF = recordDF.withColumn("Amount", expr("Amount / 100"))
  recordDF.coalesce(1).write.format("json").save(temp_path)

  tempJson = os.path.join(temp_path, dbutils.fs.ls(temp_path)[3][1])
  dbutils.fs.cp(tempJson, destination_path)
  dbutils.fs.rm(temp_path, True)
```

Figure 2.25 – The writeJsonFile function generates a set of records

The set contains amounts generated as integers, so we divide by 100 to turn the amounts into dollars and type float. The function writes out the JSON file to a `temp` directory and then moves the single file to the final directory.

With everything set up, create the dataset with the code provided. Feel free to increase the iterations to build a larger dataset. Then, move on to the next project!

Project: Retrieval-Augmented Generation Chatbot

The RAG Chatbot project will ingest PDF documents to build the knowledge base for the chatbot. We use a volume to store the PDFs. To follow along in your workspace, please open the following notebook:

- `CH2-01-Downloading PDF Documents`

Files can be uploaded to a volume directly in the Databricks console via the user interface, as shown in *Figure 2.26*; however, this project uses the code provided in the notebook to download and save the data to a volume programmatically.

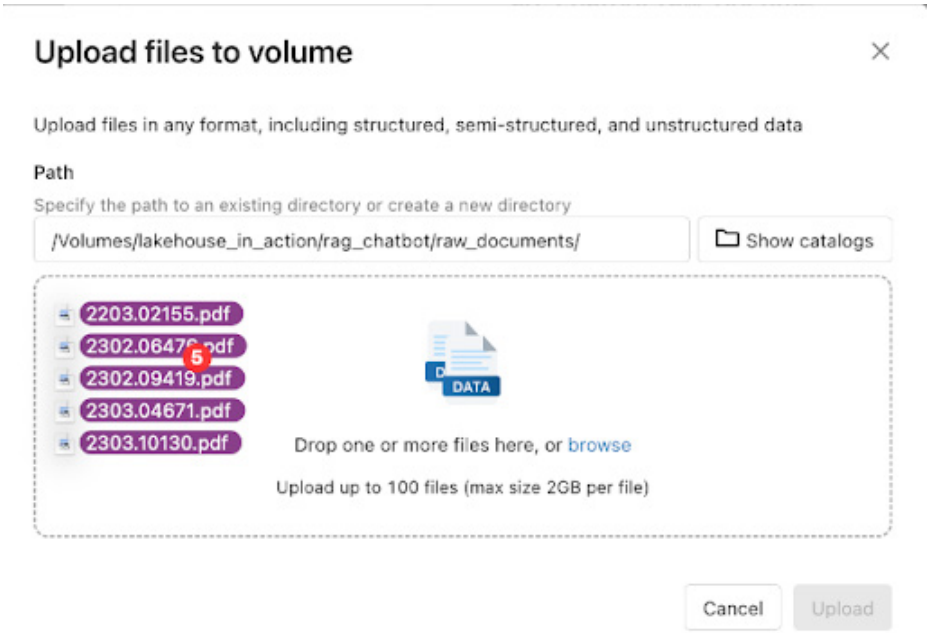


Figure 2.26 – Manually uploading documents into a volume

The code for this chapter begins with setup cells and helper functions, and in *Figure 2.27* we designate `library_folder` where we will save the PDFs we download.

```
library_folder = "{}/raw_documents".format(volume_file_path)

user_agent = "mlaction_book"
```

Figure 2.27 – Designating the library folder to hold the files for this project

We are using open articles published on the **Arxiv** page that relate to **Generative AI** (**GenAI**) and how it can impact human labor markets and economics. We pass the URLs to be used as documents for our chatbot and load these files into our volume.

```
# we are getting our documents, you could directly upload it to the volumes using UI
pdfs = {'2203.02155.pdf':'https://arxiv.org/pdf/2203.02155.pdf',
        '2302.09419.pdf': 'https://arxiv.org/pdf/2302.09419.pdf',
        '2303.10130.pdf':'https://arxiv.org/pdf/2303.10130.pdf',
        '2302.06476.pdf':'https://arxiv.org/pdf/2312.00506.pdf',
        '2302.06476.pdf':'https://arxiv.org/pdf/2302.06476.pdf',
        '2303.10130.pdf':'https://arxiv.org/pdf/2303.10130.pdf',
        '2209.07753.pdf':'https://arxiv.org/pdf/2209.07753.pdf',
        '2304.07683.pdf':'https://arxiv.org/pdf/2304.07683.pdf'}

for pdf in pdfs.keys():
    load_file(pdfs[pdf], pdf, library_folder)
```

Figure 2.28 – Download PDF files and save them to our volume

Now that we have the documents downloaded and, they are ready to be processed for our chatbot. With that completed, we can move on to our final project: **Multilabel Image Classification**.

Project: Multilabel Image Classification

The MIC project ingests images into Delta tables to fine-tune a pre-trained model from the *Restnet* family to improve its accuracy. We will programmatically download the images from Kaggle and save the data to a volume. To follow along in your workspace, please open the `CH2-01-Downloading Images` notebook:

```
%sh
cd /local_disk0/
kaggle datasets download -d puneet6060/intel-image-classification
```

Figure 2.29 – Downloading data from Kaggle using Databricks magic commands

Now we create the volume folder and unzip the images for our classification project into our volumes. It will take around one hour (as it contains 80K images!) to extract the images from ZIP to `Volumes`.

```
!mkdir {volume_file_path}/intel_image_clf/
!mkdir {volume_file_path}/intel_image_clf/raw_images

# this can take up to a few hours
# or load a few examples to UC on your own if time is a constraint
!unzip -n /local_disk0/intel-image-classification.zip -d {volume_file_path}/intel_image_clf/raw_images
```

Figure 2.30 – Unzipping images into the volumes for this project

We have downloaded or generated all four datasets, and they are ready to be brought into our Bronze layer in the next chapter.

Summary

This chapter covered a wide range of setup decisions, options, and processes for planning your Data Intelligence Platform. We took you through an overview of the main components of the DI Platform, from early design choices to important features that we will dive into further in upcoming chapters. You also learned how to set up your workspace and project code base. We hope you feel more comfortable with the basics of the platform. With Databricks ready and the project data downloaded, we are now ready to get into what it means to build out the Bronze data layer.

In *Chapter 3*, we cover the essentials of building out the Bronze data layer within the Databricks Intelligence Platform. We will format our data into the most optimized format, learn about schema evolution, change data capture using Delta, and more.

Questions

The following questions are meant to solidify key points to remember and tie the content back to your own experience.

1. How do Databricks runtimes enable stability?
2. How can we make our data more discoverable?
3. What are some common steps needed to set up a Databricks workspace?

Answers

After putting thought into the questions, compare your answers to ours.

1. Databricks runtimes enable stability by providing a consistent set of libraries.
2. Utilizing the built-in functionality for metadata, such as table and column descriptions, makes our data more discoverable.
3. Some common steps for setting up a workspace are activating Databricks through the marketplace and setting up user groups and governance.

Further reading

In this chapter, we identified specific libraries, technical features, and options. Please take a look at these resources to delve deeper into the areas that interest you most:

- *What is Unity Catalog?*: `https://docs.databricks.com/data-governance/unity-catalog/index.html`
- *Lakehouse Monitoring demo*: `https://youtu.be/3TLBZSKeYTk?t=560`

- *UC has a more centralized method of managing the model lifecycle than HMS*: https://docs.databricks.com/machine-learning/manage-model-lifecycle/index.html
- *Share Models across workspaces*: https://docs.databricks.com/en/machine-learning/manage-model-lifecycle/multiple-workspaces.html
- *In-depth UC setup on Azure*: https://youtu.be/itGKRVHdNPo
- *Connecting external HMS to UC*: https://www.databricks.com/blog/extending-databricks-unity-catalog-open-apache-hive-metastore-api
- *Unity Catalog limitations*: https://docs.databricks.com/en/data-governance/unity-catalog/index.html#unity-catalog-limitations
- *Best practices: Cluster configuration | Select Cloud in the dropdown*: https://docs.databricks.com/clusters/cluster-config-best-practices.html
- *Databricks Notebooks*: https://docs.databricks.com/en/notebooks/index.html
- *Databricks Autologging | Select Cloud in the dropdown*: https://docs.databricks.com/mlflow/databricks-autologging.html#security-and-data-management
- *Kaggle API GitHub*: https://github.com/Kaggle/kaggle-api
- *Lakehouse Monitoring product page*: https://www.databricks.com/product/machine-learning/lakehouse-monitoring
- *System Tables*: https://www.databricks.com/resources/demos/tutorials/governance/system-tables
- *Opendatasets Python package*: https://pypi.org/project/opendatasets/
- *Kaggle API*: https://www.kaggle.com/docs/api
- *GitHub*: https://github.com/
- *Databricks ML in Action GitHub Repository*: https://github.com/PacktPublishing/Databricks-ML-In-Action
- *Databricks Secrets API*: https://docs.databricks.com/en/security/secrets/secrets.html
- *Databricks CLI*: https://docs.databricks.com/en/dev-tools/cli/index.html
- *Databricks Utilities*: https://docs.databricks.com/en/dev-tools/databricks-utils.html
- *Workspace libraries*: https://docs.databricks.com/en/libraries/workspace-libraries.html

- *Data Mesh and the DI Platforms Blog Posts*: https://www.databricks.com/blog/2022/10/10/databricks-lakehouse-and-data-mesh-part-1.html, https://www.databricks.com/blog/2022/10/19/building-data-mesh-based-databricks-lakehouse-part-2.html
- *Short YouTube video on managed vs external tables in UC*: https://youtu.be/yt9vax_PH58?si=dVJRZHAOnrEUBdkA
- *Query Federation*: https://docs.databricks.com/en/query-federation/index.html
- *Centralized model registry workspace for HMS*: https://docs.databricks.com/applications/machine-learning/manage-model-lifecycle/multiple-workspaces.html
- *Manage model lifecycle in Unity Catalog*: https://docs.databricks.com/machine-learning/manage-model-lifecycle/index.html
- *Terraform https*: https://github.com/databricks/terraform-provider-databricks
- *Widgets*: https://docs.databricks.com/notebooks/widgets.html
- *Kaggle API GitHub*: https://github.com/Kaggle/kaggle-api.

3

Building the Bronze Layer

"Data is a precious thing and will last longer than the systems themselves." – Tim Berners-Lee

In this chapter, you'll embark on the beginning of your data journey in the Databricks DI Platform by exploring the fundamentals of the Bronze layer of the Medallion architecture. The Bronze layer is the first step in transforming your data for downstream projects, and this chapter will focus on the Databricks features and techniques you have available for the necessary transformations. We start with an introduction to **Auto Loader**, a tool to automate data ingestion, which you can implement with or without **Delta Live Tables** (**DLT**) to insert and transform your data. The benefits of using Auto Loader include quickly transforming new data into Delta format and enforcing or evolving schemas, which are essential for maintaining consistent data delivery to machine learning models, the business, and customers. . We find that these topics are best explained through the example projects, so the section on *Applying our learning* is the main focus of this chapter.

Here is what you will learn as part of this chapter:

- Revisiting the Medallion architecture pattern
- Transforming data to Delta with Auto Loader
- Delta Live Tables starting with Bronze
- Maintaining and optimizing Delta Tables
- Applying our learning

Revisiting the Medallion architecture pattern

We introduced the Medallion architecture in *Chapter 2*. As a reminder, this refers to the data design pattern used to logically organize data transformations. A typical Medallion architecture has three layers: Bronze, Silver, and Gold. The Bronze layer contains raw data, the Silver layer contains cleaned and transformed data, and the Gold layer contains aggregated and curated data. Curated data refers to datasets selected, cleaned, and organized for specific business or modeling purposes. There are

also cases where additional levels of refinement are required, so your data design could extend to Diamond and Platinum layers as well. The Medallion architecture is a good fit for data science projects. By selecting, cleaning, and organizing data for a specific purpose, curated data can help improve the data's accuracy, relevance, and usability.

> **Note**
>
> This book will introduce quite a few project tasks, such as *building curated datasets*, that often fall under the domain of a Data Engineer. There will also be tasks that Machine Learning Engineers, data analysts, and so on commonly perform. We include all this work in our examples because roles are blurred in today's fast-moving world and will vary by company. Titles and expectations quickly evolve, and it's imperative to have a handle on the entire data lifecycle.

Maintaining a Bronze layer allows us to go back to the original data when we want to create a different feature, solve a new problem that requires us to look at historical data from another point of view, or simply maintain the raw level of our data for governance purposes. As the world of technology evolves, it's crucial to stay current and follow trends, but the core principles remain constant. Each of the following sections in this chapter (*Transforming data to Delta with Auto Loader*, *Delta Live Tables for the Bronze layer*, and *Maintaining and optimizing Delta Tables*) will cover DI platform features that facilitate building the Bronze layer. In the *Applying our learning* section, we apply the features to our example projects.

Transforming data to Delta with Auto Loader

Before we can transform our raw data, we must first ingest it. Auto Loader is a feature that incrementally and efficiently processes new data files as they arrive in cloud storage. You can use Auto Loader to automate your data ingestion process, significantly enhancing your data product's workflow efficiency. It can ingest data from cloud storage and streaming data sources and be configured to run on a schedule or be triggered manually.

Here are some benefits of using Databricks Auto Loader:

- **Reduces overhead**: Auto Loader maintains checkpoints, removing the need to track which data is new. Auto Loader handles that all on its own.
- **Improves data quality**: Auto Loader can automatically detect schema changes and rescue any new data columns, so you can be confident that your data is accurate.
- **Increases data agility**: Auto Loader can help you quickly and easily ingest new data sources so you can be more agile in responding to changes in your business.
- **Flexible ingestion**: Auto Loader can stream files in batches or continuously. Meaning, it can consume batch data as a stream to reduce the overhead of a more manual batch pipeline.

Auto Loader is a powerful tool. It can be used standalone, in combination with **Spark Structured Streaming** (a near real-time processing engine that we'll return to in *Chapter 5*), or as the underlying technology for Delta Live Tables (which we'll introduce shortly). For an example of streaming ingestion, please refer to the Streaming Transactions project in *Applying our learning*.

Once you have set up your data ingestion process, it's important to build a resilient pipeline that helps you monitor your data quality. Let's discuss how Auto Loader's ability to evolve a schema over time plays a key role in data quality.

Schema Evolution

Auto Loader's schema evolution allows you to add or modify fields in streaming data seamlessly. Databricks automatically adjusts relevant data to fit the new schema while preserving existing data integrity. This automated schema handling makes it easy to evolve your data schema over time as your business needs and data sources change without worrying about data loss or downtime. Furthermore, schema handling reduces the amount of work and headache associated with managing data pipelines that could change unexpectedly.

In the Applying our learning section's Transaction Streaming project, we put Schema Evolution to work. You'll see how no data is lost despite the transaction data's schema changing without notice. In our case, the default schema evolution mode for Auto Loader, `.option("cloudFiles.schemaEvolutionMode", "addNewColumns")`, in combination with `.option("mergeSchema", "true")` together perform schema evolution. Auto Loader will handle changes in the schema for us. Auto Loader tracking the changes is beneficial when new data fields become available with or without prior notice; therefore, code changes are unnecessary.

There is documentation on all schema options. The method we use for the Streaming Transactions project is the only option for automatic schema evolution. Every other option will require manual intervention. For example, you can use the "`rescue`" mode to rescue data from being lost. Alternatively, you can use the "`failOnNewColumns`" mode to cause the pipeline to fail and keep the schema unchanged until the production code is updated. There are numerous options and patterns for Auto Loader. Check out *Common loading patterns with Auto Loader* docs linked in Further reading for more information.

Delta Live Tables for the Bronze layer

Delta Live Tables (**DLT**) is a Databricks feature that actively simplifies your pipeline operations, empowering you to focus on setting clear objectives for your pipeline rather than getting bogged down in operational details. Auto Loader's schema evolution capabilities are integrated with DLT, underscoring DLT's utility in handling dynamic data schemas with minimal manual intervention.

Delta Live Tables benefits and features

DLT is a sophisticated framework designed to construct reliable data pipelines. DLT automates and streamlines complex operations like orchestration and cluster management, significantly boosting the efficiency of data workflows. All you need to do is specify your transformation logic to be up and running. We will focus on a subset of the DLT's benefits as they pertain to creating the Bronze data layer, but we've included a link to DLT documentation in the *Further reading* section at the end of this chapter as well. DLT is a great tool in your toolbox for ingesting data because it provides several benefits over traditional ETL pipelines, including:

- **Declarative pipeline development**: Delta Live Tables allows you to define your data pipelines using SQL or Python, which makes them easier to understand and maintain.
- **Automatic data quality testing**: Delta Live Tables can automatically apply your tests to your data, preventing quality issues, which helps to ensure that your pipelines are producing accurate results.
- **Deep visibility for monitoring and recovery**: Delta Live Tables provides detailed tracking and logging information to make troubleshooting problems and recovering from failures easier.
- **Cost-effective streaming through efficient compute autoscaling**: Delta Live Tables can automatically scale your compute resources up or down based on demand, which helps to reduce costs.

Delta Live Tables is a powerful tool for building reliable and scalable data pipelines in batch or streaming. It can help you improve the quality, efficiency, and visibility of your data processing workflows.

Here are some of the specific features of Delta Live Tables that provide these benefits:

- **Streaming tables**: Delta Live Tables uses streaming and live tables to process data in real-time and to keep your pipelines up-to-date with the latest data.
- **Materialized views**: Delta Live Tables uses materialized views to create snapshots of your data. You can query your data in real-time and use it for downstream processing.
- **Expectations**: Delta Live Tables uses expectations to test your data for quality issues automatically and to take action if the data does not meet your expectations.
- **Autoscaling**: Delta Live Tables can automatically scale your compute resources up or down based on demand, reducing costs, and improving the performance of your pipelines.

Delta Live Tables is a good way to build reliable, scalable, and testable data pipelines. Next, we focus on DLT in the context of the Bronze layer.

Bronze data with DLT

Delta Live Tables uses Auto Loader to support schema evolution, offering significant benefits in terms of data quality, a key aspect of the Bronze layer in a Medallion architecture. In this architecture, the Bronze layer serves as the foundational stage where raw data is initially ingested and stored. DLT contributes to this layer by ensuring that each transformation applied to the raw data is precisely captured and managed. As a data scientist, understanding these transformations is crucial for maintaining the integrity of the data processing workflow. A powerful feature of DLT is its ability to automatically generate an accurate workflow **DAG** (**Directed Acyclic Graph**). This DAG not only visualizes the sequence and relationships of these data transformations but also enhances the reliability of the entire data pipeline.

The following image shows an example Delta Live Tables pipeline DAG:

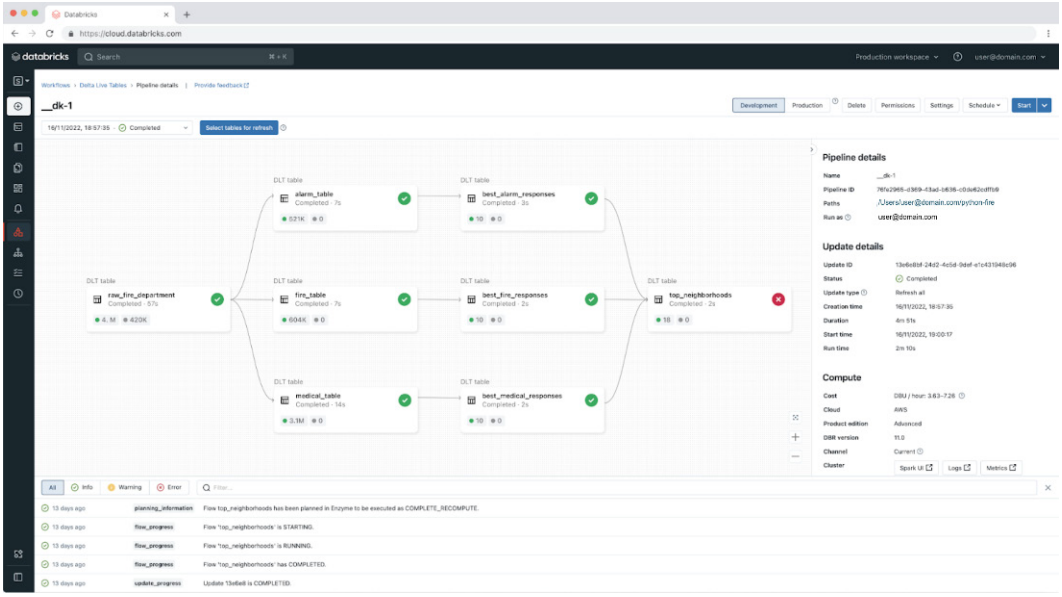


Figure 3.1 – An example Delta Live Tables pipeline workflow

Figure 3.1 is an example DLT workflow with multiple dependencies. DLT is an excellent way to build and maintain ETL pipelines, delivering high-quality data on the DI Platform. Once we have our data landed in a Delta table, there are actions we can take to ensure we are taking full advantage of Delta's benefits.

Maintaining and optimizing Delta Tables

While the primary focus of this book is not on the intricate details of Delta table optimization, understanding these techniques is crucial for building data-centric machine learning projects. Efficient management of Delta tables directly impacts the performance and reliability of ML models, as these

models heavily rely on the quality and accessibility of the underlying data. You can use techniques like `VACUUM`, liquid clustering, `OPTIMIZE`, and bucketing to store, access, and manage your data efficiently. Optimized tables ensure that the data feeding into ML algorithms is processed efficiently. We'll cover these briefly here, but also suggest that you refer to the Delta Lake documentation for a comprehensive understanding of each technique.

Vacuum

The `VACUUM` command is crucial in managing resources within Delta tables. It works by cleaning up invalidated files and optimizing the metadata layout. If your Delta table undergoes frequent **Data Manipulation Language** (**DML**) operations (update, insert, delete, merge), we recommend running the `VACUUM` operation periodically. DML operations can generate numerous small files over time. Failure to run `VACUUM` could eventually result in many small files with minimal data remaining in each, leading to poor performance.

Note that `VACUUM` doesn't run automatically; you must explicitly schedule it. Consider scheduling `VACUUM` at regular intervals, such as weekly or monthly, depending on your data ingestion frequency and how often you update the data. Additionally, you have the option to configure the retention period to optimize storage and query performance. The `delta.logRetentionDuration` Delta configuration command allows you to control the retention period by specifying the number of days you want to keep the data files. This means you will delete all transaction log data for a table that goes back beyond the retention setting, e.g., only the last seven days of metadata remain. The transaction log is not very big, but this way you can control the duration the Delta table retains data files before the `VACUUM` operation removes them. Note that the retention duration affects your ability to look back in time to previous versions of a table. Keep that in mind when deciding how long you want to retain the metadata and transaction log.

Liquid Clustering

Liquid Clustering is a great stand-in for partitioning; see how to implement it in *Figure 3.2*. Frequently, data gets over-partitioned, leaving too few files in a partition or unbalanced partitions. You do not need partitioning unless a table is a terabyte or larger. In addition to replacing manual partitioning strategies, liquid clustering also replaces the need for Z-ordering on Delta tables. Z-ordering is not compatible with clustered tables. When choosing columns to cluster on, include high cardinality columns you use for query filters. Another commonly given example is a timestamp column. Instead of creating a derived column date to minimize the cardinality, simply cluster on the timestamp. Liquid clustering benefits Delta tables with skewed data distribution or changing access patterns. It allows the table to adapt to analytical needs by redefining clustering keys without rewriting data, resulting in optimized query performance and a flexible, maintenance-efficient structure. With liquid clustering enabled, you must use DBR 13.3+ to create, write, or `OPTIMIZE` Delta tables. The following figure shows how to put liquid clustering into practice.

```
CREATE TABLE my_clustered_table (id INT, name STRING, timestamp TIMESTAMP)
USING DELTA
CLUSTER BY (id, timestamp);
```

Figure 3.2 – Example code to create a table optimized with liquid clustering

Once the clustering is setup on your table, you want to trigger it occasionally, which you can do with the OPTIMIZE command, up next.

Optimize

The `OPTIMIZE` command will trigger clustering. This is extra important when streaming data as tables are not clustered on write. `OPTIMIZE` also compacts data files, which is vital for Delta tables with numerous small files. It merges these files into larger ones, enhancing read query speed and storage efficiency, which is especially beneficial for large datasets. Knowing when to run OPTIMIZE can be an analysis exercise. Luckily, we now have predictive optimization.

Predictive optimization

Predictive optimization is a feature that runs `VACUUM` and `OPTIMIZE` for you. Your admin can enable it in **Settings** if you have the premium version of Databricks, serverless enabled, and Unity Catalog setup. Enabling predictive optimization enables you to focus on other tasks.

Now that we've covered the basics of the Medallion architecture, Auto Loader, Delta Live Tables, and some techniques for optimizing your Delta tables, get ready to follow along in your own Databricks workspace as we work through the *Chapter 3* code by project.

Applying our learning

Before we can build models, we must first ingest the data required for each project. We've discussed how Auto Loader, schema evolution, and Delta Live Tables can format your data in storage. You'll notice that the projects below use different patterns to load data into the Bronze layer.

Technical requirements

Before we begin, please review the technical requirements needed to complete the hands-on work in this chapter:

- Databricks ML Runtime includes several pre-installed libraries useful for machine learning and data science projects. For this reason, we will be using the cluster from *Chapter 2*, as it has an ML Runtime.

Project: Streaming Transactions

In *Chapter 2*, you generated streaming transaction data. In this chapter, you build out the Bronze layer for the Streaming Transactions project. This is the raw data that you will clean, transform, and eventually use to train a machine learning model. We specifically created this streaming project to practice using Auto Loader, schema evolution, Spark Structured Streaming, and Delta Live Table features, so we'll use those throughout this part of the project. To follow along in your workspace, please refer to the following notebooks:

- `CH3-01-Generating Records and Schema Change`
- `CH3-02-Auto Loader and Schema Evolution`
- `delta_live_tables/CH3-03-Formatting to Delta with DLT`

Here's where you are in the project flow:

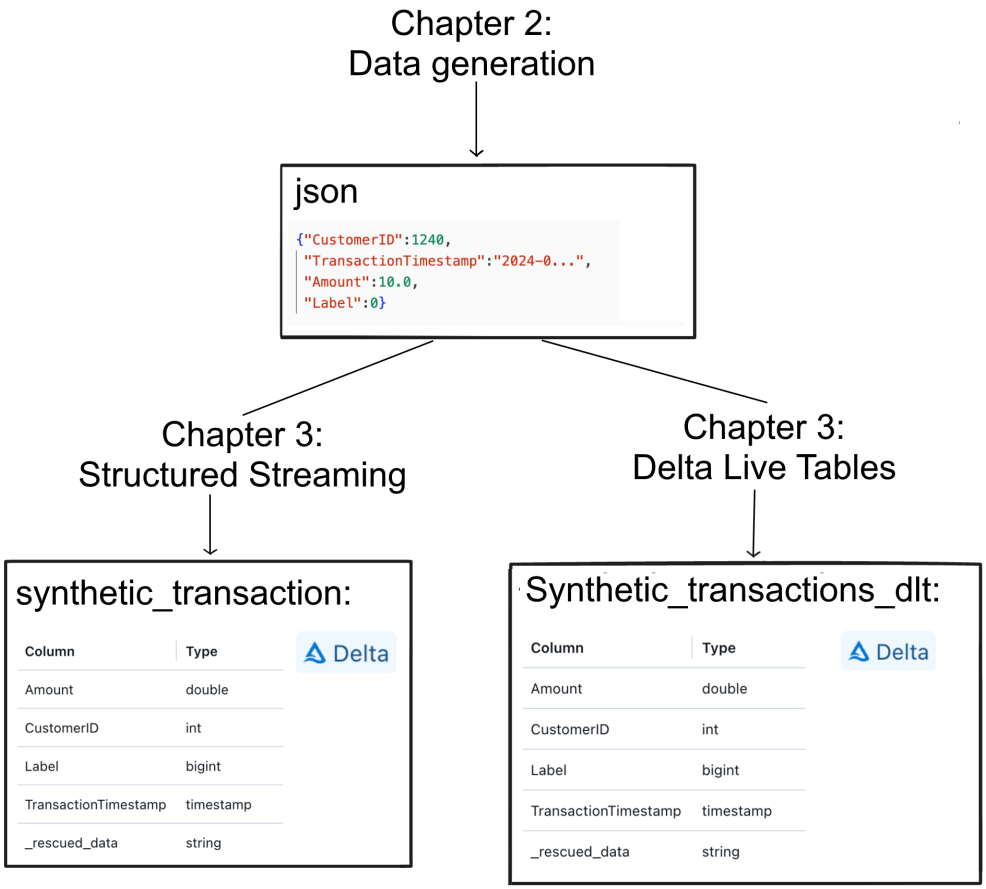


Figure 3.3 – The project plan for the Synthetic Streaming Transactions Project

In *Chapter 2*, we generated and stored the synthetic transaction data in JSON files. Now, we generate similar data and read that data into a Delta table. First, notice the updates to the data generation notebook, `CH3-01-Generating Records and Schema Change`. Updates include a product string column, which will demonstrate schema evolution, as mentioned previously in the section on *Schema Evolution*. The `Product_vars` dictionary has been updated to include new products A, B, and C. The new products are then added to the transaction record generation mid-stream. In order to run the second notebook, you need to have this data generation notebook running first. Notice that we provide a list of possible products to the `writeJsonFile`. The result is an additional field, `Product`, with a product string added to the record (`Figure 3.4`).

```
import time

Products = ["None"]
t=1
total = 100
while(t<total):
  writeJsonFile(destination_path,Products=Products)
  t = t+1
  if not (t % 4):
    print(f"t={t}")
  time.sleep(sleepIntervalSeconds)
  if (t > total/4):
    Products = ["A","B","C"]
```

Figure 3.4 – Updated method for generating data with or without a product string

As a result of these changes, you now have a stream of data that adds an additional column midstream. You can start the Auto Loader notebook to see the schema evolution in action. The stream stops when it detects the extra column, providing an exception: `[UNKNOWN_FIELD_EXCEPTION.NEW_FIELDS_IN_RECORD_WITH_FILE_PATH]`.

Don't worry; upon restart, the schema evolution takes over. Run the cell with the stream in it again to restart. Restarts can be handled in workflows for production. You will see that in *Chapter 7*.

Let's walk through two options for reading and writing the data stream to a Delta table. Both options use Auto Loader to ingest. The files are then handled and written out by either Spark Structured Streaming or Delta Live Tables. There are two ingestion notebooks for this section. We start with `CH3-02-Auto Loader and Schema Evolution`.

Continuous data ingestion using Auto Loader with Structured Streaming

The code in this section uses Auto Loader plus Structured Streaming to format the incoming `JSON` files as `Delta`. The table name we use is `synthetic_transactions`. Before processing data, we create a widget, see *Figure 3.5*, to determine if we want to reset the schema and checkpoint history. Resetting the history can be helpful when altering or debugging pipelines. If you reset (remove) your checkpoint history, you will reprocess all historical data.

```
dbutils.widgets.dropdown(name='Reset', defaultValue='False', choices=['True', 'False'], label="Reset Checkpoint and Schema")
```

Figure 3.5 – Creating a widget to reset the checkpoint and schema

Next, we set our variables for the notebook, mainly paths:

```
table_name = "synthetic_transactions"
raw_data_location = f"{volume_file_path}/schema_change_data/"
schema_location = f"{volume_file_path}/{table_name}/schema"
checkpoint_location = f"{volume_file_path}/{table_name}/checkpoint"
```

Figure 3.6 – Setting path variables

The setup file provides us with the `volume_file_path`, where we store the transaction data, schema, and checkpoint folders.

As shown in *Figure 3.7*, we add spark configurations to optimize and reduce the sample size for schema inference:

```
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", True)
spark.conf.set("spark.databricks.delta.autoCompact.enabled", True)
spark.conf.set("spark.databricks.cloudFiles.schemaInference.sampleSize.numFiles",1)
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", True)
```

Figure 3.7 – Setting configurations to optimize and reduce sample size

You can set these Spark configurations in the cluster's advanced options. These configurations will automatically compact sets of small files into larger files for optimal read performance. The streaming command is composed of a `readStream` and a `writeStream`. Let's walk through each line of code:

1. Starting with the readStream, the incoming stream format is `CloudFiles`. This means the files located in cloud storage. In this case, we generate data and write it to cloud storage. However, creating a stream with `.format("kafka")` is also possible. In that case, the stream would ingest directly from the data stream without writing to cloud storage first. Back to our project, we designate the file format as JSON.

```
stream = spark.readStream \
  .format("cloudFiles") \
  .option("cloudFiles.format", "json") \
```

Figure 3.8 – Creating the stream using stream format CloudFiles and file format JSON

2. The default is to set column types to `string`. However, we can provide schema hints so that the columns we are sure about get typed appropriately. While inferring the schema, we also want to infer the column types. The new column is a `string`, so we do not see this option in action as new columns default to a `string`.

```
.option("cloudFiles.schemaHints","CustomerID int, Amount double,
TransactionTimestamp timestamp") \
.option("cloudFiles.inferColumnTypes","true") \
```

Figure 3.9 – Setting schema hints to reduce type mismatches; we want to infer the data type for columns that are not in the schema hint

3. Auto Loader uses the rescue column to catch the change and quickly puts the new column into play without data loss! Note that the stream will fail and need to be restarted. The schema location is needed because we want Auto Loader to keep track of the schema and evolve it over time.

```
.option("cloudFiles.schemaEvolutionMode", "addNewColumns") \
.option("cloudFiles.schemaLocation", schema_location) \
.load(raw_data_location) \
.select("*") \
```

Figure 3.10 – Setting schema evolution to add new columns and designating the schema location so Auto Loader keeps track of schema changes over time

4. Next, we specify the location of the raw data files. We need to select the fields we want. We select all data fields, see *Figure 3.10*, but you could be selective on the fields you pull in or add the metadata column, see File metadata column in Further reading for more information.
5. The code in *Figure 3.11* corresponds to the "write" portion of the stream. Delta is the default file format, but we prefer to set it explicitly for clarity. We also designate that this stream is append-only, as we are not yet performing any updates or inserts.
6. A checkpoint location is important. A checkpoint is a mechanism in Spark Structured Streaming that allows you to save the state of your stream. This is where it keeps track of its place in the processing of files. If your stream fails, when it restarts, it uses the checkpoint to resume processing where it left off. The merge schema option is essential for schema evolution. Merging the schema adds the new columns as they arrive without intervention.

```
.writeStream \
.format("delta") \
.outputMode("append") \
.option("checkpointLocation", checkpoint_location) \
.option("mergeSchema", "true") \
.trigger(processingTime="10 seconds") \
.toTable(tableName=table_name)
```

Figure 3.11 – Creating our checkpoint location, setting merge schema to true, and specifying processing to occur in 10 second micro batches

7. The trigger is the last line that may need explanation, as shown in *Figure 3.11*. The trigger setting refers to the processing pace. Our data from the generation notebook is continuous. Here, we efficiently handle it with ten-second micro-batches.

In *Step 7*, we use a trigger with a processing time. The processing time means micro-batches of data will be processed every ten seconds. Suppose you do not require micro-batches. If you need your data processed once an hour or once per day, then the `trigger.availableNow` option is best. When you use the `trigger.AvailableNow` as your pipeline trigger, the pipeline will kick off as scheduled in Jobs, aka Workflows, using a job cluster. At that time, Auto Loader will process all data available and then shut down.

You are finished reading and writing with Auto Loader plus Spark Structured Streaming. Next, we'll show you how to accomplish the same task with Delta Live Tables (in less code!).

Continuous data ingestion using Auto Loader with Delta Live Tables

This section uses the Auto Loader code to read the stream of JSON files and then DLT to write the files to a table. This notebook is under the `delta_live_tables` folder and is titled `CH3-03-Formatting to Delta with DLT`.

The only import you need is DLT: `import dlt`. The DLT code is relatively short partly because the pipeline configuration occurs in the pipeline object rather than the code. Before we look through our source code, let's navigate to the Delta Live Tables tab of the Worklows pane by selecting **Delta Live Tables** in the left-hand navigation bar. Next, click **Create Pipeline**, which opens the Pipeline settings page, as shown in *Figure 3.12*.

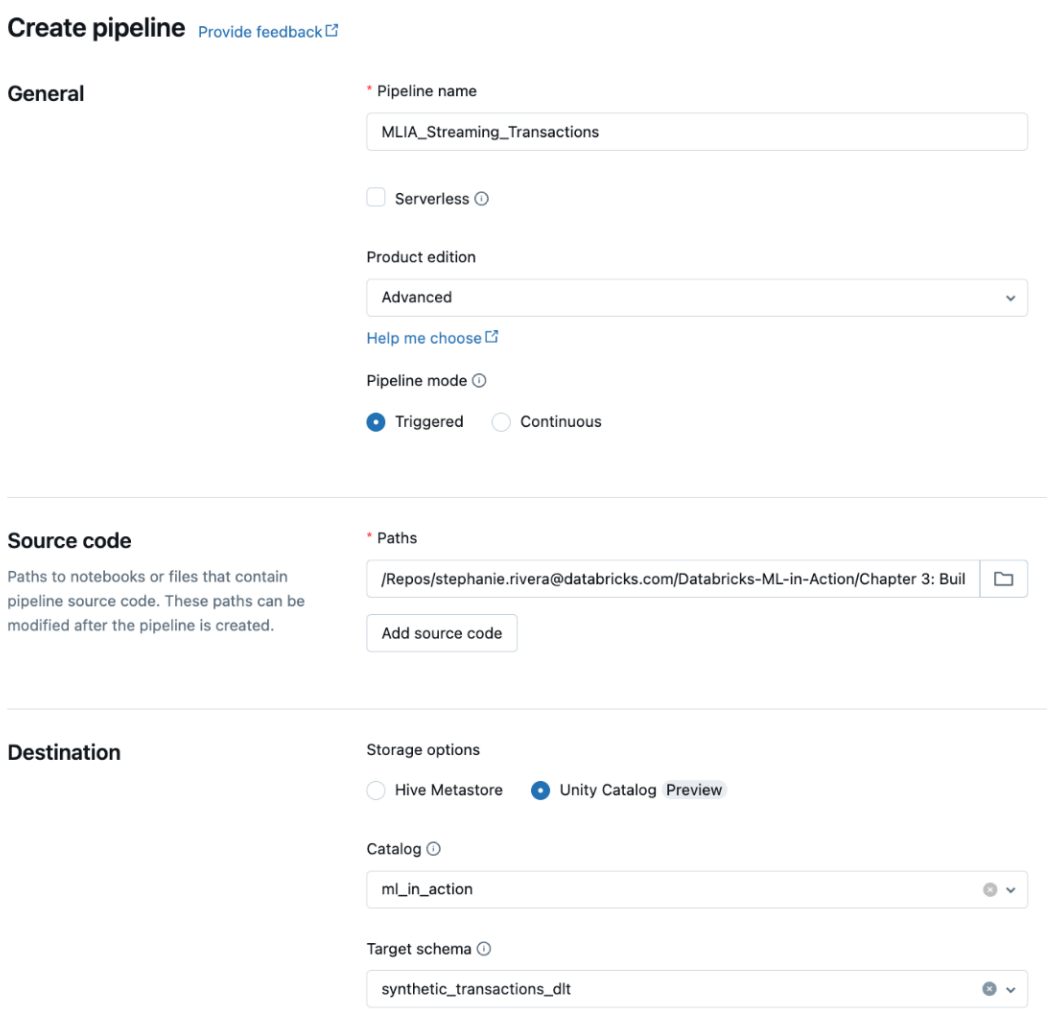


Figure 3.12 – The DLT pipeline setup in the workflow UI

Pipeline configuration for DLT

The pipeline settings are minimal, given that DLT does the optimization, so the setup instructions are minimal: The parameters entered into the pipeline configuration are accessible in the pipeline code using `spark.conf.get`, as shown in *Figures 3.13* and *3.14*.

1. Enter a **Pipeline name**. We use `MLIA_Streaming_Transactions`.
2. For **Product edition**, select **Advanced**. This DLT edition comes with the most features, including DLT's Expectation Rules.

3. For **Pipeline mode**, select **Triggered**. This will ensure the pipeline stops processing after a successful run.
4. For Paths, select this notebook:

```
Databricks-ML-in-Action/
Chapter 3: Building Out Our Bronze Layer/
Project: Streaming Transactions/
delta_live_tables/
CH3-03-Formatting to Delta with DLT
```

   from the repository for the source code.
5. Select **Unity Catalog** as your **Storage option**.
6. Select `ml_in_action` for the catalog and `synthetic_transactions_dlt` as the **Target schema**.
7. Enter the `table_name` and `synthetic_transactions_dlt` as configurations in the **Advanced** section. Additionally, specify your `raw_data_location` as `/Volumes/ml_in_action/ synthetic_transactions/files/synthetic_transactions`, as shown in *Figure 3.13*.

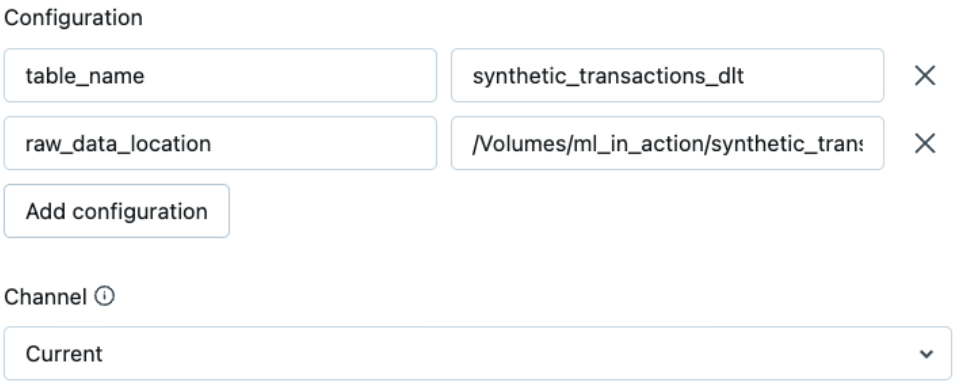


Figure 3.13 – Advanced pipeline configuration settings for variables

Methods in the DLT pipeline

Now that we have filled in the pipeline UI, let's focus on the methods used as the source code of the pipeline. In this function, we mostly reuse our previous code (*Figure 3.14*). You will notice that we do not use the setup file in this notebook. Instead, we use the variables we set in the pipeline settings' advanced configuration section.

```
def build_autoloader_stream():
  raw_data_location = spark.conf.get('raw_data_location')
  return spark.readStream.format('cloudFiles') \
      .option("cloudFiles.format", "json") \
      .option("cloudFiles.inferColumnTypes","true") \
      .option("cloudFiles.schemaEvolutionMode", "addNewColumns") \
      .option("cloudFiles.schemaHints","CustomerID bigint, Amount double, TransactionTimestamp timestamp") \
      .load(f"{raw_data_location}")
```

Figure 3.14 – The Auto Loader stream method details

Generating the bronze table in a DLT pipeline

The `generate_table()` function feeds our read stream created using Auto Loader into DLT. We use `spark.conf.get('variable_name')` to access the variable values we defined in the pipeline's Advanced settings (*Figure 3.13*). In the notebook, you will see the final step, a one-line cell called `generate_table()`. The function definition for `generate_table()` is in *Figure 3.15*.

```
def generate_table():
  table_name = spark.conf.get('table_name')
  @dlt.table(name=f'{table_name}',table_properties={"quality":"bronze"})
  def create_table():
    return build_autoloader_stream()
```

Figure 3.15 – Generating the Bronze table in a DLT pipeline

Back in the DLT UI, we save and click the **start** button. After the setup process, you will see a graph that includes our table. **Start** not only starts the pipeline but creates it as well. Once it finishes, you will have a screen like that shown in *Figure 3.1*.

Querying streaming tables created with DLT

You can query a streaming table created with DLT just like you query other tables. This is handy for populating dashboards, which we cover in *Chapter 8*.

> **Important note**
>
> If you are in a notebook, trying to query your streaming table and get this error:
>
> `ExecutionException: org.apache.spark.sql.AnalysisException: 403: Your token is missing the required scopes for this endpoint.`
>
> It is because to query streaming tables created by a Delta Live Tables pipeline, you must use a shared cluster using Databricks Runtime 13.1 and above or a SQL warehouse. Streaming tables created in a Unity Catalog enabled pipeline cannot be queried from assigned or no isolation clusters. You can change your cluster or use the DBSQL query editor.

The pipeline creates our Bronze table, wrapping up our learning applied to the Streaming Transaction data.

Project: Favorita Store Sales - Time Series Forecasting

You downloaded the Favorita Sales Forecasting dataset from Kaggle using `opendatasets` in the last chapter. We will use that data now to create Delta tables. To follow along in your own workspace, please refer to the following notebook:

- CH3-01-Loading Sales CSV Data as Delta

You can **Run all** at the top of this notebook. We utilized the data profile capability (*Figure 3.16*) to check the data types before writing to a table. In cell 6, click the **+** to see the data profile. This lets us look at the data types and distributions quickly.

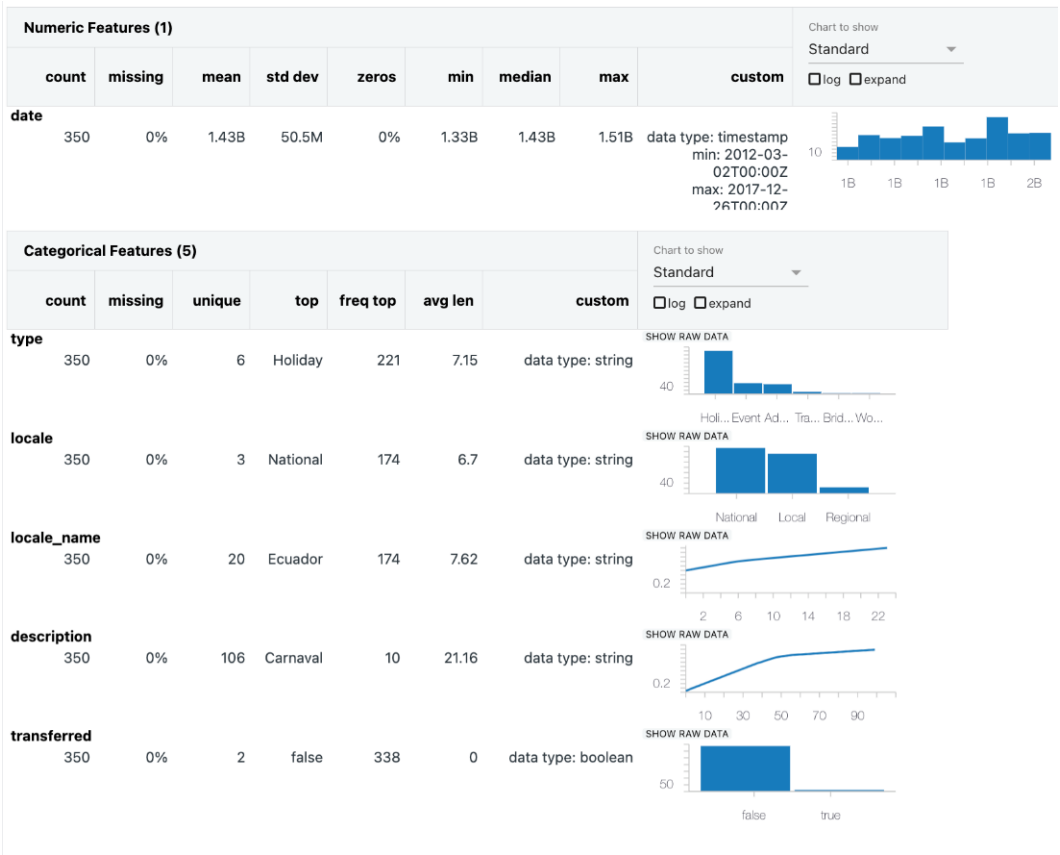


Figure 3.16 – Utilizing display(df), you can click the + to see the data profile. This lets us look at the data types and distributions quickly

Profiling shows the inferred data type for the date field is a string rather than a date or timestamp. Therefore, we alter the data type before writing to a Delta table.

The downloaded data is in single CSV files. We use Pandas to read the datasets and Spark to write to a Delta table. We demonstrate this for only the first file in the following code block (*Figure 3.17*).

```
df = pd.read_csv(f'{cloud_storage_path}/holidays_events.csv', sep=',', decimal='.')
df['date'] = pd.to_datetime(df['date'])
spark.createDataFrame(df).write.mode("overwrite").saveAsTable("holiday_events")
display(df)
```

Figure 3.17 – Reading in the sales holiday events datasets with Pandas

Each table for the Favorita project is transformed into Delta tables similarly. As a result, we only include the first table in the book's pages. However, the code that transforms each table is in the repository.

Project: Retrieval Augmented Generation Chatbot

RAG stands for **Retrieval Augmented Generation**. A RAG system often consists of your data, a vector database, a search algorithm, and a **generative AI** model to generate an answer to a user's query. *Figure 3.18* shows the pipeline we will build throughout this book.

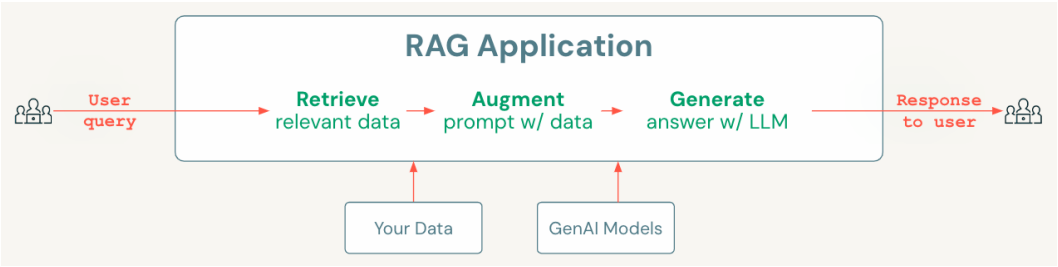


Figure 3.18 – A RAG example pipeline flow diagram

We see from *Figure 3.18* that we require retrieving relevant data according to the user query. That is where our search method would access a vector database and conduct a semantic or hybrid search. At this stage in the RAG Chatbot project, we have our PDFs in their raw, unstructured form. We want our users to access this knowledge via our chatbot, so we must bring relevant content from the PDFs into it, running in real-time. To do so, we'll convert all our PDFs into a machine-readable format. This chapter shows how to extract unstructured text from PDF files, chunk your data, and transform your text into embeddings. Then, we store the embeddings in a Delta table.

To follow along in your workspace, please open the following notebook:

- CH3-01-Creating Embedded Chunks

Our first step for data preparation is to extract the unstructured information from the PDF files. You may have many PDFs, which can be very long. Long documents can potentially be a problem for our future chatbot because they can easily exceed most Large Language Models' maximum context lengths (check out the blog in *Further reading* for more information on context lengths). Additionally, we likely don't need an entire document's worth of text to answer a specific question. Instead, we need a section or "chunk" of this text.

In the context of LLMs, the term "token" refers to a chunk of text that the model reads or generates. A token is typically not a word; it could be a smaller unit, like a character or a part of a word, or a larger one like a whole phrase. For this project, we create chunks of no more than 500 tokens with a chunk overlap of 50 tokens using the `SentenceSplitter` module of the `LlamaIndex` library. You could also use the `LangChain` library or any library you choose to split the content into chunks. We use the open-source `Llama-tokenizer` as this will be our main family of models across our project. Note that tokenizers may play a crucial role in your RAG quality.

If you are unfamiliar with tokenizers or struggling with any of the LLM verbiage, the Databricks LLM learning path (see Further reading) can provide a solid foundation of understanding to build upon. There other sources for learning about LLMs and NLP in general. However, we can only recommend the content of course we know.

Let's go through the steps to create the embedded chunks.

1. To start building the Bronze data layer, create an empty Delta table that includes the table's schema. Use the `GENERATED BY DEFAULT AS IDENTITY` feature to leverage Delta table capabilities to index the newly arriving data automatically. Also, add a table property to set the Change Data Feed to true.

```
%sql
--Note that we need to enable Change Data Feed on the table to create the index
CREATE TABLE IF NOT EXISTS pdf_documentation_text (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY,
  pdf_name STRING,
  content STRING,
  embedding ARRAY <FLOAT>
  ) TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

Figure 3.19 – Create an empty Delta table with the name pdf_documentation_text

2. For the next step, read the raw PDFs from the volume folder and save them to a table named `pdf_raw` (*Figure 3.20*). Store the PDFs in binary format in the `content` column to extract the text later. We will come back to the `pdf_documentation_text` table.

```
df = (
        spark.read.format("binaryfile")
        .option("recursiveFileLookup", "true")
        .load('dbfs:'+ documents_folder)
        )

df.write.mode("overwrite").saveAsTable(f"{catalog}.{database_name}.{table_name}")
```

Figure 3.20 – Read the PDF file in binary format and write it into a Bronze layer table

3. Let's see how it looks in a Delta table view. The binary format of each PDF is in the content column:

```
display(sql(f"SELECT * FROM {table_name} LIMIT 2"))
```

▸ (3) Spark Jobs

Table + New result table: ON Search

		modificationTime	length	content
1	f	2024-03-10T00:51:17.000+00:00	2119311	JVBERi0xLjUKJY8KMTc5IDAgb2JqCjw8IC9GaV
2	f	2024-03-10T00:51:18.000+00:00	1051746	JVBERi0xLjUKJY8KNjcgMCBvYmoKPDwgL0Zpl

Figure 3.21 – Displaying ingested PDF content into Delta Table

4. Next, we write a helper function using the `unstructured` library to extract the text from the PDF bytes. The function is in the `mlia_utils.rag_funcs` script.

```
from unstructured.partition.auto import partition
import re
import io

def extract_doc_text(x : bytes) -> str:
  # Read files and extract the values with unstructured
  sections = partition(file=io.BytesIO(x))
  def clean_section(txt):
    txt = re.sub(r'\n', '', txt)
    return re.sub(r' ?\.', '.', txt)
  # Default split is by section of document
  # concatenate them all together because we want to split by sentence instead.
  return "\n".join([clean_section(s.text) for s in sections])
```

Figure 3.22 – The helper function definition to extract document text

5. Let's apply this function to one of the PDF files we have to check the content of our document, see *Figure 3.23*. Great! Our function works on one of the ingested documents.

```
with open(f"{documents_folder}2303.10130.pdf", mode="rb") as pdf:
  doc = extract_doc_text(pdf.read())
  print(doc)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
Sorensen, T., Robinson, J., Rytting, C., Shaw, A., Rogers, K., Delorey, A., Khalil, M., Fulda, N., and Wingate, D. (202
2). An information-theoretic approach to prompt engineering without ground truth labels. In Proceedings of the 60th Annu
al Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Lingu
istics.
```

Figure 3.23 – Applying a helper function to extract the text from the PDF

Now we need to scale out. We leverage a Pandas **User Defined Function** (**UDF**) to scale the helper function across all pages of all documents.

```
@pandas_udf("array<string>")
def read_as_chunk(batch_iter: Iterator[pd.Series]) -> Iterator[pd.Series]:
    #set llama2 as tokenizer
    set_global_tokenizer(
      AutoTokenizer.from_pretrained("hf-internal-testing/llama-tokenizer")
    )
    #Sentence splitter from llama_index to split on sentences
    splitter = SentenceSplitter(chunk_size=500, chunk_overlap=50)
    def extract_and_split(b):
      txt = extract_doc_text(b)
      nodes = splitter.get_nodes_from_documents([Document(text=txt)])
      return [n.text for n in nodes]

    for x in batch_iter:
        yield x.apply(extract_and_split)
```

Figure 3.24 – Pandas UDF definition for our extractor function

6. Now, let's apply this function to our Delta table, see *Figure 3.25*. We encourage you to expand the content using the > symbol to see the chunk overlap.

Table + New result table: ON Search

	pdf_name	content
1	dbfs:/Volumes/ml_in_action/rag_chatbot/files/raw_documents/2312.14565.pdf	The Economics of Human Oversight: How Norms and Incentives A
2	dbfs:/Volumes/ml_in_action/rag_chatbot/files/raw_documents/2312.14565.pdf	However, both groups require more time to complete tasks, with a
3	dbfs:/Volumes/ml_in_action/rag_chatbot/files/raw_documents/2312.14565.pdf	The high economic value of AI skills (Stephany & Teutloff, 2024) is
4	dbfs:/Volumes/ml_in_action/rag_chatbot/files/raw_documents/2312.14565.pdf	It contains more than 14 million hand-labelled images of objects (

Figure 3.25 – The result from applying our Pandas UDF

Your chunks are ready. We need to convert them into embeddings. Embeddings are representations of information like text or images. Think of them as a type of machine learning feature. This format allows algorithms to perform a semantic search when ingesting our content into the Silver layer later.

We use a model to convert our chunks into embeddings. Databricks Model Serving has set of **Foundation Model APIs** (**FMAPI**), which allow you to access and query state-of-the-art open models from a serving endpoint, see *Figure 3.26*.

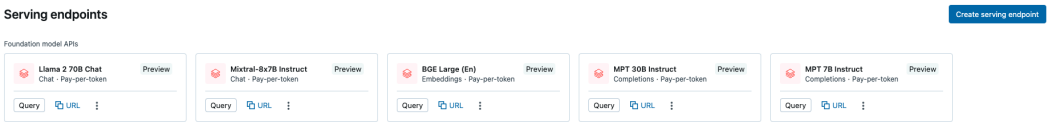


Figure 3.26 – Available models for access via FMAPI of Databricks

With Foundation Model APIs, you can quickly and easily build applications that leverage a high-quality generative AI model without maintaining your own model deployment (for more information, see *Deploy provisioned throughput Foundation Model APIs* in *Further Reading*).

The Foundation Model APIs are available in two access modes:

- **Pay-per-token**: This is the easiest way to start accessing Foundation Models on Databricks and is recommended for beginning your journey with Foundation Models.
- **Provisioned throughput**: This mode is recommended for workloads that require performance guarantees, fine-tuned models, or have additional security requirements.

Next, we leverage the DI Platform's pay-per-token capability from FMAPI that provides you with access to the `BGE_large_En` endpoint through the new functionality recently added to the `mlflow >=2.9` - mlflow deployments (previously known as AI Gateway). This functionality unifies the model serving endpoint management on Databricks.

> **Note**
>
> At the time of writing, the FMAPI is available only to US regions. If your workspace is not yet in a supported region, you can use any model of your choice (OpenAI, BERT, Llama tokenizer, etc.) to convert your content into embeddings. You may also need to fine-tune your model embedding to learn from your own context for better retrieval results.

Let's walk through the text to embedding conversion process:

1. We start by testing out our chosen model on a short sentence:

```
from mlflow.deployments import get_deploy_client

deploy_client = get_deploy_client("databricks")

embeddings = deploy_client.predict(endpoint="databricks-bge-large-en", inputs={"input": ["What is ChatGPT?"]})
pprint(embeddings)
```

```
{'data': [{'embedding': [0.03155517578125, -0.0080108642578125, -0.01099395751953125,
                         0.06463623046875, -0.007534027099609375, -0.0513916015625,
                         -0.01045989990234375, 0.03021240234375, 0.01142120361328125,
                         0.0228271484375, -0.044403076171875, -0.0018405914306640625,
```

Figure 3.27 – Applying FMAPI with the BGE endpoint to convert "What is ChatGPT?" into an embedding

2. Now we scale out to apply this embedding conversion across all our chunks. Again, we create a `pandasUDF` for scalability purposes.

```
@pandas_udf("array<float>")
def get_embedding(contents: pd.Series) -> pd.Series:
    import mlflow.deployments
    deploy_client = mlflow.deployments.get_deploy_client("databricks")
    def get_embeddings(batch):
        response = deploy_client.predict(endpoint="databricks-bge-large-en", inputs={"input": batch})
        return [e['embedding'] for e in response.data]

    max_batch_size = 150
    batches = [contents.iloc[i:i + max_batch_size] for i in range(0, len(contents), max_batch_size)]
    all_embeddings = []
    for batch in batches:
        all_embeddings += get_embeddings(batch.tolist())

    return pd.Series(all_embeddings)
```

Figure 3.28 – PandasUDF to apply embedding conversion across all chunks

3. Apply our UDF will append our `raw_table` chunks with the corresponding embeddings.
4. Now that our data preparation process is completed, we save our table in the initially pre-created Delta table `pdf_documentation_text` using append mode.

We've ingested our PDFs for this project in one big batch, which works perfectly fine as an example. However, it also means that anytime you want to add a new PDF to your chatbot's knowledge base, you must manually rerun all the above steps. We recommend a workflow for production-grade solutions to automate the steps above and incrementally ingest PDFs as they arrive in storage.

We now have a dataset ready for a Vector Search index, which we'll cover in *Chapter 4*.

Project: Multilabel Image Classification

In *Chapter 2*, we extracted and stored our raw image data in our volume. In this chapter, we prepare our image dataset and save training and validation sets into Delta tables. To follow along in your workspace, please open the following notebook:

- Ch3-01-Loading Images 2 DeltaTables

We start by creating variables and removing existing data if the **Reset** widget value is "True".

```
delta_train_name = "train_imgs_main.delta"
delta_val_name = "valid_imgs_main.delta"

if bool(dbutils.widgets.get('Reset')):
  !rm -rf {main_dir_2write}{delta_train_name}
  !rm -rf {main_dir_2write}{delta_val_name}
```

Figure 3.29 – Cleaning up existing data if Reset == True

Next, we create a function to ingest all our images in one table. Initially, each label is in its own `folder_label_name`. We extract the `image_name`, `image_id`, and `label_id`, as well as create a `label_name` . Then we write the DataFrame as Delta.

```
from pyspark.sql import functions as f
def prep_data2delta(dir_name,outcomes,name2write,path2write="YOUR_PATH",write2delta=True,returnDF=None,):
    mapping_dict = {"buildings": 0,"sea": 1,"glacier": 2,"forest": 3,"street": 4,"mountain": 5,}
    # As we have multi label problem we will loop over labels to save them all under 1 main training set
    for label_name in outcomes:
        df = (
            spark.read.format("binaryfile")
            .option("recursiveFileLookup", "true")
            .load(f"{dir_name}/{lable_name}")
            .withColumn("label_name", f.lit(f"{label_name}"))
            .withColumn("label_id", f.lit(f"{mapping_dict[label_name]}").astype("int"))
            .withColumn("image_name", f.split(f.col("path"), "/").getItem(10))
            .withColumn("id", f.split(f.col("image_name"), ".jpg").getItem(0).astype("int"))
            )
        if write2delta:
            df.write.format("delta").mode("append").save(f"{path2write}{name2write}")
        if returnDF:
            return df
```

Figure 3.30 – The prep_data2delta function definition

We use the `prep_data2delta` function to load and prepare our training and validation data sets (*Figure 3.30*). Notice that the function will save a Delta table if the `write2delta` flag is "True" and will return a DataFrame if the value for the `returnDF` flag is "True." Next, in the notebook, we call `prep_data2delta` for the training and validation sets.

Let's talk about data loaders

We load data in a fixed-size batch when fine-tuning or training our deep learning models. Each framework natively supports specific data types; some expand their native formats to other open-source formats. Data scientists sometimes prefer to keep their data (images, in our case) in blob storage and read it directly from storage rather than using Delta tables because they think this avoids additional work. However, we recommend storing images in a Delta table unless you are working with images larger than one gigabyte per image. Storing your images in a Delta table allows you to take advantage of Delta and Unity Catalog's additional benefits, such as lineage of data and models, data version control, duplicate data checks, and quality assurance.

As of today, you have a few options to read your data while working with a PyTorch or PyTorch Lightning framework:

- DeltaTorchLoader (recommended)
- Petastorm
- Reading images directly from Blob/Disk/Volumes

Our recommendation is to use DeltaTorchLoader. It handles data batching, sampling, and multiprocessing while training PyTorch pipelines without requiring a temporary copy of files like with Petastorm. See *Further reading* for more information.

As of now, DeltaTorchLoader, which we are going to use to load and transform our data from Delta into the PyTorch/Lightning dataloader framework for training our model in *Chapter 6*, requires you to have tables in the unmanaged Delta format in Volumes when using UC. Don't worry, the lineage is associated with the same path to the Volume as your main dataset. We'll talk more about lineage in *Chapter 6*. This requirement is due to UC's read/write security permissions with blob storage. The blob storage maintainers do not support these security settings yet. If you are not using UC, you should be able to read Delta tables directly from the managed tables.

> **Note**
>
> Petastorm requires a deeper understanding of certain pitfalls. The most common are memory usage issues due to data caching and the fact that it's using Apache parquet files rather than Delta files, so it consumes all versions of your parquet files.

The creators of the `DeltaTorchLoader` performed a few benchmarks with Petastorm. The benchmark was shared at the **Data & AI Summit** (**DAIS**) and is featured in *Figure 3.31*. In this project, we compare Petastorm to the classic Torch loader in *Chapter 6* to show the performance gain. The comparison demonstrates an incredible speed increase when reading the batch of data. We've also included a great video on `TorchDeltaLoader` in *Further reading* if you want to learn more.

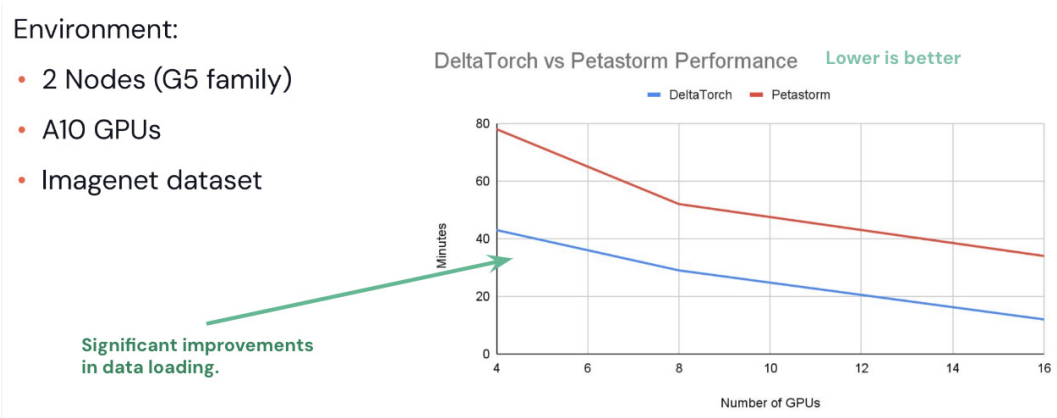


Figure 3.31 – DeltaTorchLoader benchmark - Performance comparison between DeltaTorch and Petastorm loaders

An alternative to using `DeltaTorchLoader`, is to keep file names in your Delta table instead of the images and collect them while passing them to the main PyTorch Loader with the trainer function. Keeping files in Delta is essential to avoid duplicates and control the list used during the training and validation, as you can pass the Delta version to the MLFlow during the tracking for full replication purposes.

Optimizing Our Data

Once our tables are created and written to the storage, we use a few functions to improve read performance on the Delta tables. We use the SQL magic command, `%sql`, to perform these operations using SQL in the Python notebook:

1. We use `OPTIMIZE` to keep an ideal number of files (*Figure 3.32*).

```
%sql
-- you can set up a widget to a notebook and consume widgets via $ variables with SQL
--OPTIMIZE delta.`/Volumes/$catalog/$database_name/files/intel_image_clf/valid_imgs_main.
delta`
OPTIMIZE delta.`/Volumes/ml_in_action/cv_clf/files/intel_image_clf/train_imgs_main.delta`
```

Figure 3.32 – Optimizing the file size and count of the training table

2. We disable Deletion Vectors because the `DeltaTorchReader` does not support Deletion Vectors yet (*Figure 3.33*).

```
%sql
ALTER TABLE delta.`/Volumes/ml_in_action/cv_clf/files/intel_image_clf/train_imgs_main.
delta` SET TBLPROPERTIES ('delta.enableDeletionVectors' = false);
ALTER TABLE delta.`/Volumes/ml_in_action/cv_clf/files/intel_image_clf/valid_imgs_main.
delta` SET TBLPROPERTIES ('delta.enableDeletionVectors' = false);
```

Figure 3.33 – Optimizing the training table

Note that the variables we saved in Python are inaccessible in SQL, so we hard-code them in this example. You could include SQL variables in the `global-setup` notebook to avoid this.

Now, we have the training and validation tables loaded and optimized to efficiently work with this image data for fine-tuning our multi-class computer vision models.

Summary

In this chapter, we focused on the essentials of building the Bronze data layer within the Databricks Intelligence Platform. Emphasizing the importance of schema evolution and the conversion of data into Delta format. We have applied these principles in our example projects. This chapter highlighted the advantages of Auto Loader and Delta Live Tables in the ETL process. Auto Loader, with its proficiency in handling file tracking and automating schema management, alongside DLT's robust capabilities in pipeline development and data quality assurance. Auto Loader and DLT are pivotal in our data management strategy. These tools facilitate an efficient and streamlined approach to data pipeline management, enabling us as data scientists to focus more on valuable tasks such as feature engineering and experimentation.

With our Bronze layer created, we now transition from this foundational work to the Silver layer. *Chapter 4, Getting to Know Your Data* will take us deeper into our data and demonstrate various Databricks tools that will aid us in tracking data quality, data exploration, and making our text machine explorable.

Questions

The following questions solidify key points to remember and tie the content back to your experience.

1. What are the names of the layers in the Medallion architecture design?
2. If you wanted to build a managed pipeline with streaming data, which product would you use, structured streaming or Delta Live Tables?
3. What feature did we use to add the product column to our Streaming Transaction data without manual intervention?

4. Do you have projects from your current position, experience, or on your roadmap that would benefit from one or more of the topics covered in this chapter?
5. What is a possible way to lessen the number of partitions when partitioning on a high cardinality column?

Answers

After putting thought into the questions, compare your answers to ours.

1. The layers of the Medallion architecture are Bronze, Silver, and Gold.
2. We recommend Delta Live Tables to build managed pipelines.
3. In the Streaming Transactions project example, we used Auto Loader's schema evolution feature to add a column without manual intervention.
4. We hope so! One example is a managed streaming data pipeline that could benefit from built-in data quality monitoring that comes with Delta Live Tables.
5. Bucketing is an optimal method specifically designed to provide an additional layer of organization in your data. It can reduce the number of output files and organize the data better for subsequent reading, and it can be especially useful when the partitioning column has high cardinality.

Further reading

This chapter covered different methods of ingesting data into your Bronze layer. Please take a look at these resources to read more about the areas that interest you most.

- *Use liquid clustering for Delta tables*: `https://docs.databricks.com/en/delta/clustering.html`
- *Spark Structured Streaming*: `https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html`
- *Delta Live Tables*: `https://docs.databricks.com/en/delta-live-tables/index.html`
- *DLT Databricks Demo*: `https://www.databricks.com/resources/demos/tutorials/lakehouse-platform/full-delta-live-table-pipeline`
- *Auto Loader options*: `https://docs.databricks.com/ingestion/auto-loader/options.html`
- *Schema evolution with Auto Loader*: `https://docs.databricks.com/ingestion/auto-loader/schema.html#configure-schema-inference-and-evolution-in-auto-loader`

- *Common loading patterns with Auto Loader*: https://docs.databricks.com/ingestion/auto-loader/patterns.html
- *Stream processing with Apache Kafka and Databricks*: https://docs.databricks.com/structured-streaming/kafka.html
- *File metadata column*: https://docs.databricks.com/en/ingestion/file-metadata-column.html
- *Large Language Models with Databricks*: https://customer-academy.databricks.com/learn/course/2267/large-language-models-with-databricks?generated_by=485783&hash=193cd8432a6ba52d98d09af917cce3b7f84d8a6d
- *Blog: Context length in LLMs: All you need to know*: https://agi-sphere.com/context-length/https://agi-sphere.com/context-length/
- *How We Performed ETL on One Billion Records for Under $1 With Delta Live Tables*: https://www.databricks.com/blog/2023/04/14/how-we-performed-etl-one-billion-records-under-1-delta-live-tables.html
- *Create tables - Managed vs External*: https://docs.databricks.com/en/data-governance/unity-catalog/create-tables.html#create-tables
- *Take full advantage of the auto-tuning available*: https://docs.databricks.com/delta/tune-file-size.html#configure-delta-lake-to-control-data-file-size
- *Import Python modules from Databricks repos*: https://docs.databricks.com/en/delta-live-tables/import-workspace-files.html
- *Deletion Vectors*: https://docs.databricks.com/en/delta/deletion-vectors.html
- *Databricks ML Runtime*: https://docs.databricks.com/runtime/mlruntime.html#introduction-to-databricks-runtime-for-machine-learning
- *Cluster advanced options*: https://docs.databricks.com/en/clusters/configure.html#spark-configuration
- *Deploy provisioned throughput Foundation Model APIs*: https://docs.databricks.com/en/machine-learning/foundation-models/deploy-prov-throughput-foundation-model-apis.html
- *Scaling Deep Learning Using Delta Lake Storage Format on Databricks*: https://www.databricks.com/dataaisummit/session/scaling-deep-learning-using-delta-lake-storage-format-databricks/
- *DeltaTorchLoader*: https://github.com/delta-incubator/deltatorch

Part 2:
Heavily Project Focused

This part introduces you to taking a set of data sources and working with them throughout the platform, from one end to another. The goal of this part is simply to demonstrate how to thoughtfully use all the bells and whistles of the platform. This part provides stories, code, lakehouse features, and best practices.

This part has the following chapters:

- *Chapter 4*, *Getting to Know Your Data*
- *Chapter 5*, *Feature Engineering on Databricks*
- *Chapter 6*, *Tools for Model Training and Experimenting*
- *Chapter 7*, *Productionizing ML on Databricks*
- *Chapter 8*, *Monitoring, Evaluating, and More*

4

Getting to Know Your Data

"Truth, like gold, is to be obtained not by its growth, but by washing away from it all that is not gold."

—Leo Tolstoy

In this chapter, we explore features within the Databricks DI Platform that help improve and monitor data quality and facilitate data exploration. There are numerous approaches to getting to know your data better with Databricks. First, we cover how to oversee data quality with **Delta Live Tables** (**DLT**) to catch quality issues early and prevent the contamination of entire pipelines. We'll take our first close look at Lakehouse Monitoring, which helps us analyze data changes over time and can alert us to changes that concern us. Lakehouse Monitoring is a big time-saver, allowing you to focus on mitigating or responding to data changes rather than creating notebooks that calculate standard metrics.

Moving on to data exploration, we will look at a couple of low-code approaches: Databricks Assistant and **AutoML**. Finally, we will touch on embeddings. We created embeddings from chunks of text in *Chapter 3*, and you'll learn how to use Databricks **Vector Search** (**VS**) to explore your embeddings.

Here is what you will learn as part of this chapter:

- Improving data integrity with DLT
- Monitoring data quality with Databricks Lakehouse Monitoring
- Exploring data with Databricks Assistant
- Generating data profiles with AutoML
- Preparing data for vector search and database indexing
- Enhancing data retrieval with Databricks Vector Search
- Applying our learning

Improving data integrity with DLT

In the last chapter, we introduced DLT as a helpful tool for streaming data and pipeline development. Here, we focus on how to use DLT as your go-to tool for actively tracking data quality. Generally, datasets are dynamic, not neat, and tidy like they often are in school and training. You can use code to clean data, of course, but there is a feature that makes the cleaning process even easier: DLT's expectations. DLT's expectations catch incoming data quality issues and automatically validate that incoming data passes specified rules and quality checks. For example, you might expect your customer data to have positive values for age or that dates follow a specific format. When data does not meet these expectations, it can negatively impact downstream data pipelines. With expectations implemented, you ensure that your pipelines won't suffer.

Implementing expectations gives us more control over data quality, alerting us to unusual data requiring attention and action. There are several options when dealing with erroneous data in DLT:

1. First, we can set a warning, which will report the number of records that failed an expectation as a metric but still write those invalid records to the destination dataset; see `@dlt.expect()` in *Figure 4.1* for an example.
2. Second, we can drop invalid records so that the final dataset only contains records that meet our expectations; see `@dlt.expect_or_drop()` in *Figure 4.1*.
3. Third, we can fail the operation entirely, so nothing new is written (note that this option requires manual re-triggering of the pipeline).
4. Finally, we can quarantine the invalid data to another table to investigate it further. The following code should look familiar to the DLT code in *Chapter 3*, but now with the addition of expectations.

```
def generate_table():
  table_name = spark.conf.get('table_name')
  @dlt.table(name=f'{table_name}',table_properties={"quality":"bronze"})
  @dlt.expect_or_drop("valid_CustomerID", "CustomerID IS NOT NULL")
  @dlt.expect("valid_Product", "Product IS NOT NULL")
  def create_table():
    return build_autoloader_stream()
```

Figure 4.1 – Using DLT expectations to enforce data quality

Let's look at our streaming transactions project as an example. In the *Applying our learning* section in *Chapter 3*, we used DLT to write the transaction data to a table. Utilizing the same DLT code, we will save ourselves the manual effort of cleaning the `CustomerID` column by adding an expectation to the original code to drop any records with a `null CustomerID`. We will set another expectation to warn us if the `Product` field is `null`.

Now, when we call `generate_table()`, the DLT pipeline will automatically clean up our table by dropping any null `CustomerID` values and flagging records without a `Product` value. Moreover, DLT will automatically build helpful visualizations to immediately investigate the data's quality.

To try this yourself, update the DLT code from *Chapter 3* (here's the path to the notebook as a reminder: `Chapter 3: Building Out Our Bronze Layer/Project: Streaming Transactions/delta_live_tables/`) to match *Figure 4.1*, and then rerun the DLT pipeline as you did before. Once the pipeline is complete, it generates the DAG. Click on the `synthetic_transactions_silver` table, then click the **Data Quality** tab from the table details. This will display information about the records processed, such as how many were written versus dropped for failing a given expectation, as shown in *Figure 4.2*.

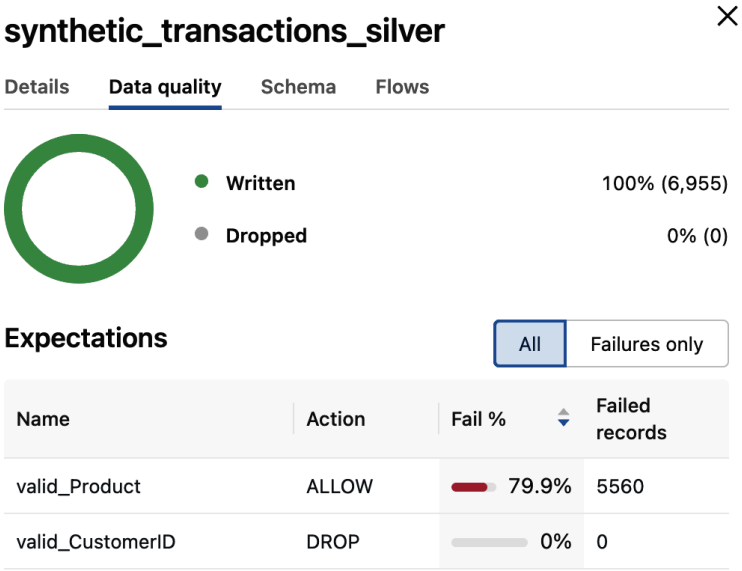


Figure 4.2 – The DLT data quality visualizations

These insights illustrate how expectations help automatically clean up our tables and flag information that might be useful for data scientists using this table downstream. In this example, we see that all records passed the `valid_CustomerID` expectation, so now we know we don't have to worry about null customer IDs in the table. Additionally, almost 80% of records are missing a `Product` value, which may be relevant for data science and **machine learning** (**ML**) projects that use this data.

Just as we've considered the correctness and consistency of incoming data, we also want to consider how we can expand our data quality oversight to include data drift, for example, when your data's distribution changes over time. Observing data drift is where Databricks Lakehouse Monitoring emerges as a vital complement to DLT, offering a configurable framework to consistently observe and verify the statistical properties and quality of input data.

Monitoring data quality with Databricks Lakehouse Monitoring

Use Databricks Lakehouse Monitoring to proactively detect and respond to any deviations in your data distribution. Over time, your data may undergo changes in its underlying patterns. This could be feature drift, where the distribution of feature data changes over time, or concept drift, where the relationship between inputs and outputs of your model changes. Both types of drift can cause model quality to suffer. These changes can occur slowly or rapidly in your production environment, which is why monitoring your data even before it becomes an input into your ML models and data products is essential.

Mechanics of Lakehouse Monitoring

To monitor a table in Databricks, you create a monitor attached to that table. To monitor the performance of a ML model, you attach the monitor to an inference table that holds the model's inputs and corresponding predictions. Databricks Lakehouse Monitoring provides the following profile types of analysis: snapshot, time series, and inference.

In addition to selecting the table to be monitored, called the **primary table**, you can optionally specify a baseline table to reference for measuring drift or the change in values over time. A baseline table is useful when you have a sample of what you expect your data to look like, such as the data with which your model was trained. Lakehouse Monitoring automatically computes drift relative to expected data values and distributions of the baseline table.

Creating a table monitor automatically creates two metric tables, `profile_metrics` and `drift_metrics`. Lakehouse Monitoring computes metric values on the table for the time windows and data subsets or "slices" you specify when you create the monitor. You can also add your own custom metrics; see *Further reading* for details.

Visualization and alerting

Lakehouse Monitoring generates an SQL dashboard automatically for every monitor. These dashboards provide a crucial platform for examining metrics and acting on results. Databricks' alert system serves as a vigilant guardian, promptly notifying you of the significant shifts in data quality or distributions you subscribe to. *Figure 4.3* shows how all the Lakehouse Monitoring components work together, using the data from the primary table and optional baseline table to generate a profile metrics table and drift table, which then populate the dashboard and power alerts.

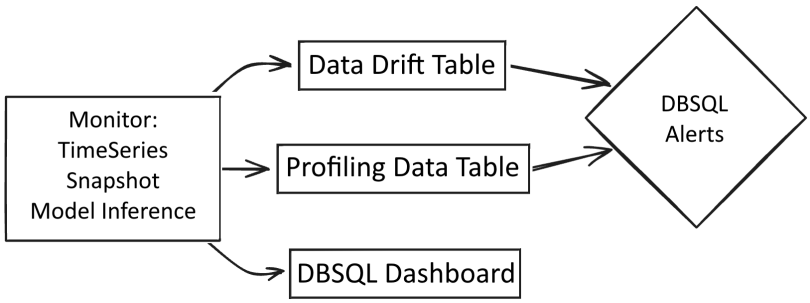


Figure 4.3 – Relationship between the input tables, the metric tables, the monitor, and the dashboard

Creating a monitor

You can create a Databricks Lakehouse Monitor using the **user interface** (**UI**) or, for a more flexible and programmable approach, you can use the API. The API method is particularly advantageous when you want to script the creation of monitors to integrate them into your automated data pipelines. The following is a high-level summary of the steps to create a Lakehouse Monitor using the API:

1. **Choose the profile type**: Decide on the profile type parameter that best suits your monitoring needs. The available types are `Snapshot`, `TimeSeries`, and `Inference`. Each type is suitable for different monitoring scenarios, with the `TimeSeries` and `Inference` types requiring a timestamp column. The inference profile also requires `prediction_col` and `model_id_col`.
2. **Create the monitor**: Use the `lakehouse_monitoring` module to call the `create_monitor` function, providing your table's catalog schema, table name, the chosen profile type, and the output schema for the monitor's results.
3. **Set the schedule**: If you wish to run the monitor at regular intervals, you can specify a schedule using the `MonitorCronSchedule` object, which takes a cron expression and a time zone ID.
4. **Control access**: After you create the monitor, you can manage access to the resulting metrics tables and dashboard using Unity Catalog privileges.
5. **Refresh and review results**: Use the `run_refresh` function to refresh and update the metric tables. You can also check the status of specific runs with the `get_refresh` function and list all refreshes associated with a monitor using `list_refreshes`.
6. **Review monitor settings**: The `get_monitor` function allows you to retrieve your monitor's current settings for review.

Figure 4.4 shows an example of how to use the Lakehouse Monitoring API to create a `TimeSeries` profile monitor:

```
from databricks import lakehouse_monitoring as lm

# Create a TimeSeries monitor
lm.create_monitor(
    table_name=f"{catalog}.{schema}.{table_name}",
    profile_type=lm.TimeSeries(
        timestamp_col="ts",
        granularities=["30 minutes"]
    ),
    output_schema_name=f"{catalog}.{schema}",
    schedule=lm.MonitorCronSchedule(
        quartz_cron_expression="0 0 12 * * ?",  # Refresh every day at noon PST
        timezone_id="PST",
    )
)
```

Figure 4.4 – Creating a simple TimeSeries table monitor

After establishing a robust framework for data quality monitoring with Databricks Lakehouse Monitoring, we can focus on enhancing our data exploration. This leads us to Databricks Assistant, a feature dedicated to helping developers be more productive in Databricks.

Exploring data with Databricks Assistant

Databricks Assistant is a feature designed to boost your productivity in Databricks. It has many capabilities, including generating SQL from English, explaining code, helping troubleshoot errors, and optimizing code. Databricks Assistant is an exciting feature to watch as more capabilities emerge, and we want to give you a taste of the possibilities. Since this chapter is about exploring and monitoring data, let's see how you can use Databricks Assistant as a low-code solution to explore your data. Suppose you are analyzing the Favorita Sales Forecasting data. You are looking to uncover insights into retail store distributions across various regions. You have a specific query in mind: you want to understand the store landscape in the Guayas region. However, SQL queries aren't your strong suit, and maybe crafting the perfect query seems daunting. In order to explore your data regardless, you can use Databricks Assistant. There is no notebook in the project repo for this section, but we encourage you to try Databricks Assistant on the Favorita Forecasting project tables. Any of the *Chapter 4* Favorita notebooks would be a great place to access Databricks Assistant. To access it, click on the icon shown in *Figure 4.5*, found in the left-hand sidebar of a notebook. Clicking on the icon will open the chat interface to the left of the notebook, where we will type in our questions.

Figure 4.5 – The Databricks Assistant icon

First, we ask Databricks Assistant how many stores and store types are in the state of Guayas in the `favorita_stores` table (*Figure 4.6*). Note that Databricks Assistant does not require Unity Catalog, but using Unity Catalog provides table information. The additional information makes Databricks Assistant's responses more helpful and specific to the table you're working with. This should sound similar to the **Retrieval Augmented Generation** (**RAG**) project. We are augmenting the generation of answers by providing relevant information. Now, let's see whether it can help us write the SQL query we need. Keep in mind that Databricks Assistant is powered by **generative AI**, so you may see different outputs when using it yourself.

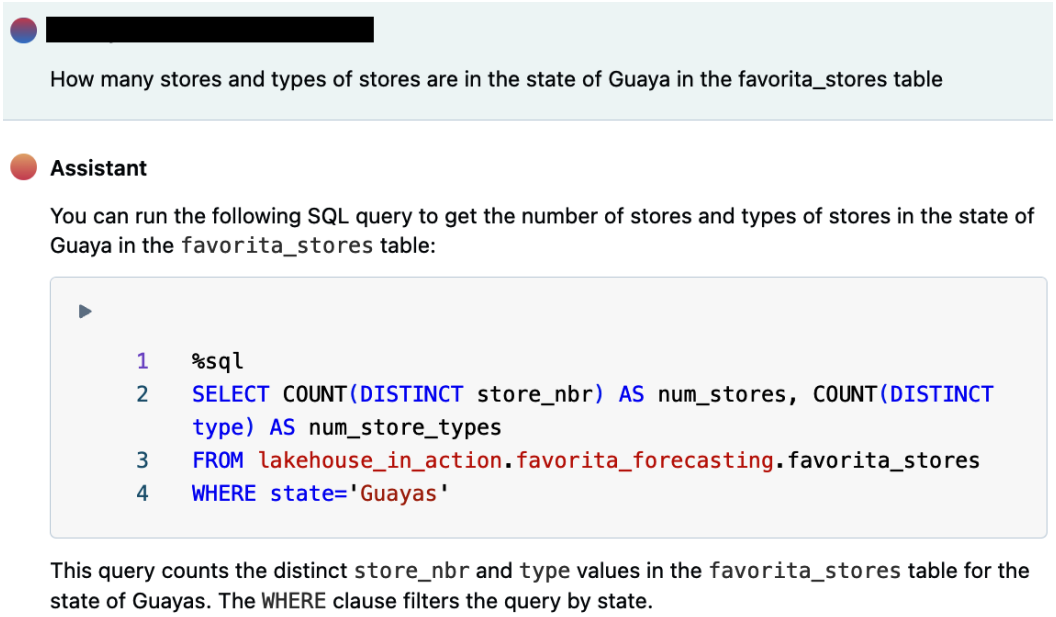


Figure 4.6 – Question and response interaction with Databricks Assistant

Nice! Databricks Assistant gave us some SQL code to paste into a notebook and run directly. However, a quick scan of the query shows us that we will only get a distinct count of stores and types rather than looking at the store type distribution we want. Let's refine our question and try again (*Figure 4.7*).

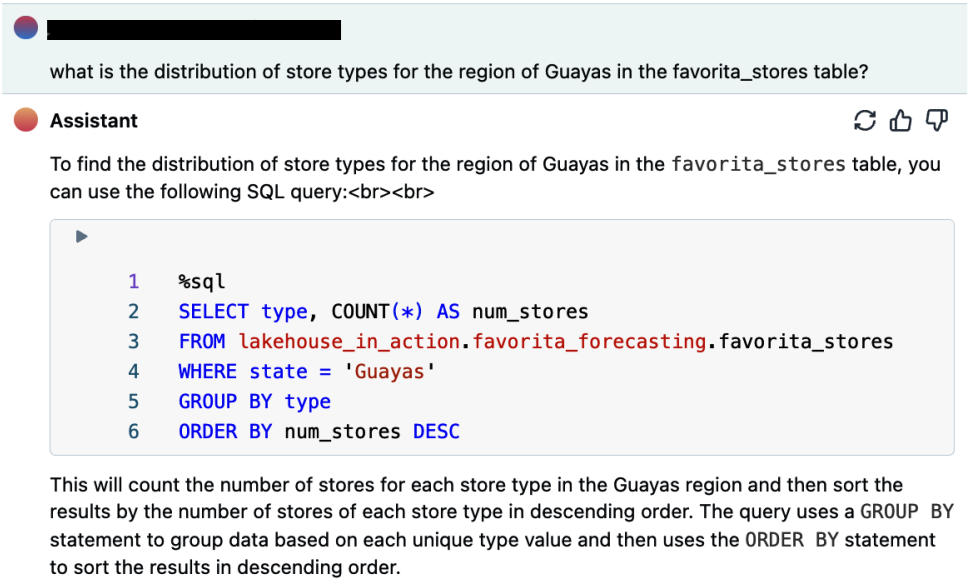


Figure 4.7 – The updated question submitted to Databricks Assistant and the results

After submitting the second question in *Figure 4.7*, Databricks Assistant provides a new query that accurately captures what we want to know. To make sure, let's copy the SQL code provided, paste it into a notebook, and run it.

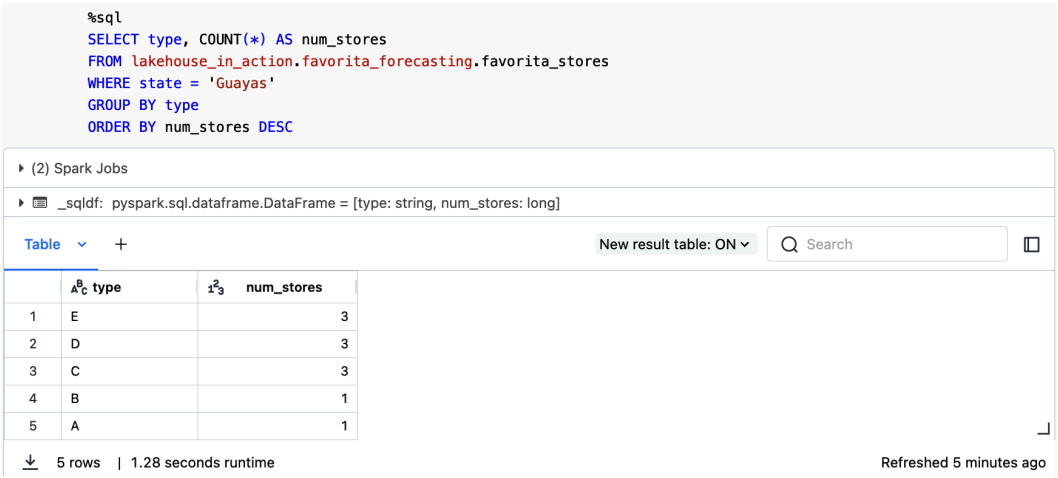


Figure 4.8 – Results from Databricks Assistant-generated SQL query

Now we see the distribution of stores by type in the Guayas region, just as we wanted. Databricks Assistant is a handy tool, and we can confirm that playing with it is also fun! We encourage you to try it out on your own to see how you can use English to explore your data.

Databricks Assistant also generates comment suggestions for your tables and fields. From the **Unity Catalog Explorer** page, navigate to the same `favorita_stores` table we explored previously. We see in *Figure 4.9* that Databricks Assistant has a suggestion for a table comment to help others understand the table's contents.

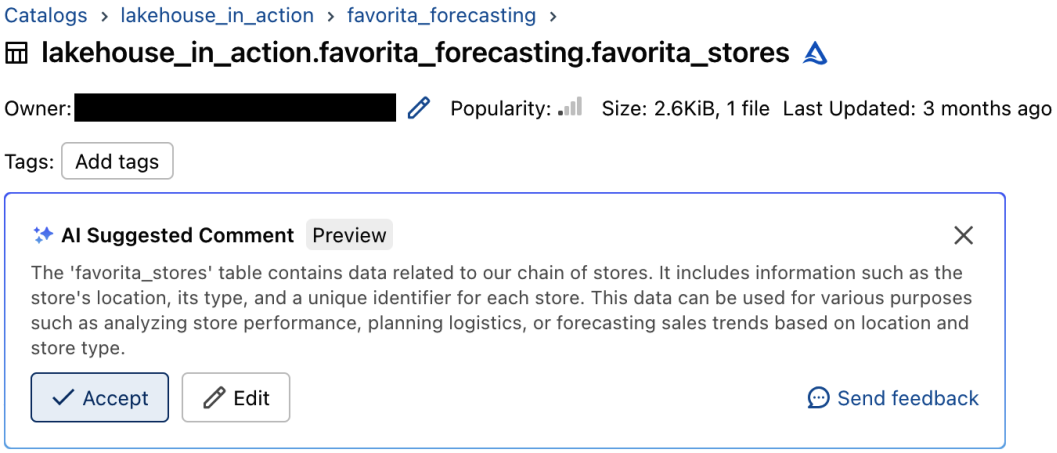


Figure 4.9 – Databricks Assistant suggests a descriptive table comment

Databricks Assistant can also generate descriptive comments for each field in the table by selecting the **AI Generate** button on the right-hand side of the table's **Columns** tab page. You can accept or edit the suggested comments. You can toggle off the suggestions by selecting the **Hide AI suggestions** button, as shown in *Figure 4.10*.

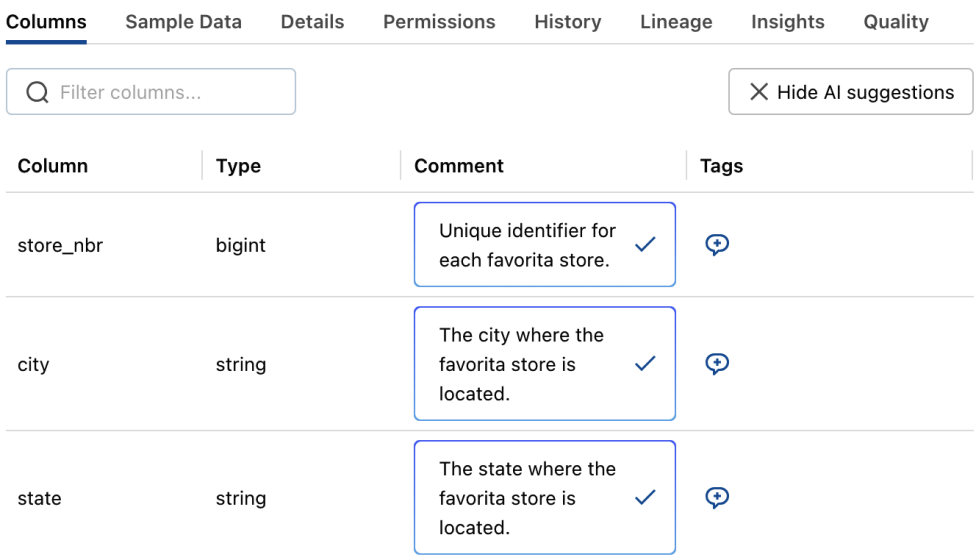


Figure 4.10 – Databricks Assistant suggests comments for the columns in a table

The generated comments might seem like they fall outside the scope of getting to know your data, but documentation is vital to making your data more easily discoverable for everyone else (and let's hope others use generated comments so you can explore their datasets more easily too). The faster you understand a dataset, the easier it is to further explore that data.

Databricks Assistant is a great way to analyze your data when you prefer to work in English rather than code directly, and when you have specific questions in mind. Now let's discuss another method for broader **exploratory data analysis** (**EDA**): autogenerated notebooks using AutoML.

Generating data profiles with AutoML

We introduced Databricks AutoML in *Chapter 1*. This tool automates ML development and augments data science workflows. AutoML is best known for generating models, but we'll get to modeling in *Chapter 6*. Since we're talking about getting to know your data, we first want to focus on one extremely useful feature built into AutoML that often flies under the radar: autogenerated Python notebooks. AutoML provides a notebook for data exploration in addition to the notebook code for every experiment it runs. We will jump right into creating an AutoML experiment, view the data exploration code, and then return to explore the modeling portion later.

We'll cover how to create an AutoML experiment via an API in the Favorita project notebooks. We encourage you to follow the instructions here to set up a simple regression experiment with the AutoML UI, so that we can take a look at the data profile created. Before you begin, make sure you have a DBR ML 9.1+ cluster running (you can use the **DBR ML 14.2** cluster set up in *Chapter 2*):

1. **Start the experiment**: Navigate to the **Experiments** tab on the platform's left-hand navigation bar. Click the **AutoML Experiment** button to initiate a new experiment.
2. **Configure the AutoML experiment**:
    - **Compute configuration**:
        i. **Cluster**: Select the **DBR ML 14.2** cluster set up in *Chapter 2*, or any other cluster with an ML runtime with a version greater than 9.1. Note that the cluster won't appear in the drop-down menu unless it's running.
    - **Experiment configuration**:
        i. **ML problem type**: Select the type of ML problem you're addressing. Select **Regression**, which is suitable for predicting continuous outcomes such as sales figures or temperature.
        ii. **Input training dataset**: Specify the dataset you will use to train the model. You can use the Favorita project's `ml_in_action.favorita_forecasting.favorite_train_set` table, or your own data (just make sure to update the problem type if `Regression` does not apply).

iii. **Prediction target**: Choose the specific column in your dataset that you want to predict. The AutoML process will use the other columns in your dataset to try and predict the values in this target column. If you're following along with the Favorita scenario, select **sales**.

Figure 4.11 shows the configuration of an AutoML experiment using the `favorite_train_set` training table. It illustrates how you can customize the AutoML process to fit the specific requirements of your ML task. By selecting the appropriate problem type, dataset, and prediction target, you're instructing the AutoML system on how to approach the model-building process.

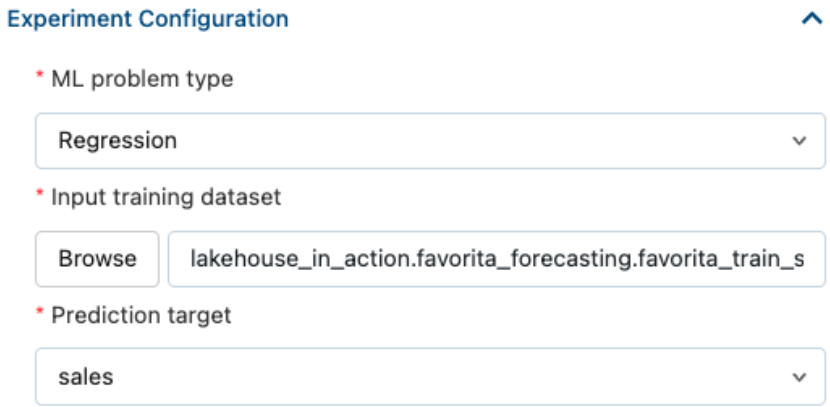


Figure 4.11 – Configuration of an AutoML experiment using the Favorita train_set table

Once you've filled out the UI and specified the Favorita table (or another table of your choice) as your input training dataset, click **Start AutoML** at the bottom of the screen. AutoML experiments may run for up to several hours as improvements in the evaluation metric continue, although you can set a shorter time limit for the experiment by changing the **Timeout** value (found under **Advanced Configuration**). As the experiment begins, a new page will open with progress updates. Once the experiment is complete, you will see links to two data artifacts: the notebook containing the code for the best model, labeled **View notebook for best model**, and the data exploration notebook, labeled **View data exploration notebook**, as shown in *Figure 4.12*.

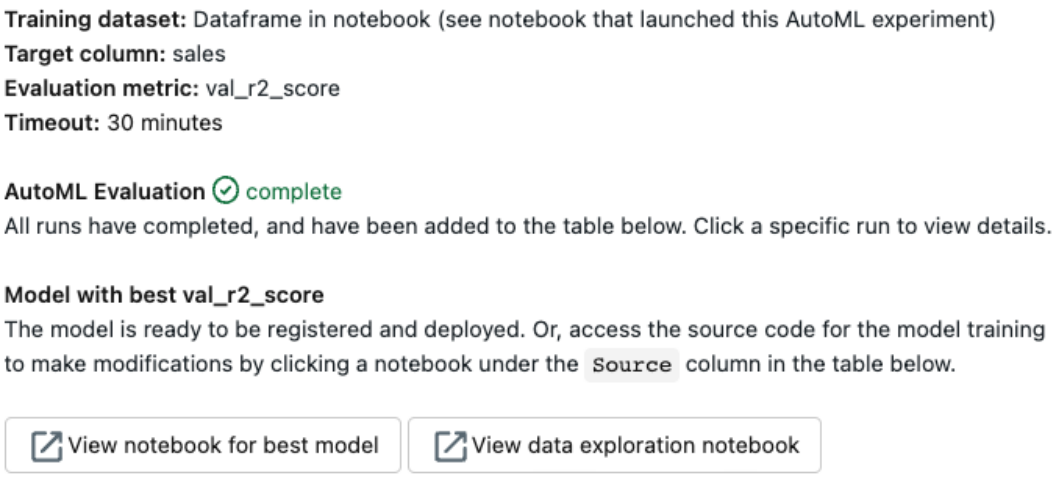


Figure 4.12 – The AutoML experiment page with the links to view the notebook for the best model and the data exploration notebook

The exploration notebook uses `ydata-profiling`, formerly referred to as pandas' profiler library, to generate statistics and summarize data for all the fields in the table. It also provides alerts on fields with high correlation issues, which could negatively impact models. These warnings are also available in the MLflow experiment UI, as shown in *Figure 4.13*.

Severity	Type	Affected Data	Action
Medium	More than 5% of values are null.	local_holiday_type national_holiday_type regional_holiday_type	AutoML imputed the null values.
Low	Data exploration notebook ran on a subset of rows because dataset is too big		Modify the data exploration notebook and rerun it to profile the entire dataset.
Low	High correlation columns	onpromotion national_holiday_type date family lag10_oil_price +4 more	Correlations found. Refer to data exploration notebook for more details.

Figure 4.13 – AutoML warnings called out during the experiment run

Look through the data exploration notebook for an overview of your data, from summary statistics to thorough profiles for each variable.

We have now explored two tools for data exploration: Databricks Assistant and the `ydata-profiling` library. These are great places to start for many classical ML projects. Next, we'll discuss a more advanced data format and how you can use the DI Platform to explore it: data embeddings and vector search. Embeddings are advanced transformations that translate complex, unstructured data into a numerical format conducive to ML algorithms, capturing intricate relationships within the data that are pivotal for sophisticated models.

Using embeddings to understand unstructured data

So far, we've focused on how to explore your structured data. What about unstructured data, such as images or text? Recall that we converted PDF text chunks into a specific format called embeddings in *Chapter 3*'s RAG chatbot project work. We require embeddings, meaning numerical vector representations of the data, to perform a similarity (or hybrid) search between chunks of text. That way, when someone asks our chatbot a question, such as "What are the economic impacts of automation technologies using LLMs?" the chatbot will be able to search through the stored chunks of text from the arXiv articles, retrieve the most relevant chunks, and use those to better answer the question. For more visual readers, see the data preparation workflow in *Figure 4.14*. We completed the **Data Preparation** step in *Chapter 3*. We'll run through the remaining setup steps in the workflow now.

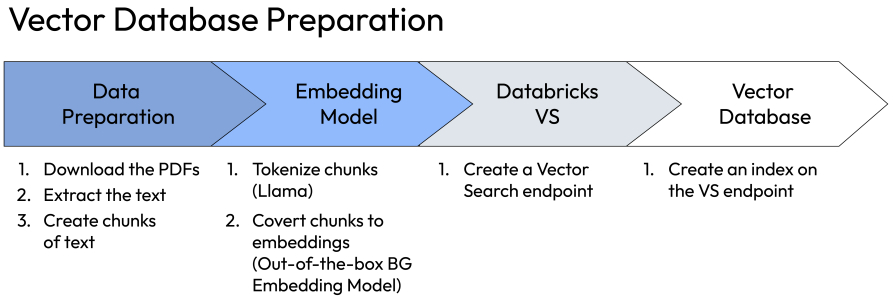


Figure 4.14 – Vector database setup is the prerequisite process supporting RAG's retrieval step

Embeddings are an essential part of building any chatbot. Pay attention to your embedding model to ensure it is relevant to the task. You wouldn't want to build a chatbot designed to answer questions in French but use an embedding model that only knows English – your chatbot's response quality will definitely suffer!

Embeddings that capture the nuances of language are essential for the chatbot to understand and generate contextually relevant responses. Equally important are the searching and filtering techniques you apply to the **vector database** itself. A vector database is similar to a SQL database, but instead of storing tabular data, it stores vector embeddings. A search algorithm can then search the embeddings. In the final flow of a RAG project, a user's question is also converted into embeddings, and the search algorithm uses those embeddings to find similar embeddings stored in the vector database. The chatbot receives the most similar embeddings from the vector database to help it craft a response to the user's question.

Let's consider the requirements of a good vector database solution:

- **Quality of the retrievals**: The correctness and completeness of embeddings returned as relevant by the search algorithm
- **Scalability of the solution**: The ability to scale per the number of requests coming to the application with dynamic traffic

- **Accessibility**: The ability to easily access, read, write to, and use the application in real time
- **Governance**: The ability to govern vector storage with the same access controls as the original sources used to create the vector embeddings and models
- **Integration**: The ability to easily integrate with current market technologies and eliminate time spent stitching technologies and solutions together

Using embeddings and vector search is a powerful way to improve a variety of ML projects. There are many uses for vector databases, the most common of which are as follows:

- **RAG systems**: Vector search facilitates efficient data retrieval, which is then used to augment a **Large Language Model** (**LLM**)'s response. Augmenting an LLM with results from vector search leads to more accurate chatbot responses and minimizes errors such as hallucinations in LLM outputs.
- **Recommendation systems**: E-commerce and streaming platforms use vector search for efficient nearest-neighbor searches, matching user behavior with relevant suggestions.
- **Image and video recognition**: Vector search facilitates quick searches for similar features in images and videos.
- **Bioinformatics**: Vector search can be applied to tasks such as DNA sequence alignment and protein structure similarity search to improve clinical research.

Enhancing data retrieval with Databricks Vector Search

Databricks VS is transforming how we refine and retrieve data for LLMs. Functioning as a serverless similarity search engine, VS enables the storage of vector embeddings and metadata in a dedicated vector database. Through VS, you can generate dynamic vector search indices from **Delta** tables overseen by Unity Catalog. Using a straightforward API, you can retrieve the most similar vectors through queries.

Here are some of Databricks VS's key benefits:

- **Seamless integration**: VS works harmoniously within Databricks' ecosystem, particularly Delta tables. This integration ensures that your data is always up to date, making it model-ready for ML applications. With VS, you can create a vector search index from a source Delta table and set the index to sync when the source table is updated.
- **Streamlined operations**: VS significantly simplifies operational complexity by eliminating the need to manage third-party vector databases. VS runs on serverless compute, meaning Databricks handles the infrastructure management for you.
- **Enhanced scalability**: Unlike standalone vector libraries, VS offers unparalleled scalability. VS handles large-scale data effortlessly, scaling automatically to meet the demands of your data and query load. This scalability is crucial for organizations with vast amounts of data and complex search requirements.

- **Unified data asset governance**: VS integrates with Unity Catalog; Unity Catalog handles data governance and access control lists. To prevent productional data leakage, you can manage access to the Databricks VS API and the underlying databases with Unity Catalog.
- **Model Serving integration**: Model Serving automates querying of the model serving endpoint for embedding generation without any overhead from users.

Flexibility in embedding model support

One of VS's major strengths is its support for any of the embedding models of your choice. VS supports hosted or fully managed embeddings. Hosted embeddings are self-managed. You create the embeddings and save them on a Delta table. For fully managed embeddings, the prepared text is saved in a Delta table, and embeddings are created by Databricks Model Serving. Model Serving will convert your incoming data into embeddings with the model of your choice. Databricks VS can support any model through the Databricks Model Serving endpoints, the Foundation Model API (as mentioned in *Chapter 3*), or external models. External models include SaaS models, for example, OpenAI's ChatGPT and Anthropic's PaLM. You can connect external models through the Databricks unified model serving gateway. See *External models in Databricks Model Serving* in *Further reading* for more information.

Setting up a vector search

As mentioned, VS is a serverless product. Hence, we require a real-time connection with our chatbot application to the relevant content stored in the vector database. We'll cover this again in the *Applying our learning*, *Project – RAG chatbot* section, but if you're ready to set up your own endpoint, you just need a few lines of code, as shown in *Figure 4.15*.

```
from databricks.vector_search.client import VectorSearchClient

vsc = VectorSearchClient()
vsc.create_endpoint(name="my_vector_search_endpoint", endpoint_type="STANDARD")
```

Figure 4.15 – Creating a Databricks VS endpoint

Once you create an endpoint, you can host multiple vector search indices under one endpoint. The endpoint will scale according to the demand. The code to host your first index is presented in the *Apply our learning* section).

There are limitations on the number of endpoints you can create indices per endpoint, and the embedding dimensions. Please review the documentation linked in *Further reading*, as Databricks may remove or update limits as the product evolves.

Technology moves fast, especially in the world of generative AI. Databricks VS was built even as we wrote this book, and we expect it to continue evolving to search not only through text but also images and audio with a more robust hybrid search engine in the future.

We've walked through various Databricks products and features geared toward helping you understand your data. Get ready to follow along in your own Databricks workspace as we work through the *Chapter 4* code by project and put these concepts into practice.

Applying our learning

It's time to apply these concepts to our example projects. We will use what we have learned to explore each project dataset, from using Databricks Assistant to AutoML, to creating a vector search index and exploring image data.

Technical requirements

Before you begin, review, and prepare the technical requirements necessary for the hands-on work in this chapter:

- We use the `missingno` library to address missing numbers in our synthetic transactions project data: `https://pypi.org/project/missingno/`
- For the RAG project, you will need to install the following either on your cluster or in the `CH4-01-Creating_VectorDB` notebook. If you choose to install them in the notebook, the code is included for you:
  - `typing_extensions==4.7.1`
  - `transformers==4.30.2`
  - `llama-index==0.9.3`
  - `langchain==0.0.319`
  - `unstructured[pdf,docx]==0.10.30`

Project: Favorita Store Sales – Time Series Forecasting

For the Favorita Store Sales project, we use many simple DBSQL queries for data exploration and to understand the relationships between datasets. Additionally, we use the `ydata_profiling` library to produce data profiles in HTML format, as shown in *Figure 4.19*. To follow along in your own workspace, please refer to the following notebooks:

- `CH4-01-Exploring Favorita Sales Data`
- `CH4-02-Exploring Autogenerated Notebook`
- `CH4-03-Imputing Oil Data`

In the last chapter, we created tables of the Favorita Sales Forecasting dataset from Kaggle. Now it's time to explore! Open up the first notebook, `CH4-01-Exploring Favorita Sales Data`, to do some initial data exploration in SQL:

1. The first cell is a simple `select *` SQL command. After running the cell, focus on the in-cell **Data Profile** and **Visualization** options.

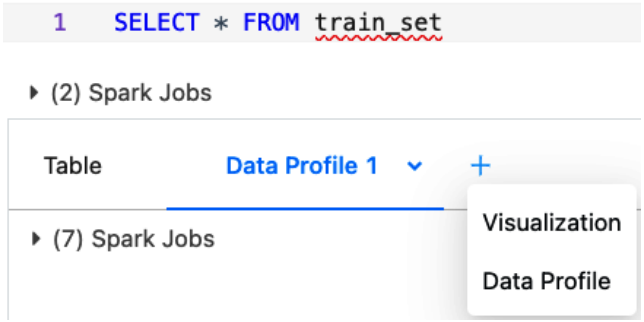


Figure 4.16 – You can create visualizations and a data profile of a SQL query result in a notebook

2. Choose **Data Profile** and investigate the information autogenerated about the data.
3. Choose **Visualization**. You can create a line chart with the date (month) versus the sum of sales and grouped by family. Notice that not all data is used to produce the visualization; we only see January at first. Once you save the visualization, the chart will display in the cell. The **Truncated data** message should be present at the bottom of the visualization. To increase the number of records in the chart, select **Aggregate over more data**.
4. Continue to play with the options. In the next cell, we filter to the top-performing product families. Investigate how different or similar the charts appear.

Now that we've done some initial exploration of the Favorita data, we can run a Databricks AutoML experiment to generate a baseline model. AutoML can be launched in the UI, which we demonstrated earlier in this chapter in the *Generating data profiles with AutoML* section (*Figure 4.11*), or you can create an experiment via an API as shown in *Figure 4.17*. For this project, we will launch both a regression experiment and a forecasting experiment. Let's start with the regression run.

Notice that at the top of the notebook, the default language is SQL rather than Python. Therefore, when we want to execute Python code, we need to include `%python` at the top of the cell. We use Python for AutoML in the last cell of the first notebook. We've set the `timeout_minutes` variable to `30`, so the experiment will run for up to 30 minutes. However, AutoML stops training models if the validation metric is no longer improving. In that case, the experiment will finish in less time. Once the run is complete, the notebook UI will display links to the MLflow experiment where the model versions are accessible, the best trial notebook with the best model's code, and the data exploration notebook. Since this chapter focuses on exploring data, we will only open the data exploration notebook for now.

```
from databricks import automl
summary = automl.regress(df, target_col="sales", timeout_minutes=30)
```

▸ (1) Spark Jobs

▾ (1) MLflow run

Logged 1 run to an experiment in MLflow. Learn more

```
2023/08/20 22:46:13 INFO databricks.automl.client.manager: AutoML will optimize for R
2 metric, which is tracked as val_r2_score in the MLflow experiment.
2023/08/20 22:46:14 INFO databricks.automl.client.manager: MLflow Experiment ID: 1570
387049089135
2023/08/20 22:46:14 INFO databricks.automl.client.manager: MLflow Experiment: http
s://e2-demo-field-eng.cloud.databricks.com/?o=1444828305810485#mlflow/experiments/157
0387049089135
2023/08/20 22:47:30 INFO databricks.automl.client.manager: Data exploration notebook:
https://e2-demo-field-eng.cloud.databricks.com/?o=1444828305810485#notebook/157038704
9089154
2023/08/20 23:16:56 INFO databricks.automl.client.manager: AutoML experiment complete
d successfully.
```

For exploratory data analysis, open the data exploration notebook

To view the best performing model, open the best trial notebook

To view details about all trials, navigate to the MLflow experiment

Figure 4.17 – Creating an AutoML experiment from a notebook

AutoML for data exploration

When you executed the final cell in `CH4-01-Exploring Favorita Sales Data`, you received several links in the results, as shown in *Figure 4.17*. Click on the link to the data exploration notebook (you can also open `CH4-02-Exploring Autogenerated Notebook` for a version that was autogenerated when we ran the AutoML experiment ourselves). Let's look at this notebook.

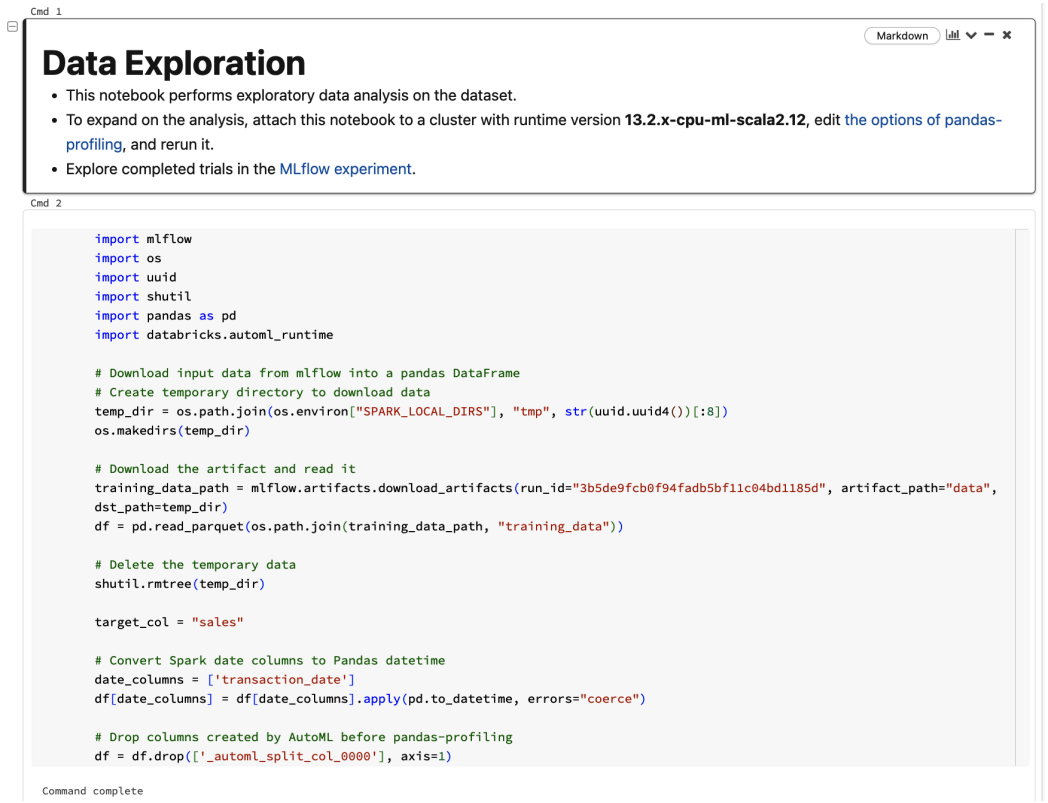


Figure 4.18 – EDA notebook created with AutoML

Figure 4.18 shows a portion of the automatically generated data exploration notebook. This notebook imports libraries and points to the training data. It also automatically converts the datetime columns to pandas datetime.

The notebook uses a pandas-based library, so the notebook limits the data to 10,000 rows.

Next, the notebook imports the `ydata_profiling` library, along with three additional correlation calculations added in this case. The `ydata_profiling` library provides similar data to what we would get using `summary()` functions, such as details about missing and correlated data. The library is easily imported into a notebook for exploring data. Once finished, you can export the details to HTML or PDF for easy sharing. It's a great timesaver when exploring new datasets.

```
from ydata_profiling import ProfileReport
df_profile = ProfileReport(df,
                           correlations={
                               "auto": {"calculate": True},
                               "pearson": {"calculate": True},
                               "spearman": {"calculate": True},
                               "kendall": {"calculate": True},
                               "phi_k": {"calculate": True},
                               "cramers": {"calculate": True},
                           }, title="Profiling Report", progress_bar=False, infer_dtypes=False)
profile_html = df_profile.to_html()

displayHTML(profile_html)
```

Figure 4.19 – ydata_profiling in the AutoML-created notebook

As shown in *Figure 4.17*, we launched a regression experiment via the API by calling `automl.regress()`. Now, we create another experiment with a forecasting problem type using the UI.

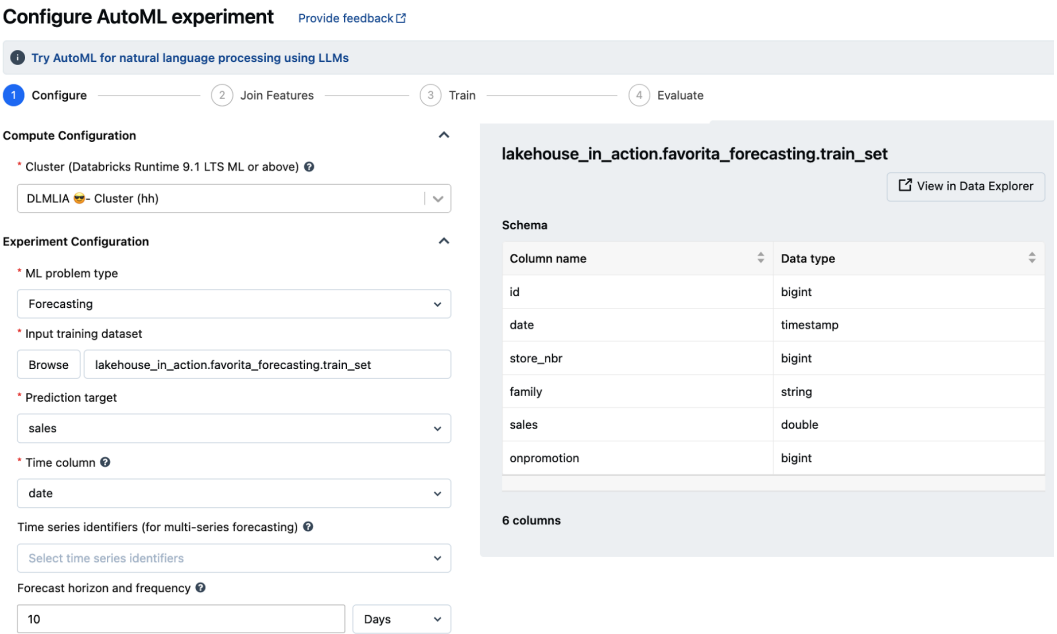


Figure 4.20 – Creating an AutoML forecasting experiment in the UI

When we create an AutoML forecasting experiment, we can incorporate a country's holidays into the model under the **Country Holidays** option in the **Advanced Configuration** section. We will choose **NONE** for this project, because although many countries are present in the drop-down menu, Ecuador is not among them. **NONE** will ensure holidays are not included as a feature in the model.

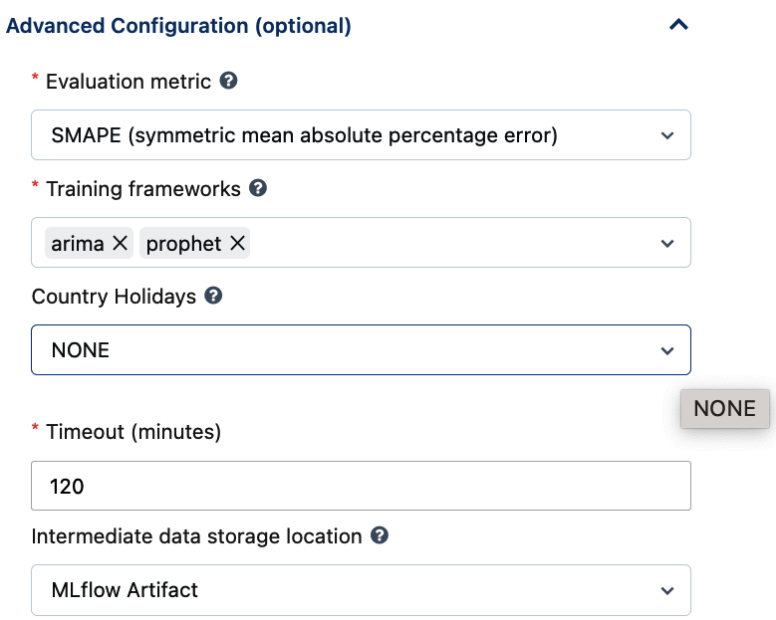


Figure 4.21 – AutoML forecasting experiment advanced configurations

Once all the fields are completed, click **Start AutoML** at the bottom of the screen to run the experiment (adjust the **Timeout** variable down if you want to ensure the run finishes within a given amount of time). When the experiment has finished running, you will see a new screen with a list of generated models.

We have now created an AutoML experiment in a notebook, used AutoML to generate a notebook for EDA, and used AutoML for time-series analysis. Databricks AutoML does a lot of the heavy lifting with minimal code for ML and EDA.

The Favorita sales dataset contains oil prices, which we think could impact sales, so let's augment the autogenerated data exploration with some of our own to explore the oil price data and see how we can use it.

Exploring and cleaning oil price data

To get started with the oil data, open the `CH4-03-Imputing Oil Data` notebook. We use the pandas API on Spark to view the data (*Figure 4.22*). We reindex the data because it's out of order and data for some dates is missing. Notice that initially, the data starts in May 2015 rather than January 2013. Also, the dates May 9th and May 10th appear to be missing.

```
import pyspark.pandas as ps

df = ps.read_table("oil_prices")
df.head(10)
```

▸ (2) Spark Jobs

	date	dcoilwtico
0	2015-05-04	58.92
1	2015-05-05	60.38
2	2015-05-06	60.93
3	2015-05-07	58.99
4	2015-05-08	59.41
5	2015-05-11	59.23
6	2015-05-12	60.72

Figure 4.22 – View the oil price data provided by Kaggle in the Favorita Sales dataset

If we add the `date` column as the index column, as shown in the notebook, you'll see that the rows are then in order. However, January 5th and January 6th are missing. This occurs because stock prices are not given for holidays or weekends.

```
df = (
    df.reindex(ps.date_range(df.index.min(), df.index.max()))
    .reset_index()
    .rename(columns={"index": "date"})
    .ffill()
)
df.to_table("oil_prices_silver")
df.head(10)
```

Figure 4.23 – Reindex to include all dates and fill forward missing prices

We use the `reindex()` command to create an updated index based on the minimum and maximum dates in the oil price data. The reindex creates rows for all missing dates. When new rows are created for the missing dates, the prices are NaNs. We use the `ffill()` function, or forward fill, to update the DataFrame by filling in dates without prices with the price from the day before. January 1st, 2013, doesn't have a previous day to fill from, so it remains `NaN`.

Now, we have a clean and consistent silver table of oil prices that we can pull into our Favorita time-series models. In the next chapter, we will do feature engineering with the Favorita sales data.

Project: Streaming Transactions

The synthetic dataset does not need cleaning (since we created it), but we can still explore the data to understand it better. To follow along in your own workspace, please refer to the following notebook:

- `CH4-01-Exploring Synthetic Transactions Data`

Open the notebook to generate visualizations with the `seaborn` library, as shown in *Figure 4.24*.

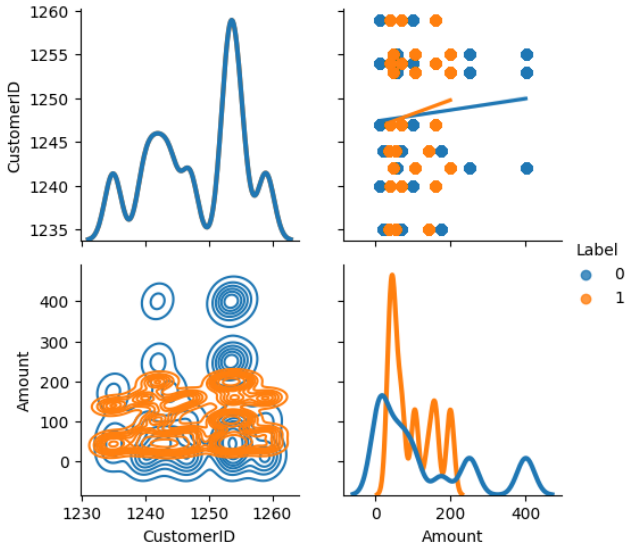


Figure 4.24 – Data visualizations of the synthetic transactions data

Feel free to explore the transaction data further. Next, we'll move on to the RAG chatbot.

Project: RAG Chatbot

In the last chapter, we extracted chunks of text from our PDF documents. We converted those chunks into embeddings using the BGE embedding model. As a result, we are now ready to take the next step in preparing our data for retrieval. To follow along in your own workspace, please refer to the following notebook:

- `CH4-01-Creating VectorDB`

We explained in the *Enhancing data retrieval with Databricks Vector Search* section that Databricks VS is a serverless managed solution. In other words, we need to create the VS endpoint to host our indices.

```
from databricks.vector_search.client import VectorSearchClient

vsc_endpoint_name = "ml_action_vs"
vsc = VectorSearchClient()

if vsc_endpoint_name not in [e['name'] for e in vsc.list_endpoints()['endpoints']]:
  vsc.create_endpoint(name=vsc_endpoint_name, endpoint_type="STANDARD")
  print(f"Endpoint named {vsc_endpoint_name} is in creation, wait a moment.")

print(f"Endpoint named {vsc_endpoint_name} is ready.")
```

Figure 4.25 – Creating the Databricks VS endpoint for the RAG chatbot

As a reminder, we will ingest the source table we prepared in *Chapter 3* into the index for retrieval search within the chatbot application.

```
display(spark.read.table(f"{catalog}.{database_name}.pdf_documentation_text"))
```

Figure 4.26 – Reading our source table containing text from the documentation

We import functions from the project's `utils` folder, `mlia_utils.rag_funcs`. The index only needs to be created once. Going forward, we will only write into it or read from it. We use an `if/else` clause to check whether an index with the name `catalog.database.docs_vsc_idx_cont` exists or not. If it exists, we just synchronize our `catalog.database.pdf_documentation_text` source table to it.

```
from databricks.sdk import WorkspaceClient
from mlia_utils.rag_funcs import *
# import databricks.sdk.service.catalog as c

# The table we'd like to index
source_table_fullname = f"{catalog}.{database_name}.pdf_documentation_text"
# Where we want to store our index
vs_index_fullname = f"{catalog}.{database_name}.docs_vsc_idx_cont"

if not index_exists(vsc, vsc_endpoint_name, vs_index_fullname):
  print(f"Creating index {vs_index_fullname} on endpoint {vsc_endpoint_name}...")
  vsc.create_delta_sync_index(
    endpoint_name=vsc_endpoint_name,
    index_name=vs_index_fullname,
    source_table_name=source_table_fullname,
    pipeline_type="TRIGGERED", # TRIGGERED or CONTINUOUS
    primary_key="id",
    embedding_dimension=1024, # Match your model embedding size (bge)
    embedding_vector_column="embedding"
  )
else:
  # Trigger a sync to update our vs content with the new data saved in the table
  vsc.get_index(vsc_endpoint_name, vs_index_fullname).sync()
```

Figure 4.27 – Check whether the index exists and create it if needed

There are a few key pieces we must make sure are in place before proceeding with the rest of our chatbot project:

- **Source data preparation**: `catalog.database.pdf_documentation_text` is the Delta table you have prepared with your self-managed embedding functions as the primary data repository for their vector search index.
- **Vector search index creation**: An index named `catalog.database.docs_vsc_idx_cont` using Databricks VS. This index is linked to the source Delta table. The index is set on a trigger base update (`pipeline_type="TRIGGERED"`), which means that any modifications or new additions to the historical texts should be synced manually to your vector search index. If you wish to have changes reflected in the vector search index automatically, choose the `CONTINUOUS` mode instead. This continuous update mechanism ensures the data is always current and ready for analysis.
- **Embedding model integration**: To transform the text data into a vector format, we are using self-managed embeddings. This means no model for embedding conversion is provided. However, you may want to choose managed embeddings, where `text_chunks` is automatically converted into embeddings with a model that is served through Databricks Model Serving. This can be specified in the setup under the `embedding_model_endpoint_name` parameter. This integration guarantees that the textual data is efficiently converted into vectors, making it suitable for advanced similarity searches.

We import the `wait_for_index_to_be_ready()` function from the `utils` folder. We run the code in *Figure 4.28* to repeatedly check the index's status until it is in an "online" state. As the VS index only needs to be created once, this function can take some time before the embeddings reach your VS index. Proceed once your index is in a "ready" state.

```
wait_for_index_to_be_ready(vsc, vsc_endpoint_name, vs_index_fullname)
```

Figure 4.28 – The wait for index function checks for the index status until it is online and ready

Once the index is online, we call `get_index()` to perform a similarity search. We also call a `describe()` method to show you the broader API options you have.

```
# Checking the information about the VS
vsc.get_index(vsc_endpoint_name, vs_index_fullname).describe()
```

```
{'name': 'ml_in_action.rag_chatbot.docs_vsc_idx_cont',
 'endpoint_name': 'ml_action_vs',
 'primary_key': 'id',
 'index_type': 'DELTA_SYNC',
 'delta_sync_index_spec': {'source_table': 'ml_in_action.rag_chatbot.pdf_documentation_text',
  'embedding_vector_columns': [{'name': 'embedding',
    'embedding_dimension': 1024}],
  'pipeline_type': 'TRIGGERED',
  'pipeline_id': 'aa31e35b-7af3-4823-ad83-2c803b59cca4'},
 'status': {'detailed_state': 'ONLINE_NO_PENDING_UPDATE',
  'message': 'Index creation succeeded using Delta Live Tables: https://adb-984752964297111.11.azuredatabricks.net#joblist/pipelines/aa31e35b-7af
3-4823-ad83-2c803b59cca4/updates/5a93e1c5-dbfc-4035-a4af-6875819cd5c6',
  'indexed_row_count': 327,
  'triggered_update_status': {'last_processed_commit_version': 1,
   'last_processed_commit_timestamp': '2024-01-03T16:20:14Z'},
  'ready': True,
  'index_url': 'adb-984752964297111.11.azuredatabricks.net/api/2.0/vector-search/endpoints/ml_action_vs/indexes/ml_in_action.rag_chatbot.docs_vsc
_idx_cont'},
 'creator': 'anastasia.prokaieva@databricks.com'}
```

Figure 4.29 – Using get_index with the describe method shows the index configuration

Because we use self-managed embeddings, our VS index is not connected to any model that can convert our input text query into embeddings to be mapped to the one under a database, so we need to convert them first! Here we are again leveraging the BGE embedding model from the Foundational Model Serving API and then passing embedded text to our index for similarity search:

```
import mlflow.deployments
deploy_client = mlflow.deployments.get_deploy_client("databricks")

question = "What are the economic impacts of automation technologies using LLMs?"

response = deploy_client.predict(endpoint="databricks-bge-large-en", inputs={"input": [question]})
embeddings = [e['embedding'] for e in response.data]

results = vsc.get_index(vsc_endpoint_name, vs_index_fullname).similarity_search(
  query_vector=embeddings[0],
  columns=["pdf_name", "content"],
  num_results=3)
docs = results.get('result', {}).get('data_array', [])
pprint(docs)
```

Figure 4.30 – Converting our query text into embeddings and passing to the index for similarity search

Here is the result of the retrieved queries (only the first found one with two columns `pdf_name` and content):

```
[['dbfs:/Volumes/ml_in_action/rag_chatbot/files/raw_documents/2303.10130.pdf',
  '7 Conclusion\n'
  'In conclusion, this study offers an examination of the potential impact of LLMs on various '
  'occupations and industries within the U.S. economy. By applying a new rubric for understanding '
  'LLM capabilities and their potential effects on jobs, we have observed that most occupations '
  'exhibit some degree of exposure to LLMs, with higher-wage occupations generally presenting more '
  'tasks with high exposure. Our analysis indicates that approximately 19% of jobs have at least '
  '50% of their tasks exposed to LLMs when considering both current model capabilities and '
  'anticipated LLM-powered software.\n'
  'Our research aims to highlight the general-purpose potential of LLMs and their possible '
  'implications for US workers. Previous literature demonstrates the impressive improvements of '
  'LLMs to date (see 2.1). Our findings confirm the hypothesis that these technologies can have '
  'pervasive impacts across a wide swath of occupations in the US, and that additional '
  'advancements supported by LLMs, mainly through software and digital tools, can have significant '
  'effects on a range of economic activities. However, while the technical capacity for LLMs to '
  'make human labor more efficient appears evident, it is important to recognize that social, '
  'economic, regulatory, and other factors will influence actual labor productivity outcomes. As '
  'capabilities continue to evolve, the impact of LLMs on the economy will likely persist and '
  'increase, posing challenges for policymakers in predicting and regulating their trajectory.\n'
  'Further research is necessary to explore the broader implications of LLM advancements, '
```

Figure 4.31– Sample of result of the retrieved query

Now, our VS index is ready to be used in our chatbot. We will explore how to connect all the tools together to generate a proper human-readable answer in *Chapter 6*!

Project: Multilabel Image Classification

In the last chapter, we saved our image data into volumes. Next, we will explore our data. To follow along in your own workspace, please refer to the following notebook:

- `CH4-01-Exploring Dataset`

We create our training dataset of images. Print the labels and a sample of the training data for a look at what we want our model to classify.

```
from mlia_utils.cv_clf_funcs import *

train_df = (spark.read.format("delta").load(train_delta_path))
print(idx_class(train_df))
display(train_df)
```

▸ (5) Spark Jobs

```
{'mountain': 0, 'buildings': 1, 'glacier': 2, 'forest': 3, 'sea': 4, 'street': 5}
```

Table + New result table: OFF

	path	modificationTime	length	content
1	dbfs:/Volumes/ml_in_action/cv_clf/files/intel_image_clf/raw_images/seg_train/seg_train/buildings/0.jpg	2023-12-29T11:56:23Z	11089	
2	dbfs:/Volumes/ml_in_action/cv_clf/files/intel_image_clf/raw_images/seg_train/seg_train/buildings/10006.jpg	2023-12-29T11:56:23Z	16802	

Figure 4.32 – Load and view the training dataset

We can use `display_image` to view a few pictures from our volumes.

```
from PIL import Image
import matplotlib.pyplot as plt

display_image(f"{train_dir}/forest/17856.jpg")
display_image(f"{train_dir}/street/15478.jpg")
```

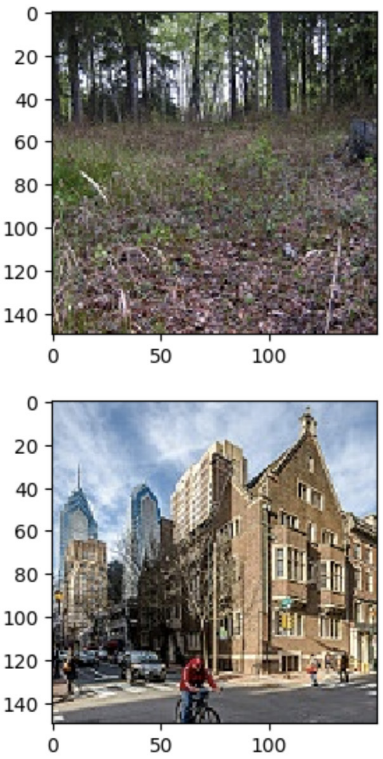


Figure 4.33 – Display images within the notebook

It's good to have an idea of the proportion of the different labels of our data. This is how you can view the proportion of data in our training dataset. Wherever you perform a multilabel classification, make sure you have a good distribution of labels. Otherwise, consider training individual models that might be combined or augment missing labels!

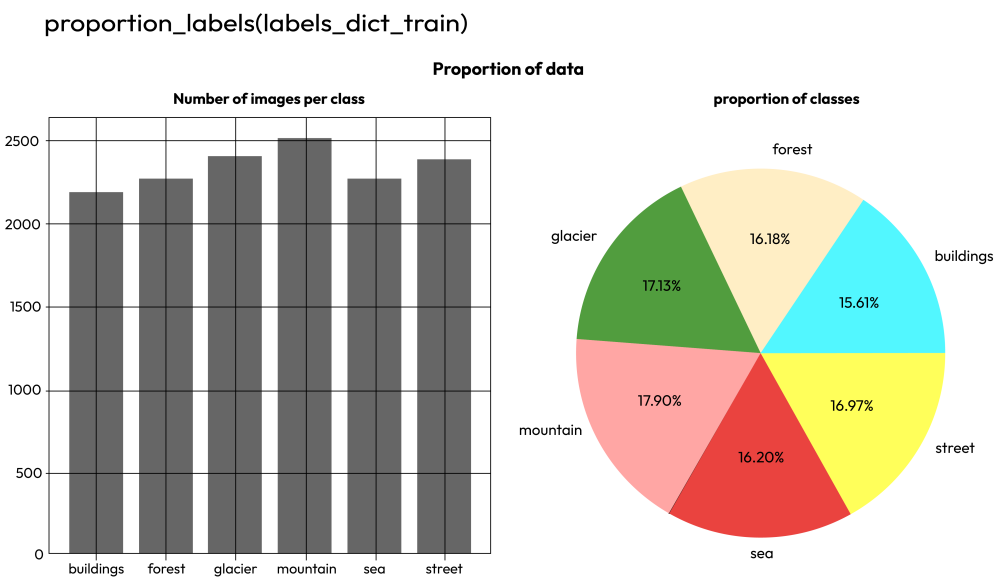


Figure 4.34 – View the proportion of labels in the training dataset

Now we have done some high-level exploration of our image classification dataset. For this project, no transformations are needed, so our next step with this data will be in *Chapter 6*.

Summary

It is critical to understand your data before using it. This chapter highlighted a variety of methods to explore and analyze our data within the Databricks ecosystem.

We began by revisiting DLT, this time focusing on how we use a feature called **expectations** to monitor and improve our data quality. We also introduced Databricks Lakehouse Monitoring as another tool for observing data quality. Among its many capabilities, Lakehouse Monitoring detects shifts in data distribution and alerts users to anomalies, thus preserving data integrity throughout its life cycle. We used Databricks Assistant to explore data with ad hoc queries written in English and showed why AutoML is an extremely useful tool for data exploration by automatically creating comprehensive data exploration notebooks. Together, all of these tools create a strong foundation to understand and explore your data. Finally, the chapter delved into Databricks VS and how using it to find similar documents can improve chatbot responses.

We have now set the foundation for the next phase of our data journey. *Chapter 5* will focus on how to build upon our bronze-layer data to create rich sets of features for data science and ML projects.

Questions

The following questions are meant to solidify key points to remember as well as tie the content back to your own experience:

1. What are some low-code options for data exploration that we discussed in this chapter?
2. When might you use Databricks Assistant for data exploration, and when might you use AutoML's data profile notebook?
3. How and why would you set expectations on your data?
4. When would you use a regular database versus a vector database? What are some common use cases for vector databases?

Answers

After putting thought into the questions, compare your answers to ours:

1. Some low-code data exploration options include using the `ydata` library, in-cell data profile, Databricks Assistant, and AutoML.
2. Databricks Assistant is useful for data exploration when you have a good idea of the analyses you want to build and you want code assistance. Databricks Assistant is a great way to speed up the coding process or augment your SQL knowledge. On the other hand, AutoML is very useful for automatically creating a profile notebook that broadly covers your dataset.
3. We would use Delta Live Tables to set expectations. Expectations are a way to flexibly handle data abnormalities and give the options to report bad data, drop that data, or fail the pipeline entirely.
4. Regular databases, or relational databases, are designed for data in tabular form, typically organized in rows or columns. A vector database is designed to store vector data, such as embeddings and high-dimensional data. Vector databases are optimized for operations based on vector space models, similarity searches, image and video analysis, and other ML problems. Some common use cases include **Retrieval Augmented Generation** (**RAG**) systems, recommendation systems, and image and video recognition.

Further reading

In this chapter, we pointed out specific technologies, technical features, and options. Please take a look at these resources to get deeper into the areas that interest you most:

- *Enabling visualizations with Aggregations in DBSQL*: `https://docs.databricks.com/sql/user/visualizations/index.html#enable-aggregation-in-a-visualization`
- Using the ydata profiler to explore data: `https://ydata-profiling.ydata.ai/docs/master/index.html`
- *Advancing Spark - Meet the new Databricks Assistant*: `https://youtu.be/Tv8D72oI0xM`
- *Introducing Databricks Assistant, a context-aware AI assistant*: `https://www.databricks.com/blog/introducing-databricks-assistant`
- *Model monitoring custom metrics creation*: `https://docs.databricks.com/en/lakehouse-monitoring/custom-metrics.html`
- Databricks Vector Search: `https://docs.databricks.com/en/generative-ai/vector-search.html`
- External models in Databricks Model Serving: `https://learn.microsoft.com/en-us/azure/databricks/generative-ai/external-models/`

5
Feature Engineering on Databricks

"Applied machine learning is basically feature engineering."

– Andrew Ng

As we progress from *Chapter 4*, where we harnessed the power of Databricks to explore and refine our datasets, we are now ready to delve into the components of Databricks that enable the next step: feature engineering. We will start by covering **Databricks Feature Engineering** (**DFE**) in Unity Catalog to show you how you can efficiently manage engineered features using **Unity Catalog** (**UC**). Understanding how to leverage DFE in UC is crucial for creating reusable and consistent features across training and inference. Next, you will learn how to leverage Sparka Structured Streaming for calculating features on a stream, which allows you to create stateful features needed for models to perform quick decision-making. Feature engineering is a broad topic. We will focus on how the DI Platform facilitates the development of certain feature categories, such as **point-in-time lookups** and **on-demand features**. You will also learn how to calculate features in real time during model inference, which is vital for scenarios requiring immediate data processing.

Here is what you will learn about as part of this chapter:

- Databricks Feature Engineering in Unity Catalog
- Feature engineering on a stream
- Employing point-in-time lookups
- Computing on-demand features
- Applying our learning

Databricks Feature Engineering in Unity Catalog

In this chapter, we will focus on several types of features. Feature types can be roughly grouped based on when they are calculated relative to the time of model prediction. The three types we cover in this chapter are batch, streaming, and on-demand:

- **Batch features**: These are features that are generated hours, days, or even further ahead of when the feature will be used as a model input. There are several reasons for this; the model may be offline and run in a batch prediction fashion, or perhaps the feature values change slowly relative to the model inference timeline. In these cases, it is not necessary to recompute the values more frequently. An example of a batch feature is the `holidays` field in our *Favorita Sales Forecasting* project, as we can compute it before we need it and don't expect the values to change frequently.
- **Streaming features**: These are features processed in real or near-real time as the source data is ingested in the data pipeline, allowing for continuous and incremental feature creation. To demonstrate this, we will calculate a streaming feature for our *Streaming Transactions* project with Spark Structured Streaming.
- **On-demand features**: Unlike batch or streaming features, on-demand features are only computed when needed, which is to say, at the time of inference. These features are crucial for scenarios where feature values are unknown beforehand and must be calculated in real time. We will delve into the mechanics of computing and storing these features, demonstrating their implementation and integration into predictive models.

Any Unity Catalog table with a defined primary key constraint can be a centralized repository for materialized, pre-computed (e.g., batch or streaming) features. These types of tables, whose explicit purpose is to centrally store features to be used throughout an organization's analytics, data science, and machine learning projects, are commonly called **feature tables**. Feature tables allow data scientists to share their engineered features and find features that other team members have built.

They are a wonderful way to ensure business logic is centrally stored, save team members from recreating already-created features, and prevent duplicate work. Plus, Unity Catalog manages all data's discoverability, lineage, and governance, so your feature tables are easily managed just like any other table. In Databricks, any table with a non-nullable primary key can be a feature table. While there is no specific API call to list all tables eligible to be a feature table or to list those officially created as a feature table, we recommend using tags or naming conventions to identify feature tables. This book's example projects will follow the convention of appending `_ft` at the end of feature table names. Feature Engineering in Unity Catalog provides the `FeatureEngineeringClient` Python client (commonly imported under the alias of `fe`) for creating and updating feature tables (*Figure 5.1*):

```
fe.create_table(
    name="lakehouse_in_action.database_name.
    feature_table_name_ft",
    primary_keys=["sequence_id","sequence_step"],
    df=feature_dataframe,
    description="This feature table was created by
    x,y,z and would be great for augmenting w for
    modeling or business use case q.")
```

Figure 5.1 – Example of creating a feature table with FeatureEngineeringClient

As we mentioned, the combination of Delta, UC, and the Feature Engineering client provides considerable benefits. Features can be shared and reused across teams, reducing the need to recreate them from scratch. Reusability can save time and resources, and it can also help to ensure that features are consistent across different teams. Centralized feature tables ensure the same code computes feature values for training and inference. This is especially important to avoid online/offline skew – that is, the discrepancy that can occur when the data transformations used during the model training phase (offline) are different from those used when the model is deployed and making predictions (online). *Figure 5.2* shows a data science project workflow and highlights the offline and online transformations that must match.

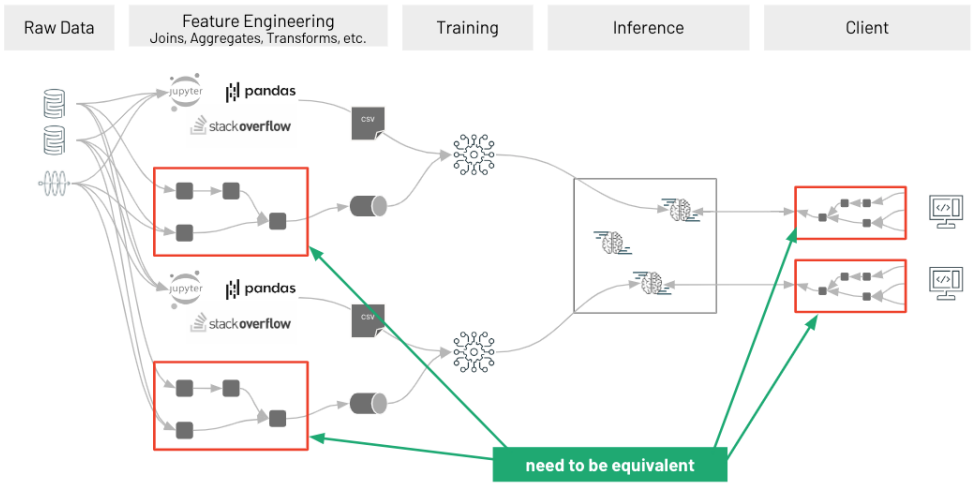


Figure 5.2 – You can avoid accuracy issues by guaranteeing that the transformations performed on training data are the same as those performed on inference data

We will prevent online/offline skew in the *Streaming Transactions* project by wrapping the data transformation processes into a packaged model in *Chapter 6*. Consistency can improve the accuracy of models, as it ensures the same input generates the same output in production as in training.

> **Note**
>
> For those who are familiar with the Databricks Feature Store, you may wonder why we do not utilize it in this chapter. Before Unity Catalog, the Databricks Feature Store provided incredible value with added lineage and metadata details. With the advancement of UC, instead of having a separate Feature Store product, every table utilizing UC automatically has these added benefits. The original Databricks Feature Store has been absorbed into UC.

In the *Applying our learning* section ahead, we will save our features to a feature table using both SQL and `FeatureEngineeringClient`. Once stored, Unity Catalog makes it easy to track the data sources used to create your features and the downstream artifacts (such as models and notebooks) that use each feature. Even better, when you log a model using a training set of features from a feature table, feature metadata is packaged with the resulting model. Watch for this when we are working on the *Streaming Transactions* project.

Feature engineering on a stream

Before diving into feature engineering on a stream, we want to clarify the difference between **streaming pipelines** and **streaming data**. If you have not used Spark Structured Streaming before, it is a stream processing engine built on the Spark SQL engine. It makes it easy to write streaming calculations or transformations like you would write expressions for static data. Structured Streaming pipelines can process batch or streaming data. Streaming pipelines have elements such as checkpoints to automate the data flow. Streaming pipelines, however, are not necessarily always running; rather, they only run when the developer chooses it. In contrast, streaming data (also known as **real-time data**) refers to continuously generated data that can be processed in real time or batch. To simplify, think of streaming pipelines as a series of automated conveyor belts in a factory set up to process items (data) as they come. These conveyor belts can be turned on or off as needed. On the other hand, streaming data is like a never-ending flow of items piling up at the beginning of the line. Depending on the volume of the stream, immediate handling may be necessary. In the context of our *Streaming Transactions* example in the *Applying our learning* section, we are using these automated conveyor belts (streaming pipelines) to efficiently manage and transform this continuous flow of items (streaming data) into a useful form.

Streaming data has several benefits:

- **Immediate insights**: Streaming data allows you to receive quick insights and make real-time decisions based on the data. Speed is essential for applications where timing is critical, such as financial trading or the real-time monitoring of industrial equipment.
- **Up-to-date analysis**: Processing streaming data in real time allows for more current data analysis. Real-time analysis can help you find patterns and trends as they occur and take proactive steps to address potential data quality issues.
- **Improved efficiency**: Streaming data can help organizations become more efficient by enabling them to respond to events quickly and proactively. Short response times can improve customer satisfaction, reduce downtime, and increase productivity.

Streaming data can provide significant benefits for organizations that need to process large volumes of data quickly and make real-time decisions based on that data. However, there are transformations that require a more complex type of streaming, specifically, **stateful streaming**. Stateful streaming refers to stream processing that consumes a continuous data stream and persists the state of past events. Persisting the state allows for the stream to "know" information from previous transactions. This is particularly helpful when calculating aggregates over a time window since the aggregate would need values from the previous window in the stream. To make this clearer, we provide an example of this in the *Streaming Transactions* project. There is also an informative video linked in the *Further reading* section that explains stateful streaming in detail.

> **Note**
>
> Streaming feature tables should be altered before you begin writing to them. Running an `ALTER TABLE` command causes the stream to quit. However, you can restart the stream if you must alter the table. Be prepared and plan ahead as much as possible!

The DFE client supports intelligent handling and lookups for time-based features, such as our timestamped streaming features. Let's go over this time-saving product feature next.

Employing point-in-time lookups for time series feature tables

Time series feature tables are any table in Unity Catalog with a `TIMESERIES` primary key. These tables are eligible for point-in-time lookups, which is a mechanism for looking up the correct feature values. Before `training_sets`, coming in *Chapter 6*, we often joined tables to connect training rows with their feature values. However, a fine-grained event timestamp is not ideal for joining. This led to rounding the timestamps to minutes, hours, or even days. Depending on the use case, this method may or may not work. For example, joining on `TransactionTimestamp` in *Figure 5.3* is not realistic in a standard join so one might create `TransactionMinute` or `TransactionHour`.

TransactionTimestamp	TransactionMinute	TransactionHour
2023-09-03 19:23:09.765676	2023-09-03 19:23:00	2023-09-03 19:00:00
2023-09-03 19:23:09.765821	2023-09-03 19:23:00	2023-09-03 19:00:00
2023-09-03 19:23:09.765899	2023-09-03 19:23:00	2023-09-03 19:00:00

Figure 5.3 – Example of timestamp rounding for easy joining

Employing point-in-time lookups fixes this problem by handling the time matching for you. For those familiar with *The Price Is Right*, it's the closest feature value without going over. More specifically, it will match with the latest feature value as of the event timestamp without ever providing a feature value calculated after the event. You don't worry about data leakage during training. Without a `Timeseries` primary key, the latest value for a feature is matched.

> **Note**
>
> It's recommended to apply Z-ordering on time series tables for better performance in point-in-time lookups. Z-ordering is covered in *Chapter 3*.

Looking up features is great and often the best choice for your use case. However, in some data science and machine learning tasks, we need feature values calculated quickly on data that we do not have ahead of time. For those projects, we need on-demand features.

Computing on-demand features

Calculating the number of transactions per customer in a brief time window works in a streaming fashion because we only need to use historical data. When we want to use a feature that requires data available only at inference time, we use on-demand features, with unknown values until inference time. In Databricks, you can create on-demand features with Python **user-defined functions** (**UDFs**). These Python UDFs can then be invoked via `training_set` configurations to create training datasets, as you will see in *Chapter 6*.

Let's consider the *Streaming Transactions* project again. We want to add a feature for the amount a product sold at, compared to its historical maximum price, and use this as part of the training data to predict the generated classification label. In this scenario, we don't know the purchase price until the transaction has been received. We'll cover how to build a Python UDF for calculating an on-demand feature for the *Streaming Transactions* project in the *Applying our learning* section.

Additionally, we recommend checking out the *How Databricks AI improves model accuracy with real-time computations* and *Best Practices for Realtime Feature Computation on Databricks* articles for in-depth advice from the on-demand experts at Databricks; see the *Further reading* section for the links.

We have discussed saving features to feature tables in Unity Catalog, the standard "offline" pattern for applications that do not have low-latency requirements. If your business problem requires low latency or fast results, you'll need your data in an online table or implement Databricks Feature Serving. Feature Serving can serve functions as well as precomputed features. For low-latency projects, we recommend Databricks Model Serving because it removes any need for Feature Serving. We don't cover Feature Serving in this book, but if you are going to serve your models externally to Databricks, Feature Serving may be of interest to you.

We've gone through ways to save feature tables, how to build streaming features, implement point-in-time lookups, and how to create on-demand features. It's time to get ready to follow along in your own Databricks workspace as we work through the *Chapter 5* project code.

Applying our learning

Now that we have learned about the feature engineering components of the DI platform, let's put these topics into action and build on the example project datasets with new features that will enhance the data science projects.

Technical requirements

Here are the technical requirements needed to complete the hands-on examples in this chapter:

- The *Streaming Transactions* project requires more compute power than is available in the single node cluster. We created a multi-node cluster to address this. See *Figure 5.4* for the multi-node CPU configuration we used.

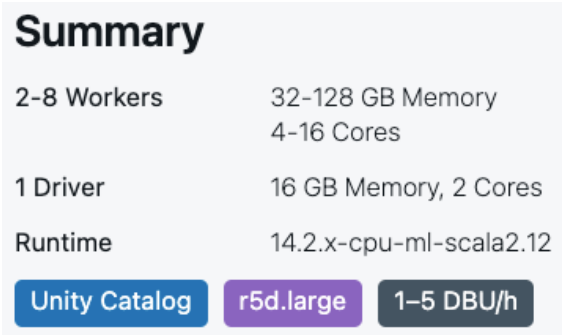


Figure 5.4 – Multi-node CPU cluster configuration (on AWS) used for this book

- We will use managed volumes to store cleaned, featurized data.

Project: Streaming Transactions

If you've been following the code in the previous chapters, at this point, you have the streaming data you need. In this chapter, we will augment that data with some of the feature engineering techniques discussed earlier. First, we will create a streaming feature to count the number of transactions that have arrived in the last two minutes for each customer.

Before we jump in, let's remember where we are and where we are going.

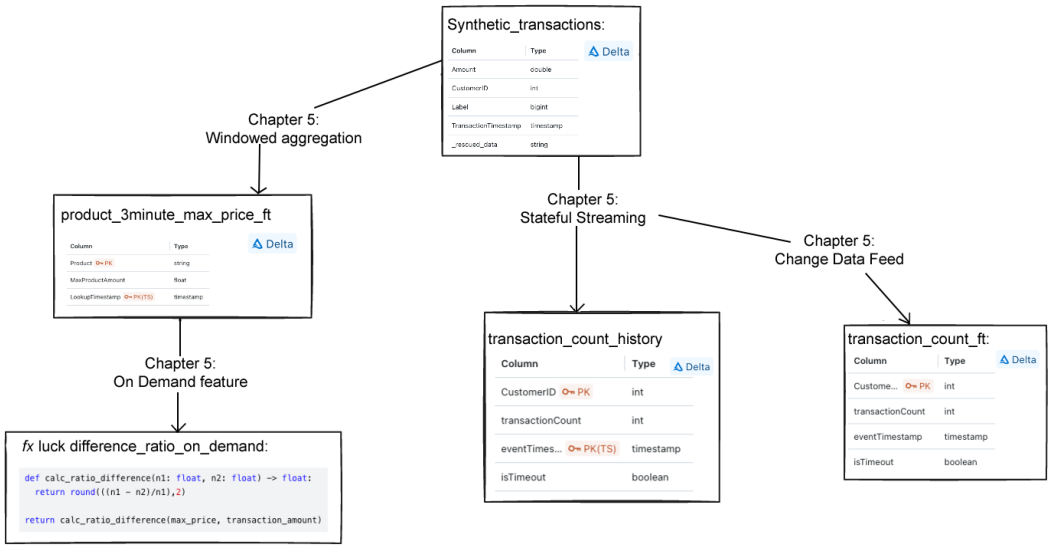


Figure 5.5 – The project pipeline for the Streaming Transactions project

We are building a streaming pipeline to process the incoming transactions into a feature. We will use stateful streaming to count the number of transactions per customer over a two-minute timeframe. Stateful streaming is required because the calculations in the stream need to know how many transactions have already occurred and when a transaction falls outside the two-minute window. This information is known as the state of a customer.

The streaming feature and the on-demand feature are created in two different notebooks. In the code repository, you have the following four notebooks:

- `CH5-01-Generating Records`: This notebook is nearly identical to the data generator used in the previous chapters. The two key differences are that there is now always a product present in the records, and the total (meaning the number of time steps) has been increased to provide a stream for a longer period of time.
- `CH5-02-Auto Loader`: The only change to the `Auto Loader` notebook is the location the data is being written to.
- `CH5-03-FE Using Spark Structured Streaming`: This notebook is explained in detail in the *Building a streaming feature with Spark Structured Streaming* subsection of this project. The code is written in Scala. Stateful streaming is now also available with PySpark. See the *Further reading* section at the end of this chapter for more details.
- `CH5-04-Building Maximum Price Feature Table`: This notebook calculates the maximum price for a product over a time window. The calculated price will be used at inference time by the Python UDF also created in this notebook.

The first two notebooks are nearly identical to their counterparts from the last chapter, so we won't cover them again. We start from the `CH5-03-FE Using Spark Structured Streaming` notebook.

Building a streaming feature with Spark Structured Streaming

We calculate the number of transactions per customer in the last two minutes and call this new feature `transactionCount`, which is a case class in the code.

CustomerID	transactionTimestamp	…	…
1	2023-09-03 19:23:09.765676		
4	2023-09-03 19:23:09.765821		
2	2023-09-03 19:23:09.765899		

Figure 5.6 – The table contains example data corresponding to readStream – namely, inputDf

We need to aggregate the transactions from the incoming stream, the `InputRow` case class, by the `CustomerID` field. *Figure 5.11* shows an example table of the incoming stream. Additionally, we must remove transactions once they have fallen outside the specified window; we will call these "expired transactions." This means we must create and maintain a state for each customer, the `TransactionCountState` case class. Each customer state consists of `CustomerID`, `transactionCount`, and `transactionList` containing the times that the transactions occurred for each transaction accounted for in the transaction count. *Figure 5.12* is a visual representation of the customer state. As a customer transaction arrives, it is added to a list of transactions with a timestamp as part of the customer state.

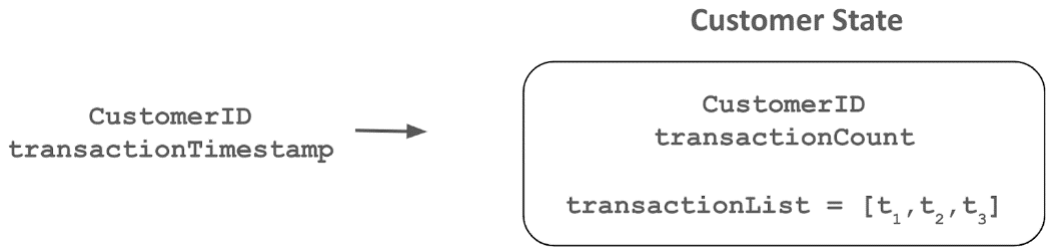


Figure 5.7 – Each transaction updates the customer state

The state is created by applying the stateful streaming logic to the input data. The customer state is then used to write the feature table, `transaction_count_ft`, shown in *Figure 5.8*:

CustomerID	transactionCount	eventTimestamp	isTimeout
5	3	2023-09-03T19:24:14.388	false
2	4	2023-09-03T19:24:16.721	true
3	0	2023-09-03T19:24:16.720	true

Figure 5.8 – This table is the result after applying the stateful streaming transformation

The feature table shown in *Figure 5.8* includes the `CustomerID` reference, `transactionCount` for that customer, `eventTimestamp`, and a Boolean variable, `isTimeout`. The `eventTimestamp` is the time the feature record was written. We call it `eventTimestamp` because a new transaction or a timeout could have triggered the update to the customer state/event. To know which type of event it is, we include `isTimeout`. A timeout occurs when no new transactions for a customer have occurred but the value of `transactionCount` has changed – an indication that the count has decreased.

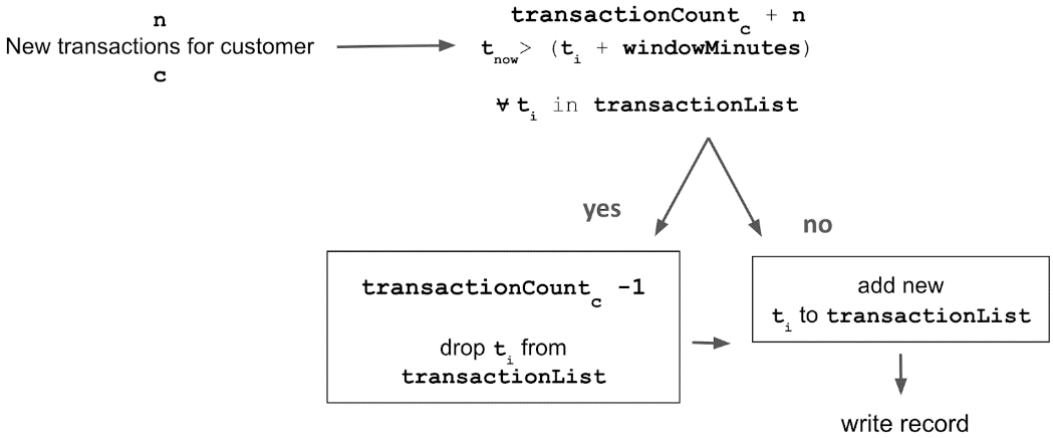


Figure 5.9 – Logic flow for the stateful streaming transformation

Figure 5.9 visually represents the update logic applied to the customer state. The logical path can be broken into steps:

1. For `n` new transactions coming in for customer `c`, `transactionCount` is incremented `n` times.
2. Then, for each `transactionTimestamp`, `ti`, in `transactionList`, we compare the current time with `expirationTimestamp` (`ti+windowMinutes`) to determine whether the transaction accounted for in `transactionCount` has expired:

    I. If any transactions have expired, we decrement `transactionCount` by one for each and drop the corresponding `transactionTimestamp` from `transactionList`.

II. If no transaction has expired or `transactionTimestamp` for the expired transactions has been dropped, then we add the new `transactionTimestamp` to `transactionList` and write out the customer record.

Now that we have outlined the goal of our transformation, let's look at the code. To follow along in your own workspace, please refer to the following notebooks:

- `CH5-01-Generating Records`
- `CH5-02-Auto Loader`
- `CH5-03-FE Using Spark Structured Streaming`
- `CH5-04-Building Maximum Price Feature Table`

Before you begin executing *notebook 3*, please open *notebooks 1* and *2* and click **Run All** for both. These two notebooks relaunch the data streams that we need running in order to run *notebook 3*. However, you do not need all streams running in order to run *notebook 4*.

The code we are jumping into is in the `CH5-03-FE Using Spark Structured Streaming` notebook. We start with the basics – imports and delta configurations – as shown in the following screenshot:

```
import java.time.Instant
import java.util.concurrent.TimeUnit
import scala.collection.mutable.ListBuffer
import org.apache.spark.sql.streaming.{GroupStateTimeout, OutputMode, GroupState}

spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")
```

Figure 5.10 – Setting delta configurations for optimized writes and compaction

These configurations can also be set in the cluster configurations, but we call them out explicitly in *Figure 5.10*. These settings will automatically compact sets of small files into larger files as it writes for optimal read performance. *Figure 5.11* primarily shows the reset commands that enable starting fresh when needed:

```
%python
table_name = "transaction_count_ft"
history_table_name = "transaction_count_history"
if bool(dbutils.widgets.get('Reset')):
  dbutils.fs.rm(f"{volume_file_path}/{table_name}/streaming_outputs/", True)
  dbutils.fs.rm(f"{volume_file_path}/{history_table_name}/streaming_outputs/", True)
  sql(f"DROP TABLE IF EXISTS {table_name}")
  sql(f"DROP TABLE IF EXISTS {history_table_name}")
```

Figure 5.11 – Setting the widget value to True runs the commands in this cell, removing the output data, including the checkpoint, and dropping the table

We define the necessary variables in this next code snippet and create our table. Notice that variables passed from the setup file are in Python and SQL, meaning they are unavailable in Scala. Although all three languages can be used in a notebook, they do not share constants or variable values between them. As a result, we define the variables we need in Scala for access in the Scala notebook. We set the volume location for files, output paths, and the `inputTable` name:

```
// variables passed from the setup file are in python
val table_name = "transaction_count_ft"
val history_table_name = "transaction_count_history"
val volumePath = "/Volumes/ml_in_action/synthetic_transactions/files/"
val outputPath = f"$volumePath/$table_name/streaming_outputs/"
val outputPath2 = f"$volumePath/$history_table_name/streaming_outputs/"
val inputTable = "ml_in_action.synthetic_transactions.raw_transactions"
```

Figure 5.12 – Setting variables, constants, and paths in Scala

Notice that we enable the CDF on our streaming feature table, `transaction_count_ft`, in *Figure 5.13*. We could publish this table to an online store if desired. Additionally, we set the table name we want to write all transactions, `transaction_count_history`:

```
sql(f"""CREATE OR REPLACE TABLE $table_name (CustomerID Int, transactionCount Int,
eventTimestamp Timestamp, isTimeout Boolean)""")
sql(f"""ALTER TABLE $table_name ALTER COLUMN CustomerID SET NOT NULL""")
sql(f"""ALTER TABLE $table_name ADD PRIMARY KEY(CustomerID)""")
sql(f"ALTER TABLE $table_name SET TBLPROPERTIES (delta.enableChangeDataFeed=true)")
```

Figure 5.13 – Creating a table and enabling CDF

Next, the `windowMinutes` constant is the number of minutes we want to aggregate transactions for each customer, whereas `maxWaitMinutes` is exactly what it sounds like. It is the minutes the stream waits for a transaction before writing out the state without new transactions. The value of `maxWaitMinutes` should always be less than the value of `windowMinutes`:

```
// aggregate transactions for windowMinutes
val windowMinutes = 2
// wait at most max_wait_minutes before writing a record
val maxWaitMinutes = 1
```

Figure 5.14 – Setting windowMinutes and maxWaitMinutes for our stream

We will initiate `FeatureEngineeringClient` and set feature table tags so we can easily see which project these tables are associated with:

```
%python
from databricks.feature_engineering import FeatureEngineeringClient
fe = FeatureEngineeringClient()

fe.set_feature_table_tag(name=f"{table_name}", key="Project", value="MLIA")
fe.set_feature_table_tag(name=f"{history_table_name}", key="Project", value="MLIA")
```

Figure 5.15 – Setting feature table tags on our table

Next, we define the case class structures. *Figure 5.16* shows that our case classes define our data structures. This is because Scala is a typed language.

```
case class InputRow(CustomerID: Long,
                    TransactionTimestamp: java.time.Instant)

case class TransactionCountState(latestTimestamp: java.time.Instant,
                                 currentTransactions: List[InputRow])

case class TransactionCount(CustomerID: Long,
                            transactionCount: Integer,
                            eventTimestamp: java.time.Instant,
                            isTimeout: Boolean)
```

Figure 5.16 – Defining class structures to support aggregations for each customer ID

In *Figure 5.17*, we see the `addNewRecords` function. `latestTimestamp` is calculated by comparing the latest timestamp in `transactionCountState` with the latest timestamp from the new records. This is in case we've received data out of order. Finally, we create and return the new `TransactionCountState` object with the newly calculated `latestTimestamp` and combine the two record lists:

```
def addNewRecords(newRecords: List[InputRow],
                  transactionCountState: TransactionCountState):
                  TransactionCountState =
  {
    val recordWithLatestTimestamp =
        newRecords.maxBy(record => record.TransactionTimestamp)
    val latestNewTimestamp = recordWithLatestTimestamp.TransactionTimestamp
    val latestTimestamp =
        if (latestNewTimestamp.toEpochMilli() > transactionCountState.
        latestTimestamp.toEpochMilli()
        ) latestNewTimestamp else transactionCountState.latestTimestamp

    new TransactionCountState(latestTimestamp,
        transactionCountState.currentTransactions ::: newRecords)
  }
```

Figure 5.17 – Defining the addNewRecords function

The next function, *Figure 5.18*, drops the records that are more than `windowMinutes` old from `TransactionCountState` by calculating the state expiration timestamp (the latest timestamp minus the transaction count minutes). Then, it loops through the list of current transactions and keeps any that occur before the expiration timestamp. This is shown in the following screenshot:

```
def dropExpiredRecords(newLatestTimestamp: java.time.Instant,
    currentTransactions: List[InputRow]): TransactionCountState = {
    val newTransactionList = ListBuffer[InputRow]()
    val expirationTimestamp = Instant.ofEpochMilli(newLatestTimestamp.
    toEpochMilli() - TimeUnit.MINUTES.toMillis(windowMinutes))

    if (currentTransactions.size > 0) {
      currentTransactions.foreach { value =>
        if (value.TransactionTimestamp.toEpochMilli() >=
        expirationTimestamp.toEpochMilli())
          newTransactionList.append(value)
      }
    }

    new TransactionCountState(newLatestTimestamp,
                              newTransactionList.toList)
  }
```

Figure 5.18 – Defining a function to drop stale records

The `updateState` function uses our helper functions to – you guessed it – update the customer state. This is the function called `flatMapGroupsWithState`. The `updateState` function receives the customer ID, values, and the current state. `CustomerID` is the key we are grouping on. `values` is an iterator of `InputRow`. In *Figure 5.6*, we see that `InputRow` is a transaction record consisting of a `CustomerID` reference and the time the transaction occurred. The `updateState` function behaves in two ways. Suppose one or more `InputRow` records for a given `CustomerID` are received. In that case, it will add those records to the state, drop any records that are older than `windowMinutes` from the state, and calculate the transaction count. See *Figure 5.13* for the notebook code.

```
if (!state.hasTimedOut) {
  val transactionList = new ListBuffer[InputRow]()
  values.foreach { value =>
    transactionList.append(value)}

  var prevState = state.getOption.getOrElse {
    val firstTransactionTimestamp = transactionList.head.
    TransactionTimestamp
    new TransactionCountState(firstTransactionTimestamp, List
    [InputRow]())}

  val stateWithNewRecords = addNewRecords(transactionList.toList,
    prevState)

  val stateWithRecordsDropped =
    dropExpiredRecords(stateWithNewRecords.latestTimestamp,
    stateWithNewRecords.currentTransactions)

  val output = new TransactionCount(CustomerID,
    stateWithRecordsDropped.currentTransactions.size,
    stateWithRecordsDropped.latestTimestamp, false)

  transactionCounts.append(output)
  state.update(stateWithRecordsDropped)
  state.setTimeoutTimestamp(
    stateWithRecordsDropped.latestTimestamp.toEpochMilli(),
    "30 seconds")
} else {
```

Figure 5.19 – If the state has not timed out, the updateState function receives and processes records

On the other hand, as shown in *Figure 5.20*, if no records are received for a given `CustomerID` within a minute since the last time this function was called, it will drop any records that are older than `windowMinutes` from the state and adjust the count:

```
} else {
  val prevState = state.get
  val newTimestamp = Instant.now
  val stateWithRecordsDropped = dropExpiredRecords(newTimestamp,
    prevState.currentTransactions)
  val output = new TransactionCount(CustomerID,
    stateWithRecordsDropped.currentTransactions.size,
    stateWithRecordsDropped.latestTimestamp, true)

  transactionCounts.append(output)
  state.update(stateWithRecordsDropped)
  state.setTimeoutTimestamp(
    stateWithRecordsDropped.latestTimestamp.toEpochMilli(),
    "30 seconds")
}
```

Figure 5.20 – If the state has timed out, the updateState function receives no records and only updates the state after dropping expired records

Note that the `transactionCounts` list buffer created at the beginning of the `updateState` function is returned as an iterator. In this case, the output will contain one record: the transaction count record for the specific customer.

Next, as we prepare to create a read stream, we define the input schema we need for our read stream; see *Figure 5.21*:

```
import org.apache.spark.sql.types.{StringType,
       StructField, StructType, IntegerType,
       FloatType, TimestampType}

// The schema for the incoming records
val schema = StructType(Array(
        StructField("Source", StringType, true),
        StructField("TransactionTimestamp", StringType, true),
        StructField("CustomerID", IntegerType, true),
        StructField("Amount", FloatType, true),
        StructField("Product", StringType, true),
        StructField("Label", IntegerType, true),
                ))
```

Figure 5.21 – Read stream input schema

We now must create the read and write components of the stream. The read stream reads in the Delta table we created in *Chapter 3*. Recall we created the Bronze layer by streaming in JSON files and writing them to our `inputTable`. The `readStream` and `writeStream` code is long so we will break it up into smaller sections in the following steps:

1. You should recognize this from earlier chapters. The difference is we are using `selectExpr` to isolate `CustomerID` and `TransactionTimestamp`. Additionally, we specifically set the type of output dataframe to the case class that `flatMapGroupsWithState` is expecting:

```
val inputDf =
  spark.readStream
    .format("delta")
    .schema(schema)
    .table(inputTable)
    .selectExpr("CustomerID", "cast(TransactionTimestamp as timestamp) TransactionTimestamp")
    .as[InputRow]
```

Figure 5.22 – InputDF is a read stream reading the table we created in Chapter 3

2. We apply the watermark and `flatMapGroupsWithState` function to `inputDf`. In Spark Streaming, a watermark is a time threshold determining the maximum allowable delay for late events. We're allowing data to be 30 seconds late before it is dropped (*Watermarking in Spark Structured Streaming*, by Thomas Treml: `https://towardsdatascience.com/watermarking-in-spark-structured-streaming-9e164f373e9`). The `flatMapGroupsWithState` function is an arbitrary stateful streaming aggregation operator. It applies our `updateState` function to each micro-batch of transactions while maintaining the state for each customer ID:

```
val flatMapGroupsWithStateResultDf =
  inputDf
    .withWatermark("TransactionTimestamp", "30 seconds")
    .groupByKey(_.CustomerID)
    .flatMapGroupsWithState(OutputMode.Append,
        GroupStateTimeout.EventTimeTimeout)(updateState)
```

Figure 5.23 – Applying the watermark and flatMapGroupsWithState function to inputDf

3. We define the `updateCounts` function for `foreachBatch` to update the counts in the write stream. It performs upserts of the new transaction counts into the `transaction_count_ft` table; this is the CDC component:

```
def updateCounts(newCountsDs: Dataset[TransactionCount], epoch_id: Long): Unit = {
  val newCountsDf = newCountsDs.toDF
  val aggregationTable = DeltaTable.forName(table_name)
  aggregationTable.alias("t")
    .merge(newCountsDf.alias("m"), "m.CustomerID = t.CustomerID")
    .whenMatched().updateAll()
    .whenNotMatched().insertAll()
    .execute()
}
```

Figure 5.24 – The updateCounts function upserts the new transaction counts into the transaction_count_ft table

4. The last piece of the stream is the write, or, more clearly, the update. The write stream applies the `updateCounts` function. Delta tables do not support streaming updates directly, so we need to use a `foreachBatch` function. The `foreachBatch` function is like a streaming `for` loop applying the function to each micro-batch of data in the stream. This write stream is similar to `flatMapGroupsWithState` without grouping the data or maintaining the state. We are simply updating the CDC result table. Notice the checkpointing is handled for us, and our stream is triggered every 10 seconds, meaning every 10 seconds is a new micro-batch of the data. The query name is optional. It shows up in the SparkUI.

```
flatMapGroupsWithStateResultDf.writeStream
  .foreachBatch(updateCounts _)
  .option("checkpointLocation", f"$outputPath/checkpoint")
  .trigger(Trigger.ProcessingTime("10 seconds"))
  .queryName("flatMapGroups")
  .start()
```

Figure 5.25 – The write stream applies the updateCounts function using foreachBatch

5. In addition to the writing out to the CDC table, we also want a historical record of the transaction values:

```
flatMapGroupsWithStateResultDf.writeStream
  .option("checkpointLocation", f"$outputPath2/checkpoint")
  .trigger(Trigger.ProcessingTime("10 seconds"))
  .queryName("flatMapGroupsHistoryTable")
  .table(history_table_name)
```

Figure 5.26 – The write stream records all of the transactionCounts values to the Delta table

6. While the stream runs, we can observe the output table, `transaction_count_ft`. While your streams are running, refresh the table view so you can follow along with the output changes as the input data changes:

```
%sql
select * from lakehouse_in_action.synthetic_streaming_features order by eventTimestamp desc;
```

▸ (2) Spark Jobs

Table ⌄ Visualization 1 +

#	CustomerID	transactionCount	eventTimestamp	isTimeout
2	1	49	2023-10-08 18:54:47.686	false
3	8	58	2023-10-08 18:54:47.686	false
4	9	64	2023-10-08 18:54:47.686	false
5	4	49	2023-10-08 18:54:47.686	false
6	7	63	2023-10-08 18:54:47.686	false
7	11	50	2023-10-08 18:54:47.686	false

Figure 5.27 – A snapshot of the transaction_count_ft table

7. Another thing you can observe while the stream runs is the stream statistics. This view is found by expanding the results section.

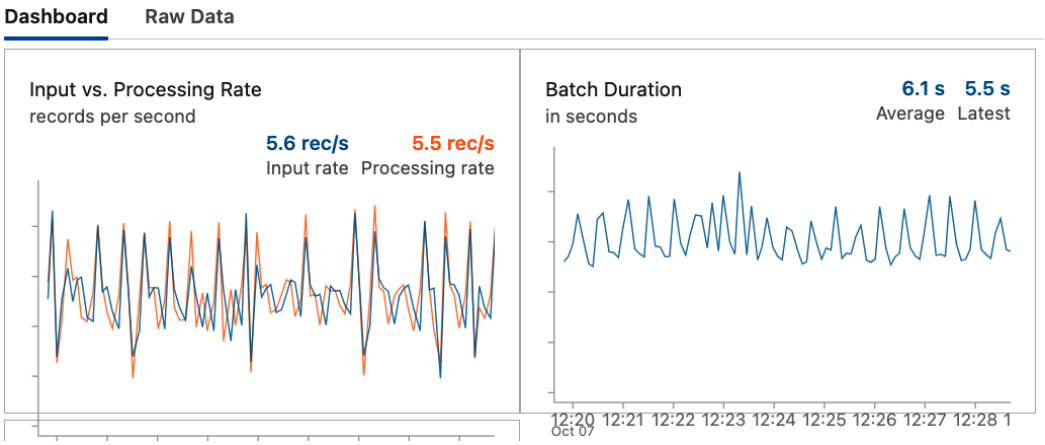


Figure 5.28 – Stream real-time statistics for writeStream

Navigate to the Catalog view to see that the two new tables have appeared. This wraps up our streaming feature. Next, we will build a Python UDF.

Building an on-demand feature with a Python UDF

Let's also create a second feature table using on-demand feature engineering. Focus on the `CH5-04-Building Maximum Price Feature Table` notebook. We briefly introduced a scenario in an early section of this chapter requiring us to calculate a transaction's difference from the maximum price on the fly. This could be useful for modeling. To get the difference as an on-demand feature, we do the following:

1. Calculate the maximum price per product over a rolling window. We begin with creating `time_window`. We want the maximum price in the last three minutes:

```
fe = FeatureEngineeringClient()

time_window = F.window(
    F.col("TransactionTimestamp"),
    windowDuration="3 minutes",
    slideDuration="1 minute",
).alias("time_window")
```

Figure 5.29 – Creating a time window to calculate the rolling maximum prices by product

2. Now that we have a window, we can calculate and save that maximum price to a DataFrame. Usually, the value of a maximum price doesn't drastically change, so hourly is an appropriate timeframe. We add a new time column called `LookupTimestamp` for joining on.

```
max_price_df = (
  raw_transactions_df
    .groupBy(F.col("Product"),time_window)
    .agg(F.max(F.col("Amount")).cast("float").alias("MaxProductAmount"))
    .withColumn("LookupTimestamp",
                F.date_trunc('minute',
                             F.col("time_window.end") + F.expr('INTERVAL 1 MINUTE')))
    .drop("time_window")
)
```

Figure 5.30 – Creating a DataFrame feature table of product maximum prices

3. Next, let's create a new feature table from our DataFrame. We will assume that the maximum price for a specific product does not vary enough to calculate this value more than hourly, so we can set this table to update on a set schedule. In the GitHub code, we'll instantiate `FeatureEngineeringClient`. In *Figure 5.31*, we use it to write the new feature table as a Delta table in Unity Catalog:

```
fe.create_table(
  df=max_price_df,
  name='product_3minute_max_price_ft',
  primary_keys=['Product','LookupTimestamp'],
  timeseries_columns='LookupTimestamp',
  schema=max_price_df.schema,
  description="Maximum price per product over the last 3 minutes for Synthetic
  Transactions. Join on TransactionTimestamp to get the max product price from last
  minute's 3 minute rolling max"
)
```

Figure 5.31 – Writing our table to a Delta table in Unity Catalog

4. Next, we need a Python UDF to calculate the discount or the difference between the transaction amount and the product's maximum price. We'll name it `product_difference_ratio_on_demand_feature`. We can use the same notebook to build and save this simple function under the same catalog and schema as our tables in Unity Catalog.

```
%sql
CREATE FUNCTION IF NOT EXISTS product_difference_ratio_on_demand_feature(max_price FLOAT, transaction_amount FLOAT)
RETURNS float
LANGUAGE PYTHON
COMMENT 'Calculate the difference ratio for a product at time of transaction (maximum price - transaction amount)/maximum price.'
AS $$
def calc_ratio_difference(n1: float, n2: float) -> float:
  return round(((n1 - n2)/n1),2)

return calc_ratio_difference(max_price, transaction_amount)
$$
```

Figure 5.32 – Building an on-demand function to calculate product discounts

5. Once we run this code, we can navigate to Unity Catalog and see `product_difference_ratio_on_demand_feature` listed. It's ready to use in a training set! We'll refer to this function in *Chapter 6*.

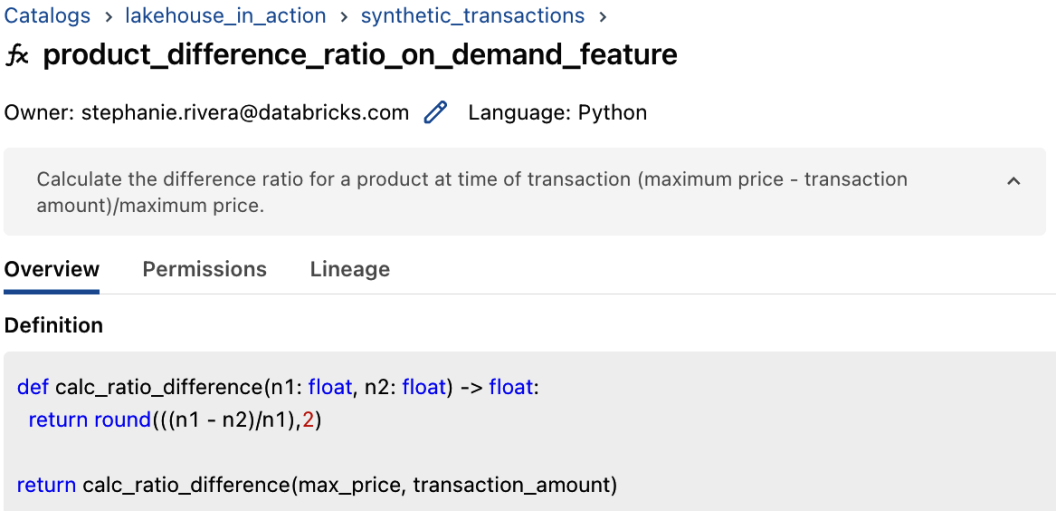


Figure 5.33 – Viewing the on-demand feature we created in Unity Catalog

And with that, we have engineered features for our *Streaming Transactions* dataset. We enriched the original data with a streaming feature called `transactionCount` and an on-demand feature function that we will use to build the training dataset in the next chapter.

In the next section, we will aggregate store sales data and save it to a feature table.

Project: Favorita Store Sales – Time Series Forecasting

In *Chapter 4*, we used AutoML to explore the *Favorita Sales* dataset and create a baseline model predicting sales. To follow along in your own workspace, please refer to the following notebook: `CH5-01-Building Favorita Feature Tables`.

To create a Databricks feature table, we can use either Python (via `FeatureEngineeringClient`) or SQL. This chapter primarily uses Python, but we start by creating a `stores_ft` feature table using SQL:

```
CREATE
OR REPLACE TABLE stores_ft AS(
  SELECT
    store_nbr,
    city,
    state,
    `type` as store_type,
    `cluster` as store_cluster
  FROM
    favorita_stores
  );
ALTER TABLE stores_ft ALTER COLUMN store_nbr SET NOT NULL;
ALTER TABLE stores_ft ADD PRIMARY KEY(store_nbr);
COMMENT ON TABLE stores_ft IS 'Favorita Store features include geography, store
cluster, and store_type.';
```

Figure 5.34 – Creating the stores_ft feature table from the favorita_stores table using SQL

Executing the code in *Figure 5.34* creates a feature table called `stores_ft` that we will use as a central repository for important store details. `store_nbr` is set to `NOT NULL` because it is the primary key. The table does not contain a date column, so this feature table is not a time series feature table. If it did, we could include an additional `TIMESERIES` primary key. Note that when adding the primary key to the table, you can name the constraint for the primary key with a unique name, as shown in the documentation. We prefer to let the DFE client name the constraint automatically. You can use `DESCRIBE TABLE EXTENDED` to see the name of your primary key constraint.

The remaining two feature tables are time series tables. After some transformations, we will create the feature tables using Python and the DFE client. Let's focus on the holidays first.

date	type	locale	locale_name	description	transferred
2012-03-02 00:00:00.000	Holiday	Local	Manta	Fundacion de Manta	false
2012-04-01 00:00:00.000	Holiday	Regional	Cotopaxi	Provincializacion de Cotopaxi	false
2012-04-12 00:00:00.000	Holiday	Local	Cuenca	Fundacion de Cuenca	false
2012-04-14 00:00:00.000	Holiday	Local	Libertad	Cantonizacion de Libertad	false
2012-04-21 00:00:00.000	Holiday	Local	Riobamba	Cantonizacion de Riobamba	false

Figure 5.35 – The holiday events table

The features we create are their respective store's local, regional, and national holidays. We can look at *Figure 5.35* to recall what the holiday data looked like.

One thing to note about our data is that there are days when multiple holidays occur for the same store. We process the three types of holidays very similarly. The transformations for national holidays are slightly different, given that there is no need to match on locale. All holiday transformations are present in the GitHub code. However, we do not cover each in detail in the book.

The following steps take you through the transformations for the local holiday type we use for the holiday feature table:

1. We begin the ETL for the local holidays by singling out the local locale and renaming the type to `holiday_type`:

```
sql("""
  CREATE OR REPLACE TABLE local_holidays_bronze AS (
    SELECT
      `date`,
      `type` as holiday_type,
      locale_name
    FROM
      holiday_events
    WHERE
      locale == "Local"
    GROUP BY
      ALL
    ORDER BY
      `date`)
""")
```

Figure 5.36 – Isolating the local holidays

Using the bronze table created in *Step 1*, we construct a silver table consisting of the date, store number, and local holiday type. We must account for the issue of multiple local holidays happening on the same day for the same stores. We do so by grouping with a `MIN` function to select a holiday type. The local holiday type is changed to `Multiple` using a case statement and `num_holidays` accounts for these instances:

```
sql(
"""
CREATE OR REPLACE TABLE local_holidays_silver AS
SELECT
  `date`, store_nbr,
  CASE
    WHEN num_holidays >= 2 THEN "Multiple"
    ELSE local_holiday_type
  END as local_holiday_type
FROM
  (
    SELECT
      h.`date`, s.store_nbr,
      MIN(h.holiday_type) as local_holiday_type,
      count(1) as num_holidays
    FROM
      favorita_stores s
      INNER JOIN local_holidays_bronze h ON (
        s.city == h.locale_name
        OR s.state == h.locale_name
      )
    GROUP BY
      ALL
    ORDER BY
      h.`date`
  )
)
""")
```

Figure 5.37 – SQL checking for multiple holiday instances and identifying them with a new holiday type for the local holidays

The resulting silver table is shown in *Figure 5.38*:

date	store_nbr	local_holiday_type
2012-03-02 00:00:00.000	52	Holiday
2012-03-02 00:00:00.000	53	Holiday
2012-04-12 00:00:00.000	37	Holiday
2012-04-12 00:00:00.000	39	Holiday
2012-04-12 00:00:00.000	42	Holiday

Figure 5.38 – The first five rows of the local_holidays_silver table

2. After completing the same process in the previous steps for regional and national holidays, we combine the silver tables into a single DataFrame. We use full joins to include all holidays. To avoid `null` dates and store numbers, we chain two `ifnull()` functions:

```
df = sql("""
    SELECT
      ifnull(n.`date`,ifnull(r.`date`, l.`date`)) as `date`,
      ifnull(n.store_nbr,ifnull(r.store_nbr, l.store_nbr)) as store_nbr,
      n.national_holiday_type,
      r.regional_holiday_type,
      l.local_holiday_type
    FROM
      national_holidays_silver n
    FULL JOIN regional_holidays_silver r ON n.`date`=r.`date` AND n.store_nbr = r.store_nbr
    FULL JOIN local_holidays_silver l ON n.`date`=l.`date` AND n.store_nbr = l.store_nbr
    ORDER BY
      `date`
    """)
```

Figure 5.39 – Combining all three silver tables into a single DataFrame

3. Now that we have prepared our DataFrame with the features we want to save, we use the DFE `create_table` method to save the DataFrame as a feature table. We specify `primary_keys`. Be sure to note that we are *not* including a time series column. This is because we want the feature table lookup only to match exact dates. Holidays would not function well with the point-in-time logic. The primary keys will also be our lookup keys when we create the training set in the next chapter. A best practice is to include a thoughtful description as well.

```
fe.create_table(
    name = f"store_holidays_ft",
    primary_keys=["date", "store_nbr"],
    df=df,
    description="Holidays in Ecuador by date and store number. Table includes
    holiday types for national, regional, and local. Days where a store has more
    than one holiday is indicated by holiday type being 'Multiple'. Nulls indicate
    non-holiday days.",
)
```

Figure 5.40 – Creating the store holidays feature table using the DFE client

4. Notice that in our feature table, *Figure 5.41*, there are `null` values when there is not a holiday of a specific holiday of such type. When we create the training set, the feature lookup functionality will have `null` for the values of dates not in the table. We do not need to create `null` rows for each date that is not a holiday, thus saving us the preprocessing time.

date	store_nbr	national_holiday_type	regional_holiday_type	local_holiday_type
2012-03-02 00:00:00.000	52	null	null	Holiday
2012-03-02 00:00:00.000	53	null	null	Holiday
2012-04-01 00:00:00.000	13	null	Holiday	null
2012-04-01 00:00:00.000	12	null	Holiday	null
2012-04-12 00:00:00.000	37	null	null	Holiday

Figure 5.41 – The store_holidays_ft feature table

5. In addition to the holiday and store data, we were provided with oil prices data. We can use this data as a proxy for the economy. Let's create one more feature table using the `oil_price_silver` table. Unlike `store_holidays_ft`, having the previous value of a stock price in the place of `null` would be helpful. This is a fitting example of when to use the point-in-time lookup functionality. To do so, a second primary key is required. Therefore, we include the date as the primary key but not as `timeseries_column`, as shown in *Figure 5.42*:

```
df = sql(
    """
    SELECT
      timestamp(`date`) as price_date,
      timestamp(date(`date`) -10) as date,
      dcoilwtico as lag10_oil_price
    FROM
      oil_prices_silver
  """
)

fe.create_table(
    name=f"oil_10d_lag_ft",
    primary_keys=["date"],
    df=df,
    description="The lag10_oil_price is the price of oil 10 days after
    date. The price date is the actual date of the oil price. The oil
    prices were imputed to replace nulls with the previous day's price.
    The stock market is not open on weekends and holidays.",
)
```

Figure 5.42 – Creating a feature table from the oil prices

We have multiple feature tables that we can use with the specific training data provided on Kaggle or any other related training data. For example, `oil_10d_lag_ft` can be used as a proxy for the economy for any dataset based in Ecuador.

For future modeling, saving the features in feature tables using the DFE client will be helpful. Doing so makes it seamless to look up the features with the model at inference time. In the next chapter, using the DFE client, we will combine our feature tables to create a training set for our model.

Summary

As we conclude *Chapter 5*, we have successfully navigated the multifaceted realm of feature engineering on Databricks. We have learned how to organize our features into feature tables, in both SQL and Python, by ensuring there is a non-nullable primary key. Unity Catalog provides lineage and discoverability, which makes features reusable. Continuing with the streaming project, we also highlighted creating a streaming feature using stateful streaming. We touched on the latest feature engineering products from Databricks, such as point-in-time lookups, on-demand feature functions, and publishing tables to the Databricks Online Store. These product features will reduce time to production and simplify production pipelines.

You are now ready to tackle feature engineering for a variety of scenarios! Next up, we take what we've learned to build training sets and machine learning models in *Chapter 6*.

Questions

The following questions solidify key points to remember and tie the content back to your experience:

1. What element of the Delta format helps with reproducibility?
2. How would you create a training set using the Feature Engineering API?
3. What distinguishes a feature table from any other table in Unity Catalog?
4. In the *Streaming Transactions dataset* section of *Applying our learning*, we created a stream with a transformation. What could be the business drivers for creating this pipeline? We did the *how*; what is the *why*?

Answers

After putting thought into the questions, compare your answers to ours:

1. Delta's ability to time travel helps with reproducibility. Delta has versioning that allows us a point-in-time lookup to see what data our model was trained on. For long-term versioning, deep clones or snapshots are appropriate.
2. Create `FeatureLookups` for each feature table you wish to include. Then, use `create_training_set`.
3. A feature table has a unique primary key, which indicates the object or entity the features describe.
4. The possibilities are vast. An example is behavior modeling or customer segmentation to support flagging fraud.

Further reading

In this chapter, we identified specific technologies, technical features, and options. Please take a look at these resources to get deeper into the areas that interest you most:

- *How Databricks AI improves model accuracy with real-time computations*: `https://www.databricks.com/blog/how-lakehouse-ai-improves-model-accuracy-real-time-computations`
- *Use time series feature tables with point-in-time support*: `https://docs.databricks.com/en/machine-learning/feature-store/time-series.html`
- *Python Arbitrary Stateful Processing in Structured Streaming* – Databricks blog: `https://www.databricks.com/blog/2022/10/18/python-arbitrary-stateful-processing-structured-streaming.html`
- *Databricks Volumes*: `https://docs.databricks.com/en/sql/language-manual/sql-ref-volumes.html`
- *Experimenting with Databricks Volumes*: `https://medium.com/@tsiciliani/experimenting-with-databricks-volumes-5666cecb166`
- *Optimize stateful Structured Streaming queries*: `https://docs.databricks.com/en/structured-streaming/stateful-streaming.html`
- Databricks blog – *Introducing Delta Time Travel for Large Scale Data Lakes*: `https://www.databricks.com/blog/2019/02/04/introducing-delta-time-travel-for-large-scale-data-lakes.html`
- YouTube video – *Arbitrary Stateful Aggregations in Structured Streaming in Apache Spark by Burak Yavuz*: `https://youtu.be/JAb4FIheP28?si=BjoeKkxP_OUxT7-K`
- Databricks documentation – *Compute features on demand using Python user-defined functions*: `https://docs.databricks.com/en/machine-learning/feature-store/on-demand-features.html`
- *Delta Lake change data feed (CDF)*: `https://docs.databricks.com/en/delta/delta-change-data-feed.html`

6

Tools for Model Training and Experimenting

In this chapter, we will focus on creating training datasets and building baseline models. We start by building training sets from a training DataFrame and feature tables. You will learn how to combine feature table data with your training data without using traditional joins. We will also return to Databricks **AutoML** and explore how to use it to establish a baseline model quickly. We then cover how to experiment with different features, **hyperparameters**, and models when searching for predictive signals in your training data. Manually tracking configurations and their corresponding evaluation metrics is time-consuming. We introduce a component of MLflow called **MLflow Tracking**, which significantly improves tracking each permutation of parameters and the corresponding outputs.

We will highlight tools to integrate external data and models into your own projects and workflows, covering the Databricks Marketplace and the new AI Playground for readers interested in **Generative AI** (**GenAI**). In the *Applying our learning* section, we showcase how to put all the tools we have discussed to use in creating training datasets, building models, and enhancing projects across a variety of task types.

Here is what you will learn as part of this chapter:

- Building a training set from a feature table
- Baselining with AutoML
- Tracking experiments with MLflow
- Integrating external data and models
- Applying our learning

Building a training set with feature tables

Recall from *Chapter 5* that the **Databricks Feature Engineering** (**DFE**) client enables us to build feature tables that serve as central repositories for features, or variables, that can be used in downstream projects. A DFE `training_set` is the combination of your training data and your features via a `FeatureLookup` object. Training sets enable you to create new training datasets with the same specifications quickly. This can reduce duplicative work by making it easy to share feature tables. For example, many people in an organization may need the same customer features (such as age, country of residence, or number of transactions for the last year) for their various projects. However, without an agreed-upon repository for these features, each person could be using different source data and calculations to build them, resulting in multiple copies of inconsistent features being used for each project. Using training sets and feature tables helps avoid these inconsistencies and prevents the inefficient re-creation of the same feature. A feature table can be created and used by everyone in an organization.

In *Chapter 5*, we looked at how to use the DFE API to build and store features. The Streaming Transactions and Favorita Store Sales projects now have engineered features that can augment the data used to train models, but how do we easily bring together these new features to create a training dataset? We use the DFE feature lookup functionality. The DFE API function `create_training_set()` has a parameter called `feature_lookups`, where you can specify a list of feature tables to include in your training set. This list will reference the `FeatureLookup` objects. For each `FeatureLookup`, you must specify the name of the feature table you want to bring into your training dataset, and the `lookup_key` (one or more) used to match the right feature values to the corresponding records in your training set. You can think of the `lookup_key` as a join key. Note that you can specify a subset of features from the feature table to include in your training set, rather than including every column, by including the feature names in the `FeatureLookup's feature_names` argument.

We also briefly mentioned point-in-time lookups in *Chapter 5*, or features that must be joined with a timestamp. You can use point-in-time lookups by storing time-dependent features in a time series feature table and including a `timestamp_lookup_key` in the `FeatureLookup` object. *Figure 6.1* is an example of creating a training set spec using a point-in-time look-up.

```
model_feature_lookups = [
    FeatureLookup(
        table_name= table_name,
        lookup_key="unique_id",
        timestamp_lookup_key="ts"
    )
]

training_set = fs.create_training_set(
    df=raw_data,
    feature_lookups=model_feature_lookups,
    label=label_name
)

training_df = training_set.load_df()
training_df.write.mode("overwrite").saveAsTable("training_set")
```

Figure 6.1 – Creating a training set using feature lookups

With a training set ready to go, it's (finally) time to model! We begin with learning how to build a baseline model using AutoML.

Baselining with AutoML

A baseline model is a simple model used as a starting point for experimentation. Data scientists often use a baseline model's metrics to measure the performance of more complex models. Baseline models are typically simple or common algorithms, such as a majority class classifier or a random forest.

Baseline models are valuable for several reasons:

- They can help you understand the predictive strength of your dataset.
- They can help you identify the most critical features of the machine learning task. If a baseline model performs well, it may indicate your dataset has predictive power and contains salient features.
- They can help you avoid overfitting. Overfitting occurs when a model learns the training data too well and cannot generalize to new data, and it is a common problem with more complex models. You can determine whether the more complex model is overfitting by comparing its performance to the baseline model. If the complex model performs better than the baseline on training data but worse on unseen test data, you know the complex model is overfitting and should introduce some regularization.

There are many options to build a baseline model. Our personal favorite way to begin the experimentation phase of a machine learning task is to use AutoML in the Databricks Platform. AutoML experiments are easy to set up and execute, automatically provide a ton of information on a model's performance, and autogenerate data products like the data exploration notebook. The complete set of code and modeling artifacts is the "glass-box" concept in action, enabling you to recreate, alter, and fully own the results. We used AutoML in *Chapter 4* to explore the Favorita Sales data via the data profile provided in the data exploration notebook. You may have noticed the model run information when generating the exploration notebook, which is what we will focus on now.

AutoML rapidly explores many model-algorithm/hyperparameter permutations to find the best baseline model for your data, along with tracking evaluation metrics and actual code for each run. Once you have created a baseline model, you can evaluate future models against its performance metrics, such as accuracy, precision, recall, confusion matrices, ROC curves, and AUC. As great as it is, we also caution that AutoML is not a magic bullet. It removes some of the overhead of coding multiple algorithms for experimentation, but you are still in charge of where to go next. Luckily, AutoML automatically tracks these model artifacts in Databricks.

Tracking the many features, models, hyperparameters, and evaluation metrics is a real headache, so using MLflow to track everything natively is a lifesaver. In the *Applying our learning* section of this chapter, we use AutoML to build traditional regression baseline models for forecasting. Next, let's explore MLflow Tracking, so we know how to apply it later in *Applying our learning*.

Tracking experiments with MLflow

MLflow is an open-source platform developed by Databricks for managing data science projects. MLflow supports data science practitioners through the entire machine learning life cycle, from the experimentation phase to packaging code to model deployment in production. Because the platform is open-source, it benefits from a robust open-source development community and continues to expand to support new features and models, including in the generative AI space. Within the DI Platform, we use a managed version of MLflow that natively integrates with the rest of the Databricks platform. We will cover the deployment-related components in *Chapter 7*. For now, let's focus on tracking.

Before we had MLflow Tracking, managing machine learning experiments required much work outside of the actual experimentation process. Tracking handles the overhead of capturing the configurations of environment, features, algorithms, and hyperparameters, plus their corresponding evaluation metrics during testing. Whenever you execute an experiment with MLflow in a notebook, every execution of that experiment is tracked with MLflow Tracking. Consequently, you can use the MLflow UI to compare experiments. We discussed this in the "Modeling in Databricks" section of *Chapter 2*. For convenience, Databricks notebooks have integrations that make it easy to manage and compare experiment results programmatically via MLflow's lightweight APIs or the Databricks notebook UI. For example, *Figure 6.2* shows how to view experiment runs while still in a notebook, by clicking the small Erlenmeyer Flask icon on the screen's far right.

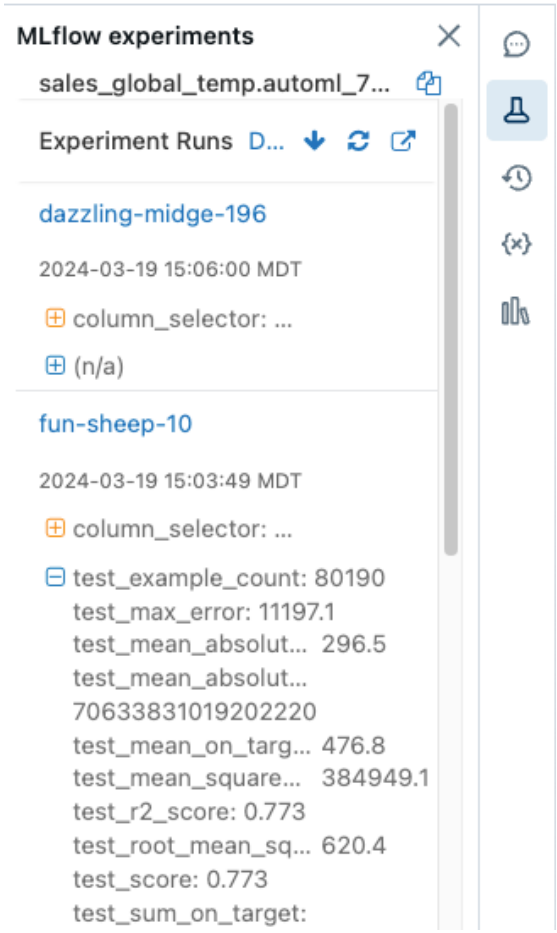


Figure 6.2 – The in-notebook user interface for viewing the experiment runs

In MLflow Tracking lingo, the execution of data science modeling code is called a "run." Each run can record the following:

- **Parameters**: Key-value pairs of input parameters, such as the features used for a given run or the number of trees in a random forest.
- **Metrics**: Evaluation metrics such as RMSE or Area Under the ROC Curve.
- **Artifacts**: Arbitrary output files in any format. This can include images, pickled models, and data files.
- **Source**: The code that originally ran the experiment and a reference to the exact version of the data used for training.

When training models in a notebook, initiating an MLflow run with `mlflow.start_run()` is the recommended practice. This function begins a new experiment run. Any subsequent training information, such as parameters, metrics, and models, is automatically tracked and logged by MLflow Tracking until the run ends. In *Applying our learning* for the Streaming Transactions project, we use autolog as well as add additional parameters to be tracked. To customize the autologging configuration, call `mlflow.autolog()` before your training code. Please note that although many common libraries have autolog support (such as Scikit-learn, XGBoost, and Keras), not all do. Check the documentation linked in *Further Reading* for the latest list.

ML Artifacts

Artifacts in the context of experiment tracking are any physical files, images, or models generated while training a machine learning model. These are stored and logged for future reference and analysis.

Grouping these runs, or executions of modeling code, into experiments when working on a particular ML task makes it easier to compare evaluation metrics and other details. In the next section, we invite you to explore options to leverage external data and models.

Integrating external data and models

The AI and ML world moves extremely fast. You will likely come across projects that benefit from innovations and data outside your DI Platform. For example, it's common to augment corporate datasets with external data. Similarly, it is also common to augment an ML solution with a pre-trained model. Your platform should never be a blocker to open-source options, which is why Databricks offers several paths to integrate external data resources into your projects. To demonstrate, we have included examples of three different methods to combine external data and models with your own data artifacts: downloading data from the **Databricks Marketplace**, accessing OpenAI's ChatGPT model via API, and exploring the **LLM Playground**. Please note that the following examples are outside of the *Applying our learning* project work, so while we encourage you to follow along, the Marketplace and LLM Playground work will not impact the example projects themselves.

Integrating data via the Marketplace

Databricks Marketplace is an open market for data, analytics, and AI artifacts. Organizations can provide and consume available artifacts to augment their internal projects. Companies can leverage this built-in ability to share, sell, or purchase data, notebooks, and models. Imagine you have a lot of data on shipping logistics, and you want to improve the accuracy of your shipping time predictions. You have a working model that does a decent job, and you have exhausted your internal data resources. However, you find a weather data set while exploring Databricks Marketplace, which might provide some interesting benefits! Let's walk through the process of getting our hands on that data.

1. Open the **Marketplace** inside of Databricks (accessible via the left-hand navigation bar). The landing page looks something like *Figure 6.3*.

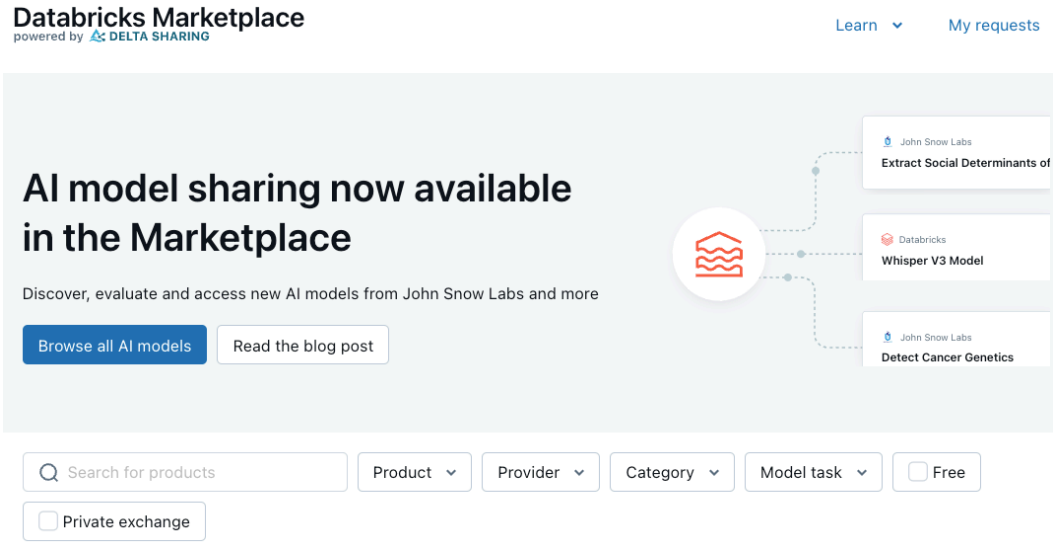


Figure 6.3 – The Databricks Marketplace landing page

2. Navigate to *Historical Weather Data - U.S. Postal Codes Sample* by searching for Weather Data in the Search for products box. When you reach the search page select the **Free** filter option.
3. Click the **Get instant access** button in the upper-right hand corner of the page to install the dataset in Unity Catalog and make it available for your project.

As you explore the Marketplace further, notice that there are more assets available beyond just data. For example, Mistral models, CodeLlama models, and many AI products from John Snow Labs models are all easily accessible through the Marketplace. We encourage you to explore further. Hopefully, you find something that enhances your projects!

Databricks LLM Playground

Integrating external models can also be beneficial to data science projects, and this is especially the case for **Large Language Models** (**LLMs**), which may be expensive and resource-intensive to build on your own. However, the LLM landscape is vast and growing, so how do you know which model would be most useful for a given task? Each type of LLM has nuances and strengths, and teams need to be able to test the model outputs to evaluate them for appropriateness for a business problem. This may sound like the start of a complex evaluation process (and indeed it can be!), but fortunately, the LLM Playground in Databricks makes things easier. This feature allows you to interact with supported large language models in a chat-like environment, and is designed for quick testing, prompting, and comparison of various LLMs. The LLM Playground provides a few features to help you select the best model, including:

- **Model evaluation**: Compare foundational or custom model responses side by side to better understand the differences in results.
- **Streaming responses**: View LLM outputs as they are generated in a stream.
- **Parameter tuning**: Easily modify temperature and top_p generation parameters to see how they impact model outputs.
- **Prompt engineering**: Store and iterate on system prompts to identify which prompt generates the best responses to your specific task.

The LLM Playground provides a user interface to test and compare different LLMs. and decide to either use a foundational model or select one to fine-tune for a specific use case. In *Figure 6.4* below, we walk through the Playground UI by testing a prompt with the *Mixtral 8x7B Instruct* model.

Once you have selected a model to test, you can also choose a sample instruction, or system prompt, based on common tasks: Sentiment analysis, unstructured text parsing, summarization, and document Q&A. These pre-built system prompts can be a useful starting point when beginning a specific task.

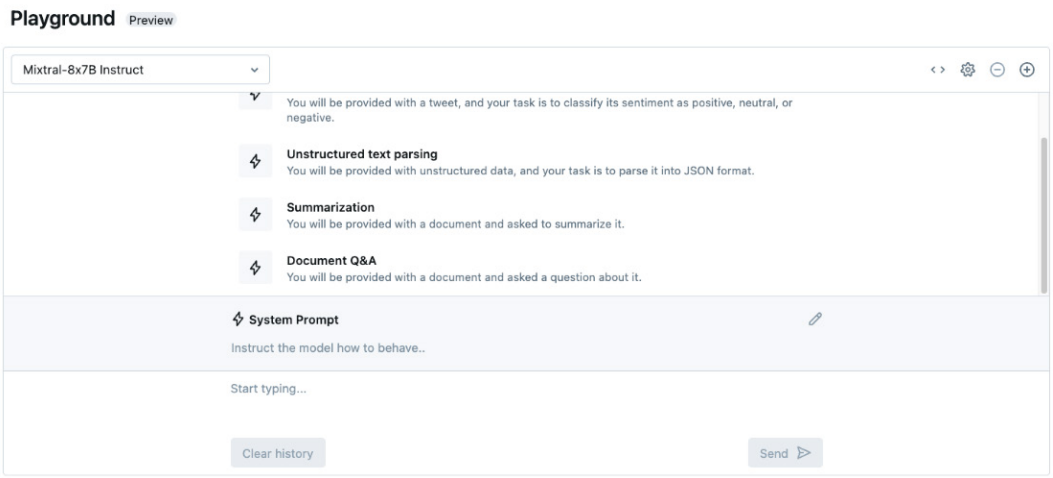


Figure 6.4 – Accessing the LLM Playground

By selecting the + sign at the top of the chat window, you can open additional chat windows (as shown in *Figure 6.5*). This allows you to compare multiple models at the same time with the same prompt or compare the same model across different prompts. This is a straightforward way to gain quick insights into the models' performance.

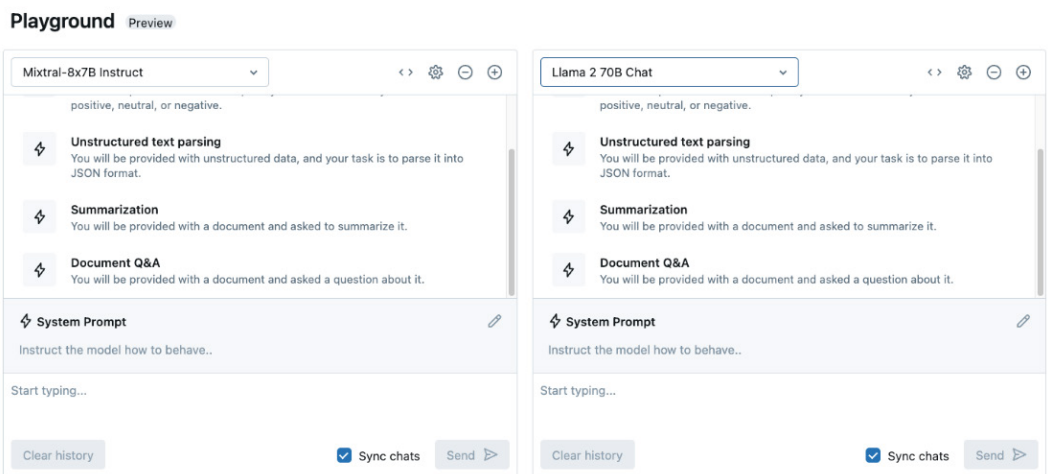


Figure 6.5 – Comparing two language models in the LLM Playground

You can also adjust parameters for the models to improve performance and understand the parameter impact on each model's output. Temperature, Top P, Top K, and maximum number of tokens per response are adjustable via the Playground UI (*Figure 6.6*), and you can also add custom parameters.

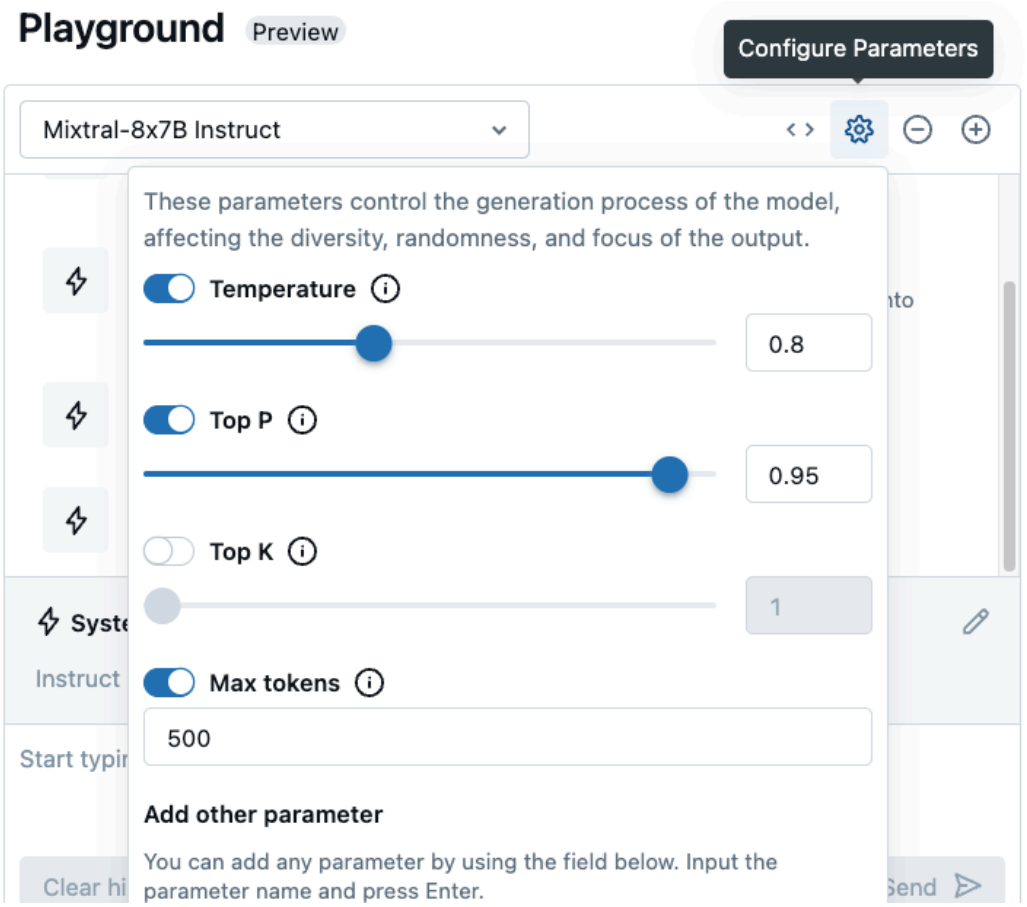


Figure 6.6 – Modifiable parameters in the LLM Playground

Playing around with temperature, for example, can give you more creative and diverse answers. On the other hand, you might end up with something nonsensical if you push the temperature too high. Using the Playground allows you to compare models and parameters to choose the best model for your project. As with everything else in this book, we encourage you to play in the Playground and explore how it might help you with your own prompt engineering and model evaluation work.

Now that we have covered assorted topics around creating training datasets, building baseline models, and integrating external data assets, we are ready to put this into practice with our example projects. Get ready to follow along in your own Databricks workspace as we work through the *Chapter 6* project code.

Applying our learning

In this chapter, we learned how to create training sets with Databricks Feature Engineering, make baseline models using AutoML, track our experiments with MLFlow, explore Databricks Marketplace, and use the LLM Playground to enhance our data science projects. Now let's apply what we have learned to the datasets we cleaned in *Chapter 4* and featurized in *Chapter 5*.

Technical requirements

Here are the technical requirements needed to complete the hands-on examples in this chapter:

- We will integrate our LLM model with the **ChatGPT** model from **OpenAI**. You will need an API Key and need to install the OpenAI Python library.
- For the Multilabel Image Classification project work in this chapter, you will need a GPU cluster. Here is the single node GPU configuration.

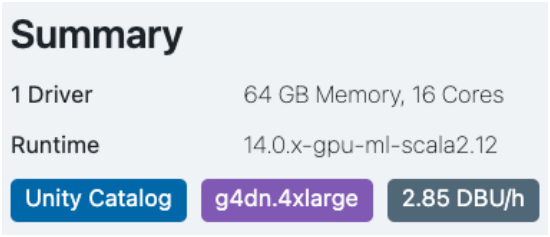


Figure 6.7 – Single node GPU cluster configuration used for this book

Project: Favorita Store Sales - Time Series Forecasting

Your goal is to create a `training_set` specification using DFE `FeatureLookups` and then create an AutoML baseline model. You will create the training set using the feature tables you created in *Chapter 5*. In *Chapter 4*, we used AutoML to jump-start our exploratory data analysis. Now, you will use AutoML to create a baseline model.

To follow along in your own workspace, please refer to the following notebook:

- `CH6-01-Building Favorita Baseline Model`

Building a training set

When you open `CH6-01-Building Favorita Baseline Model`, you will see that the repository code includes imports and variable names. It also includes a quick look at your data to remind you what the tables look like. Then we create the `model_feature_lookups` by specifying the `FeatureLookups` for each feature table, as shown in Figure 6.8. The `FeatureLookup` objects always include the `table_name` and `lookup_key`. Specifying the `feature_names` is optional.

Because the default behavior is to use all columns in the table as features except the primary keys, and we only want some of the features from those tables, we want to include the `feature_names` argument for the `oil_10d_lag_ft` and `stores_ft` tables. For example, in the oil price feature table, there is an additional date column that we do not want to include in the modeling training set.

```
model_feature_lookups = [
    FeatureLookup(
      table_name="oil_10d_lag_ft",
      lookup_key="date",
      feature_names="lag10_oil_price"
    ),
    FeatureLookup(
      table_name="store_holidays_ft",
      lookup_key=["date","store_nbr"]
    ),
    FeatureLookup(
      table_name="stores_ft",
      lookup_key="store_nbr",
      feature_names=["store_cluster","store_type"]
    ),
]
```

Figure 6.8 – The list of feature lookups for the Favorita Sales Forecasting model

Combine the feature lookups with the raw training data using `create_training_set`. The `training_set` specification can be reused to create a fresh DataFrame of training data with the same logic.

```
training_set = fe.create_training_set(
    df=raw_data,
    feature_lookups=model_feature_lookups,
    label=label_name,
)
training_df = training_set.load_df()
```

Figure 6.9 – Create the training set specification using training data as df, the feature lookups, and the column name of the label

To populate a DataFrame with the training data, call the `load_df()` method. A DataFrame is returned with the current version of features and training data for modeling. For this reason, saving the spec details in a util file can be advantageous.

With that, we have the training dataset for the Favorita project. We save a snapshot of the training data to a table, to follow best practice and ensure we can train future models on the exact same data if we need to reproduce the project at any point. *Figure 6.10* shows the catalog view of our Favorita `training_data` table. Here, we have opened the Lineage tab of the table to see what data was used to build this training data.

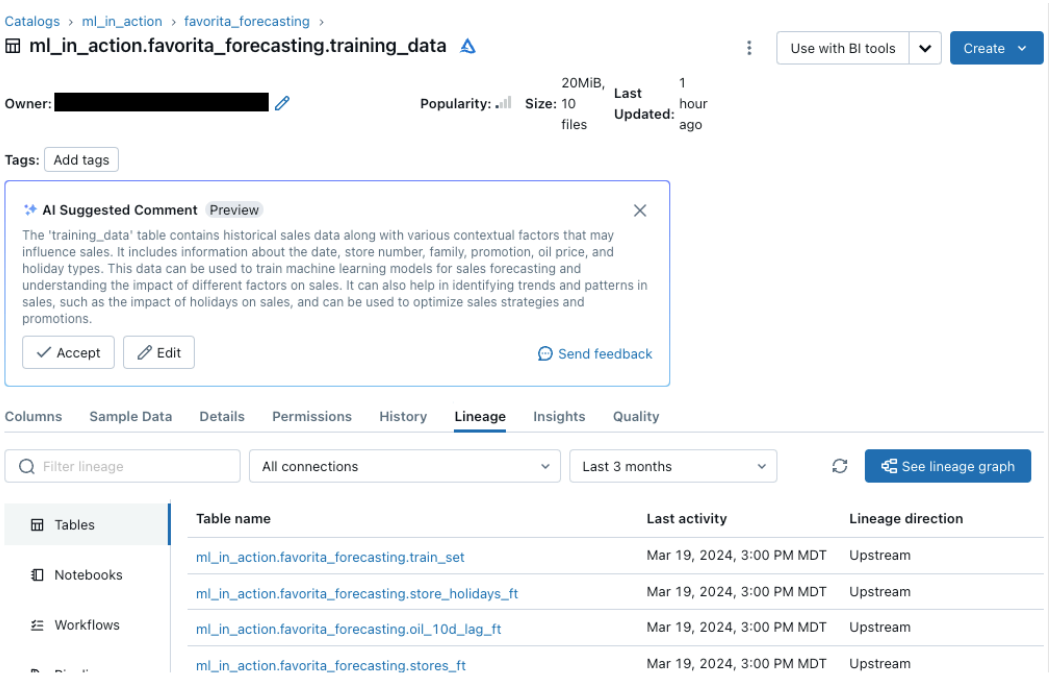


Figure 6.10 – The UI displaying the Lineage tab for the feature table in the Catalog

The Catalog pane is available by clicking the Catalog on the left side menu bar. Then search for the `ml_in_action` catalog. Select the `favorita_forecasting` database (also called a schema). In the list of tables (you may have to scroll), select the `training_data` table. Once you your screen matches *Figure 6.10*, click the **See lineage graph** button to see the lineage DAG, which takes us to something close to *Figure 6.11*.

Figure 6.11 – The data lineage DAG for our training_set table

For our DAG, we clicked the left **+** button on the `store_holidays_ft` table and again on the `regional_holidays_silver` to expand the lineage even further. You can continue to explore by clicking the **+** buttons on the lineage DAG in your workspace as well. The DAGs for every table saved in Unity Catalog are updated whenever the table itself or its sources are altered, making it living documentation (this is great news for those of us who despise creating and maintaining documentation!).

Creating a baseline model with AutoML

Now, let's create a baseline model using AutoML. There are two ways to complete this. You can use the UI or the API. The project notebook `CH6-01-Building Favorita Baseline Model` has the necessary Python code to run the baseline experiment using the API. If you choose to run the code, check the notebook for the experiment information as in *Figure 6.2*. If you want to create the baseline model using the UI, you can start by selecting the **+ New** button on the left sidebar. Then in the Machine Learning section, choose **AutoML Experiment**. See the experiment configurations in *Figure 6.12* to set up the experiment. Please note that you need to have a cluster running before you can select it from the Cluster dropdown menu. You do not need GPUs for this part; A CPU-based cluster works fine.

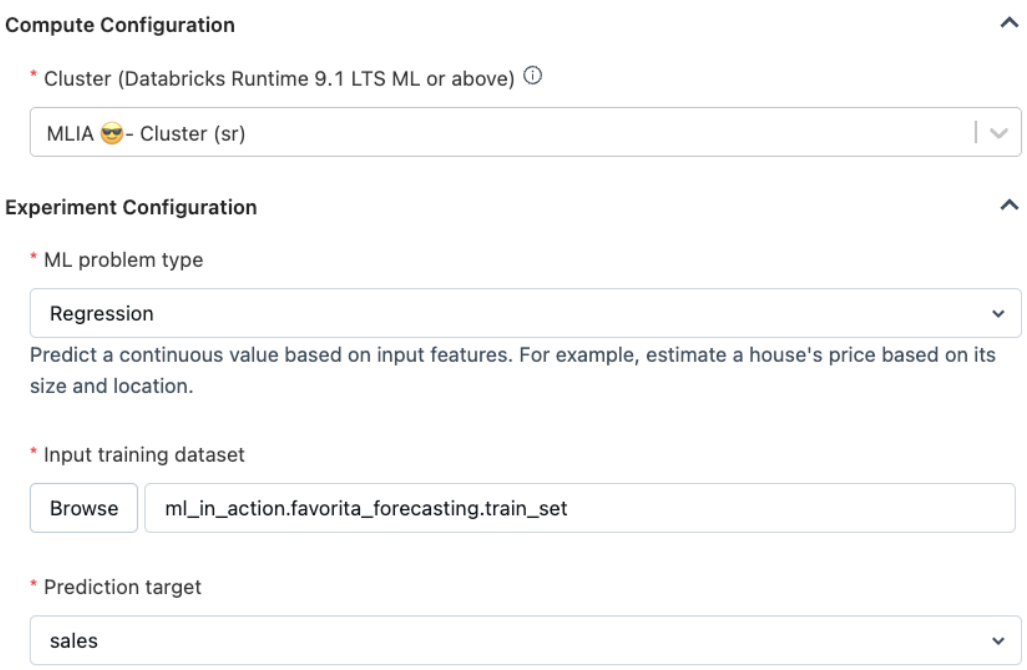


Figure 6.12 – AutoML experiment configuration for the Favorita Sales Forecasting project

Notice that during this experiment, we treat the forecasting problem like a regression problem by selecting the ML problem type as **Regression**. As a result, you also must include the **date** variable in the **Advanced Configuration** section as the **Time column**.

You can use the training dataset you just created, `training_data`, as the **Input training dataset**. You can also choose to use the **Join features** (**optional**) button, where you can add the features tables to be joined to the original `train_set` (see *Figure 6.13*). Notice how using the `_ft` naming convention at the end of our feature table names makes searching for feature tables easier.

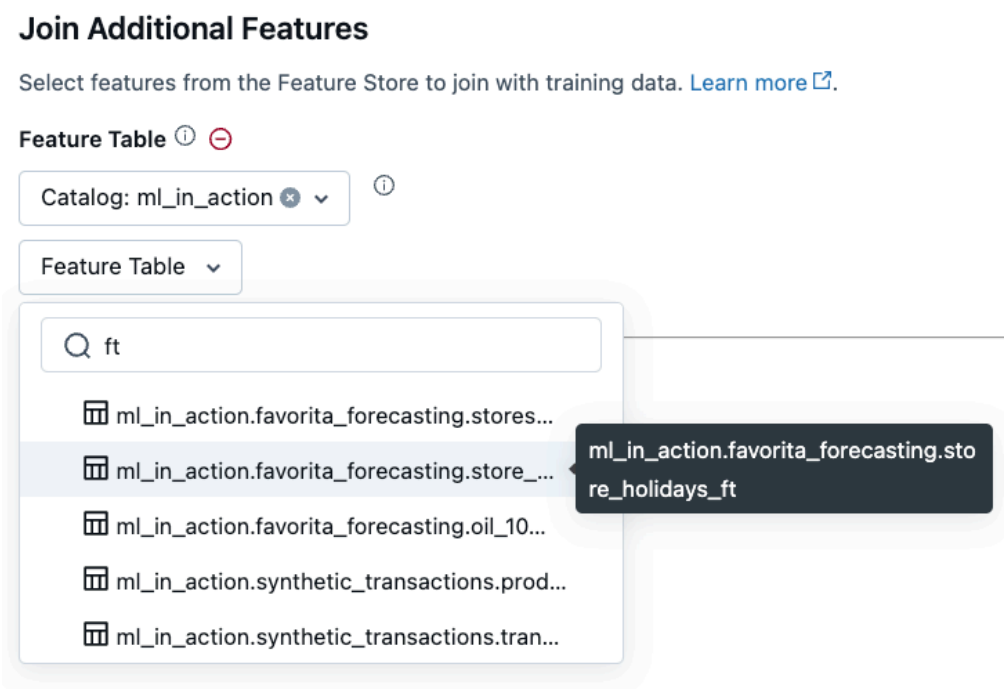


Figure 6.13 – Joining additional features from our feature tables

Once you have your experiment set up, click **Start AutoML** to begin the experiment. When we ran this, the experiment created about 100 runs before reaching the point where it was no longer making progress against the metric of choice, in our case, R-squared, as shown in *Figure 6.14*.

Run Name	Created	Dataset	Duration	Source	Models	val_r2_score	model_type
useful-grouse-161	11 minutes ago	-	8.2s	Noteb...	sklearn	0.854	lightgbm_r...
salty-shad-332	10 minutes ago	-	9.6s	Noteb...	sklearn	0.852	lightgbm_r...
adventurous-deer-223	10 minutes ago	-	9.9s	Noteb...	sklearn	0.851	lightgbm_r...
casual-carp-736	10 minutes ago	-	8.2s	Noteb...	sklearn	0.851	lightgbm_r...
sassy-dove-915	9 minutes ago	-	6.5s	Noteb...	sklearn	0.85	lightgbm_r...
defiant-swan-537	12 minutes ago	-	8.4s	Noteb...	sklearn	0.85	lightgbm_r...
abundant-fly-694	10 minutes ago	-	9.9s	Noteb...	sklearn	0.849	lightgbm_r...
rambunctious-kit-989	9 minutes ago	-	8.6s	Noteb...	sklearn	0.849	lightgbm_r...

Figure 6.14 – AutoML created almost 100 models for Favorita Sales project

Out of the hundred models, only six have an R-squared value of `0.85` or higher. Databricks AutoML saved us considerable time and effort! During the experiment, MLflow tried a variety of algorithms and hyperparameters, tuning with **HyperOpt**. This experiment is also distributed with the power of Spark, meaning we have a solid model that was tuned efficiently. Our best-performing model, which will become the baseline model, performs reasonably well, and indicates the dataset has predictive power. In practice, the next step might be to improve performance by creating new features, gathering more data points, or enriching the dataset. In this book, we move on to preparing for production.

Project: Streaming Transactions

You will create a `training_set` spec similar to the Favorita Sales project for the Streaming Transactions project. Before we jump in, let's remember where we are and where we are going by reviewing the project pipeline for our streaming use case.

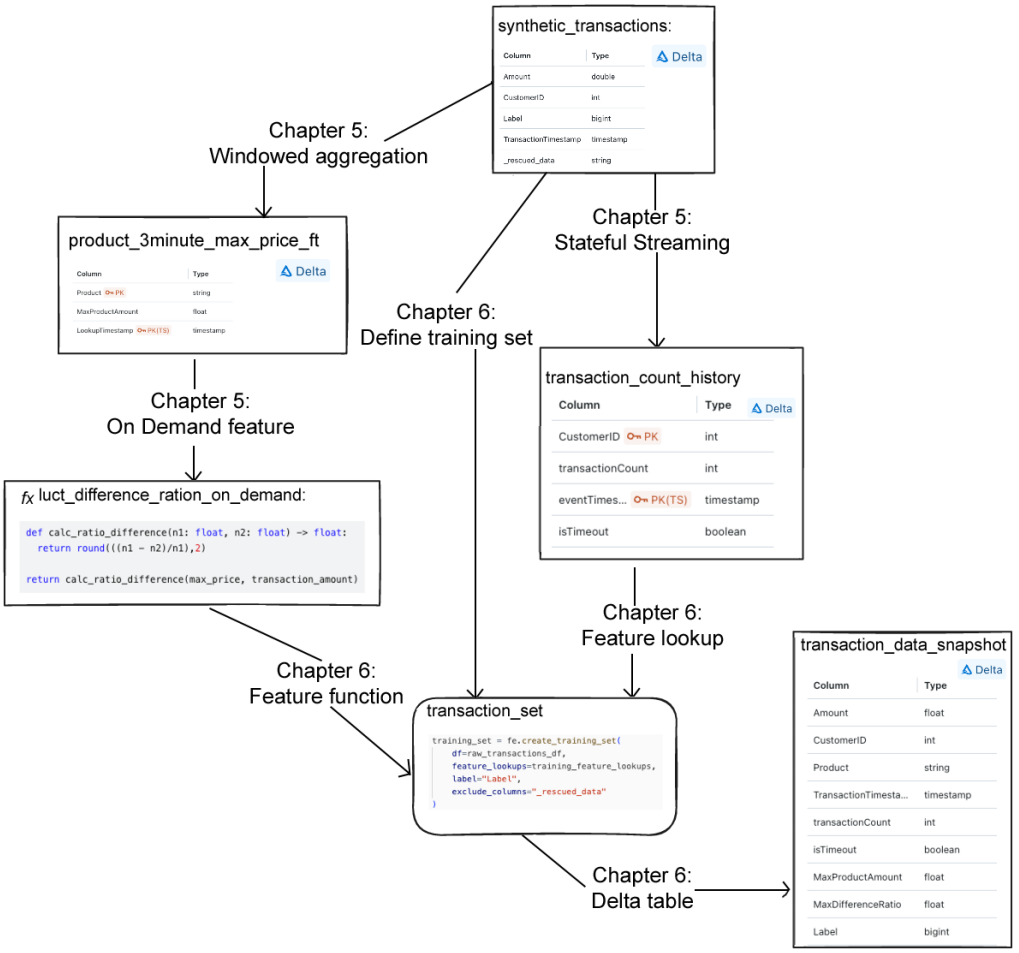


Figure 6.15 – The project pipeline for the Streaming Transactions Project

To create a snapshot training set of the streaming data using the Databricks Feature Engineering API client, we need the data from *Chapter 5*. To double-check that you have all the data necessary to complete this chapter's project work, you can use the Catalog view. Navigate to the `ml_in_action` catalog, then click on the `synthetic_transactions` schema, and confirm that you see the following tables: `product_3minute_max_price_ft`, `raw_transactions`, `transaction_count_history` and function `product_difference_ratio_on_demand_feature`. If you have all the necessary tables, you can skip the following steps and start with the *Chapter 6* notebook. Otherwise, please follow these steps first to stream in the Transactions data:

1. Start the multiple CPU cluster from *Chapter 5*.
2. Open the *Chapter 5* notebooks. Feel free to edit `CH5-01` to a smaller number of total steps by reducing the value of the `total` variable in the final cell of the notebook.
3. Hit **Run All** at the top of `CH5-01`. Confirm that the final cell titled "Loop for Generating Data" has begun to execute before moving on to the next step.
4. Hit **Run All** at the top of `CH5-02`. Confirm that the final cell title "readStream" has begun to execute before moving on to the next step.
5. Hit **Run All** at the top of `CH5-03`. Watch to see that the final two cells titled "Read and Write Streams" and "Writing the history of transaction counts to a table" have begun to execute before moving on to the next step.
6. While you are waiting for the data streams to finish, add a monitor to the `raw_transactions` table, as shown in previous chapters. You can refer back to the *Monitoring data quality with Databricks Lakehouse Monitoring* section in *Chapter 4*.
7. Once the generating notebook, `CH5-01`, finishes, you can sequentially terminate notebooks `CH5-02` and `CH5-03`.
8. Hit **Run All** at the top of notebook `CH5-04`.

Once all the Streaming Transactions tables are ready, you can begin this chapter's project work. To follow along in your own workspace, please refer to the following notebook:

- `CH6-01-Building Synthetic Transactions Training Set`

We begin by creating a training set, which we will build into a snapshot of data for training. To do so, we specify the feature lookups for the base DataFrame. In our case, the base DataFrame is the `raw_transactions` table. However, we cannot train on data without feature values present in the right tables. Data is continuously flowing into the `raw_transactions` table. Therefore, we need the base DataFrame to contain only transactions that have a calculated feature value. For that, we take the maximum of the two minimum timestamps. This is shown in the notebook code.

Figure 6.16 shows the feature objects we will use to build the training dataset. Note that for this project we're using Feature Lookups as well as a Feature Function. First, we include our feature tables `transaction_count_history` and `product_3min_max_price_ft`. Then we add the feature function `product_difference_ratio_on_demand_feature`.

```
training_feature_lookups = [
    FeatureLookup(
      table_name="transaction_count_history",
      rename_outputs={
          "eventTimestamp": "TransactionTimestamp"
        },
      lookup_key=["CustomerID"],
      feature_names=["transactionCount", "isTimeout"],
      timestamp_lookup_key = "TransactionTimestamp"
    ),
    FeatureLookup(
      table_name="product_3minute_max_price_ft",
      rename_outputs={
          "LookupTimestamp": "TransactionTimestamp"
        },
      lookup_key=['Product'],

      timestamp_lookup_key='TransactionTimestamp'
    ),
    FeatureFunction(
      udf_name="product_difference_ratio_on_demand_feature",
      input_bindings={"max_price":"MaxProductAmount", "transaction_amount":"Amount"},
      output_name="MaxDifferenceRatio"
    )
]
```

Figure 6.16 – Creating training feature lookups

We combine the Feature Lookups and the Feature Function into a list to provide to the DFE client. Also note that we rename `LookupTimestamp` to `TransactionTimestamp` for the `product_3min_max_price_ft` table using the `rename_outputs` argument. This is because the timestamp column we want to key off of is present in our base DataFrame as `TransactionTimestamp`. Both feature tables use the point-in-time lookup functionality. We use point-in-time because, as a time series, we want the feature values that are the most recent relative to a given lookup timestamp. We do not want nulls when a timestamp doesn't match perfectly, and we certainly don't want information from the future sneaking into our training data. Implementing this product feature is as simple as adding a timestamp column as the `timestamp_lookup_key`.

Following the feature lookups, we build the `training_set` spec; see *Figure 6.17* for the code. The spec needs a base DataFrame, `raw_transactions_df`, that begins once the features have non-null values. We ensure we only have non-null values by using the max timestamp of the two minimums as mentioned previously. Looking at your SQL results, pick the max and copy the timestamp. Then paste it into place of the timestamp in *Figure 6.17*.

```
raw_transactions_df = sql("SELECT * FROM raw_transactions WHERE timestamp
(TransactionTimestamp) > timestamp('2024-03-22T17:40:59.246+00:00')")

training_set = fe.create_training_set(
    df=raw_transactions_df,
    feature_lookups=training_feature_lookups,
    label="Label",
    exclude_columns="_rescued_data"
)
training_df = training_set.load_df()
```

Figure 6.17 – Creating training set

Running the code in *Figure 6.17* gives us our Streaming Transactions training data. As you can see in *Figure 6.18*, the dataset contains all the feature table columns, the label, and the resulting feature column created by the *on-demand feature* function.

```
training_df.write.mode('overwrite').saveAsTable('training_data_snapshot')
display(training_df)
```

▸ (11) Spark Jobs

Table + New result table: ON Search

	Amount	Custo...	Product	TransactionTimesta...	transa...	isTime...	MaxPr...	MaxDif...	Label
1	69.99	1235	Product A	2024-03-22T17:41:15.3...	30	false	250.01	0.72	0
2	52.72	1235	Product A	2024-03-22T17:41:20.8...	30	false	250.01	0.79	1
3	21.25	1235	Product B	2024-03-22T17:41:22.7...	30	false	55.01	0.61	0
4	35.27	1235	Product B	2024-03-22T17:41:25.5...	30	false	55.01	0.36	1
5	174.98	1235	Product C	2024-03-22T17:41:29.9...	30	false	400.01	0.56	0

Figure 6.18 – Viewing the training data

You now have a training set spec using point-in-time lookups and an on-demand feature function. We save our snapshot to a table, but because we have the training spec saved to a notebook, we can also rest easy knowing we have followed data science best practices for reproducibility. We are ready for modeling in the next chapter.Project: Retrieval Augmented Generation Chatbot

Back in *Chapter 4*, we created a **Vector Search** (**VS**) endpoint and index to run similarity searches against our dataset of processed *Arxiv* article contents. Now, we are going to build a chatbot by combining the Vector Search components with a Large Language Model that can use the *Arxiv* data to generate answers to user questions.

To follow along in your own workspace, please refer to the following notebook:

- `CH6-01-Assembling ChatBot`

Our notebook begins with some instructions for setting up your secret for this project. First, we connect our Vector Database and LLM using the LangChain framework. We are using the integration from Databricks with LangChain to create a function called `get_retriever`, which calls the VS index that you created in *Chapter 4*. We include the host and access token in the function because our goal is for this chatbot to connect it to a hosted application we deploy in later chapters. The application will require the host and access token to be able to authenticate. *Figure 6.19* shows the code to call the previously created index, create the retriever, and run a quick test see the top relevant document returned for a user's question.

```
from databricks.vector_search.client import VectorSearchClient
from langchain.vectorstores import DatabricksVectorSearch
from langchain.embeddings import DatabricksEmbeddings

# NOTE: your question embedding model must match the one used in the chunk in the previous model
embedding_model = DatabricksEmbeddings(endpoint="databricks-bge-large-en")
vsc_endpoint_name = "ml_action_vs"
index_name = f"{catalog}.{database_name}.docs_vsc_idx_cont"

def get_retriever(persist_dir: str = None):
    os.environ["DATABRICKS_HOST"] = host
    #Get the vector search index
    vsc = VectorSearchClient(workspace_url=host, personal_access_token=os.environ["DATABRICKS_TOKEN"])
    print("\n")
    vs_index = vsc.get_index(
        endpoint_name=vsc_endpoint_name,
        index_name=index_name)
    # Create the retriever
    vectorstore = DatabricksVectorSearch(
        vs_index, text_column="content", embedding=embedding_model)
    return vectorstore.as_retriever()

# test our retriever
vectorstore = get_retriever()
similar_documents = vectorstore.get_relevant_documents("What is GPT? ")
print(f"Relevant documents: {similar_documents[0]}")
```

Figure 6.19 – Testing the retrieval component with the LangChain

Here is the first element that was retrieved from the Databricks VS for the question, "What is GPT?"

```
Relevant documents: page_content='7.1 LLM Conclusion (GPT-4's Version)\nGenerative Pre-trained Transformers (GPTs) generate profound transformations, garnering poten
tial technolog- ical growth, permeating tasks, greatly impacting professions. This study probes GPTs' potential trajectories, presenting a groundbreaking rubric to g
auge tasks' GPT exposure, particularly in the U.S. labor market.\n7.2 LLM Conclusion (Author-Augmented Version)\nGenerative Pre-trained Transformers (GPTs) generate
profound transformations, garnering potential techno- logical growth, permeating tasks, gutting professional management. Gauging possible trajectories? Generate pion
eering taxonomies, gather policymakers together, generalize past today.\nAcknowledgments\nThank you to the group of annotators who helped us annotate task exposure,
including Muhammad Ahmed Saeed, Bongane Zitha, Merve Özen Şenen, J.J., and Peter Hoeschele. We also thank Lauryn Fuld, Ashley Glat, Michael Lampe, and Julia Susser f
or excellent research assistance. We thank Miles Brundage for significant feedback on this paper.\nWe thank Todor Markov and Vik Goel for setting up the infrastructu
re we use to run our rubrics with GPT-4. We thank Lama Ahmad, Donald Bakong, Seth Benzell, Erik Brynjolfsson, Parfait Eloundou-Enyegue, Carl Frey, Sarah Giroux, Gill
ian Hadfield, Johannes Heidecke, Alan Hickey, Eric Horvitz, Shengli Hu, Ashyana Kachra, Christina Kim, Katya Klinova, Daniel Kokotajlo, Gretchen Krueger, Michael Lam
pe, Aalok Mehta, Larissa Schiavo, Daniel Selsam, Sarah Shoker, Prasanna Tambe, and Jeff Wu for feedback and edits at various stages of the project.' metadata={'id':
179.0}
```

Figure 6.20 – Output example from the get_relevant_documents function

Once the integration works, we create our chatbot with a custom prompt where LangChain will pass the context and the question, as shown in *Figure 6.21*.

Modifying your Prompt

Please pay attention to the prompt because you will have to modify it according to your use cases. For this project, we create a simple RetrievalQA chain without a conversation history buffer or complex RAG chains. The idea is to familiarize you with tools that can be applied to various scenarios on your own using domain expertise in RAG.

```
import langchain
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langchain.chat_models import ChatDatabricks

# Using Foundational Model from Databricks
chat_model = ChatDatabricks(endpoint="databricks-mixtral-8x7b-instruct",
                            max_tokens = 200
                            )

TEMPLATE = """
You are an assistant for the AI Swat Team. You are answering questions related to
the GenerativeAI and LLM's and how they impact humans life, labour, economic and
financial impact. If the question is not related to one of these topics, kindly
decline to answer. If you don't know the answer, just say that you don't know, don't
try to make up an answer. Keep the answer as concise as possible. Use the following
pieces of context to answer the question at the end:
{context}
Question: {question}
Answer:
"""
prompt = PromptTemplate(template=TEMPLATE, input_variables=["context", "question"])

chain = RetrievalQA.from_chain_type(
    llm=chat_model,
    chain_type="stuff",
    retriever=get_retriever(),
    chain_type_kwargs={"prompt": prompt}
)
```

Figure 6.21 – Creating a RetrievalQA chain with a custom prompt

Let's try it out! We submit a question to the chatbot using `chain.run()` and then view the response.

```
question = {"query": "Will AI impact work forces in the US ? "}
answer = chain.run(question)
```

```
 Yes, AI, including LLMs, is projected to have a significant impact on the US workforce. According to a study, approximatel
y 19% of jobs have at least 50% of their tasks exposed to LLMs when considering both current model capabilities and anticip
ated LLM-powered software. The impact is expected to be more pronounced in higher-wage occupations and could have various e
ffects such as augmenting or displacing human labor, affecting job quality, and influencing inequality. However, social, ec
onomic, regulatory, and other factors will also play a role in determining the actual outcomes. Further research is necessa
ry to understand the broader implications of AI advancements on the workforce.
```

Figure 6.22 – Testing our chatbot with chain.run()

Once our chatbot model is ready, we track and log our model with MLflow. We do this for the tracking and version control but also because there is a direct integration with the MLflow LangChain flavor that will help us seamlessly serve this model using Databricks Model Serving, and then later connect to hosted applications outside of the Databricks platform.

```
from mlflow.models import infer_signature
from mlflow.tracking.client import MlflowClient

mlflow.set_registry_uri("databricks-uc")
model_name = f"{catalog}.{database_name}.mlaction_chatbot_model"

with mlflow.start_run(run_name="mlaction_chatbot_rag") as run:
    signature = infer_signature(question, answer)
    model_info = mlflow.langchain.log_model(
        chain,
        loader_fn=get_retriever,
        artifact_path="chain",
        registered_model_name=model_name,
        pip_requirements=[
            "mlflow==" + mlflow.__version__,
            "langchain==" + langchain.__version__,
            "databricks-vectorsearch",
        ],
        input_example=question,
        signature=signature
    )
    #------------------------
    import mlflow.models.utils
    mlflow.models.utils.add_libraries_to_model(
        f"models:/{model_name}/{get_latest_model_version(model_name)}"
    )
```

Figure 6.23 – Tracking and logging our model

Now we have used the LangChain framework to connect our Vector Database and LLM. We created a RetrievalQA chain with a custom prompt for our chatbot. Last, we tracked and logged our model with MLflow and have it ready for model serving in *Chapter 7*.

Project: Multilabel Image Classification

In this section, we will fine-tune our multilabel classification model. To follow along in your own workspace, please refer to the following notebooks:

- `Ch6-01-Training Base Model`
- `Ch6-02-Scoring Best Model`

Before we begin, note the compute recommendations in the *Technical requirements* section for this project, as you will need a GPU cluster for running the `Ch6-01-Training Base Model` notebook. We would also like to remind you that this example project is focused on code execution, rather than a technical deep dive into domain knowledge. We anticipate that most readers will have no problem running the Image Classification notebooks, but some prior familiarity with the Lightning/PyTorch frameworks will be helpful.

> **Databricks ML Runtimes**
>
> The Databricks Machine Learning Runtimes include pre-built **Deep Learning** (**DL**) infrastructure and libraries like PyTorch, TensorFlow, and Hugging Face transformers, and CUDA libraries. Deep learning models are computationally intensive and distributed deep learning frameworks like **Distributed Data Parallel** (**DDP**) or **Fully Shared Data Parallel** (**FSDP**), DeepSpeed, and Composer are often used for more efficient training. Be sure to check out the Spark PyTorch/DeepSpeed Distributor on Databricks! There is a blog titled "Introducing the Spark PyTorch Distributor" linked in *Further reading* if you are working with PyTorch and you are interested in performing distributed Deep Learning.

Training

We begin our deep learning journey by downloading libraries using `!pip install` in the `Ch6-01-Training Base Model` notebook.

```
!pip install pytorch-lightning==2.1.2 deltalake==0.14.0 deltatorch==0.0.3 evalidate==2.0.
2 pillow==10.1.0
dbutils.library.restartPython()
```

Figure 6.24 – Installing notebook scoped libraries and restarting Python

After installing and loading libraries, we peek at our training data. Then we define our experiment and log paths using some utility functions that you can find in the `mlia_utils` folder.

```
import mlflow
from mlia_utils import mlflow_funcs

experiment_path = f"/Users/{current_user}/intel-clf-training_action"
mlflow_funcs.mlflow_set_experiment(experiment_path)
log_path = f"{volume_file_path}/intel_image_clf/intel_training_logger_board"
!mkdir {log_path}
```

Figure 6.25 – Defining experiment and log paths

Rather than working our way through the code top-to-bottom, we find it helpful to start by explaining the main training function `train_distributed`. This function contains the parameters we pass to the `pl.Trainer` to initialize and fit our trainer. *Figure 6.26* shows the code that you will find towards the end of the notebook in cell 12.

```
def train_distributed(max_epochs: int = 1, strategy: str = "auto"):
    # import logging
    # logging.basicConfig(level=logging.DEBUG)

    # this is required if you want to log with MlFlow
    # if you do not have access to a Token, simply save models under a checkpoint and log them back
    # Track your loss with the Tensoflow Board
    os.environ['DATABRICKS_HOST'] = db_host
    os.environ['DATABRICKS_TOKEN'] = db_token
    torch.set_float32_matmul_precision("medium")

    checkpoint_callback = pl.callbacks.ModelCheckpoint(…
    )
    early_stop_callback = pl.callbacks.EarlyStopping(…
    )

    tqdm_callback = pl.callbacks.TQDMProgressBar(…
    )
    # Initialize a trainer
    trainer = pl.Trainer(…
    )

    print(f"Global Rank: {trainer.global_rank}")
    print(f"Local Rank: {trainer.local_rank}")
    print(f"World Size: {trainer.world_size}")

    dm = DeltaDataModule()
    model = LitCVNet(num_classes=6)
    #trainer.fit(model, dm)
    trainer.fit(model, train_dataloaders=dm.train_dataloader(), val_dataloaders=dm.val_dataloader())
    print("Training done!")
```

Figure 6.26 – Defining training function

We are passing two variables to the trainer: `dm` and `model`, where `dm` is a variable that represents the `DeltaDataModule` class, and the `model` is assigned to the `LitCVNet` class with the corresponding number of classes we are expecting to train our model on. Note that `trainer.fit(model, dm)` is another way of calling the same thing (we have included this alternative code as a comment). Keeping this structure of your training function helps you to evaluate various strategies while selecting the best parameters, model families, and data loaders for your task.

Now let's move on to the DataLoader. As we explained in *Chapter 3*, we are going to store our images under a Delta Lake format and use a new open-source project called DeltaTorch to load in our images (see *Further Reading* for more details on DeltaTorch).

```
from deltatorch import create_pytorch_dataloader, FieldSpec

class DeltaDataModule(pl.LightningDataModule):
    """
    Creating a Data loading module with Delta Torch loader
    """
    def __init__(self):
        super().__init__()
        self.num_classes = 6
        # Here we are applying the same Transformation
        # in Production case scenario you probably would like to have to separate transformers for your data
        # 1 for Train and another one for test.
        # See the Native Torch example below.

        self.transform = transforms.Compose([
                transforms.Resize((150,150)),
                transforms.RandomHorizontalFlip(p=0.5), # randomly flip and rotate
                transforms.ColorJitter(0.3,0.4,0.4,0.2),
                transforms.ToTensor(),
                transforms.Normalize((0.425, 0.415, 0.405), (0.205, 0.205, 0.205))
                ])

    def dataloader(self, path: str, batch_size=32):

        return create_pytorch_dataloader(
            path,
            id_field="id",
            fields=[
                FieldSpec("content", load_image_using_pil=True, transform=self.transform),
                FieldSpec("label_id"),
            ],
            shuffle=True,
            batch_size=batch_size,
        )

    def train_dataloader(self):
        return self.dataloader(train_delta_path, batch_size=64)

    def val_dataloader(self):
        return self.dataloader(val_delta_path, batch_size=64)

    def test_dataloader(self):
        return self.dataloader(val_delta_path)
```

Figure 6.27 – A class to store our images using DeltaTorch

Let's talk about the function `create_pytorch_dataloader`, as this is where the DeltaTorch loader is used. This is the main differentiator from classical PyTorch code. `Create_pytorch_loader` is a function to create a PyTorch DataLoader, which can be used directly during training. During training, this function checks if the Delta Table with a specified path exists, if the provided columns exist under the schema of a table, and if the table is not empty. It also performs a shuffle function under the hood, so you do not need to take care of it (which is why the `id_field` is required; it's used for sharding and parallelization of loading). As with the classic PyTorch DataLoader, you can apply transformations to your batch of images, modify the batch size, modify the number of workers that are used to load the data, and choose whether you want to apply the shuffle or not. We have also provided an example with a classical PyTorch Dataloader inside the `Ch6-01` notebook under the header "Native Torch Loader" in case you would like to store your images in a volume instead.

Now let's move on to the model! *Figure 6.28* shows the code in cell 7, though we have minimized some functions for easier reading. The main difference between a classic PyTorch code and the one you see below are the x and y definitions from the batch.

```
class LitCVNet(pl.LightningModule):

    def __init__(self, num_classes = 6, learning_rate= 0.0001, family = "resnext", momentum = 0.1,  dropout_rate = 0):…

    def get_model(self, num_classes):
        """
        This is the function that initialises our model.
        If we wanted to use other prebuilt model libraries like timm we would put that model here
        """
        backbone = torchvision.models.wide_resnet50_2(pretrained=True)
        for param in backbone.parameters():
            param.required_grad = False
        num_ftrt = backbone.fc.in_features
        backbone.fc = nn.Linear(num_ftrt, num_classes)
        return backbone

    # We do not overwrite our forward pass
    def forward(self, x):…

    def training_step(self, batch, batch_idx):
        x = batch["content"]
        y = batch["label_id"]
        logits = self.forward(x)
        loss = self.loss(logits,y)
        # Track accuracy
        acc = self.accuracy(logits, y)
        # required format
        self.log("loss", torch.tensor([loss]), on_step=True, on_epoch=True, logger=True)
        self.log("acc", torch.tensor([acc]), on_step=True, on_epoch=True, logger=True)
        return  {"loss": loss, "acc": acc}

    def validation_step(self, batch, batch_idx):…

    def test_step(self, batch, batch_idx):…

    # predict_step is optional unless you are doing some predictions
    def predict_step(self, batch, batch_idx, dataloader_idx=0):…

    def configure_optimizers(self):…
```

Figure 6.28 – Function for initializing and training model

Some of you may already guess that this is simply because our DataLoader is reading the column's `content` that corresponds to the binary image representation and `label_id` corresponding to the original label. We selected one of the most common Computer Vision family models: WideResnet50 from Wide Residual Networks (see the *Arxiv paper* on *Wide Residual Networks* linked in *Further reading* for more information on this family of models). To execute your training, we simply call the training function.

Let's briefly touch on how to move from a single node into the distributed mode. The good news is that you don't have to change much. You simply update your implementation to use a distributed library such as `TorchDistributor`. Here is an example of how you can leverage `TorchDistributor` (please note that if you need to run distributed training for a GenAI task, there is also a similar library for the `DeepSpeed` framework).

```
from pyspark.ml.torch.distributor import TorchDistributor

distributed = TorchDistributor(num_processes=4, local_mode=True, use_gpu=True)
distributed.run(train_distributed, 1, "ddp")
```

Figure 6.29 – Using TorchDistributor to distribute model training

From the code above, you can see that you just need to add a few lines of code to run the same function using a DDP strategy.

Scoring

We perform scoring in the second notebook, `Ch6-02-Scoring Base Model`. It starts with installing all necessary libraries we used for model training in the notebook using pip.

```
!pip install pytorch-lightning==2.1.2 deltalake==0.14.0 deltatorch==0.0.3
evalidate==2.0.2 pillow==10.1.0
dbutils.library.restartPython()
```

Figure 6.30 – Installing notebook scoped for scoring libraries and restarting Python

After installing and loading libraries, we need to find the best model that was trained. For this, we leverage the MLflow SDK, which requires our `experiment_path` name. We are again using utility functions that you can find in the `mlia_utils` folder. Once the best model is found, we check if the GPU cluster is available. If found, we move the model into the GPU.

```
import os
import mlflow
from mlflow.store.artifact.models_artifact_repo import ModelsArtifactRepository
from mlia_utils.cv_clf_funcs import select_best_model

experiment_path = f"/Users/{current_user}/intel-clf-training_action"
local_path = select_best_model(experiment_path)

requirements_path = os.path.join(local_path, "requirements.txt")
if not os.path.exists(requirements_path):
  dbutils.fs.put("file:" + requirements_path, "", True)

device = "cuda" if torch.cuda.is_available() else "cpu"
loaded_model = torch.load(local_path+"/data/model.pth", map_location=torch.device
(device))
loaded_model
```

Figure 6.31 – Defining experiment, requirements, and loading our model into GPU

Figure 6.32 runs simple example to confirm that our model performs well. We have again minimized some of the code for improved readability, but you can find the full code in the notebook.

```
  import matplotlib.pyplot as plt
  from torch.autograd import Variable
  from mlia_utils import cv_clf_funcs as cv_funcs

  transform_tests = cv_funcs.transform_imgs()

> def pred_class(img):…

  classes = {k:v for k , v in enumerate(sorted(outcomes))}
  loaded_model.eval()

  plt.figure(figsize=(20,20))
> for i, images in enumerate(pred_files[:50]):…
```

Figure 6.32 – Simple example to verify model predictability

When you execute this cell, you will see a sample of images and their predicted labels, similar to what is shown in *Figure 6.33*. Scan the results to get an idea of the model's accuracy.

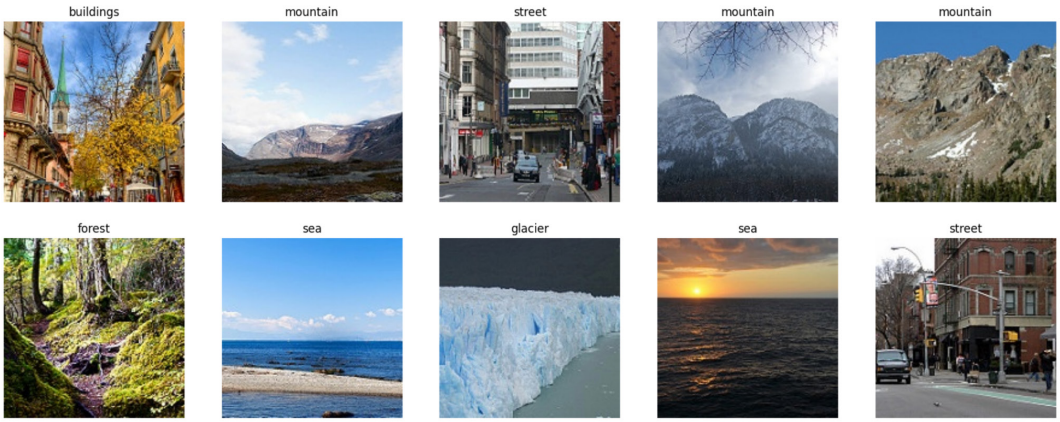


Figure 6.33 – The images and their labels

The `Ch6-02` notebook includes a markdown example of code that can calculate your score on the exact label prediction. In our case, we leverage binary classification by scoring the model using the one-vs-all method. Recall that while training our model we were transforming our images, and it is a good practice to keep this transformation when scoring as well. To do so, we create the function `feature_extractor`.

```
def feature_extractor(img):
    image = Image.open(io.BytesIO(img))
    transform = transform_tests = torchvision.transforms.Compose([
        transforms.Resize((150,150)),
        transforms.RandomHorizontalFlip(p=0.8), # randomly flip and rotate
        transforms.ColorJitter(0.3,0.4,0.4,0.2),
        transforms.ToTensor(),
        transforms.Normalize((0.425, 0.415, 0.405), (0.205, 0.205, 0.205))
        ])
    return transform(image)

# to reduce time on loading we broadcast our model to each executor
model_b = sc.broadcast(loaded_model.model)
```

Figure 6.34 – Creating a feature extractor function and broadcasting our model to each executor

In many cases, you will have a sizable number of files to score against, and for this you might want to leverage the distributed capacity of Apache Spark. We create a Pandas UDF to score our model across all of the labels. Pay attention to *Figure 6.34*, where we use the broadcast option. Broadcast is a functionality of Apache Spark that would copy the object across all nodes. Why do we need a broadcast? It is not a requirement, but the way Pandas UDF works is that they will have to load your model in the memory of each node each time a chunk of data arrives. Broadcasting your model can accelerate that process. Consider the size of your model and your node memory capacity before doing this for any other use case.

```
@pandas_udf("struct< label: int, labelName: string>")
def apply_cv(images_iter: Iterator[pd.Series]) -> Iterator[pd.DataFrame]:

    model = model_b.value.to(torch.device("cuda"))
    model.eval()

    id2label = {0: 'buildings',
                1: 'forest',
                2: 'glacier',
                3: 'mountain',
                4: 'sea',
                5: 'street'}

    with torch.set_grad_enabled(False):
        for images in images_iter:
            pil_images = torch.stack(
                [
                    feature_extractor(b)
                    for b in images
                ]
            )
            pil_images = pil_images.to(torch.device("cuda"))
            outputs = model(pil_images)

            preds = torch.max(outputs, 1)[1].tolist()
            probs = torch.nn.functional.softmax(outputs, dim=-1)[:, 1].tolist()

            yield pd.DataFrame(
                [
                    {"label": pred, "labelName":id2label[pred]} for pred in preds
                ]
            )
```

Figure 6.35 – Creating a Pandas UDF with an Iterator to yield results across chunks

With the Pandas UDF set up, we read our Delta tables with images from our volumes and score them with the `apply_cv` function.

```
spark.conf.set("spark.sql.execution.arrow.maxRecordsPerBatch", 32)
predictions_df = spark.read\
  .format("delta")\
  .load(f"/Volumes/{catalog}/{database_name}/files/intel_image_clf/valid_imgs_main.delta")\
  .withColumn("prediction", apply_cv("content"))
```

Figure 6.36 – Scoring all images in our volume

Once your scores are done, you can save them to another table for further investigation. With that, we have trained and scored our Multilabel classification model. We will add some infrastructure around the model in *Chapter 7* to make it production-ready.

Summary

We have now turned all our hard work from previous chapters into predictive models. From building training sets with the Databricks Feature Engineering client, to baselining models with AutoML and tracking experiments in MLflow, we covered core stages of all data science projects. We also discussed the Databricks Marketplace and Databricks LLM Playground, two tools that can further enhance data science and machine learning tasks by incorporating external data and models with your existing work.

You now have the skills required to take full advantage of the data available to you and gain powerful, predictive insights from the models you train. Of course, building the model is just the start – in *Chapter 7, Productionizing ML* on Databricks, we show you how to get your models in the hands of your users.

Questions

The following questions solidify key points to remember and tie the content back to your experience.

1. Why is it important that MLflow is open-source?
2. Why would you use a baseline model? Which Databricks Platform tool makes it easy to experiment and build baseline models rapidly?
3. What are some tools you can use to incorporate external data and models into your own data science projects? When might external data artifacts be useful?

Answers

After putting thought into the questions, compare your answers to ours.

1. It is important that MLflow is open-source because it is vendor agnostic, so you won't be locked into using a specific vendor or model. MLflow continues to develop and expand to support generative AI with the help of its robust open-source development community.
2. Baseline models are simple or common models you can use to learn valuable information about your data, such as if it contains salient features. You can also use baseline models to evaluate more complex ones. You can use AutoML to quickly build a baseline model.
3. You can use the Databricks Marketplace to incorporate external data artifacts, like datasets, notebooks, and models, into your own projects. You can also use the Databricks LLM Playground to experiment with Large Language Models within the platform. Using external data is useful when your project could benefit from data sources like weather, which might not be available internally. Incorporating LLMs via the Playground is beneficial to start experimenting rapidly without the typical development overhead.

Further reading

In this chapter, we pointed out specific technologies, technical features, and techniques. Please look at these resources to go deeper into the areas that interest you most.

- *MLFlow Tracking*: `https://mlflow.org/docs/latest/tracking.html#tracking`
- *MLFlow Built in Models and Autolog*: `https://mlflow.org/docs/latest/models.html#built-in-model-flavors`.
- *Introducing AI Functions*: *Integrating Large Language Models with Databricks SQL*: `https://www.databricks.com/blog/2023/04/18/introducing-ai-functions-integrating-large-language-models-databricks-sql.html`
- *LLM Training*: `https://www.databricks.com/blog/enroll-our-new-expert-led-large-language-models-llms-courses-edx`
- *Deploy Your LLM Chatbot With Retrieval Augmented Generation (RAG), llama2-70B (MosaicML inferences) and Vector Search*: `https://www.databricks.com/resources/demos/tutorials/data-science-and-ai/lakehouse-ai-deploy-your-llm-chatbot`
- *Best Practices for LLM Evaluation of RAG Applications | Databricks Blog*: `https://www.databricks.com/blog/LLM-auto-eval-best-practices-RAG`
- *Unifying Your Data Ecosystem with Delta Lake Integration*: `https://www.databricks.com/blog/integrating-delta-lakehouse-other-platforms`
- *Reading and Writing from and to Delta Lake from non-Databricks platforms*: `https://www.databricks.com/blog/integrating-delta-lakehouse-other-platforms`

- *Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM*: https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm
- *Fully Sharded Data Parallel*: https://engineering.fb.com/2021/07/15/open-source/fsdp/ https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/
- *OpenAI developer site*: https://platform.openai.com
- *OpenAI API Key*: https://platform.openai.com/account/api-keys
- *OpenAI Python library*: https://pypi.org/project/openai/
- *Databricks CLI*: https://docs.databricks.com/en/dev-tools/cli/install.html
- *Databricks Secrets API*: https://docs.databricks.com/en/security/secrets/secrets.html
- *Using Databricks Utilities with secrets*: https://docs.databricks.com/en/dev-tools/databricks-utils.html#dbutils-secrets
- *Blog: Introducing the Spark PyTorch Distributor*: (https://www.databricks.com/blog/2023/04/20/pytorch-databricks-introducing-spark-pytorch-distributor.html)
- *Pytorch documentation*: https://pytorch.org/
- *WideResnet50 Model*: https://pytorch.org/vision/main/models/generated/torchvision.models.wide_resnet50_2.html
- *Wide Residual Networks*: https://arxiv.org/abs/1605.07146
- *TorchDistributor*: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.torch.distributor.TorchDistributor.html
- *DeepSpeed framework*: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.deepspeed.deepspeed_distributor.DeepspeedTorchDistributor.html
- *DeltaTorch*: https://github.com/delta-incubator/deltatorch

7

Productionizing ML on Databricks

"Production is 80% of the work." — Matei Zaharia

Once you've refined your model and have satisfactory results, you are ready to put it into production. We've now entered the field of **Machine learning operations** (**MLOps**)! Unfortunately, this is where many data scientists and ML engineers get stuck, and it's not uncommon for companies to struggle here. Implementing models in production is much more complex than running models ad hoc because MLOps requires distinct tools and skill sets and sometimes entirely new teams. MLOps is an essential part of the data science process because the actual value of a model is often only realized post-deployment.

You can think of MLOps as combining **DevOps**, **DataOps**, and **ModelOps**. MLOps is often divided into two parts: inner and outer loops. The inner loop covers the data science work and includes tracking various stages of the model development and experimentation process. The outer loop encompasses methods to orchestrate your data science project through its life cycle, from testing to staging and ultimately into production.

Fortunately, the path from model development to production doesn't have to depend entirely on another team and tool stack when using the Databricks **Data Intelligence** (**DI**) Platform. Productionizing a machine learning model using Databricks products makes the journey more straightforward and cohesive by incorporating functionality like the Unity Catalog Registry, Databricks workflows, Databricks Asset Bundles, and model serving capabilities. This chapter will cover the tools and practices for taking your models from development to production.

Here is what you will learn as part of this chapter:

- Deploying the MLOps inner loop
- Deploying the MLOps outer loop
- Deploying your model
- Applying our learning

Deploying the MLOps inner loop

In Databricks, the MLOps inner loop uses a variety of tools within the DI platform that we've already touched upon throughout this book, such as MLflow, Feature Engineering with Unity Catalog, and Delta. This chapter will highlight how you can leverage them together to facilitate MLOps from one place. MLOps is covered in even more depth by Databricks' ebook, *The Big Book of MLOps*, which we highly recommend if you wish to learn more about the guiding principles and design decisions when architecting your own MLOps solution. We use **GitHub** to help facilitate DevOps and code reproducibility. For the DataOps portion, we use **Unity Catalog** and **Delta**. These tools help us track the versions of data and the code associated with the features created. This is the data reproducibility piece of DataOps. We use Delta time travel to query data from previous versions of the same table in the short term. For long term reproducibility, we recommend saving the training dataset as a Delta table and logging the table path with the `mlflow.log_input()` method. The upstream sources of the feature tables we create are automatically tracked by UC lineage.

Registering a model

Let's jump into model registries and their role in productionalization. A model registry is a centralized model store that helps to manage the entire model lifecycle, including versioning or aliasing, CI/CD integration, webhooks, and notifications. In Databricks, the **Unity Catalog Registry** (**UC Registry**) extends the governance features of Unity Catalog to the model registries, including centralized access control, auditing, lineage, and model discovery across workspaces. The UC Registry is a centralized repository for your models and their chronological lineage. This means that the experiment and experiment run that created each respective model is linked to the respective code and data source. Once you are satisfied with your machine learning model, the first thing to do is register it in Databricks under your central model registry. A registered model is a logical grouping of a model's version history. The UC Registry tracks different model versions, including each version's training data, hyperparameters, and evaluation metrics. A model rarely, if ever, has only one version. In addition to experimentation of model types and hyperparameter values, there are also cases where we want to experiment with different versions of a model. We can refer to these different model versions using model aliases. For example, deploying different versions simultaneously can be helpful for A/B testing; we'll cover this in more detail in the *Model Inference* section below. With the UC Registry, you can easily create and manage multiple model aliases, making it easier to track changes, compare performance, and revert to earlier versions if needed.

Collaborative Development

The UC Registry provides a collaborative model development environment, enabling teams to share and review models. This collaboration is also tracked, allowing multiple teams or leads to stay abreast of the model development process. Team or project leads can also require documentation before allowing a model to continue through the lifecycle. We find it easier to add snippets of documentation throughout the project rather than trying to remember everything and take the time to write it all down afterward.

The UC Registry allows for the tagging of models and model versions. In the Streaming Transaction project, we use tags to track the validation status of a model. The approval process can be automated or may require human interaction. An approval process can ensure that models are high quality and meet business requirements.

Figure 7.1 is a screenshot of the UC Model Registry, note that there are built-in locations for tags, aliases, and comments.

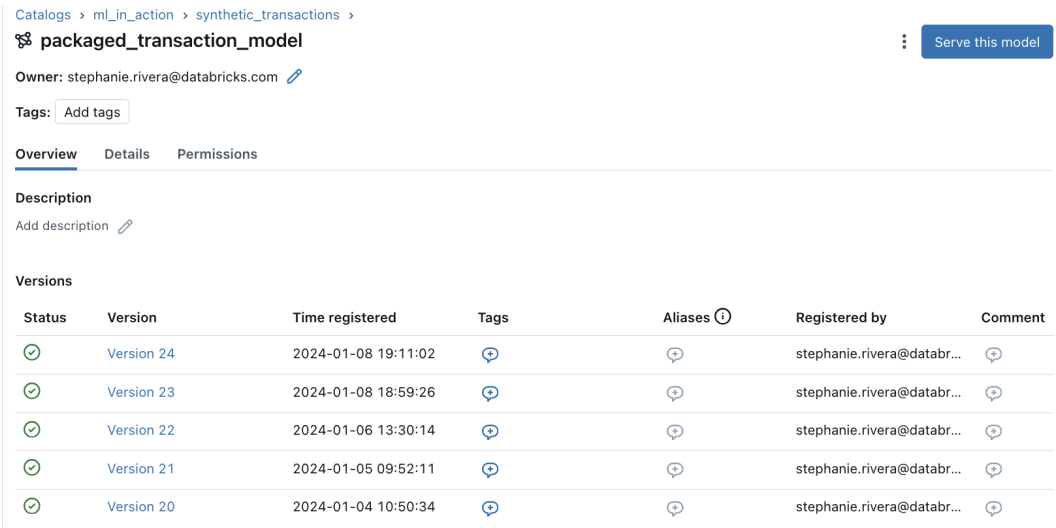


Figure 7.1 – The UC Model Registry UI shows each model version

The UC Registry is tightly integrated with the DI Platform. This provides a single technical stack, a unified environment for moving a model through experimentation to deployment. The seamless integration lets you leverage other Databricks components like Databricks Notebooks, Databricks Jobs, Databricks Lakehouse Monitoring, and Databricks Model Serving.

Next, let's move on to product features that support the outer loop process.

Deploying the MLOps Outer Loop

The ML lifecycle looks different for different use cases. However, the set of tools available in the Databricks Platform makes it possible to automate as you like and supports your MLOps. The outer loop connects the inner loop products with the help of Workflows, Databricks Terraform Provider, REST API, Databricks Asset Bundles, and more. We covered automating the tracking process through MLflow Tracking and the Unity Catalog Registry. The UCR is tightly integrated with the Model Serving feature and has a robust API that can easily be integrated into the automation process using webhooks. Each of these features can play a role in automating the ML lifecycle.

Workflows

Databricks Workflows is a flexible orchestration tool for productionizing and automating machine learning projects. Workflows help the ML lifecycle by providing a unified way to chain together all aspects of machine learning, from data preparation to model deployment. With Databricks Workflows, you can designate dependencies between tasks to ensure tasks are completed in the required order. These dependencies are visualized with the arrows connecting the tasks, as shown in *Figure 7.2*.

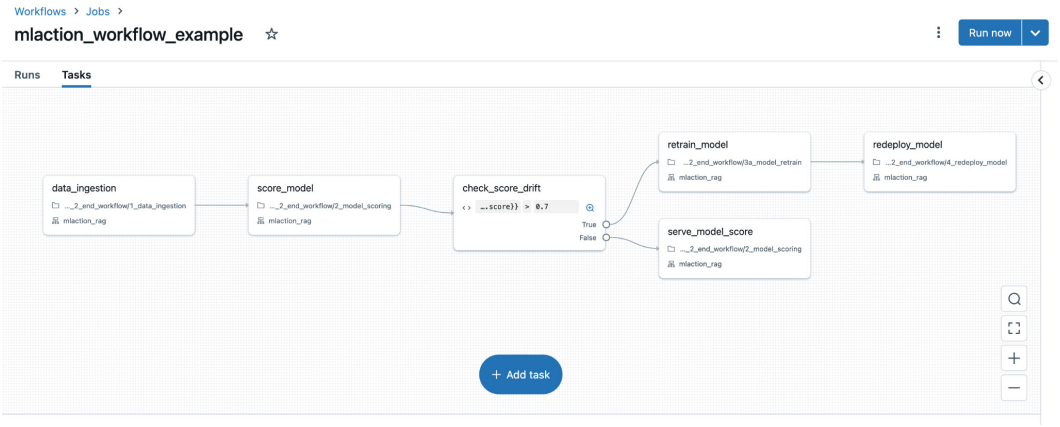


Figure 7.2 – A Databricks workflow with five tasks, with the feature engineering task having two dependencies

Tasks in a workflow are not limited to notebooks. In *Chapter 3*, we prepared a DLT pipeline to prepare data in the Bronze layer, and DLT pipelines can be a workflow component. Additional objects, like JAR files, python scripts, and SQL, can be tasks within a workflow, as shown in *Figure 7.3*.

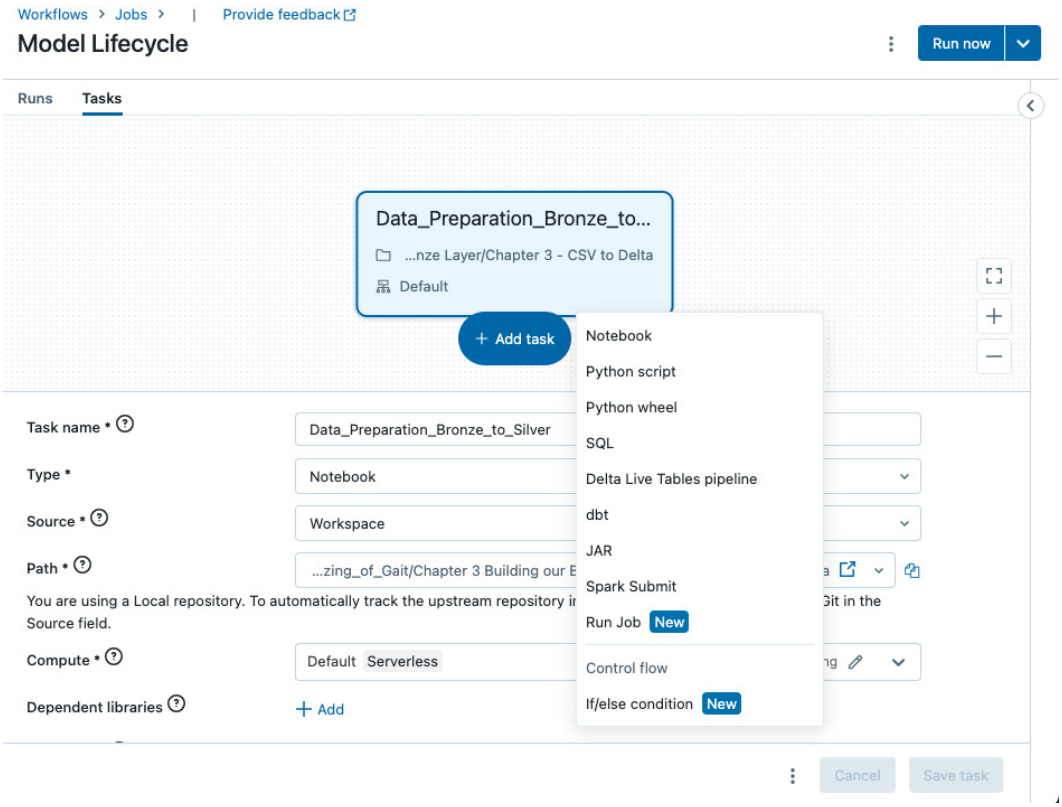


Figure 7.3 – Examples of objects that can be used as tasks within a workflow

Workflows are robust and play an important role in automating work within Databricks. Databricks Asset Bundles are another tool for productionization in Databricks.

Databricks Asset Bundles

Databricks Asset Bundles, or **DABs**, are a way to bring uniformity and standardization to the deployment approach for all data products built on the Databricks Platform. They are an **infrastructure-as-code** (**IaC**) approach to managing your projects, allowing developers to outline a project's infrastructure and resources using a YAML configuration file. DABs are especially useful for managing complex projects that involve a lot of collaborators and require automation. Where **continuous integration and deployment** (**CI/CD**) are necessary. DABs are a great way to manage machine learning pipeline resources across environments and to help your team follow best practices throughout the development and productionalization processes.

Under the hood, DABs are collections of Databricks artifacts (including jobs, DLT pipelines, and Machine Learning models) and assets (e.g., notebooks, SQL queries, and dashboards). These bundles are managed through YAML templates created and maintained alongside source code. You can build DAB YAML files manually or use templates to automate. You can also build custom templates for more complex processing tasks.

Using DABs requires the Databricks CLI. We discussed installing the CLI in *Chapter 2* if you want to review it again. Also, DABs are fairly new and not incorporated into the projects. However, great resources are listed in Further reading that cover this new product feature in depth.

Rest API

Everything you can do in the UI can be accomplished via the API as well. The UI is great for exploring product features and building workflows for the first time. For example, we automated the process after building out our AutoML experiment in the UI. Additionally, we saw how secrets are completely contained in the API and not available via the UI. As we see in the next section, it is possible to deploy your models via API.

Deploying your model

Deploying a model can be done in many ways, depending on the use case and data availability. For example, deployment may look like packaging a model in a container and deploying at an endpoint or a model that runs daily in a production workflow to provide predictions into tables consumed by applications. Databricks has product features to pave the way to production for all inference types.

Model Inference

We've walked through the methods and tools that help you set up your model in production, and finally, you have a model ready for inference! But one key question you should consider as part of this process is how your model should be used. Do you need the results once a day? Is the model powering an application that requires real-time results? Your model's purpose will help you decide the type of deployment you need. You've seen the words "batch" and "streaming" a few times in this chapter already, so let's quickly define what those mean in the context of model inference.

- **Batch Inference**: Batch inference (also known as offline inference) refers to a job that generates predictions on a group (or "batch") of data all at once. Batch jobs are scheduled to run on a specified cadence, such as once a day. This type of inference is best when there are no/low-latency requirements and allows you to take advantage of scalable compute resources.
- **Streaming Inference**: Streaming inference (also known as online inference) refers to a job that generates predictions on data as it is streamed. This is possible in the Databricks platform via Delta Live Tables.

- **Real-time Inference**: Real-time inference (a.k.a. model serving) refers to the process of exposing your models as REST API endpoints. This enables highly available, low-latency predictions and is especially useful when deploying models that generate predictions required by real-time applications.

All these options are available via the Databricks UI, shown below. Let's use the Favorita Store example again. Perhaps we've met with our beverages business team, and they would like to see weekly forecasts to help decide how much of each product they should purchase. We will opt for batch inference since we only need to produce an updated forecast once a week. It just takes a few clicks to set up a model for batch processing. Follow along in the "Applying our learning" section of the Favorita Sales dataset for detailed instructions on deploying your model for batch inference.

Model Serving

Let's dive deeper into real-time inference or **Model Serving**. The process of model serving can be complex and costly, involving additional tools and systems to achieve real-time needs. Fortunately, deploying a registered model as an endpoint only takes a single click from the Databricks platform! Because Model Serving is tightly integrated with MLflow, the path from development to production is much faster. Using a model registered in the MLflow Model Registry, Databricks automatically prepares a production-ready container and deploys the container to serverless compute.

It's also easy to deploy models via API, as shown below. A forecasting problem doesn't make much sense as a model serving use case, so let's consider instead the MIC problem we've been working on throughout the book. We will use model serving serve our classification model in real-time. The following example code snippet creates an endpoint that serves a version of a model named `asl_model`.

```
POST /api/2.0/serving-endpoints

{
  "name": "lakehouse-in-action-endpoint",
  "config":{
   "served_models": [{
     "model_name": "asl_model",
     "model_version": "1",
     "workload_size": "Small",
     "scale_to_zero_enabled": true
    }]
  }
}
```

Figure 7.4 – Deploying a model through a serving endpoint

Model Serving lets you build workflows around model endpoints, which provides plenty of flexibility in model deployment. For example, an organization may want to A/B test two or more models. Model serving makes it easy to deploy multiple models behind the same endpoint and distribute traffic among them. In another pattern, you can deploy the same model behind multiple endpoints. Below is an example of the UI and the analysis you can perform when A/B testing models.

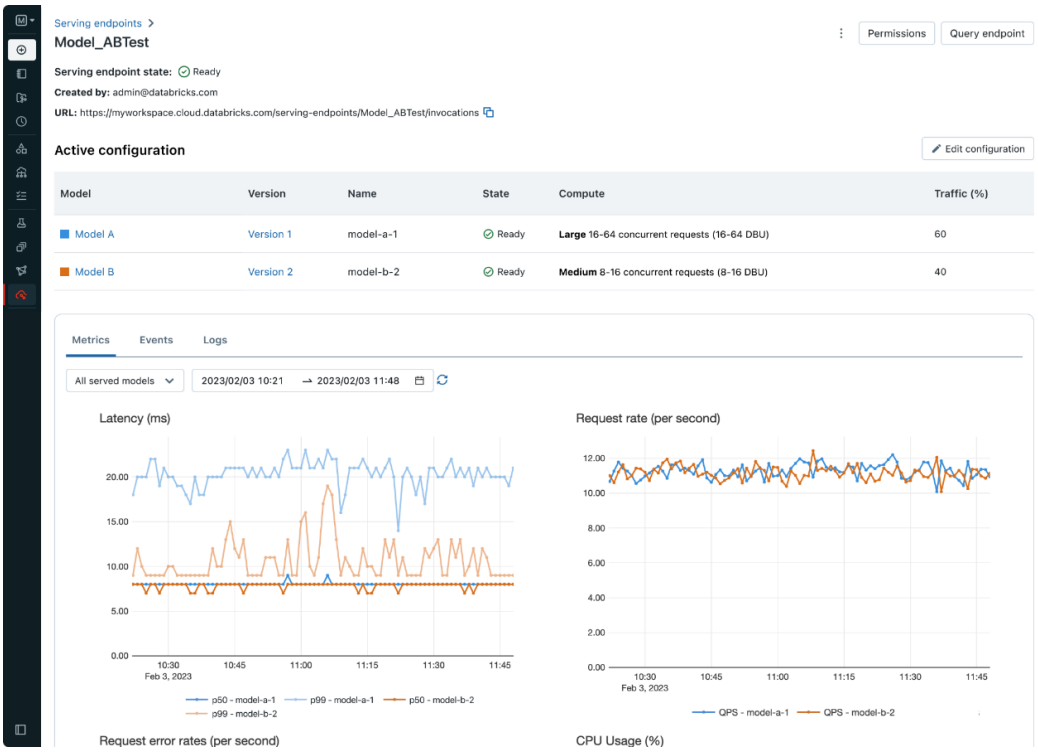


Figure 7.5 – Model serving gives us the flexibility to A/B test models deployed behind the same endpoint

Get ready to follow along in your own Databricks workspace as we review the *Chapter 7* code project by project.

Applying our learning

Let's use what we have learned to productionalize our models.

Technical requirements

Here are the technical requirements needed to complete the hands-on examples in this chapter:

- On-demand features require the use of DBR ML 13.1 or higher.
- RAG and CV parts require DBR ML 14.2 and higher
- Python UDFs are created and governed in UC; hence, Unity Catalog must be enabled for the workspace - no shared clusters.
- The Streaming Transactions project uses `scikit-learn==1.4.0rc1`. The notebooks that need it install it.
- The Streaming Transactions project, again, performs better with parallel compute. We use the multi-node cluster from *Chapter 5*. See *Figure 7.6* for the multi-node CPU configuration shown as follows:

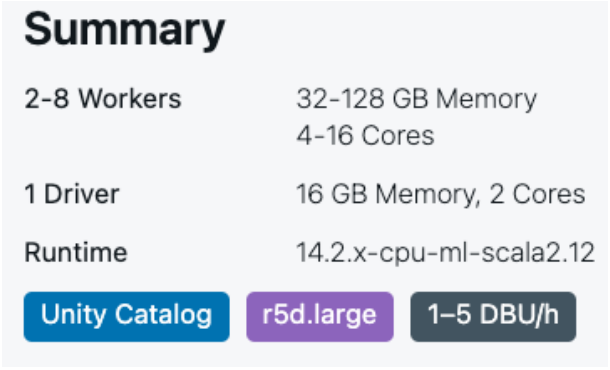


Figure 7.6 – Multi-node CPU cluster configuration (on AWS)

Project: Favorita Sales Forecasting

In this chapter, we discussed using Managed MLflow and the UCMR to register and prepare models for deployment. We start by walking through the UI, so please open the following notebooks and a tab with the **Experiments UI** page.

- `CH7-01-Registering the Model`
- `CH7-02-Batch Inference`

As a reminder, in *Chapter 6*, we ran experiments to find a baseline model, and you can review those experiments from the Experiments UI page in Databricks. To follow along in your own workspace, please open the Experiments UI page from *Chapter 6* as shown in *Figure 7.7*.

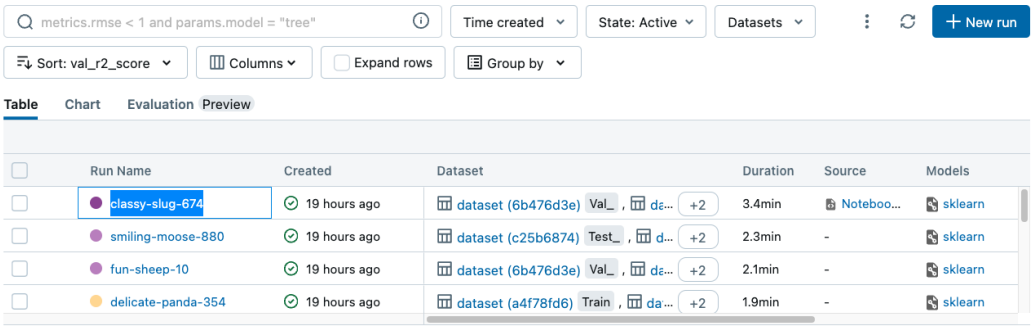


Figure 7.7 – Experiments UI page for exploring the experiment runs

For the sake of the project, we will move forward as though the best baseline model is the model we want in production. To productionalize the model, follow these steps:

1. In the AutoML experiment page, click on the best run (the run at the top when sorting by the evaluation metric in descending order). This will open the run details page. As you can see in *Figure 7.8*, there are four tabs: overview, model metrics, system metrics, and artifacts.

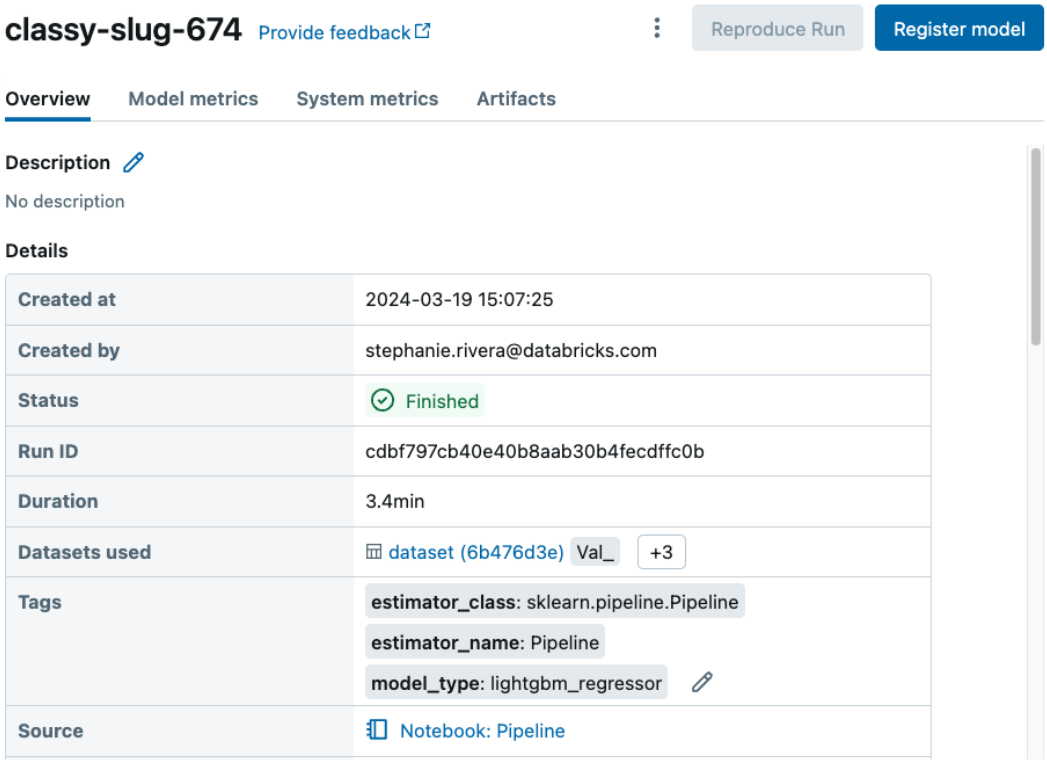


Figure 7.8 – Exploring the model details page

2. Click **Register model** in the top corner. This opens a dialogue box where you can register the model in the workspace or UC Model Registry. Of course, we want to use Unity Catalog. Selecting UC provides the code to register the model via API. This code is already included in the `CH7-01-Registering the Model` notebook. When running the notebook in your own workspace, you must update the `runs:/` URL in the notebook before executing it.

Figure 7.9 – Registering a new model

3. While still in your notebook, execute the final cell a second time. This creates a second version of the same model.
4. Navigate to the `favorita_forecasting` database in the Catalog. Selecting the **Models** tab opens a new UI, as shown in *Figure 7.10*.

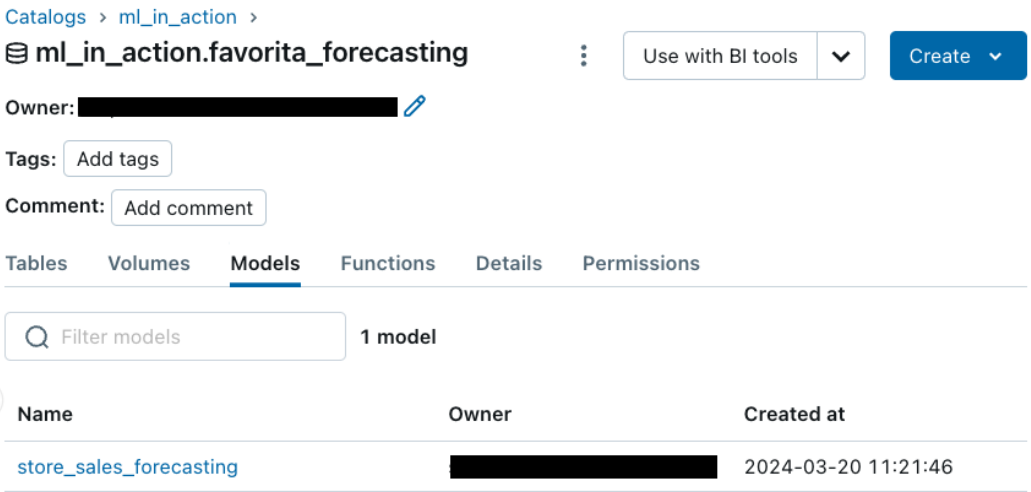


Figure 7.10 – View the model in the Catalog

5. Select the forecasting model. You'll notice that we have two versions of the model. Add an alias to each: **champion** and **challenger** are common to indicate the current best model and the newer version being considered to replace the current best model.

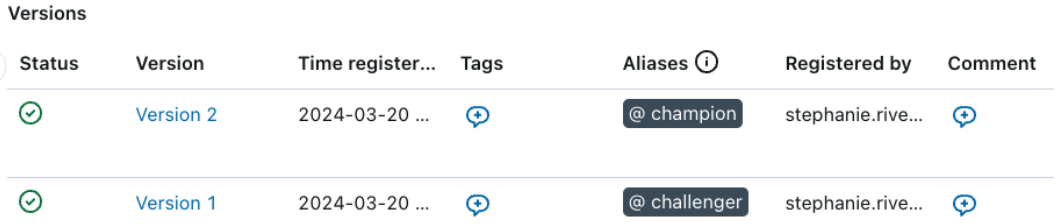


Figure 7.11 – Assign aliases to identify model status

Now it's time to think about how we want to deploy this model. Since this is a sales forecasting use case predicting 10+ days in advance, real-time inference doesn't make the most sense.

6. Open `CH7-02-Batch Inference` to run inference on the test set. Notice in *Figure 7.12* that we define our `model_uri` using the alias rather than the model version number.

```
import mlflow
from pyspark.sql.functions import struct, col
mlflow.set_registry_uri("databricks-uc")
model_name = "store_sales_forecasting"

logged_model = f"models:/{model_name}@Champion"

# Load model as a Spark UDF.
loaded_model = mlflow.pyfunc.spark_udf(spark, model_uri=logged_model)

df = sql("SELECT * FROM test_set")

# Predict on a Spark DataFrame.
df.withColumn('predictions', loaded_model(struct(*map(col, df.columns))))
```

Figure 7.12 – Batch forecasting on the test set using the Champion model

This inference code will always run inference on the data provided using the Champion model. If we later determine that a new version is better, we can change the model alias and run the correct model without making any code changes to the inference code.

The next practical step is to set up a workflow on a schedule. Please refer to the streaming project to see this demonstrated. This wraps up the end of this project. In *Chapter 8*, we will use the Favorita Sales data to show how easily an SQLbot can be created using the Foundational Model API.

Project: Streaming Transactions

We have much to do to wrap up the streaming transactions project! We will build our model, wrap it, validate it, and implement batch inference. To accomplish this, we begin ingesting the labels into a different table. This allows us to set up the inference table and merge the actual labels after the predictions happen. We will create two workflows: **Production Streaming Transactions** and **Production Batch Inference** and Model Retraining.

As with any project, we refine previously written code as we work toward production. The notebooks that look familiar from earlier in the book may have some minor updates, but most of the code remains unchanged.

Before we jump in, let's remember where we are and where we are going with a quick look at the project pipeline in *Figure 7.13*.

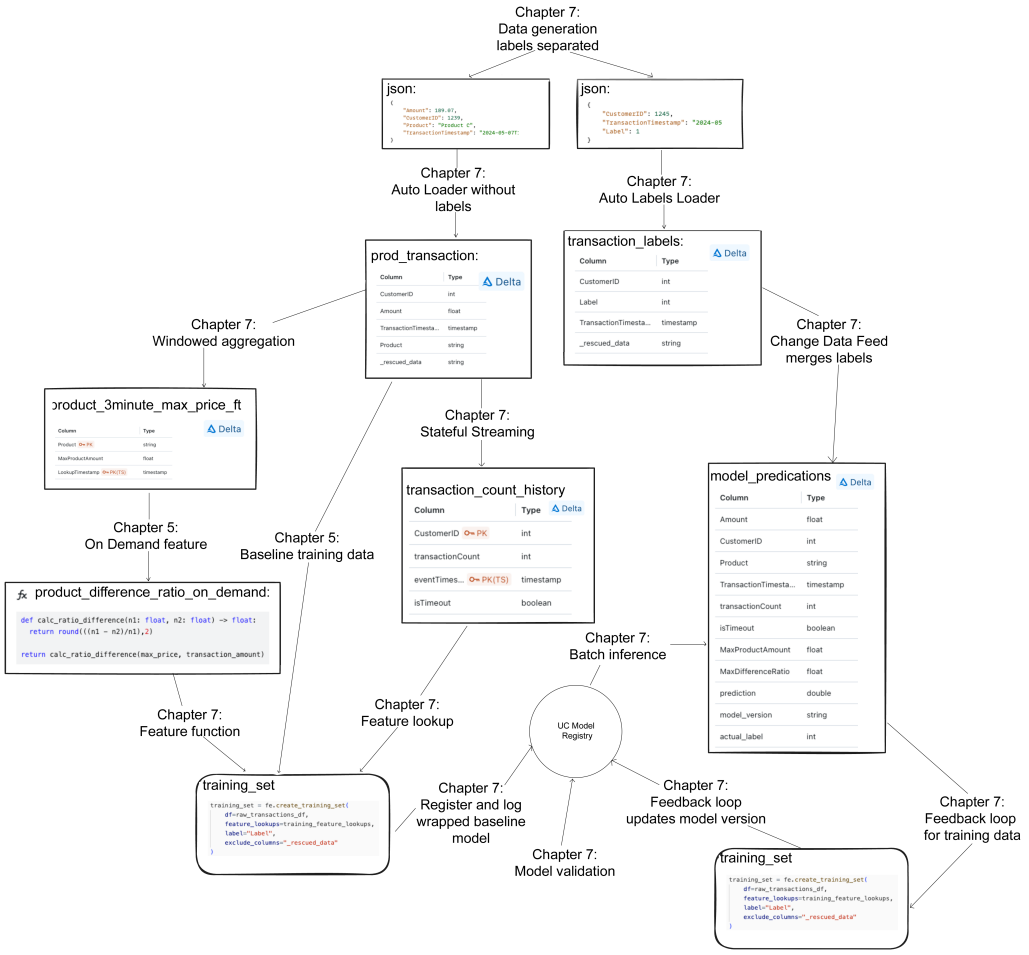


Figure 7.13 – The project pipeline for the Synthetic Streaming Transactions Project

To follow along in your own workspace, please open the following notebooks:

- `CH7-01-Generating Records`
- `CH7-02-Auto Loader`
- `CH7-03-Feature Engineering Streams`
- `CH7-04-Update Maximum Price Feature Table`
- `CH7-05-Wrapping and Logging the Baseline Model`
- `CH7-06-Wrapping and Logging the Production Model`
- `CH7-07-Model Validation`
- `CH7-08-Batch Inference`
- `CH7-09-Production Batch Inference`
- `CH7-10-Auto Label Loader`
- `mlia utils/transactions funcs.py`
- `production streams.yaml`
- `model retraining n inference workflow.yaml`

We recommend using a multi-node CPU cluster for this project. The first four notebooks (`CH7-01` through `CH7-04`) are nearly identical to their previous versions in *Chapter 6*, but the *Chapter 7* versions all point to the production rather than the development catalog. Table names are parameterized in widgets so they can be set in workflows. Here is a list of the essential notebook-specific changes that have been made to the first four notebooks:

- `CH7-01-Generating Records`: Labels and transactions are now being written to separate folders. The data generation components have also been moved to the `transactions_funcs` file in the utils folder.
- `CH7-02-Auto Loader`: The label column is no longer being added to the transactions table. For production, we ensure that the table being written to has `delta.enableChangeDataFeed = true` before the stream starts. If the table property is set after the stream starts, the stream is interrupted and will require a restart. Lastly, if the table property is never set, Lakehouse Monitoring is negatively impacted and will not support continuous monitoring.
- `CH7-03-Feature Engineering Streams`: Similar to the `CH7-02-Pipeline Auto Loader` notebook, table properties are set before any data is written.
- `CH7-04-Update Maximum Price Feature Table`: The code is cleaned up for a step toward production. Specifically, the feature table is updated rather than created once it exists.

You will need transaction data in the production catalog to create the model. Note the new setup variable `$env=prod`. The pipeline workflow notebooks: `CH7-01, CH7-02, and CH7-03` are ready to be added to a workflow in the jobs section of Databricks. Start by clicking the **Workflows** icon on the left-side menu bar. Next, click **Create job**. We have provided you with a `YAML` file, `production streams.yaml`, to guide you. Notice, in the `YAML` and *Figure 7.14*, the tasks do not depend upon one another.

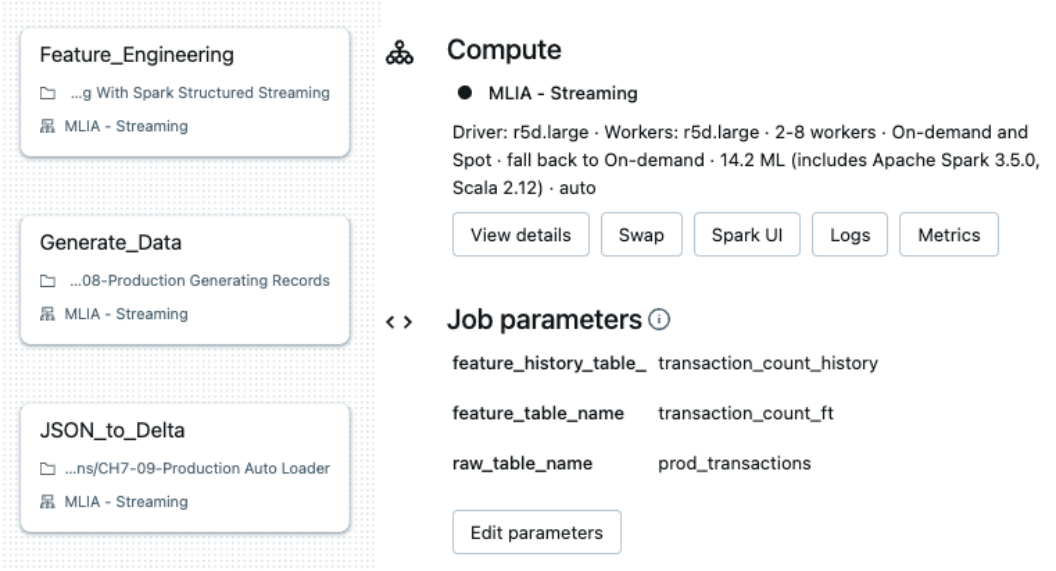


Figure 7.14 – The task DAG (L) and some settings (R) for the Production Streaming Transactions job

As shown in *Figure 7.14*, you can use the compute cluster shown in *Figure 7.6* for the job and the other notebooks. Also, we chose to add parameters at the job level rather than the individual tasks. You can now easily generate the data needed to build the model.

We will use the `training_set` created in *Chapter 6* for training a new version of the model. Recall that logging a model in Unity Catalog packages the feature metadata with the model. Hence, at inference time, it automatically looks up features from feature tables provided in the specified training set. We added this spec to both `Wrapping and Logging the Model` notebooks. We will not go through these notebooks separately. The difference between them is the baseline model is trained on data in the development catalog. Then the model is re-trained in production using production data from the inference table.

We know that we don't have an inference table yet. Don't worry—it's coming! Back to the `training_set`, being able to match up the features is only useful if we have feature values. We use the time boundaries of our feature table to ensure that the raw transactions being used for training have feature values. Also, we require the label column to be present, as shown in *Figure 7.15*.

```
ft_name = "product_3minute_max_price_ft"

if not spark.catalog.tableExists(ft_name) or spark.table(tableName=ft_name).isEmpty():
  raise Exception("problem")
else:
  min_time = sql(f"SELECT MIN(LookupTimestamp) FROM {ft_name}").collect()[0][0]
  max_time = sql(f"SELECT MAX(LookupTimestamp) FROM {ft_name}").collect()[0][0]
  raw_transactions_df = sql(f"""
    SELECT Amount,CustomerID,actual_label as Label,Product,TransactionTimestamp FROM {table_name}
    WHERE TransactionTimestamp >= '{min_time}' AND TransactionTimestamp <= '{max_time}' AND actual_label IS NOT NULL
    """)
```

Figure 7.15 – Defining the feature lookups for the inference model

The transactions in *Figure 7.15* come from the inference table, `packaged_transaction_model_predictions`, created in `CH7-08-Batch Inference`. The baseline model does something similar with the `raw_transactions` table. The baseline model also sets the model description, as shown in *Figure 7.16*.

```
import pandas as pd
import mlflow
mlflow.set_registry_uri("databricks-uc")

model_name = "packaged_transaction_model"
full_model_name = f'{catalog}.{database_name}.{model_name}'
model_description = "MLflow custom python function wrapper around a LightGBM model with embedded
pre-processing. The wrapper provides data preprocessing so that the model can be applied to input
dataframe directly without training/serving skew. This model serves to classify transactions as 0/1 for
learning purposes."

model_artifact_path = volume_model_path + _model_name
dbutils.fs.mkdirs(model_artifact_path)
```

Figure 7.16 – Setting the model description

We are now ready to focus on the model. Let's walk through the rest of the process.

1. Set the model registry to UC with the following line of code: `mlflow.set_registry_uri("databricks-uc")`.
2. Use pip to save a `requirements.txt` file.
3. Create a PyFunc Wrapper for Transaction Model `TransactionModelWrapper`.

Most of the code for the `TransactionModelWrapper` should look familiar to those who have created a model using Sklearn. The initialization function, `__init__(self, model, X, y, numeric_columns, cat_columns)`, accepts a model and DataFrames. The data preprocessing for training and inference data is standardized within the wrapper. The `TransactionModelWrapper` consists of four methods: `init`, `predict`, `preprocess_data`, `fit`.

The initialization method performs the following:

1. Splits the data into train and test sets.
2. Initializes and fits a `OneHotEncoder` for the categorical feature columns provided.
3. Initializes and fits a `StandardScaler` for the numerical feature columns provided.
4. Applies the `preprocess_data` method on the training and test data.
5. Defines an `evaluation` method to calculate the log loss on the X and y sets provided.
6. Invoke the `evaluation` method on preprocessed test data.
7. Defines and invokes the `_model_signature` method to easily provide the signature when logging the model.

The `predict` method calls the `preprocess_data` (*Figure 7.17)* method on the input DataFrame before performing and returning the prediction. This method is used for processing the training data and the inference data ensuring identical preprocessing for predictions.

```
@staticmethod
def preprocess_data(df, numeric_columns,fitted_scaler,cat_columns, encoder):
  if "TransactionTimestamp" in df.columns:
    try:
      df = df.drop("TransactionTimestamp",axis=1)
    except:
      df = df.drop("TransactionTimestamp")
  one_hot_encoded = encoder.transform(df[cat_columns])
  df = pd.concat([df,one_hot_encoded],axis=1).drop(columns=cat_columns)
  df["isTimeout"] = df["isTimeout"].astype('bool')
  ## scale the numeric columns with the pre-built scaler
  if len(numeric_columns):
    ndf = df[numeric_columns].copy()
    df[numeric_columns] = fitted_scaler.transform(ndf[numeric_columns])
  return df
```

Figure 7.17 – The preprocessing method for the model

As you can see in *Figure 7.17*, the fitted numeric scaler and one hot encoder are passed as input. This protects against skew between training and inference features. Notice how the Transaction timestamp is dropped. This is done because after the features from the feature tables are present, we no longer need a timestamp. The input DataFrame can be a Spark or Pandas DataFrame. This is the need for different syntax for dropping the timestamp column.

In the following command cell, you customize the `mlflow.autolog` and start the mlflow experiment for training, testing, wrapping, and logging the model. You will use `mlflow.evaluate` to handle the evaluation. The logging process is easy. You call `log_model` with the model name, wrapped model, `pyfunc` flavor, and previously created training set. This process also registers the model in UC. Last in this notebook is a quick test on the predict function showing how to pass in the Spark context with the input data. You are now ready to validate the model.

Next, focus on the `CH7-07-Model Validation` notebook, which checks to ensure the input model has the correct metadata to be ready for production. This notebook can be used to test any model. Ideally, you will add numerous checks, including the ability to predict and possibly the testing accuracy on specific slices of data. For example, you could check the model performance on each product or geography. You can pass those columns with tags when slices need testing.

Collect the model details as shown in *Figure 7.18*. Notice the use of the util function, which is imported in a previous cell: `from mlia_utils.mlflow_funcs` import `get_latest_model_version`.

```
model_details = mlfclient.get_registered_model(model_name)
model_version = str(get_latest_model_version(model_name))
model_version_details = mlfclient.get_model_version(name=model_name, version=model_version)
```

Figure 7.18 – Use MLflow Client and util functions to access the model details

Each time you train and log the model, a new model version is created. Using tags, you can indicate which model version(s) must be tested and validated before deployment.

```
assert 'validation_status' in model_version_details.tags.keys(), f"the model, model={model_name},
specfied does not have validation_status tag"
if model_version_details.tags['validation_status'] == 'passed_tests':
  dbutils.notebook.exit("No validation needed!")

assert model_version_details.tags['validation_status'] == 'needs_tested', f"the latest version,
version={model_version}, of model, model={model_name} is not tagged as validation_status=needs_tested"
```

Figure 7.19 – Assertions to be sure the model needs to be tested

As shown in *Figure 7.20*, you need an informative model description for all production models. We recommend you include information about the use case the model is used for. Metadata hygiene is becoming increasingly important as companies want to leverage Generative AI on internal data, as LLMs use the metadata fields to find relevant information in the models.

```
if model_details.description == "":
  print("No model description found. Please add a model description to pass this test.")
  validation_results['description'] = 0
elif len(model_details.description) < 15:
  print("Model description is too short. Perhaps include a note about the business impact.")
  validation_results['description'] = 0
else:
  validation_results['description'] = 1
```

Figure 7.20 – Validating a model description is present

Similar to *Figure 7.20*, the notebook checks for tags. These are examples to get you started. This is an ideal section to expand on the current code; and continue adding validation results and update tags, *Figure 7.21*.

```
  if sum(validation_results.values()) == len(validation_results.values()):
    mlfclient.set_model_version_tag(name=model_name, key="validation_status", value="passed_tests",
    version=model_version)
    print(f"Success! Your model, {model_name} version {model_version}, passed all tests.")
  else:
    mlfclient.set_model_version_tag(name=model_name, key="validation_status", value="failed_tests",
    version=model_version)
    print(f"Fail! Check the results to determine which test(s) the model failed. {validation_results}")
```

```
Success! Your model, ml_in_prod.synthetic_transactions.packaged_transaction_model version 8, passed all tests.
```

Figure 7.21 – Processing the results of the validation tests

With a tested model, you can progress to inference; see the `CH7-08-Batch Inference` notebook. Review the examples of performing batch inference on a DataFrame (*Figure 7.22*) and a JSON data string.

```
from databricks.feature_engineering import FeatureEngineeringClient
from mlia_utils.mlflow_funcs import get_latest_model_version
import mlflow
mlflow.set_registry_uri("databricks-uc")
fe = FeatureEngineeringClient()

scoring_df = sql("SELECT * FROM raw_transactions ORDER BY TransactionTimestamp DESC").drop("Label").limit(100)

print(f"Scoring model={model_name} version={get_latest_model_version(model_name)}")

scored = fe.score_batch(
  model_uri=f"models:/{model_name}/{get_latest_model_version(model_name)}",
  df=scoring_df,
  env_manager="conda"
)

display(scored)
```

Figure 7.22 – Load and score the model in batch on a DataFrame

Now look at the code in `CH7-09-Production Batch Inference`. The big changes include the `scoring_df`, the dataframe we apply our model to for predictions. Notice in *Figure 7.23* that the `min_time` and `max_time` variables provide boundaries for the transactions, ensuring the batch feature values have been calculated. Additionally, the inference table also provides a boundary preventing duplicate prediction calculations.

```
min_time = sql(f"SELECT MIN(LookupTimestamp) FROM {ft_name}").collect()[0][0]
max_time = sql(f"SELECT MAX(LookupTimestamp) FROM {ft_name}").collect()[0][0]

if not spark.catalog.tableExists(inference_table) or spark.table(tableName=inference_table).isEmpty
():
  scoring_df = sql(f"""
                   SELECT Amount,CustomerID,Product,TransactionTimestamp FROM {table_name}
                   WHERE TransactionTimestamp <= '{max_time}' AND TransactionTimestamp >= '
                   {min_time}'
                   """)
  sql(f"""
      CREATE TABLE IF NOT EXISTS {inference_table}
      (Amount FLOAT, CustomerID INT, Product STRING, TransactionTimestamp TIMESTAMP,
      transactionCount INT, isTimeout BOOLEAN, MaxProductAmount FLOAT, MaxDifferenceRatio FLOAT,
      prediction DOUBLE, model_version STRING, actual_label INT) USING delta TBLPROPERTIES (delta.
      enableChangeDataFeed = true)
  """)
else:
  last_inf_time = sql(f"SELECT MAX(TransactionTimestamp) FROM {inference_table}").collect()[0][0]
  scoring_df = sql(f"""
                   SELECT Amount,CustomerID,Product,TransactionTimestamp FROM {table_name}
                   WHERE TransactionTimestamp <= '{max_time}' AND TransactionTimestamp >= '
                   {min_time}'
                   AND TransactionTimestamp > '{last_inf_time}'
                   """)
```

Figure 7.23 – The scoring_df query configuration

The inference table in `CH7-08` needs to be updated to fit the requirements for the inference table monitoring provided by Lakehouse Monitoring. This means adding the `model_version` and `actual_label` columns. The `actual_label` column is set to NULL so that it is clear the value has not been updated yet; see *Figure 7.24*.

```
scored = fe.score_batch(
  model_uri=f"models:/{model_name}/{model_version}",
  df=scoring_df
)
scored.withColumn("model_version",lit(model_version))\
  .withColumn(colName="actual_label",col=lit(None))\
  .write.mode('append').format('delta').saveAsTable(inference_table)
```

Figure 7.24 – The addition of the model version and actual label columns

These two additional columns to the inference table are requirements for the Lakehouse Monitor. The `InferenceLog` monitor comes with autogenerated dashboards. However, you need to populate the table. Begin by creating a bronze table for the transaction labels. The Auto Loader is back again, focusing on labels; see `CH7-10-Auto Label Loader` and *Figure 7.25*. In the notebook, you see the creation of the `transaction_labels` table which is very similar to code from *Chapter 3*. In *Figure 7.25*, you use the CDF and CDC Delta features to update the new inference table with the ground truth label.

```
def merge_stream(df, i):
  df.createOrReplaceTempView("labels_cdc_microbatch")

  df._jdf.sparkSession()\
    .sql(f"""MERGE INTO {inference_table} target
        USING
        (SELECT CustomerID, TransactionTimestamp, Label as actual_label
        FROM labels_cdc_microbatch) as source
        ON source.CustomerID = target.CustomerID
        AND source.TransactionTimestamp = target.TransactionTimestamp
        WHEN MATCHED THEN UPDATE SET target.actual_label == source.actual_label
        """)

if spark.catalog.tableExists(inference_table):
  (stream.writeStream
    .foreachBatch(merge_stream)
    .option("checkpointLocation", volume_file_path+inference_table+"/checkpoint_cdc")
    .trigger(availableNow=True)
    .start()
  )
```

Figure 7.25 – The merging of transaction labels into the inference table

You now have a bronze table and the ability to merge new labels into the inference table. However, the inference table is still empty. Let's move forward to create a workflow job, as shown in *Figure 7.26*, to generate predictions every 15 minutes.

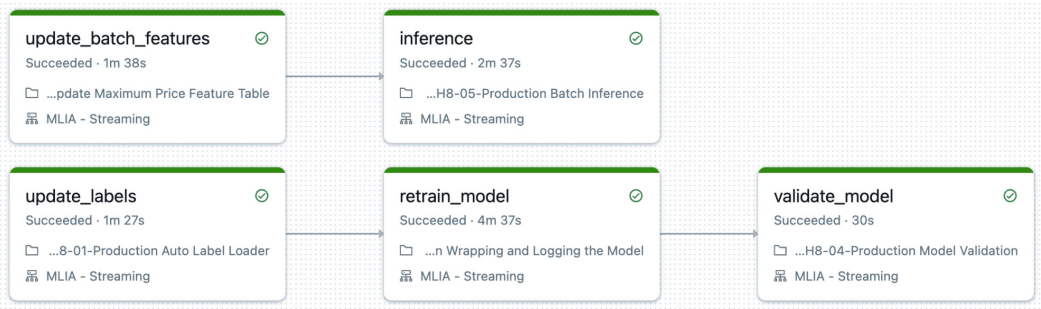


Figure 7.26 – The DAG for the inference (upper) and model retraining (lower) job

Walk through the two workflow paths, starting with the upper path. The batch features are updated, thus providing feature data for inference. The data is ready for predictions, and the inference task can begin.

The lower path has the updating of labels as the first task. It adds the latest data to the `transaction_labels` table and merges all new labels that match previous predictions into the inference table. Skip forward beyond the first iteration, the inference table contains not only previous predictions but also the labels for those predictions. The model training is performed using the updated table containing the features. The retraining of the model only occurs if there is data in the inference table, as shown in *Figure 7.27*. The retraining is, of course, followed by validation when needed. The validation notebook exits when it detects that the latest version of the model has already been tested.

```
if not spark.catalog.tableExists(table_name) or spark.table(tableName=table_name).isEmpty():
  dbutils.notebook.exit("No inference table exists yet")
```

Figure 7.27 - The retraining notebook exits if there is no data to retrain on

Create the workflow job shown in *Figure 7.29*. You can reference the configuration for the job in the `model retraining n inference workflow.yaml` file. The workflow automatically provides the lineage of all upstream and downstream tables. You can see these in *Figure 7.26*. This saves us time on documentation.

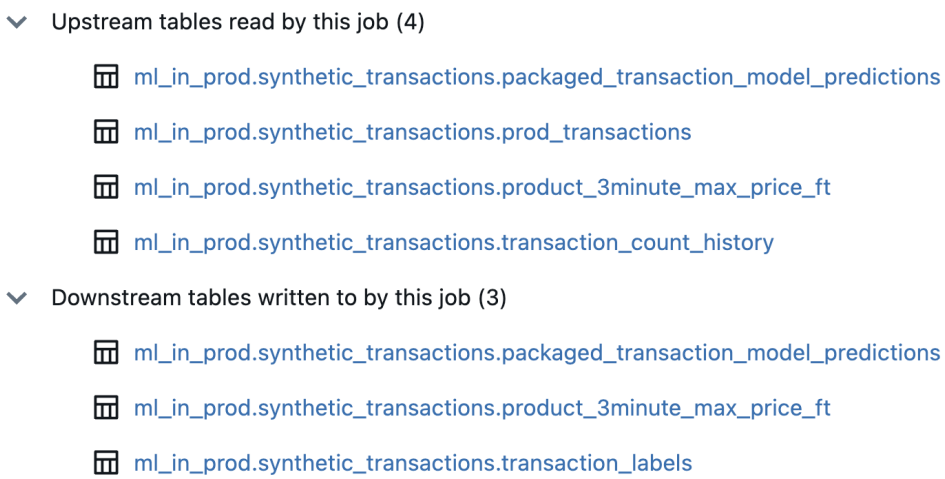


Figure 7.28 – The workflow table lineage

All that is left is to run both workflows simultaneously. After letting both run for a bit (don't forget to schedule the Production Batch Inference and Model Retraining workflow), you should have a screen that looks very similar to *Figure 7.29*.

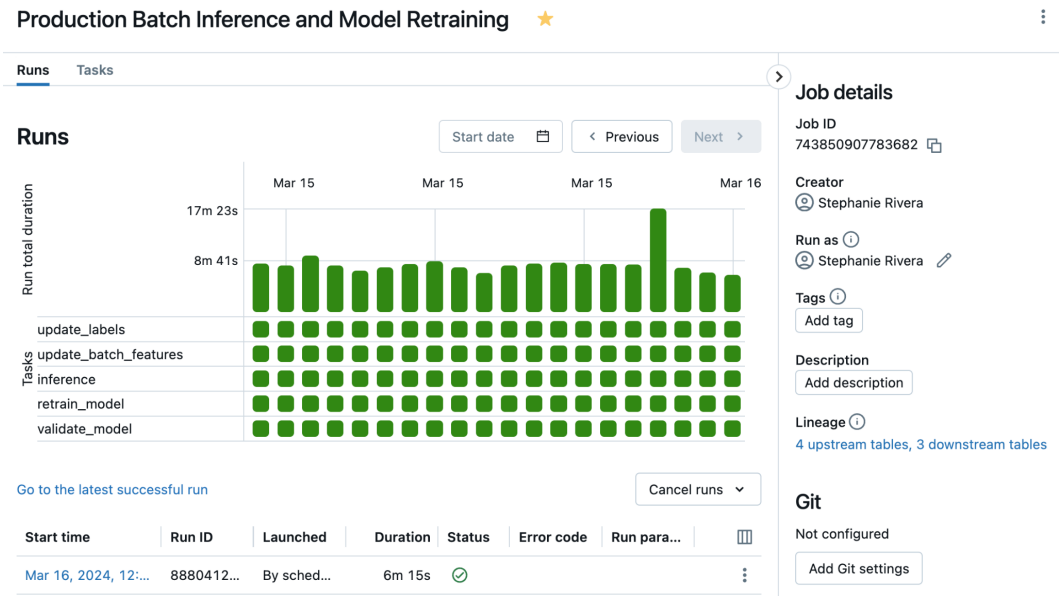


Figure 7.29 – The historical view of successful job runs

You now have all of the pieces for productionizing this project.

Project: Multilabel Image Classification

We currently have a working Image Classification model that we trained and evaluated in *Chapter 6*. Now let's add some infrastructure around our code to serve our model and make it available to downstream applications and, ultimately, our end users. To follow along in your own workspace, please open the following notebooks:

- `Ch7-01-Create Final Wrapper Production`
- `Ch7-02-Serve In Production`

We start out by creating our model class `wrapper`. This wrapper includes two functions, `feature_extractor` and `predict`. The `feature_extractor` function is required because otherwise, our fine-tuned model would not contain the same preprocessing step used during the fine-tuning and would, therefore, not be consistent during serving. Of course, you can simply serve your original model if you do not require any custom modifications to your model and only need the raw format outputs.

```
from io import BytesIO
import torchvision

class CVModelWrapper(mlflow.pyfunc.PythonModel):
    def __init__(self, model):
        # instantiate model in evaluation mode
        model.to(torch.device("cpu"))
        self.model = model.eval()

    def feature_extractor(self, image, p=0.5):…

    def predict(self, context, images):…
```

Figure 7.30 – Creating the model class wrapper

The `feature_extractor` function, which transforms an image into the format required by the served model, is the same code we used to score the model in *Chapter 6*. Let's dive into the prediction part, it's a similar function to what we have created in *Chapter 6* to score our model.

The `predict` function is similar to the one we used to score our model in *Chapter 6* using `pandas_udf`. Note that we are not returning only the predicted label but also a label corresponding to it in a dictionary format (this isn't required, but we wanted to show the output format's flexibility).

```
class CVModelWrapper(mlflow.pyfunc.PythonModel):
    def __init__(self, model):
        # instantiate model in evaluation mode
        model.to(torch.device("cpu"))
        self.model = model.eval()

    def feature_extractor(self, image, p=0.5):…

    def predict(self, context, images):
        id2label = {
            0: 'buildings',
            1: 'forest',
            2: 'glacier',
            3: 'mountain',
            4: 'sea',
            5: 'street'
            }
        with torch.set_grad_enabled(False):

          # assuming this is a DataFrame
          pil_images = torch.stack([self.feature_extractor(row[0]) for _, row in images.iterrows()])
          pil_images = pil_images.to(torch.device("gpu"))
          outputs = self.model(pil_images)
          preds = torch.max(outputs, 1)[1]
          probs = torch.nn.functional.softmax(outputs, dim=-1)[:, 1]
          labels = [id2label[pred] for pred in preds.tolist()]

          return pd.DataFrame(
                                data=dict(
                                  label=preds,
                                  labelName=labels
                                  )
                              )
```

Figure 7.31 – Including feature_extractor and predict functions in the model class wrapper

Now, we are ready to wrap our fine-tuned model from *Chapter 4* into the wrapper. To do this, we load the artifact back from MLflow and pass it to the pre-created class `CVModelWrapper` above.

```
import os
import mlflow
from mlflow.store.artifact.models_artifact_repo import ModelsArtifactRepository
from mlia_utils.cv_clf_funcs import select_best_model

experiment_path = f"/Users/{current_user}/intel-clf-training_action"
local_path = select_best_model(experiment_path)

requirements_path = os.path.join(local_path, "requirements.txt")
if not os.path.exists(requirements_path):
  dbutils.fs.put("file:" + requirements_path, "", True)

device = "cuda" if torch.cuda.is_available() else "cpu"
loaded_model = torch.load(local_path+"/data/model.pth", map_location=torch.device(device))

wrapper = CVModelWrapper(loaded_model)
```

Figure 7.32 – Load our model from the existing MLFlow experiment

Let's test whether our wrapper is functioning as expected. To do so, we encode a few images (as the model serving accepts strings and cannot yet accept images and convert them) and save them into a Pandas DataFrame. We then use our model wrapper to get a prediction.

```
import base64
import pandas as pd

images = spark.read.format("delta").load(val_delta_path).take(25)

b64image1 = base64.b64encode(images[0]["content"]).decode("ascii")
b64image2 = base64.b64encode(images[1]["content"]).decode("ascii")
b64image3 = base64.b64encode(images[3]["content"]).decode("ascii")
b64image4 = base64.b64encode(images[4]["content"]).decode("ascii")
b64image24 = base64.b64encode(images[24]["content"]).decode("ascii")

df_input = pd.DataFrame(
    [b64image1, b64image2, b64image3, b64image4, b64image24], columns=["data"])

df = wrapper.predict("", df_input)
display(df)
```

Figure 7.33 – Using our model wrapper to create predictions on a few images

Next, we will use MLflow to log and serve the model via Databricks Model Serving.

```
import mlflow
# Set the registry URI to "databricks-uc" to configure
# the MLflow client to access models in UC
mlflow.set_registry_uri("databricks-uc")
model_name = f"{catalog}.{database_name}.cvops_model_mlaction"

from mlflow.models.signature import infer_signature,set_signature
img = df_input['data']
predict_sample = df[['label', 'labelName']]
# To register models under UC you require to log signature for both
# input and output
signature = infer_signature(img, predict_sample)

print(f"Your signature is: \n {signature}")

with mlflow.start_run(run_name=model_name) as run:
    mlflowModel = mlflow.pyfunc.log_model(
        artifact_path="model",
        python_model=wrapper,
        input_example=df_input,
        signature=signature,
        registered_model_name=model_name,
    )
##Alternatively log your model and register later
# mlflow.register_model(model_uri, "ap.cv_ops.cvops_model")
```

Figure 7.34 – Logging and running models using MLflow

During the production phase, you will usually operate on the Aliases and the latest model version, so here we simulate setting the alias "Champion" to the best-performing model and getting the latest model version to be deployed.

```
from mlflow import MlflowClient
import mlflow

mlflow.set_registry_uri('databricks-uc')
client = MlflowClient()
model_name = f'{catalog}.{database_name}.cvops_model_mlaction'

client.set_registered_model_alias(model_name, "Champion", 1)

model_version_uri = f"models:/{model_name}@Champion"
# Or another option: model_version_uri = f"models:/{model_name}/1"
loaded_model_uc = mlflow.pyfunc.load_model(model_version_uri) # runiid / model_registry_name+version
# champion_version = client.get_model_version_by_alias("prod.ml_team.iris_model", "Champion")
latest_model = client.get_model_version_by_alias(model_name, "Champion")
```

Figure 7.35 – Loading our champion model

Next, we create our model serving endpoint using the Databricks SDK. You could also create your endpoint using the UI. If you decide to use SDK, you must create a configuration file for your endpoint. The example below is for a CPU container with a small workload size. If you are unfamiliar with this option, please check out *Create model serving endpoints* in the *Further reading* section.

```
import mlflow
from mlia_utils.mlflow_funcs import *

# Create or update serving endpoint
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import EndpointCoreConfigInput, ServedModelInput

serving_endpoint_name = "cvops_model_mlaction_endpoint"
latest_model_version = get_latest_model_version(model_name)


w = WorkspaceClient(host=db_host, token = db_token)

endpoint_config = EndpointCoreConfigInput(
    name=serving_endpoint_name,
    served_models=[
        ServedModelInput(
            model_name=model_name,
            model_version=latest_model_version,
            workload_size="Small",
            scale_to_zero_enabled=True
        )
    ]
)
```

Figure 7.36 – Setting config input for the endpoint to serve our model

Once the settings are provided, you are ready to deploy or to update your endpoint in the event that it already exists.

```
import os
serving_endpoint_url = f"{db_host}/ml/endpoints/{serving_endpoint_name}"

try:
  w.serving_endpoints.get(serving_endpoint_name)
  print(f"Updating the endpoint {serving_endpoint_url} to version {latest_model_version}, this will take a few minutes to package and
  deploy the endpoint...")
  w.serving_endpoints.update_config_and_wait(served_models=endpoint_config.served_models, name=serving_endpoint_name)
except:
  print(f"Creating the endpoint {serving_endpoint_url}, this will take a few minutes to package and deploy the endpoint...")
  w.serving_endpoints.create_and_wait(name=serving_endpoint_name, config=endpoint_config)

displayHTML(f'Your Model Endpoint Serving is now available. Open the <a href="/ml/endpoints/{serving_endpoint_name}">Model Serving Endpoint
page</a> for more details.')
```

Your Model Endpoint Serving is now available. Open the Model Serving Endpoint page for more details.

Figure 7.37 – Deploying a new endpoint if does not exist or updating the existing one

Keep in mind that if your endpoint did not exist, it will take a bit of time to deploy.

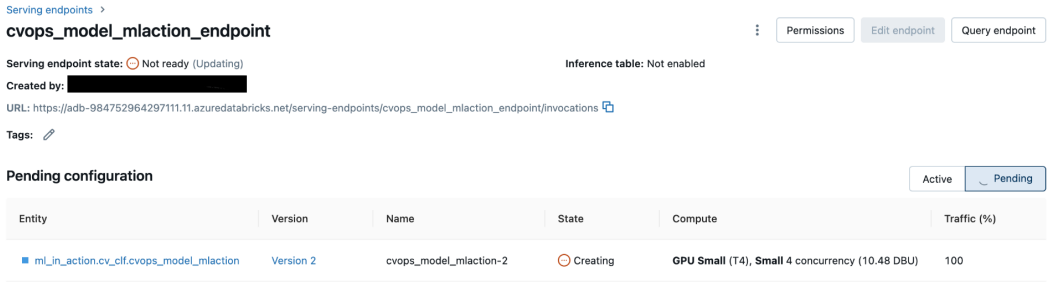


Figure 7.38 – UI example, while the GPU endpoint is deploying/updating

Now we can score our model, and again, for simplicity and reusability, we are covering the serving call into a function `score_model`.

```
def create_tf_serving_json(data):
  return {'inputs': {name: data[name].tolist() for name in data.keys()} if isinstance(data, dict) else data.tolist()}

def score_model(dataset):
  url = f"{db_host}/serving-endpoints/{serving_endpoint_name}/invocations"
  headers = {'Authorization': f'Bearer {db_token}', 'Content-Type': 'application/json'}
  ds_dict = {'dataframe_split': dataset.to_dict(orient='split')} if isinstance(dataset, pd.DataFrame) else create_tf_serving_json(dataset)
  data_json = json.dumps(ds_dict, allow_nan=True)
  response = requests.request(method='POST', headers=headers, url=url, data=data_json)
  if response.status_code != 200:
    raise Exception(f'Request failed with status {response.status_code}, {response.text}')
  return response.json()
```

Figure 7.39 – Defining a model scoring function

Lastly, we scored our model using our model scoring function, as shown in *Figure 7.40*.

```
score_model(df_input)
```

Figure 7.40 – Scoring our model

Here is an example of scoring under the UI page.

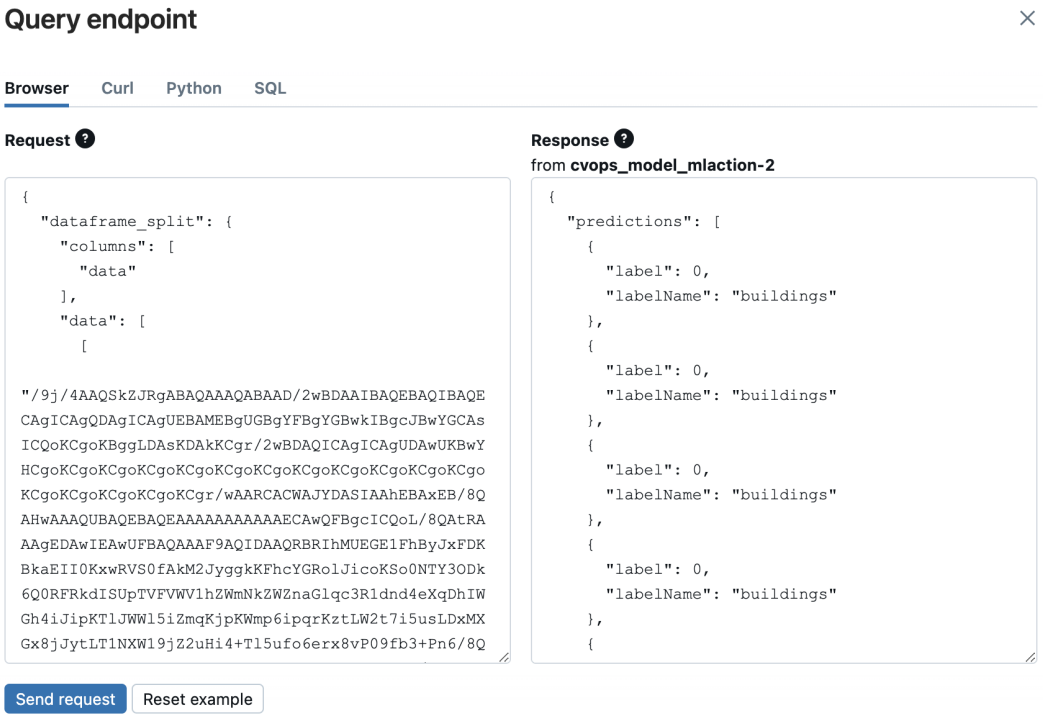


Figure 7.41 – Scoring our model under UI of the Model Serving

Now that our model is ready for production, we can train and create it, designate our champion model, serve it on an endpoint, and score. The next steps might include setting up a monitor to track image pixel distributions, the number of images, label distributions, and the distribution of the response. We will talk more about Model Monitoring in *Chapter 8*.

Project: Retrieval Augmented Generation Chatbot

In the last chapter, we completed our chatbot and tested it out in a notebook. It might be tempting to jump to the final deployment step immediately, but that would be skipping a critical step in the process - evaluation! Evaluating projects for correctness before providing them to the end users should always be part of the development process. However, it can be especially tricky to build automated evaluation solutions for newer technologies and techniques, such as when working with LLMs. This is a continually developing area of research, and we recommend reading Databricks' recommendations on *Best Practices for LLM Evaluation* for more information (linked in *Further Reading*). We'll walk through the MLflow evaluation capability to evaluate the RAG chatbot we've built.

To follow along in your own workspace, please open the following notebooks:

- `CH7-01-GetData`
- `CH7-02-Evaluating ChatBot`

Loading the ground truth labels

We start in the first notebook, `CH7-01-GetData`. To evaluate our model, we must have ground truth labels, a.k.a. the correct answers. This generally involves human effort to write out some typical questions you expect users to ask your chatbot and the answers you expect your chatbot to respond with. To simplify this process, we created a file with thirty-five sample questions and the corresponding human-generated answers for this project, saved to `Questions_Evaluation.csv`. Let's load this file and examine the question and answer pairs.

```
import pandas as pd

df_evaluation = pd.read_csv("/Volumes/ml_in_action/rag_chatbot/files/Questions_Evaluation.csv").dropna().
drop(columns=["id_question"])
df_eval = spark.createDataFrame(df_evaluation)
display(df_eval)
```

Figure 7.42 – Reading our pre-created evaluation set

Take a look at some of the records to get a sense of the questions a user might ask and the expected answers. You can also add your own examples to augment the evaluation data further.

question_asked	answer_given
How does the choice between rules and standards impact content moderation on social media platforms?	Rules can provide clear guidelines for content moderators to follow, reducing the need for human involvement an potentially lowering error costs. However, rules may not be effective in addressing complex or dynamic issues, su hate speech, and may require frequent updates to remain effective. Standards, on the other hand, can provide fle for content moderators to adjust their policies to specific contexts, but may require more human involvement and potentially increase error costs d...
What are large language models (LLMs) and how do they differ from other generative AI models?	Large language models (LLMs) are artificial intelligence models that can process and produce various forms of se data, including but not limited to natural language. They differ from other generative AI models as they can be tra using a range of architectures and are not limited to transformer-based models. LLMs can generate a wide range sequences, such as assembly language, protein sequences, and chess games.

Figure 7.43 – Question-answer pair examples

Let's save these examples to a Delta table named `evaluation_table`. That way, if a new `Question_Evaluation.csv` file with different examples is uploaded, you can append the new examples to the existing table without risking losing the original data.

```
%sql
--Note that we need to enable Change Data Feed on the table to create the index
CREATE TABLE IF NOT EXISTS evaluation_table (
  id_question BIGINT GENERATED BY DEFAULT AS IDENTITY,
  question_asked STRING,
  answer_given STRING,
  pdf_name STRING,
  article_name STRING
  ) TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

Figure 7.44 – Creating evaluation_table to store the question/answer pairs

We save the table and are now ready to evaluate our model.

```
df_eval.write.mode("append").saveAsTable(f"{catalog}.{database_name}.evaluation_table")
```

Figure 7.45 – Saving questions_evaluation.csv data to a Delta table

Setting up the evaluation workflow

We're now ready to open the second notebook, `CH7-02-Evaluating ChatBot`, and evaluate our chatbot against the ground truth labels we saved to `evaluation_table`. Let's briefly discuss how we will compare our model's outputs to the human-generated answers. While there is plenty of ongoing research in the realm of automated LLM evaluation methods, we will focus on one technique in particular: *LLM-as-a-judge*. This method brings in a second large language model to evaluate the performance of the first, automating the tedious task of comparing the generated answer to the true answer that a human would traditionally have to do.

To use an LLM as a judge, we first load the original model we created in *Chapter 6*.

```
import mlflow
from mlia_utils.mlflow_funcs import *
from pyspark.sql.functions import col, udf, length, pandas_udf, explode

os.environ['DATABRICKS_TOKEN'] = dbutils.secrets.get("mlaction", "rag_sp_token")
model_name = f"{catalog}.{database_name}.mlaction_chatbot_model"

model_version_to_evaluate = get_latest_model_version(model_name)
mlflow.set_registry_uri("databricks-uc")
rag_model = mlflow.langchain.load_model(f"models:/{model_name}/{model_version_to_evaluate}")
```

Figure 7.46 – Loading the mlaction_chatbot_model as rag_model

We run a quick test to verify that our RAG model works as expected.

```
question = "What is GPT ?"
dialog = {"query": question}
rag_model.invoke({"query":"What is GPT? "})["result"]
```

```
'\nGenerative Pre-trained Transformers (GPTs) are a type of artificial intelligence model that
uses machine learning to generate human-like text based on input data. They are a subset of a
broader category of technologies known as general-purpose technologies (GPTs), which have the
potential to significantly impact a wide range of industries and applications. GPTs are capabl
e of performing a variety of tasks, including text generation, translation, summarization, and
question answering. They are trained on large amounts of text data and can generate coherent a
nd contextually relevant responses to a wide range of prompts. GPTs have the potential to sign
ificantly impact the labor market by automating certain tasks and potentially creating new typ
es of work.'
```

Figure 7.47 – Verifying our loaded model works as expected

Next, we use our RAG chatbot to generate answers for all of the example questions we stored in the `evaluation_table`. These responses are what we will compare against the ground truth answers. We build a `pandas_udf` to make this part run faster.

```
import mlflow
from mlia_utils.mlflow_funcs import *
from pyspark.sql.functions import col, udf, length, pandas_udf, explode

os.environ['DATABRICKS_TOKEN'] = dbutils.secrets.get("mlaction", "rag_sp_token")
model_name = f"{catalog}.{database_name}.mlaction_chatbot_model"

model_version_to_evaluate = get_latest_model_version(model_name)
mlflow.set_registry_uri("databricks-uc")
rag_model = mlflow.langchain.load_model(f"models:/{model_name}/
{model_version_to_evaluate}")

@pandas_udf("string")
def predict_answer(questions):
    def answer_question(question):
        dialog = {"query": question}
        return rag_model.invoke(dialog)['result']
    return questions.apply(answer_question)
```

Figure 7.48 – Creating a function to receive a question and return an answer using our RAG model

We use Llama-2-70B for our judge, but you could use GPT4 or any other LLM you prefer (though we cannot guarantee good results!). Our code leverages the Databricks Foundational Model API, which we also used in *Chapter 3* when creating the embeddings of *ArXiv* article text chunks. As a reminder, the Databricks Foundation Model APIs provide direct access to state-of-the-art open models from a serving endpoint, allowing you to incorporate high-quality generative AI models into your application without the need to maintain your own model deployment. We call the Llama-2-70b endpoint in *Figure 7.49*.

```
from mlflow.deployments import get_deploy_client
deploy_client = get_deploy_client("databricks")
endpoint_name = "databricks-llama-2-70b-chat"
#Let's query our external model endpoint
answer_test = deploy_client.predict(endpoint=endpoint_name, inputs={"messages":
[{"role": "user", "content": "What is GPT?"}]})
answer_test['choices'][0]['message']['content']display(spark.table
('evaluation_dataset'))
```

Figure 7.49 – Testing the Foundation Model endpoint for Llama-2-70b

We build a DataFrame from the `evaluation_table` questions and answers. If you have added many more question/answer examples to this dataset, you may want to downsample the number of questions and speed up the prediction process. We then call our UDF `predict_answer`.

```
df_qa = (spark.read.table('evaluation_table')
                .selectExpr('question_asked as inputs', 'answer_given as targets')
                .where("targets is not null")
                #.sample(fraction=0.005, seed=40) # if your dataset is very big you
                could sample it
                )

df_qa_with_preds = df_qa.withColumn('preds', predict_answer(col('inputs'))).cache()
```

Figure 7.50 – Building a DataFrame with questions and ground truth answers, and adding RAG chatbot answers

Now that we have our DataFrame with the chatbot's responses to each question, we save this to a Delta table as well. We continue to reference the DataFrame throughout the rest of this code, but this way we won't have to generate the chatbot's responses again if we want to query this data in the future.

```
df_qa_with_preds.write.saveAsTable(f"{catalog}.{database_name}.
evaluation_table_preds")
```

Figure 7.51 – Writing our evaluation DataFrame to a new table

As we mentioned at the beginning of this section, we are using MLflow's Evaluate capability to facilitate our model evaluation. Before we evaluate our RAG chatbot, let's load the methods and verify how MLflow Evaluate works by default. We first load the "answer correctness" metric, which we will use as-is.

```python
from mlflow.metrics.genai.metric_definitions import answer_correctness
from mlflow.metrics.genai import make_genai_metric, EvaluationExample

# Because we have our labels (answers) within the evaluation dataset, we can evaluate the answer correctness as part
of our metric. Again, this is optional.
answer_correctness_metrics = answer_correctness(model=f"endpoints:/{endpoint_name}")
print(answer_correctness_metrics)
```

```
Example Input:
How is MLflow related to Databricks?

Example Output:
MLflow is a product created by Databricks to enhance the efficiency of machine learning processes.

Additional information used by the model:
key: targets
value:
MLflow is an open-source platform for managing the end-to-end machine learning (ML) lifecycle. It was developed by Databricks, a
company that specializes in big data and machine learning solutions. MLflow is designed to address the challenges that data scie
ntists and machine learning engineers face when developing, training, and deploying machine learning models.

Example score: 4
Example justification: The output provided by the model is mostly correct. It correctly identifies that MLflow is a product crea
ted by Databricks. However, it does not mention that MLflow is an open-source platform for managing the end-to-end machine learn
ing lifecycle, which is a significant part of its function. Therefore, while the output is mostly accurate, it has a minor omiss
ion, which is why it gets a score of 4 according to the grading rubric.
```

Figure 7.52 – Viewing the answer correctness metrics

The out-of-the-box metrics are good, but professionalism is also an important criterion for our use case, so we create a custom professionalism metric. *Figure 7.53* shows how you can use the `make_genai_metric()` function to build out our professionalism evaluation metric.

```python
professionalism = make_genai_metric(
    name="professionalism",
    definition=(
        "Professionalism refers to the use of a formal, respectful, and appropriate style of communication that is "
        "tailored to the context and audience. It often involves avoiding overly casual language, slang, or "
        "colloquialisms, and instead using clear, concise, and respectful language."
    ),
    grading_prompt=(
        "Professionalism: If the answer is written using a professional tone, below are the details for different scores: "
        "- Score 1: Language is extremely casual, informal, and may include slang or colloquialisms. Not suitable for "
        "professional contexts."
        "- Score 2: Language is casual but generally respectful and avoids strong informality or slang. Acceptable in "
        "some informal professional settings."
        "- Score 3: Language is overall formal but still have casual words/phrases. Borderline for professional contexts."
        "- Score 4: Language is balanced and avoids extreme informality or formality. Suitable for most professional contexts. "
        "- Score 5: Language is noticeably formal, respectful, and avoids casual elements. Appropriate for formal "
        "business or academic settings. "
    ),
    model=f"endpoints:/{endpoint_name}",
    parameters={"temperature": 0.0},
    aggregations=["mean", "variance"],
    examples=[professionalism_example],
    greater_is_better=True
)
```

Figure 7.53 – Adding a custom professionalism metric

As you can see from the `grading_prompt`, we've designed this metric to give a score between one and five, where a score of one identifies text as casual and a five evaluates text as noticeably formal. This is a powerful tool to evaluate your model based on criteria important to your business use case. You can modify the template according to your needs. We also add in examples of the metric, defined below in *Figure 7.54*.

```
# Adding custom professionalism metric
professionalism_example = EvaluationExample(
    input="What is MLflow?",
    output=(
        "MLflow is like your friendly neighborhood toolkit for managing your machine
        learning projects. It helps "
        "you track experiments, package your code and models, and collaborate with
        your team, making the whole ML "
        "workflow smoother. It's like your Swiss Army knife for machine learning!"
    ),
    score=2,
    justification=(
        "The response is written in a casual tone. It uses contractions, filler words
        such as 'like', and "
        "exclamation points, which make it sound less professional. "
    )
)
```

Figure 7.54 – Adding an example for the custom professionalism metric

With our professionalism metric, let's run the model evaluation using MLfLow. To run an evaluation, we call `mlflow.evaluate()` against the Pandas DataFrame containing the questions, ground truth answers, and chatbot-generated answers. We include the answer correctness and professionalism metrics as extra metrics. The code below will calculate many other metrics in addition to the two we specified, such as token count, toxicity, and Automated Readability Index grade (the approximate grade level required to comprehend the text).

```
from mlflow.deployments import set_deployments_target

set_deployments_target("databricks")

#This will automatically log all
with mlflow.start_run(run_name="chatbot_rag") as run:
    eval_results = mlflow.evaluate(data = df_qa_with_preds.toPandas(), # evaluation
    data,
                                   model_type="question-answering", # toxicity and
                                   token_count will be evaluated
                                   predictions="preds", # prediction column_name from
                                   eval_df
                                   targets = "targets",
                                   extra_metrics=[answer_correctness_metrics,
                                   professionalism])

eval_results.metrics
```

Figure 7.55 – Running an MLflow experiment with mlflow.evaluate()

Once we've run the experiment, the metrics can be accessed in our notebook and via the UI to easily see how our chatbot is performing in terms of accuracy and professionalism.

Evaluating the chatbot responses

Let's first look at the MLfLow UI in Databricks to compare results between the human-generated and chatbot-generated responses. To do so, navigate to the **Experiments** page and open the experiment matching your notebook name (in this case, `CH7-02-Evaluating Chatbot`). Then navigate to the **Evaluation** tab. We're using this view to see some examples of how closely the bot's answers match what we would expect to see, but it is also very useful when you want to test and compare outputs across different models or chunking strategies.

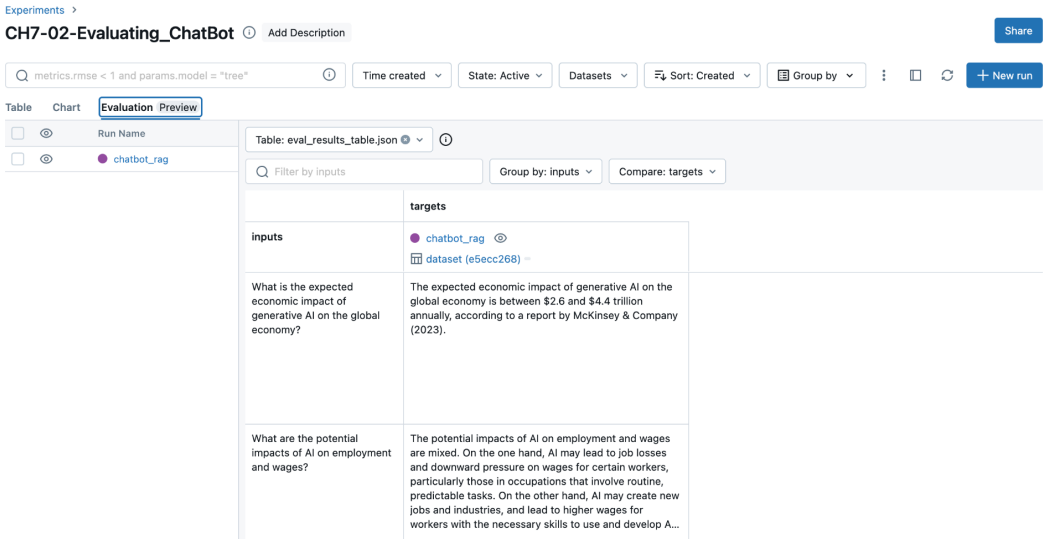


Figure 7.56 – Using the Evaluation view in Databricks MLflow

Take a look at a few examples from the evaluation dataset. We see that our model, according to a quick human assessment, is performing reasonably well. Of course, this form of evaluation is not scalable, so let's dig into the other metrics as well. We use the common visualization library `plotly` to take a closer look at our model results. First, we look at the distribution of token counts in our chatbot's responses.

```
import plotly.express as px
px.histogram(df_genai_metrics, x="token_count", labels={"token_count": "Token Count"}, title="Distribution of Token Counts in Model Responses")
```

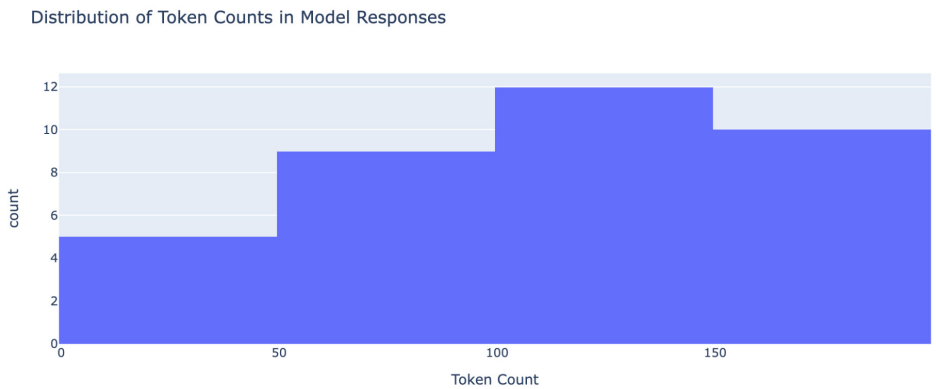


Figure 7.57 – Plotting token counts from our chatbot's responses

While interesting, what we really care about are the two metrics discussed earlier: correctness and professionalism. Take a look at the distribution of the correctness scores below.

```
# Counting the occurrences of each answer correctness score
px.bar(df_genai_metrics['answer_correctness/v1/score'].value_counts(), title='Answer Correctness Score Distribution')
```

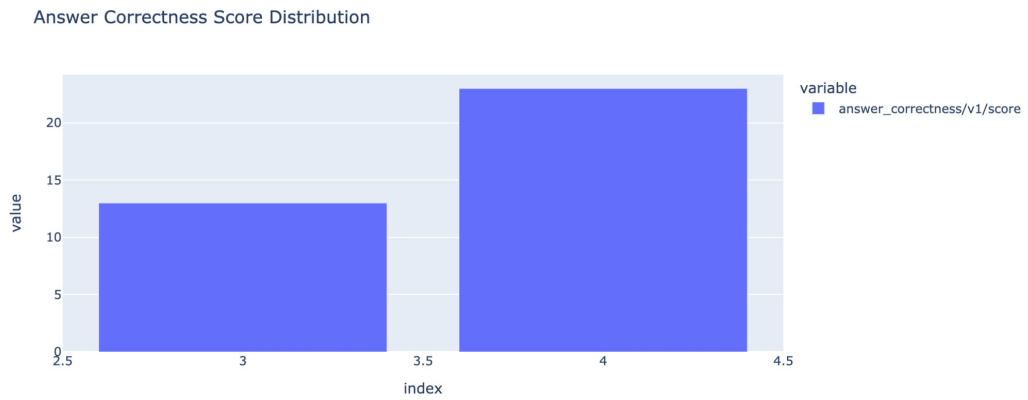


Figure 7.58 – Plotting the correctness score distribution

Let's also view the professionalism score distribution. Our distribution is threes and fours, which means the tone is most often "borderline professional," as we defined it in our custom metric.

```
# Counting the occurrences of each answer correctness score
px.bar(df_genai_metrics['professionalism/v1/score'].value_counts(), title='Answer Professionalism Score Distribution')
```

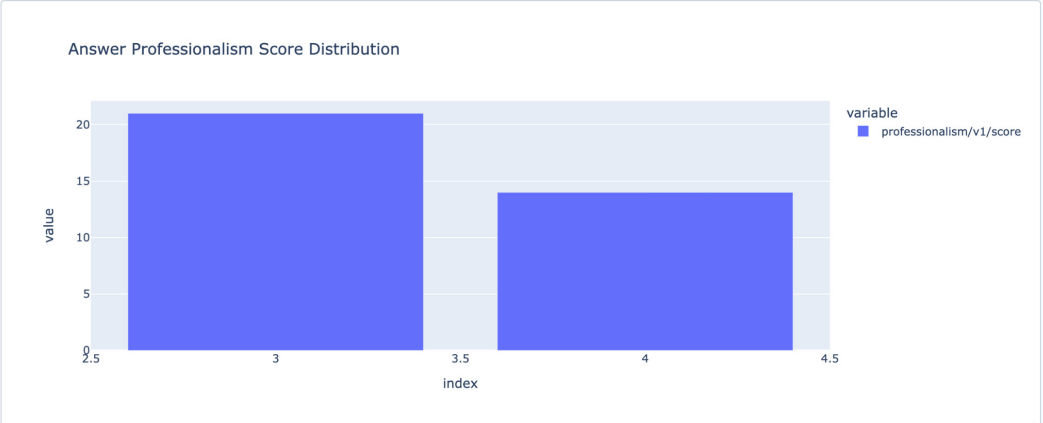


Figure 7.59 – Plotting the professionalism score distribution

Overall, our model is looking good! If we're satisfied with the accuracy and professionalism scores, we mark this model as ready for production by giving it a production alias.

```
client = MlflowClient()
client.set_registered_model_alias(name=model_name, alias="Production",
version=model_version_to_evaluate)
```

Figure 7.60 – Aliasing our model to show it is production-ready

We evaluated our chatbot by creating predictions from our question-and-answer dataset, created a custom evaluation metric to evaluate the professionalism of each response, and visualized information about our model outputs. In the final chapter, we will demonstrate how to build a Gradio app so that you can bring your RAG chatbot to your end users!

Summary

Implementing models in production can be challenging, but with tools designed to support productionizing models and automating the MLOps process, it is much easier. In this chapter, we looked at using the Unity Catalog Model Registry to manage the lifecycle of a machine learning model. We showcased MLflow and how it can be used to create reproducible, modularized data science workflows that automatically track parameters and performance metrics. We discussed techniques for calculating features at the time of inference. To make the end-to-end MLOps process more manageable, we showed how to use workflows and webhooks to automate the ML Lifecycle. We also showed how to serve models and make inferences using the MLFlow and the Databricks Platform.

In the final chapter, *Monitoring, Evaluating, and More*, we will look at monitoring our data and machine learning models within the Databricks Lakehouse so you can get the most value from your data.

Questions

Let's test ourselves on what we've learned by going through the following questions.

1. Can more than one model be in production simultaneously? When would you want to use two models in production?
2. What component of MLflow could you use to route approvals?
3. Can you use an MLflow API to serve your model?

Answers

After putting thought into the questions, compare your answers to ours.

1. Yes, having multiple models in production simultaneously is possible, and this is appropriate for use cases like comparing models in a challenger/champion test or running A/B tests.
2. The Unity Catalog model registry can be used to route approvals.
3. The model serving API within MLFlow can be used for model serving.

Further reading

This chapter discussed tools and technologies to help productionize machine learning. Please take a look at these resources to go deeper into the areas that interest you most- and help you get more of your machine learning projects into production!

- *Best Practices for Using Structured Streaming in Production - The Databricks Blog*: `https://www.databricks.com/blog/streaming-production-collected-best-practices`
- *The big book of machine learning use cases*: `https://www.databricks.com/resources/ebook/big-book-of-machine-learning-use-cases`
- *Databricks Model Serving*: `https://www.databricks.com/blog/2023/03/07/announcing-general-availability-databricks-model-serving.html`
- *Create and manage serving endpoints using Mlflow*: `https://docs.databricks.com/en/machine-learning/model-serving/create-serving-endpoints-mlflow.html`
- *Model Evaluation in MLFLow*: `https://www.databricks.com/blog/2022/04/19/model-evaluation-in-mlflow.html`
- *The big book of MLOps*: `https://www.databricks.com/resources/ebook/the-big-book-of-mlops`
- *Databricks Asset Bundles - Programmatically define, deploy, and run Databricks jobs, Delta Live Tables pipelines, and MLOps Stacks using CI/CD best practices and workflows*: `https://docs.databricks.com/en/dev-tools/bundles/index.html`
- *Model Registry Webhooks*: *MLflow Model Registry Webhooks on Databricks*
- *Webhooks*: `https://docs.databricks.com/en/mlflow/model-registry-webhooks.html`

- *CI/CD workflows with Git and Databricks Repos - Use GitHub and Databricks Repos for source control and CI/CD workflows*: https://docs.databricks.com/en/repos/ci-cd-techniques-with-repos.html
- *Continuous integration and delivery using GitHub Actions - Build a CI/CD workflow on GitHub that uses GitHub Actions developed for Databricks*: https://docs.databricks.com/en/dev-tools/ci-cd/ci-cd-github.html
- *CI/CD with Jenkins on Databricks – Develop a CI/CD pipeline for Databricks that uses Jenkins*: https://docs.databricks.com/en/dev-tools/ci-cd/ci-cd-jenkins.html
- *Orchestrate Databricks jobs with Apache Airflow – Manage and schedule a data pipeline that uses Apache Airflow*: https://docs.databricks.com/en/workflows/jobs/how-to/use-airflow-with-jobs.html
- *Service principals for CI/CD – Use service principals instead of users with CI/CD systems*: https://docs.databricks.com/en/dev-tools/ci-cd/ci-cd-sp.html
- *DFE client*: https://docs.databricks.com/en/machine-learning/feature-store/python-api.html#use-the-clients-for-unit-testing
- *Unity*: https://docs.databricks.com/en/udf/unity-catalog.html
- *Best Practices for LLM Evaluation of RAG Applications, Part 1*: https://www.databricks.com/blog/LLM-auto-eval-best-practices-RAG
- *Best Practices for LLM Evaluation of RAG Applications, Part 2*: https://www.databricks.com/blog/announcing-mlflow-28-llm-judge-metrics-and-best-practices-llm-evaluation-rag-applications-part
- *Create model serving endpoints*: https://docs.databricks.com/en/machine-learning/model-serving/create-manage-serving-endpoints.html

8

Monitoring, Evaluating, and More

"Focus on how the end-user customers perceive the impact of your innovation – rather than on how you, the innovators, perceive it." — Thomas A. Edison

Congratulations, you've made it to the final chapter! We've come a long way, yet there is still more to explore in Databricks. As we wrap up, we will take another look at Lakehouse Monitoring. We'll focus on monitoring model inference data. After all the work you've put in to build a robust model and push it into production, it's essential to share the learnings, predictions, and other outcomes with a broad audience. Sharing results with dashboards is very common. We will cover how to create visualizations for dashboards in both the new Lakeview dashboards and the standard Databricks SQL dashboards. Deployed models can be shared via a web application. Therefore, we will not only introduce Hugging Face Spaces but also deploy the RAG chatbot using the Gradio app in *Applying our learning*. Lastly, we'll demonstrate how analysts can invoke LLMs via SQL AI Functions! By the end of this chapter, you will be ready to monitor inference data, create visualizations, deploy an ML web app, and use the groundbreaking DBRX open source LLM with SQL.

Here is the roadmap for this chapter:

- Monitoring your models
- Building gold layer visualizations
- Connecting your applications
- Incorporating LLMs for analysts
- Applying our learning

Monitoring your models

The ML lifecycle does not end at deployment. Once a model is in production, we want to monitor the input data and output results of the model. In *Chapter 4*, we explored two key features of Databricks Lakehouse Monitoring integrated with Unity Catalog: Snapshot and TimeSeries profiles. Snapshot profiles are designed to provide an overview of a dataset at a specific point in time, capturing its current state. This is particularly useful for identifying immediate data quality issues or changes. On the other hand, TimeSeries profiles focus on how data evolves over time, making them ideal for tracking trends, patterns, and gradual changes in data distributions.

Expanding on these capabilities, Databricks also provides an Inference profile, tailored for monitoring machine learning models in production. This advanced profile builds upon the concept of TimeSeries profiles, adding critical functionalities for comprehensive model performance evaluation. It includes model quality metrics, essential for tracking the accuracy and reliability of predictions over time. It also records predictions and, optionally, ground truth labels, directly comparing expected and actual outcomes. This functionality is key for identifying model drift, where shifts in input data or the relationship between inputs and outputs occur.

Inference Tables in Databricks further bolster this monitoring capability. They contain essential elements such as model predictions, input features, timestamps, and potentially ground truth labels. Building a monitor on top of InferenceTables with the corresponding `InferenceLog` allows us to monitor model performance and data drift continuously.

In the event of drift detection, immediate actions should be taken – data pipeline verification or model retraining and evaluation are recommended. These steps ensure the model adapts to new data patterns, maintaining accuracy and effectiveness. Continuous monitoring against your baseline and cross-model versions is a strategy to adopt when trying to ensure a stable process across various deployed solutions.

Figure 8.1 is a code sample for creating an Inference monitor with model quality metrics using the `InferenceLog` profile type. This illustrates a practical application of this monitoring setup. We specify the `schedule` argument to make sure that this monitor is refreshed hourly.

```
lm.create_monitor(
    table_name=f"{catalog}.{database_name}.packaged_transaction_model_predictions",
    profile_type=lm.InferenceLog(
        timestamp_col="TransactionTimestamp",
        granularities=["5 minutes","30 minutes","1 hour"],
        model_id_col="model_version",
        prediction_col="prediction",
        label_col="actual_label",
        problem_type="classification"
    ),
    slicing_exprs=["Product","CustomerID"],
    schedule=lm.MonitorCronSchedule(
        quartz_cron_expression="0 0 * * * ?", # schedules a refresh every hour
        timezone_id="MST",
    ),
    output_schema_name=f"{catalog}.{database_name}"
)
```

Figure 8.1 – Creating an Inference profile monitor that refreshes hourly

Model monitoring is an effective way to ensure your models are working for you as expected. We hope this gets you thinking about how you use monitoring in your MLOPs process.

Next, we'll learn about ways to create dashboards.

Building gold layer visualizations

The gold layer in your lakehouse is the consumption-ready layer. In this layer, final transformations and aggregations crystallize the insights within your data so it is ready for reporting and dashboarding. Being able to share your data with an audience is critical, and there are several options for doing so in the DI Platform. In fact, both Lakeview and Databricks SQL dashboards allow you to transform and aggregate your data within visualizations. Let's walk through how to do that.

Leveraging Lakeview dashboards

Lakeview dashboards in Databricks are a powerful tool for creating data visualizations and sharing insights hidden in data. Visualizations can be made in the English language, making dashboard creation available to more users. To create a Lakeview dashboard, first click **Dashboards** in the left-hand navigation bar, and then open the **Lakeview Dashboards** tab. You will see a **Create Lakeview dashboard** button at the top right of the screen. Clicking this will open a canvas to create your dashboard. The canvas is where the visualization magic happens. It includes an enhanced visualization library to bring your data to life and makes it easy to add widgets for interactive filtering. The **Data** tab, shown in *Figure 8.2*, is where you can define your dataset with SQL or simply by selecting a table.

For example, you could select the `ml_in_action.favorita_forecasting.train_set` table. This creates a dataset by selecting all records from the provided table. Notice how we do not *have to* write any SQL or create aggregates in order to visualize data aggregations.

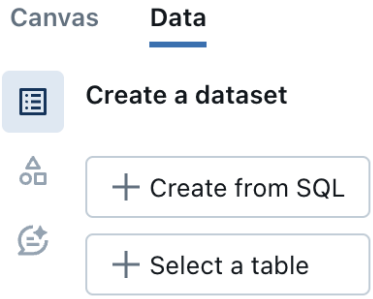


Figure 8.2 – The Data tab for adding data to your Lakeview dashboard

Once you have a dataset, return to the **Canvas** tab. Select the **Add a visualization** button found on the blue bar toward the bottom of your browser window. This gives you a widget to place on your dashboard. Once placed, your widget will look similar to *Figure 8.3*.

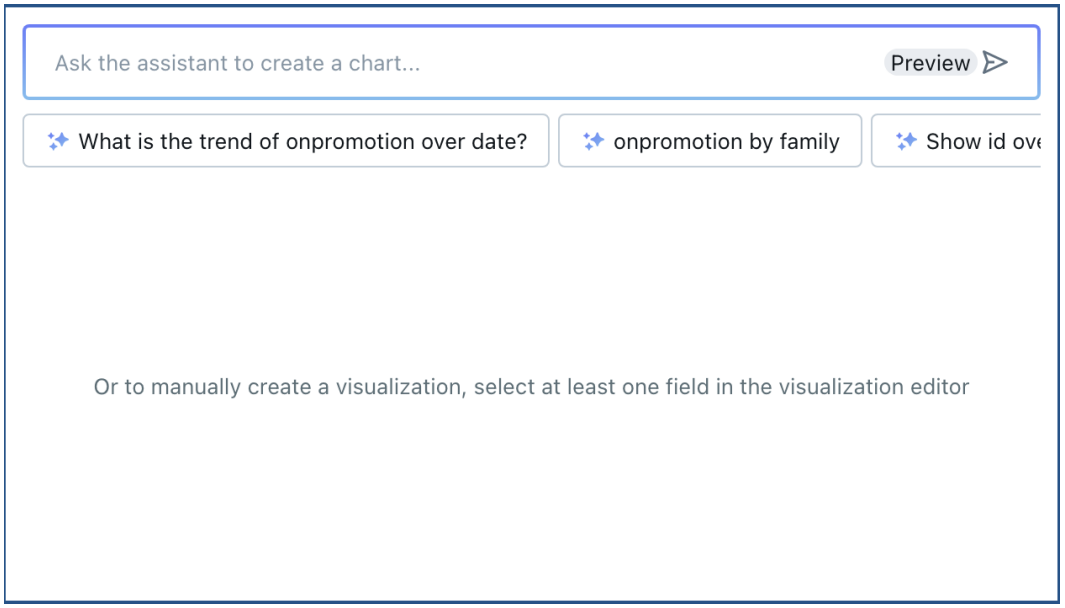


Figure 8.3 – A new Lakeview widget

In the new widget, you can manually create a visualization using the options on the right-side menu. Alternatively, Databricks Assistant can help you rapidly build a chart using just English. You can write your own question, or explore the suggested queries. We selected the suggested question *What is the trend of onpromotion over date?* to automatically generate a chart, and *Figure 8.4* is the result.

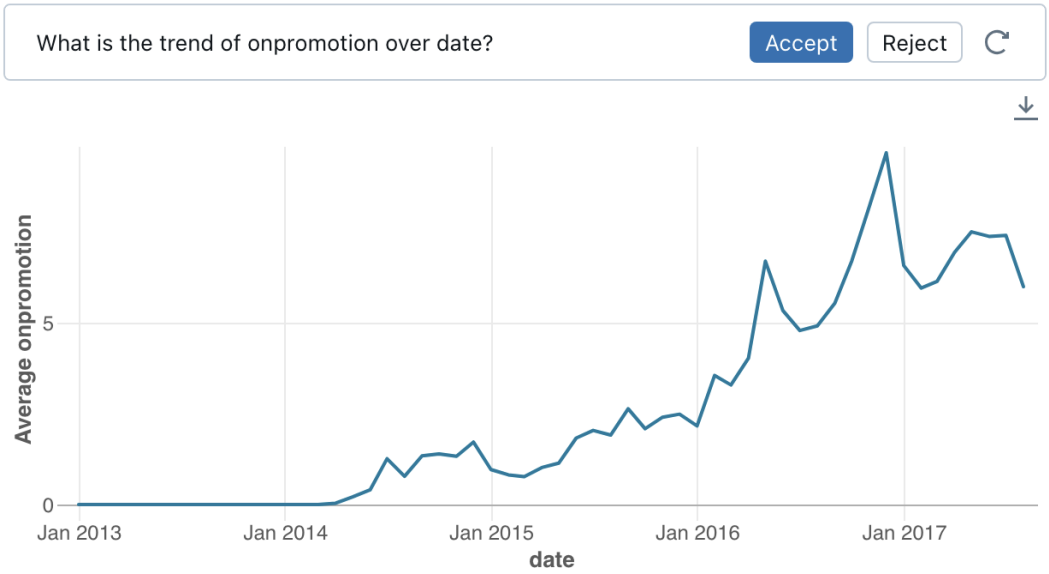


Figure 8.4 – English text generated Lakeview widget

When you are ready to share your dashboard, you can publish it! The engine powering Lakeview dashboards is optimized for performance, driving faster interactive charts for your data. It's also powerful enough to handle streaming data. Furthermore, Lakeview dashboards are unified with the DI Platform through Unity Catalog, providing data lineage. They are designed for easy sharing across workspaces, meaning users in other workspaces can access your curated dashboard.

Visualizing big data with Databricks SQL dashboards

Lakeview dashboards are the future of Databricks. However, you can also build dashboards with **Databricks SQL** (**DBSQL**). The SQL editor built into the DI Platform is another way to quickly create visualizations. With a single `select` statement, you can produce the data and use it for multiple visualizations.

> **Note**
>
> To recreate *Figure 8.5* in your own workspace, you will want to uncheck the **LIMIT 1000** box. There is still a limit for the visualizations of 64,000 rows. The best way to get around this is by filtering or aggregating.

Figure 8.5 is an example visualization we created from a simple SQL query against the *Favorita Store Sales* data.

```
select * from lakehouse_in_action.favorita_forecasting.training_set
```

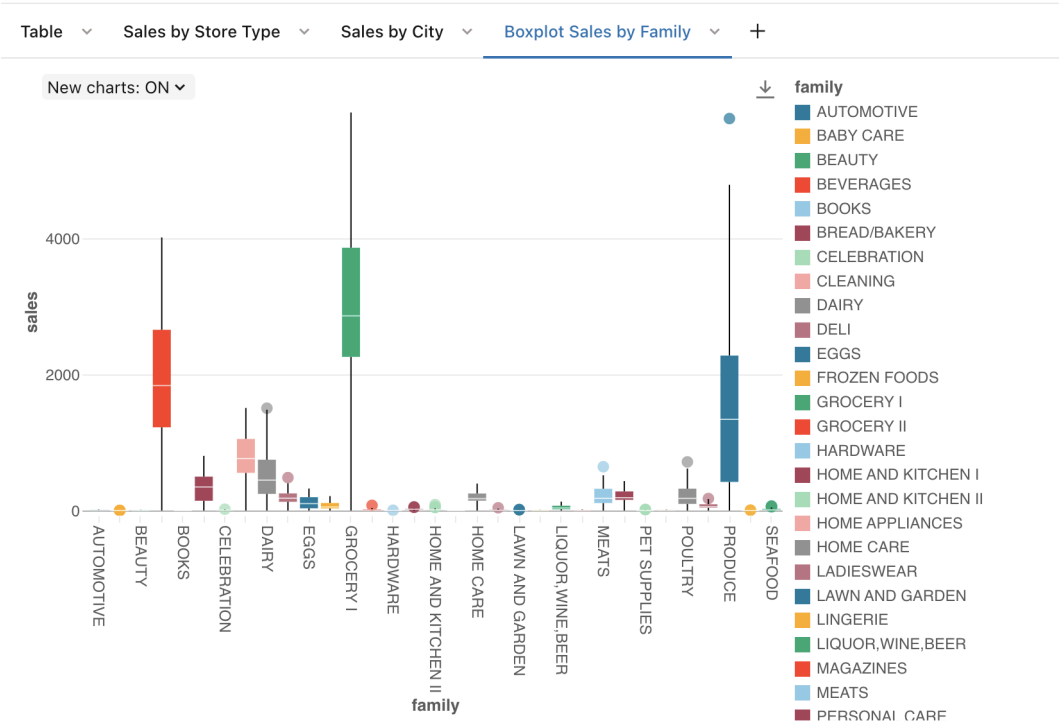


Figure 8.5 – After executing the select statement in the DBSQL editor, we create the visualizations without writing any code

Suppose your dataset has categorical variables that you want to use to filter and compare features, as with the *Favorita* sales data. You can add filters within the DBSQL editor without revising the query. To add a filter, click the **+** and choose either **filter** or **parameter**. Both options provide widgets for filtering, as shown in *Figure 8.6*. You can use the widgets with any visualizations or dashboards associated with the query.

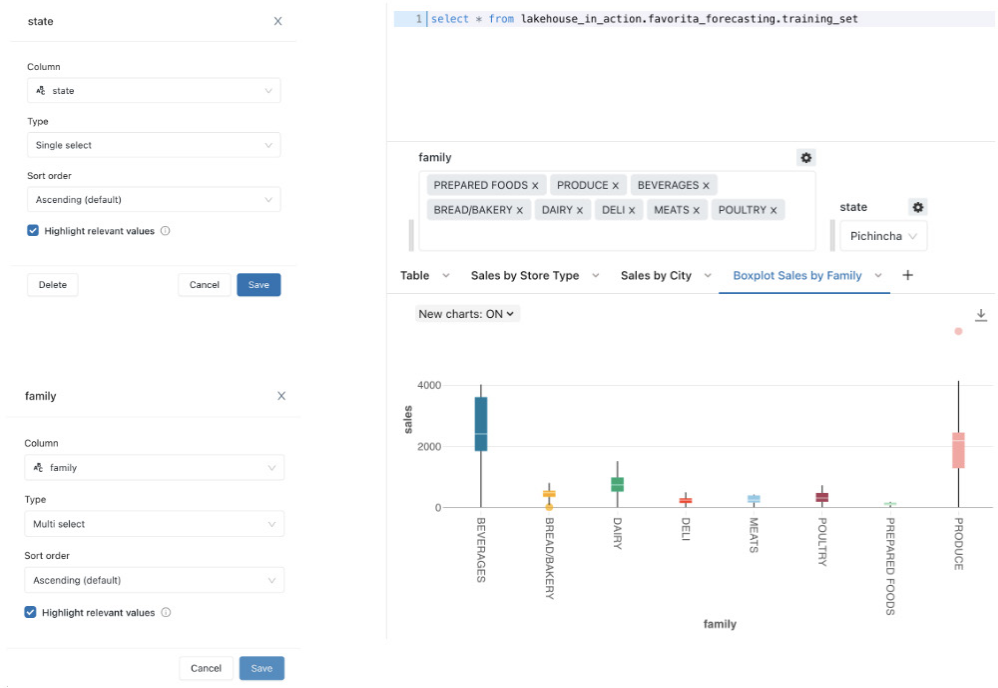


Figure 8.6 – (L) The configuration for the state and family filter; (R) the result of adding two filters to the Favorita sales query

The dashboard functionality shown in *Figure 8.7* is built into Databricks SQL as a way to present the charts and other visualizations created from one or many queries.

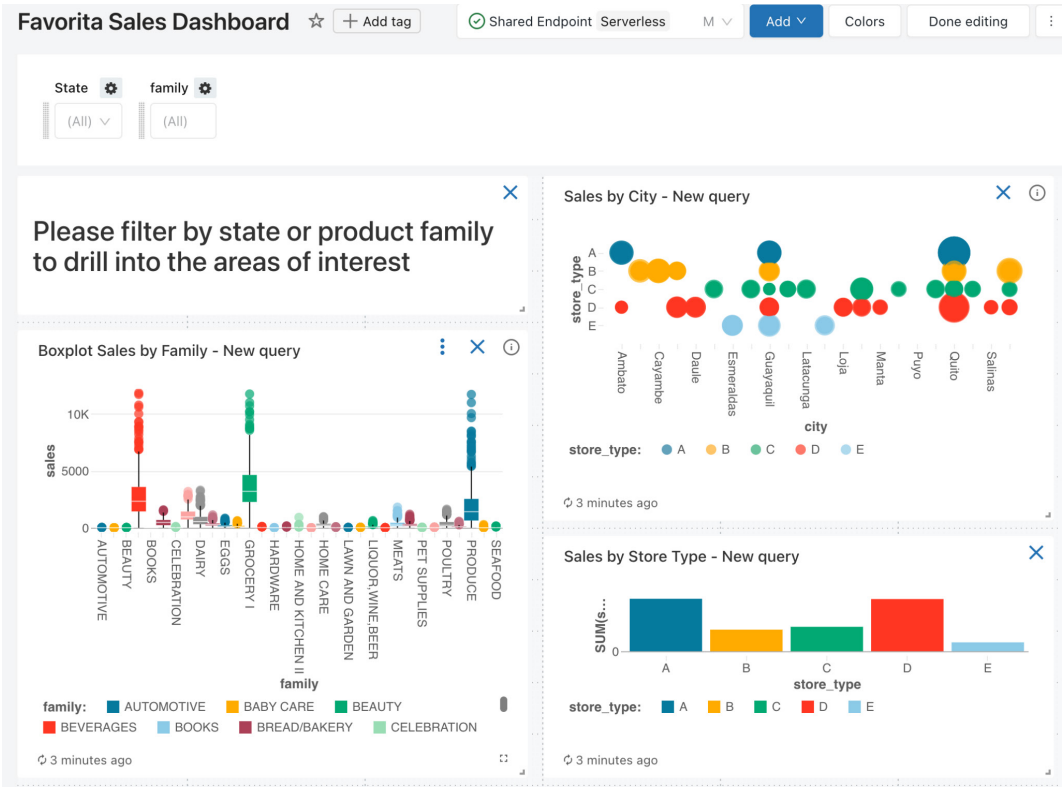


Figure 8.7 – A dashboard with charts created from the Favorita sales data query

The built-in visualization capabilities of DBSQL are a quick way to explore data without connecting to an external dashboarding or data visualization tool.

Next, we'll look at an example of using Python **User-Defined Functions** (**UDFs**) for reusable Python code within DBSQL.

Python UDFs

Python UDFs are a way to create reusable snippets of code in Python, and these can be used in DBSQL. In this example, we'll create a UDF for sales analysts to redact information in a customer record. Line five indicates the language syntax for the function is Python between `$$` signs:

```
--Create a SQL UDF
CREATE FUNCTION redact(a STRING)
RETURNS STRING
LANGUAGE PYTHON
AS $$
import json
keys = ["phone", "email"]
obj = json.loads(a)
for k in obj:
  if k in keys:
    obj[k] = "REDACTED"
return json.dumps(obj)
$$;

-- Grant execute so sales analysts can use the SQL UDF
GRANT EXECUTE ON sales.customer_db.redact TO sales-analysts

--Members of the sales-analyst group can use the redact UDF in their SQL expressions
-- Using the redact UDF to remove phone and email from the contact_info field
SELECT customer_id, redact(contact_info) FROM sales.customer_db.customer_data
```

Figure 8.8 – Creating a Python UDF in DBSQL

UDFs are defined and managed as part of Unity Catalog. Once a UDF is defined, you can give teams the ability to execute the UDF using `GRANT EXECUTE`.

```
--Create a SQL UDF
CREATE FUNCTION redact(a STRING)
RETURNS STRING
LANGUAGE PYTHON
AS $$
import json
keys = ["phone", "email"]
obj = json.loads(a)
for k in obj:
  if k in keys:
    obj[k] = "REDACTED"
return json.dumps(obj)
$$;

-- Grant execute so sales analysts can use the SQL UDF
GRANT EXECUTE ON sales.customer_db.redact TO sales-analysts

--Members of the sales-analyst group can use the redact UDF in their SQL expressions
-- Using the redact UDF to remove phone and email from the contact_info field
SELECT customer_id, redact(contact_info) FROM sales.customer_db.customer_data
```

Figure 8.9 – Granting permissions for the sales-analysts groups to execute a UDF

In this SQL query, we are applying the `redact` UDF to the `contact_info` field.

```
--Create a SQL UDF
CREATE FUNCTION redact(a STRING)
RETURNS STRING
LANGUAGE PYTHON
AS $$
import json
keys = ["phone", "email"]
obj = json.loads(a)
for k in obj:
  if k in keys:
    obj[k] = "REDACTED"
return json.dumps(obj)
$$;

-- Grant execute so sales analysts can use the SQL UDF
GRANT EXECUTE ON sales.customer_db.redact TO sales-analysts

--Members of the sales-analyst group can use the redact UDF in their SQL expressions
-- Using the redact UDF to remove phone and email from the contact_info field
SELECT customer_id, redact(contact_info) FROM sales.customer_db.customer_data
```

Figure 8.10 – Using the Python UDF in a SQL query

Now that we have the basics for visualizing data and applying Python UDFs in SQL, let's cover a couple of tips and tricks.

Tips and tricks

This section covers our tips and tricks related to DBSQL. Some tips apply to DBSQL and Lakeview, but not all:

- **Use managed compute (a.k.a. serverless compute) when possible**: Query performance using Databricks' SQL warehouses is record-setting, as mentioned in *Chapter 1*. The new managed compute for DBSQL puts the new first query performance at roughly 10 seconds. This means that idle time has drastically been reduced, which translates into cost savings.
- **Use a subquery as a parameter filter**: In your query visualizations and dashboards, you can prepopulate drop-down filter boxes. You can do this by creating and saving a query in the SQL editor. For example, you could create a query that returns a distinct list of customer names. In *Figure 8.11*, we select a query called **Customer Name Lookup Qry** as a subquery to filter the query visualization by customer name. Therefore, we can filter on **Customer** using a drop-down list.

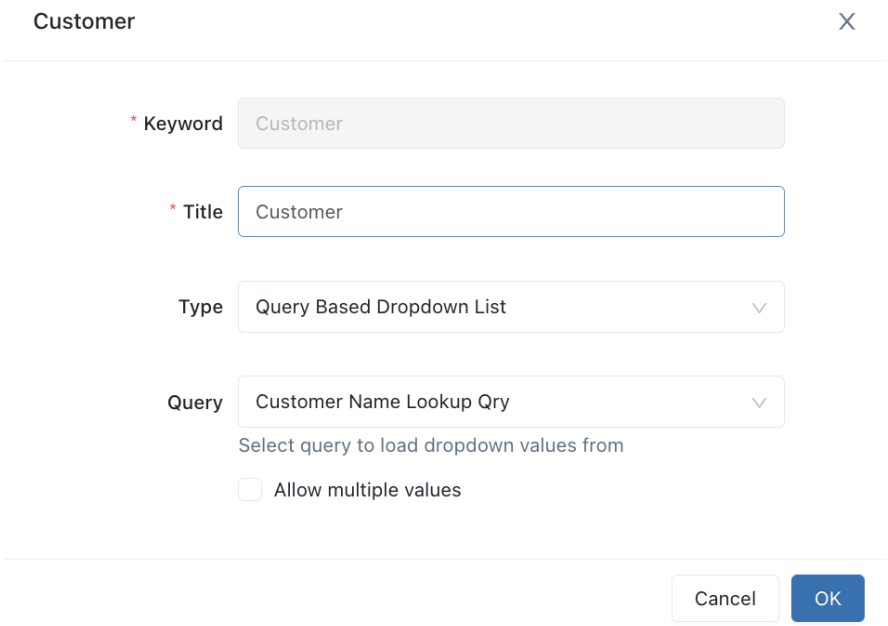
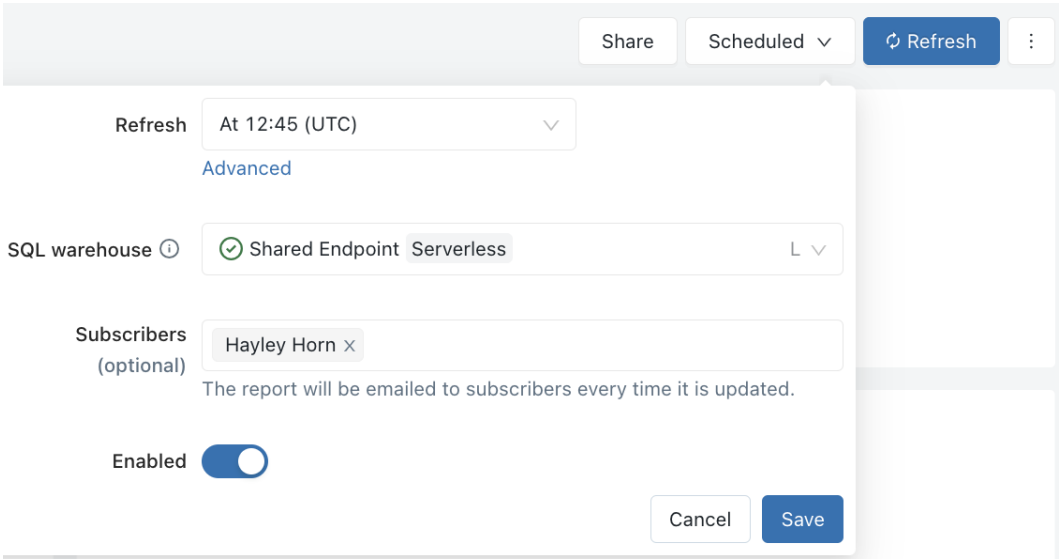


Figure 8.11 – Using a subquery as a parameter for a query

- **Schedule report delivery**: If you have users who want to receive an up-to-date dashboard regularly, you can schedule the refresh and have it sent to subscribers. For DBSQL dashboards, remember to turn off **Enabled** when you are developing so that users don't get too many updates.



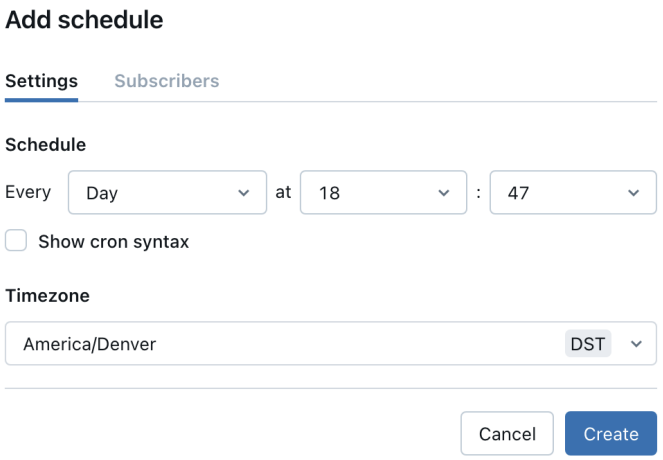


Figure 8.12 – Scheduling a dashboard report with subscribers (T) DBSQL (B) Lakeview

- **Speed up development with Databricks Assistant**: As we covered in *Chapter 4*, Databricks Assistant is an AI-based interface that can help generate, transform, fix, and explain code. The Assistant is context-aware, meaning it uses Unity Catalog to look at the metadata of your tables and columns, personalized in your environment. In *Figure 8.13*, we ask the Assistant to help write a query with syntax for grouping. It sees the metadata of the **Favorita Stores** table and provides code specific to that table and the column of interest.

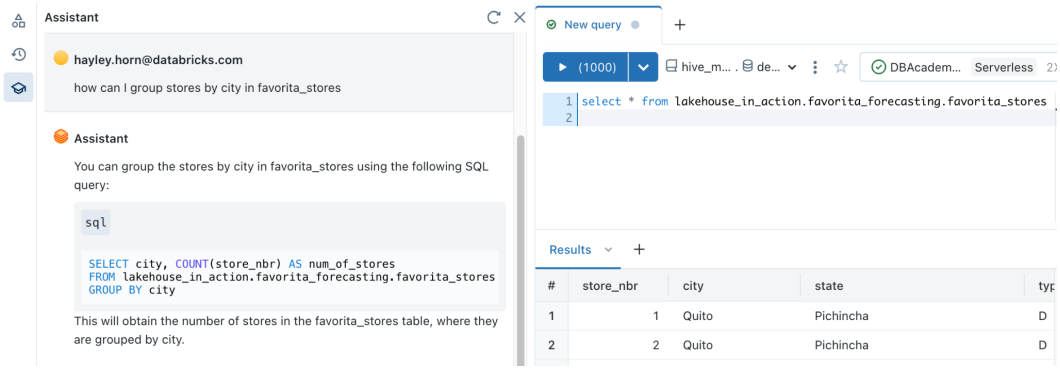


Figure 8.13 – Using Databricks Assistant for help writing a query

- **Be informed**: Keep an eye out for important data changes with alerts. Use SQL to calibrate the alert and schedule the condition evaluation at specific intervals through the UI shown in *Figure 8.14*. You can use HTML to create a formatted alert email.

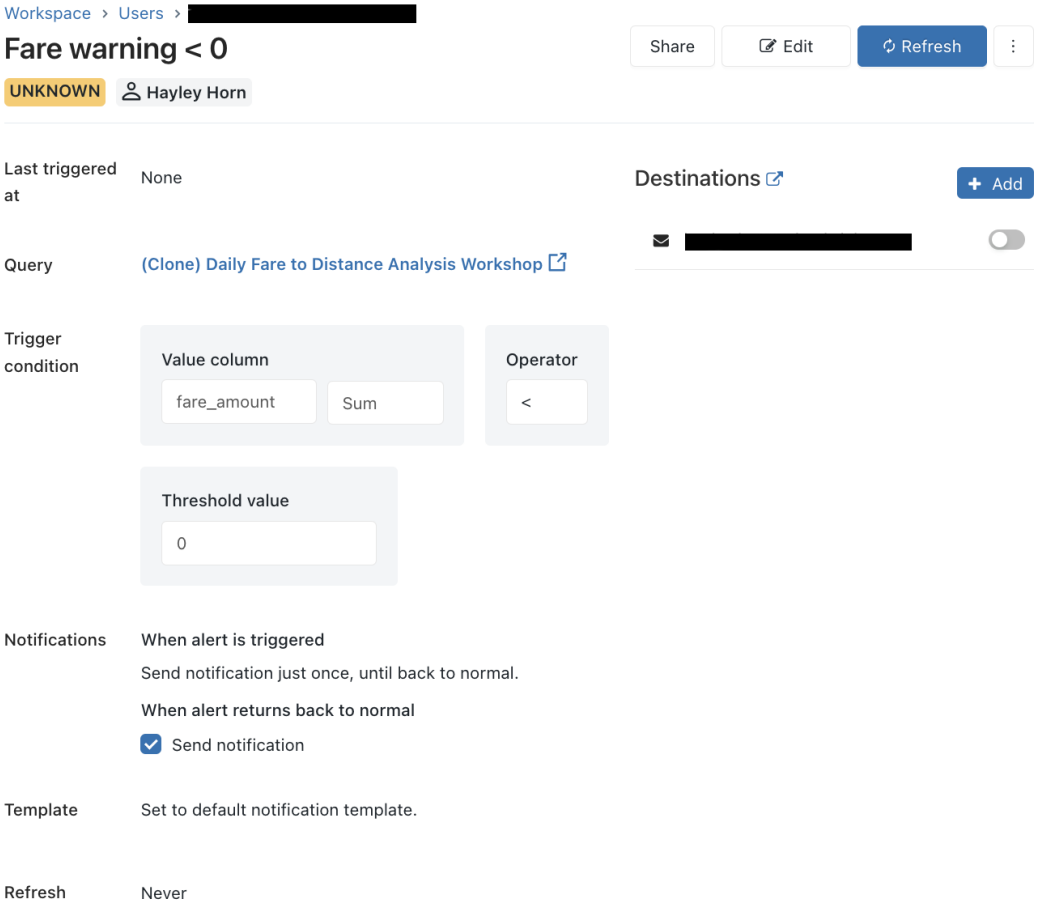


Figure 8.14 – Scheduling alerts to trigger when certain conditions are met

- **Use tags to track usage**: When creating a new SQL warehouse, use tags to code your warehouse endpoint with the correct project. Tagging is a great way to understand usage by project or team. System tables contain the information for tracking usage.

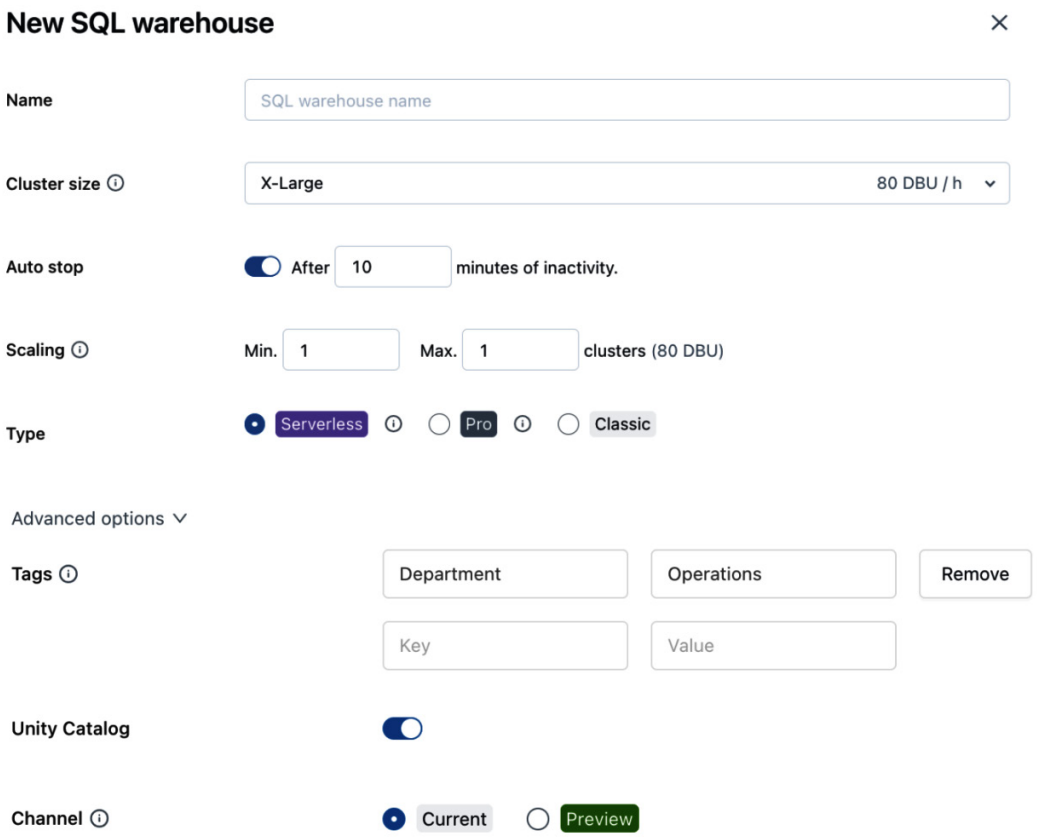


Figure 8.15 – Using tags to connect an endpoint to a project

Next, you'll learn how to connect your models to applications.

Connecting your applications

You can deploy your model anywhere using Databricks Model Serving, which is how you deployed your RAG chatbot model in *Chapter 7*. In this section, we will introduce how to host ML demo apps in **Hugging Face** (**HF**). Having an easy way to host ML apps allows you to build your ML portfolio, showcase your projects at conferences or with stakeholders, and work collaboratively with others in the ML ecosystem. With HF Spaces, you have multiple options for which Python library you use to create a web app. Two common ones are Streamlit and Gradio.

We prefer Gradio. It is an open source Python package that allows you to quickly build a demo or web application for your machine learning model, API, or any arbitrary Python function. You can then share a link to your demo or web application in just a few seconds using Gradio's built-in sharing features. No JavaScript, CSS, or web hosting experience is needed – we love it!

We will walk you through deploying a chatbot to an HF Space in the *Applying our learning* section's RAG project work.

Incorporating LLMs for analysts with SQL AI Functions

There are many use cases where you can integrate an LLM, such as DBRX or OpenAI, for insights. With the Databricks Data Intelligence Platform, it's also possible for analysts who are most comfortable in SQL to take advantage of advances in machine learning and artificial intelligence.

Within Databricks, you can use **AI Functions**, which are built-in SQL functions to access LLMs directly. AI Functions are available for use in the DBSQL interface, SQL warehouse JDBC connection, or via the Spark SQL API. In *Figure 8.16*, we are leveraging the Databricks SQL editor.

Foundational Models API

The storage and processing of data for Databricks-hosted foundation models occur entirely within the Databricks Platform. Importantly, this data is not shared with any third-party model providers. This is not necessarily true when using the External Models API, which connects you to services such as OpenAI that have their own data privacy policies. Keep this in mind when you are concerned about data privacy. You may be able to pay for a tier of service that restricts the use of your data.

Let's do some simple emotion classification. Since the three datasets we've been working with don't include any natural language, we'll create a small dataset ourselves first. You can also download a dataset (such as the Emotions dataset from Kaggle) or use any other natural language source available to you:

1. First, let's explore the built-in `AI_QUERY` DBSQL function. This command will send our prompt to the remote model configured and retrieve the result. We're using Databricks' DBRX model, but you can use a variety of other open source and proprietary models as well. Open up the Databricks SQL editor and type in the code as shown in *Figure 8.16*. Let's write a query to give us a sample sentence that we can classify.

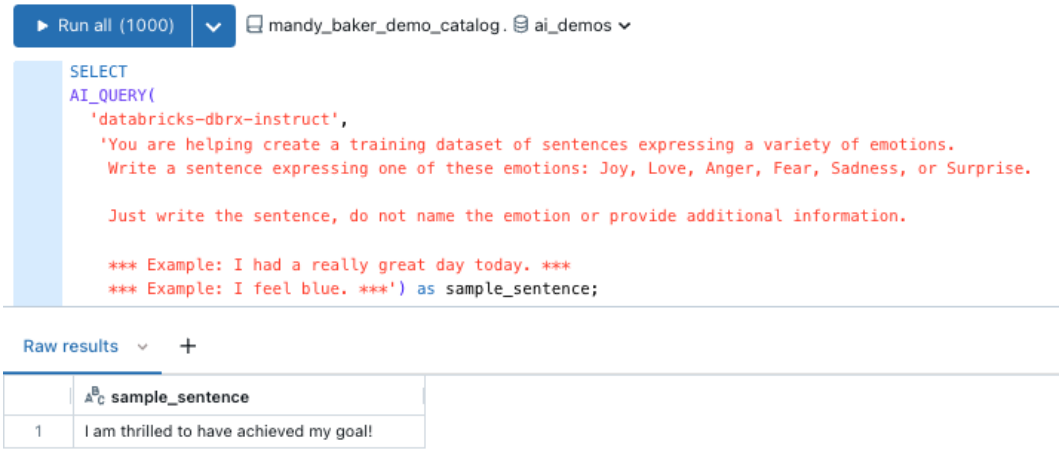


Figure 8.16 – Crafting a prompt using the AI_QUERY function

2. If you don't have a dataset ready and don't want to download one, you can build a function to generate a dataset for you, as shown here. We're expanding on the prompt from *Step 1* to get several sentences back in JSON format.

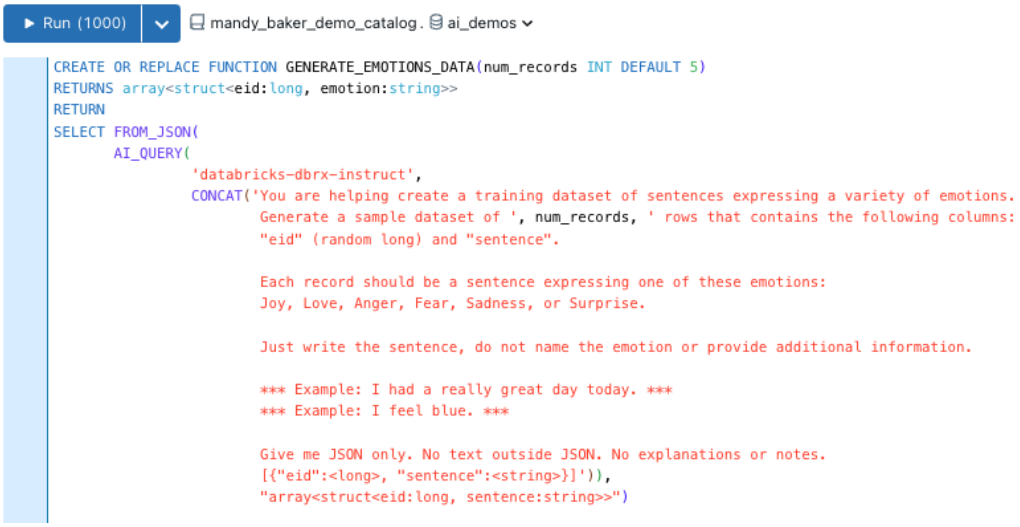


Figure 8.17 – Creating a function to generate fake data

3. Now use the `GENERATE_EMOTIONS_DATA` function to build a small dataset. After a quick review of the data, it looks like we have a good sample of emotions.

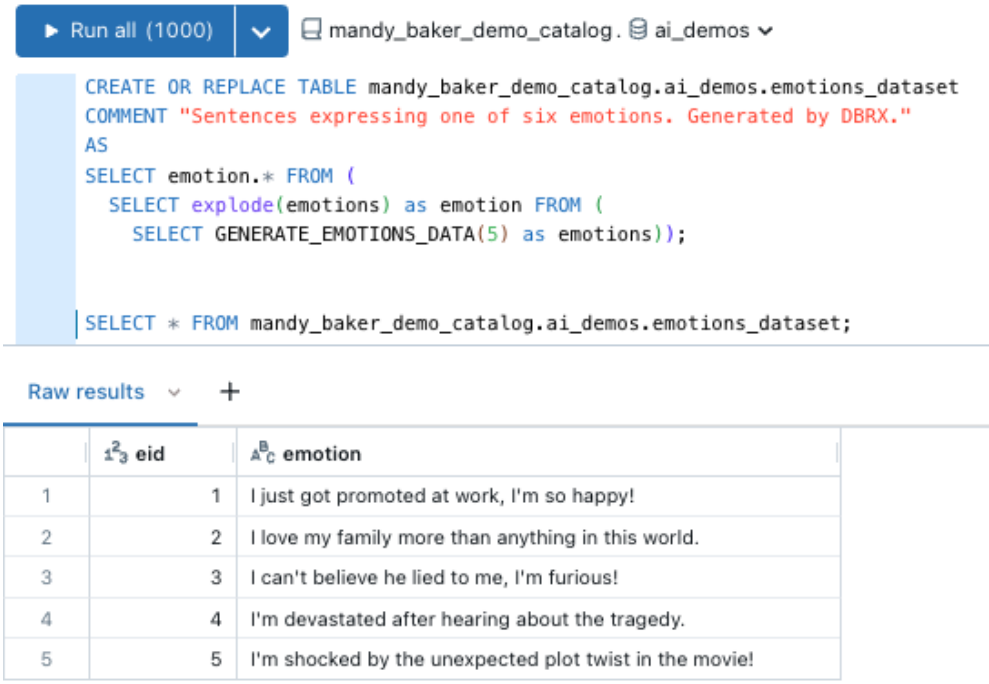


Figure 8.18 – Generating fake emotion data

4. Now, we're going to write a function called `CLASSIFY_EMOTION`. We're using the AI Function `AI_QUERY` again, but this function will use a new prompt asking the model to classify a given sentence as one of six emotions.

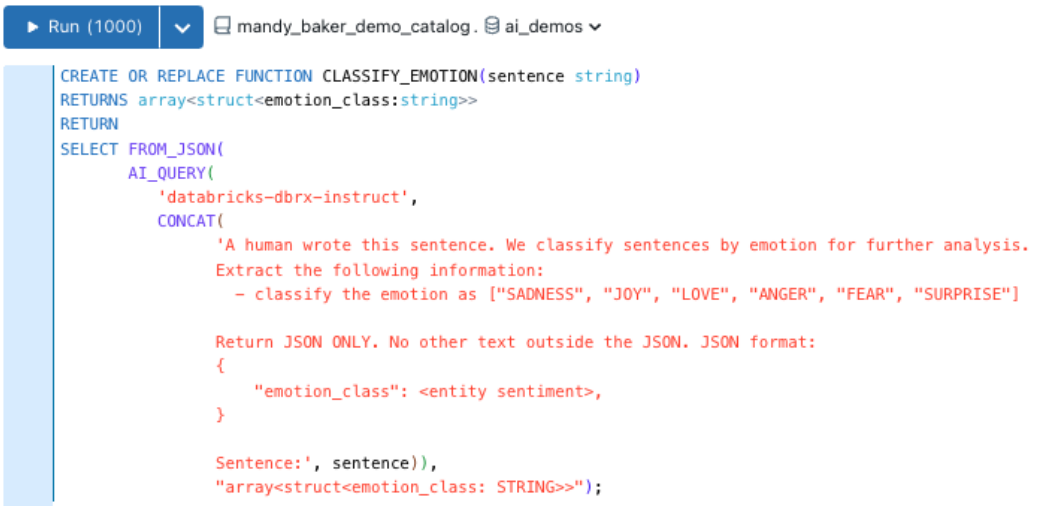


Figure 8.19 – Creating a function to classify sentences by emotion

5. Let's call our function to evaluate an example sentence and take a look at the results.

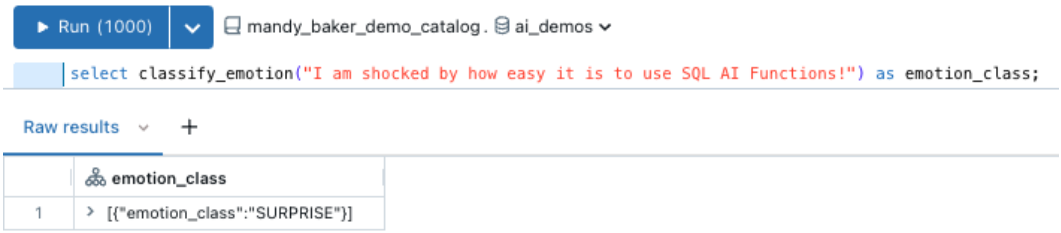


Figure 8.20 – Calling the CLASSIFY_EMOTION function

6. Finally, to classify all records in a table, we call the `CLASSIFY_EMOTION` function on the records in our table and view the results.

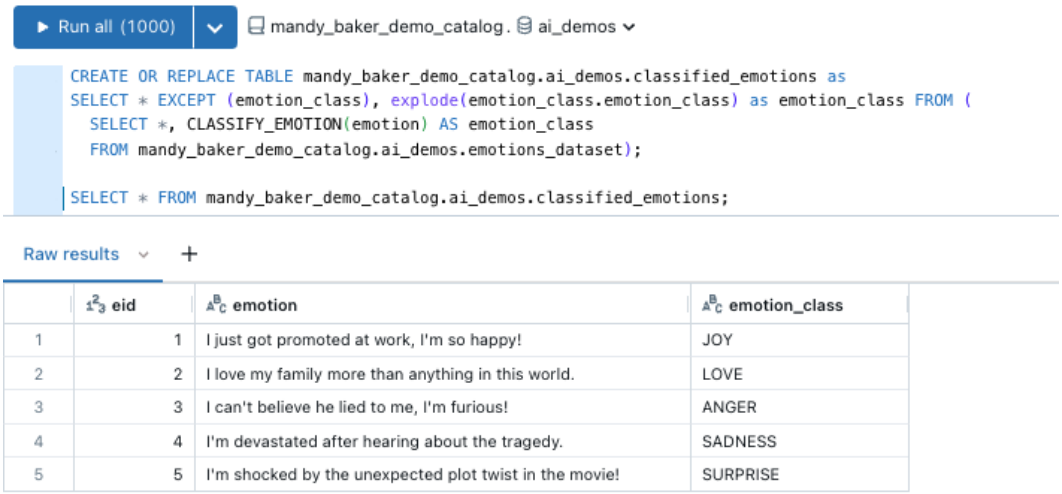


Figure 8.21 – Calling the CLASSIFY_EMOTION function on a table

SQL AI Functions are a great way to put the power of LLMs in the hands of SQL users. Solutions like SQL AI Functions still require some technical knowledge. Databricks is researching ways to allow business users direct access to data, with less upfront development required to get your team moving even faster. Keep an eye out for exciting new product features that remove the programming experience barrier to unlock the value of your data!

Applying our learning

Let's use what we have learned to build a SQL chatbot using the Favorita project's table metadata, monitor the streaming transaction project's model, and deploy the chatbot that we have assembled and evaluated.

Technical requirements

The technical requirements needed to complete the hands-on examples in this chapter are as follows:

- The SQLbot will require OpenAI credentials.
- We will use the Databricks Secrets API to store our OpenAI credentials.
- You will need a **personal access token** (**PAT**) to deploy your web app to HF. See *Further reading* for detailed instructions.

Project: Favorita Store Sales

Let's build a simple **SQLbot** using OpenAI's GPT to ask questions about our Favorita Sales tables. Please note that while this section continues to use the *Favorita Store Sales* data, it is not a continuation of the earlier project work. In this example, you'll create instructions on how the bot can ask for a list of tables, get information from those tables, and sample data from the tables. The SQLbot will be able to build a SQL query and then interpret the results. To run the notebooks in this example, you will need an account with OpenAI on the OpenAI developer site and request a key for the OpenAI API.

To follow along in your own workspace, please open the following notebook:

- `CH8-01-SQL Chatbot`

Keeping secret API keys in your Databricks notebook is far from best practice. You can lock down your notebook access and add a configuration notebook to your `.gitignore` file. However, your ability to remove people's access may not be in your control, depending on your role. Generally, the permissions of admins include the ability to see all code. The OpenAI API key ties back to your account and your credit card. Note that running the notebook once cost us $0.08.

We added our API key to Databricks secrets. The Secrets API requires the Databricks CLI. We set up our CLI through Homebrew. If you haven't already, we suggest getting Secrets set up for your workspace. This may require admin assistance. Start by installing or updating the Databricks CLI. You know the CLI is installed correctly when you get version v0.2 or higher. We are working with `Databricks CLI v0.208.0`.

We followed these steps to set up our API key as a secret:

1. Create a scope:

```
databricks secrets create-scope dlia
```

2. Create a secret within the scope:

```
databricks secrets put-secret dlia OPENAI_API_KEY
```

3. Paste your API key into the prompt.

Once your secret is successfully saved, we can access it via `dbutils.secrets` in our notebooks.

We are all set up to use OpenAI via the API now. We do not have to worry about accidentally committing our API or a coworker running the code, not knowing it costs you money.

Next, let's focus on creating our SQLbot notebook, step by step, beginning with the setup:

1. First, we install three libraries: `openai`, `langchain_experimental`, and `sqlalchemy-databricks`.
2. To create a connection to OpenAI, pass the secret we set up previously and open a `ChatOpenAI` connection.
3. In *Figure 8.22*, we create two different models. The first is the default model and the second uses GPT 3.5 Turbo.

```
from langchain_openai import OpenAI
from langchain_openai import ChatOpenAI
import os

os.environ['OPENAI_API_KEY'] = dbutils.secrets.get(scope="dlia", key="OPENAI_API_KEY")
llm = OpenAI(temperature=0)
chat = ChatOpenAI(model_name="gpt-3.5-turbo")
```

Figure 8.22 – OpenAI API connection

4. The setup file does not set your schema variable. Define your schema; we chose `favorita_forecasting`. We have been using `database_name` rather than a schema. However, we specify the database we want to ask SQL questions against, which is different.

```
schema = "favorita_forecasting"

table_schemas = spark.sql(
f"""
select
  *
from
  system.information_schema.columns
where
  table_catalog = '{catalog}'
  and table_schema = '{schema}'
order by
  table_name,
  ordinal_position;
"""
)
```

```
table_schemas = table_schemas.drop(
        "table_catalog",
        "table_schema",
        "ordinal_position",
        "character_octet_length",
        "numeric_precision",
        "numeric_precision_radix",
        "numeric_scale",
        "datetime_precision",
        "interval_type",
        "interval_precision",
        "identity_start",
        "identity_increment",
        "identity_maximum",
        "identity_minimum",
        "identity_cycle",
        "is_system_time_period_start",
        "is_system_time_period_end",
        "system_time_period_timestamp_generation",
        "is_updatable"
    )
```

Figure 8.23 – (L) Collecting the table schema and system information schema; (R) dropping unnecessary and repetitive columns

5. Next, we create two helper functions. The first function organizes the schema information provided, `table_schemas`, creating a table definition. The second collects two rows of data as examples.

```
def table_def(table):
    table_schema = table_schemas.drop("table_name").where(f'table_name = "{table}"')

    return f"Table Schema for {table}: \ncolumn_index" + table_schema.toPandas().to_csv(
        sep="|"
    )

def table_records(table):
    records = spark.sql(f"select * from {catalog}.{schema}.{table} limit 2")

    return f"Example records for {table}: \ncolumn_index" + records.toPandas().to_csv(
        sep="|"
    )
```

Figure 8.24 – Helper functions for organizing table information

6. Iterate through the table and column data, leveraging our helper functions to format the SQL database input.

```
metadata = {}
table_list = []

for row in table_schemas.select("table_name").distinct().collect():
  tbl = row['table_name']
  table_list.append(tbl)
  meta = table_def(tbl) + '\n\n' + table_records(tbl)
  metadata[tbl] = meta
```

Figure 8.25 – Iterating through the tables and leveraging the helper functions

7. We now have all of our data ready to be able to create a SQL database for OpenAI to talk to. You will need to edit `endpoint_http_path` to match the path of an active SQL warehouse in your workspace. The database is passed to both the default OpenAI model and the GPT 3.5 model.

```
from sqlalchemy.engine import create_engine
from langchain.sql_database import SQLDatabase
from langchain_experimental.sql import SQLDatabaseChain

table = table_list

databricks_token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().get()
workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
# Obtain this from a SQL endpoint under "Connection Details", HTTP Path
endpoint_http_path = "/sql/1.0/warehouses/5ab5dda58c1ea16b"

engine = create_engine(
  f"databricks+connector://token:{databricks_token}@{workspace_url}:443/{database_name}",
  connect_args={"http_path": endpoint_http_path, "catalog": catalog}
)

db = SQLDatabase(engine, schema=None, include_tables=table)
db_chain = SQLDatabaseChain.from_llm(llm, db, verbose=True)
chat_chain = SQLDatabaseChain.from_llm(chat, db, verbose=True)
```

Figure 8.26 – Create a database for OpenAI to query containing only the information we provided

With the setup complete, we can now interact with our SQL chatbot models! Let's start with a basic question: *Which store sold the most?*

In *Figure 8.27*, we run both models on the question and get back two different answers.

```
> Entering new  chain...
Which store sold the most?
SQLQuery:SELECT store_nbr, SUM(transactions) AS total_transactions FROM favorita_transactions
GROUP BY store_nbr ORDER BY total_transactions DESC LIMIT 5;
SQLResult: [(44, 7273093), (47, 6535810), (45, 6201115), (46, 5990113), (3, 5366350)]
Answer:Store 44 sold the most with 7273093 transactions.
> Finished chain.
'Store 44 sold the most with 7273093 transactions.'


> Entering new  chain...
Which store sold the most?
SQLQuery:SELECT store_nbr, SUM(sales) AS total_sales
FROM train_set
GROUP BY store_nbr
ORDER BY total_sales DESC
LIMIT 1
SQLResult: [(44, 62087553.25008899)]
Answer:Store number 44 sold the most with a total sales of 62,087,553.25
> Finished chain.
'Store number 44 sold the most with a total sales of 62,087,553.25'
```

Figure 8.27 – The SQL chatbot model's responses to our question "Which store sold the most?". (T) db_chain.run(question) (B) chat_chain.run(question)

As new versions of OpenAI's GPT model are released, the results and behavior of your SQLbot may change. As new models and approaches become available, it is good practice to test them and see how the changes impact your work and the results of your chatbot. Leveraging MLflow with your SQLbot experiment will help you track and compare the different features and configurations throughout your production process.

Project: Streaming Transactions

You are ready to wrap up this project. The production workflow notebooks are the `CH7-08-Production Generating Records`, `CH7-09-Production Auto Loader`, and `CH7-10-Production Feature Engineering` components in the workflow job created in *Chapter 7*. You will run that same job once the new workflow is in place. To follow along in your own workspace, please open the following notebook: `CH8-05-Production Monitoring`

In the `CH8-05-Production Monitoring` notebook, you create two monitors – one for the `prod_transactions` table and one for the `packaged_transaction_model_predictions` table. See *Figure 8.28* for the latter.

```
lm.create_monitor(
    table_name=f"{catalog}.{database_name}.packaged_transaction_model_predictions",
    profile_type=lm.InferenceLog(
        timestamp_col="TransactionTimestamp",
        granularities=["5 minutes","30 minutes","1 hour"],
        model_id_col="model_version",
        prediction_col="prediction",
        label_col="actual_label",
        problem_type="classification"
    ),
    slicing_exprs=["Product","CustomerID"],
    schedule=lm.MonitorCronSchedule(
        quartz_cron_expression="0 0 * * * ?", # schedules a refresh every hour
        timezone_id="MST",
    ),
    output_schema_name=f"{catalog}.{database_name}"
)
```

Figure 8.28 – The inference table monitor

Congratulations! The streaming project is complete. We encourage you to add improvements and commit them back to the repository. Here are a few possible examples: add more validation metrics to the validation notebook, incorporate the inference performance results into the decision to retrain, and make adjustments to the configuration of data generation to simulate drift.

Project: Retrieval-Augmented Generation Chatbot

To follow along in your own workspace, please open the following notebooks and resources:

- CH8-`app.py`
- `CH8-01-Deploy Your Endpoint with SDK`
- The **Hugging Face Spaces** page

First, we need to ensure our chatbot is deployed using Model Serving, as shown in *Figure 8.30*. Here, we are using the fastest way by going through the UI of the Model Serving page. To follow along and serve, we are selecting the registered model that we registered in *Chapter 7*. Select the latest version – in our case, it's version 4. For this demo project, we expect minimal concurrency so the endpoint will have only four possible concurrent runs and will scale to 0 when no traffic is available. We are enabling inference tables under the same catalog to track and potentially further monitor our payload. We are not going to demonstrate in this chapter how to set a monitor or data quality pipeline for the RAG project, as it was demonstrated for the streaming project. We encourage you to apply it on your own!

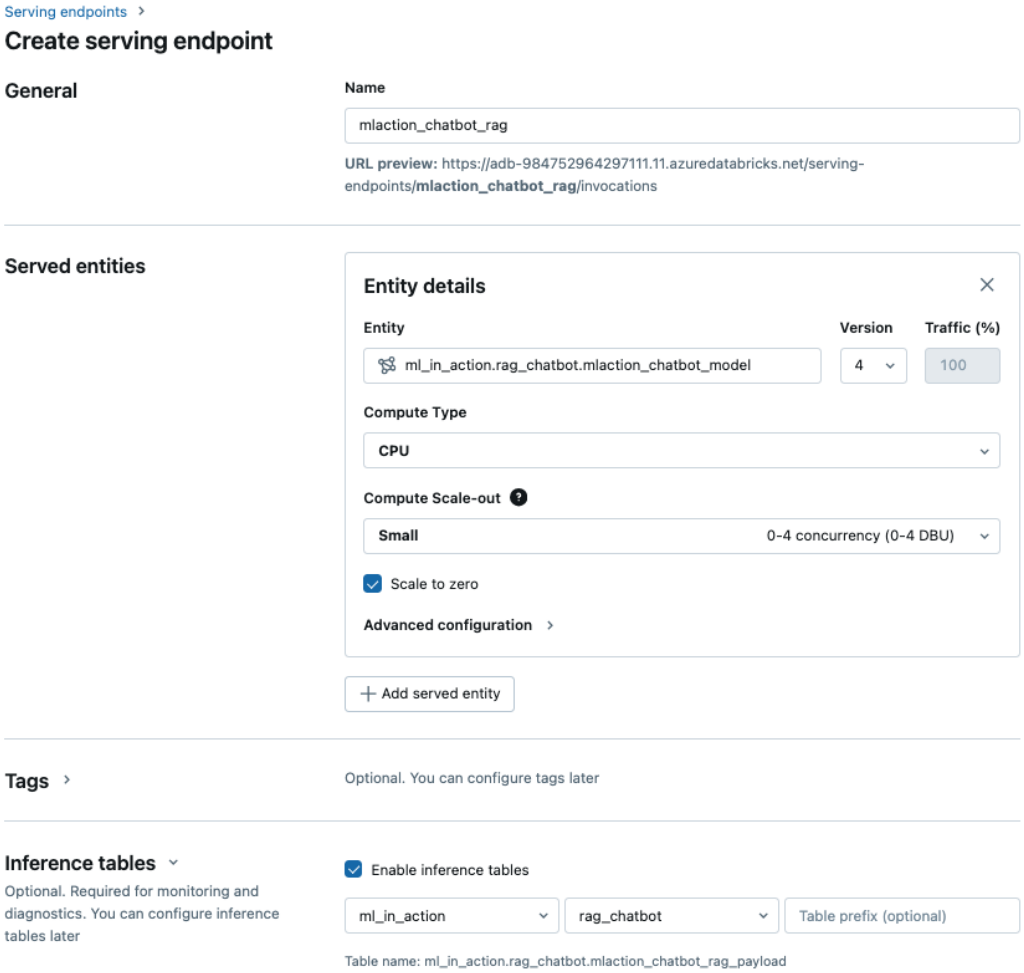


Figure 8.29 – Example of the Model Serving deployment via UI

In order for your application to be able to connect to the resources attached to it, such as Vector Search, the endpoint requires you to provide additional configurations to the endpoint such as your PAT and host under **Advanced configuration**:

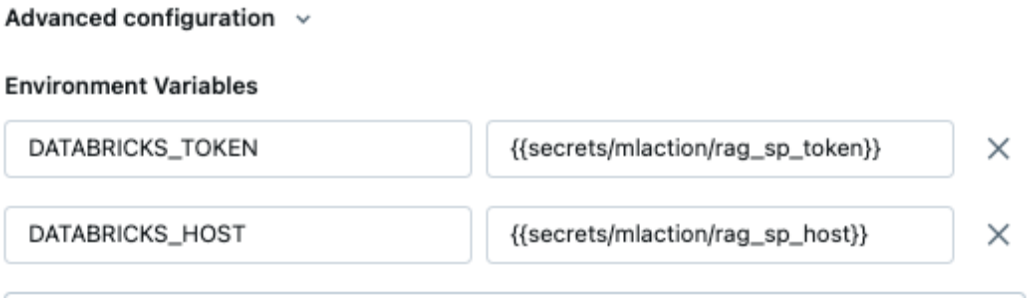


Figure 8.30 – Advanced configuration requirements

> **Note**
>
> You could also use the Databricks SDK service to deploy your endpoints. If you are interested in seeing how to use SDK deployment, please use the notebook attached under `CH8 - 01 -Deploy Your Endpoint with SDK`.

Jump over to the Hugging Face Spaces website. Instructions on how to deploy your first HF Space are explained very well on the main page of HF Spaces, so we will not duplicate them here. We would like to highlight that we are using a free deployment option of Spaces with 2 CPUs and 16 GB of memory.

When you deploy your Space, it will look like this:

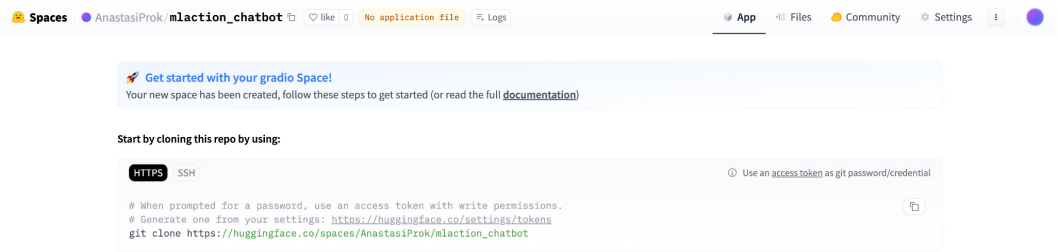


Figure 8.31 – Empty Hugging Face Space

We would like to highlight a few things that are important in order to connect to your chatbot, which is served in real time with Databricks Model Serving. To connect the chatbot to your HF Space, you must set `API_TOKEN` and an `API_ENDPOINT`. Here's how to set these values:

1. Go to **Settings** in the HF Space you created.
2. Scroll down to **Variables and secrets**.
3. Set your API_ENDPOINT as the URL from the REST API provided on the Databricks Model Serving page.
4. Set your API_TOKEN using a personal access token generated by Databricks. This is required to connect to the endpoint.

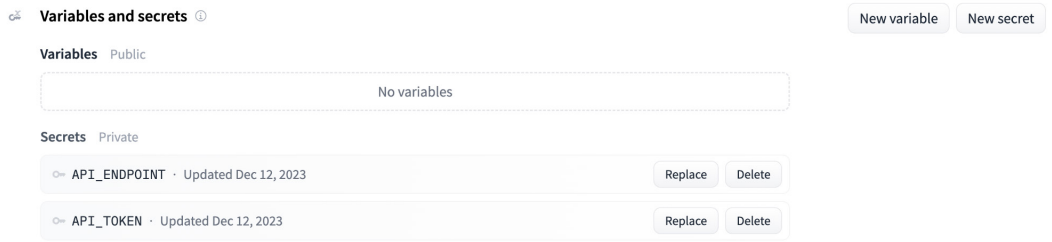


Figure 8.32 – Example of Variables and secrets on HF Spaces

5. Once this is set, you are ready to bring your Gradio web app script into your HF Space.

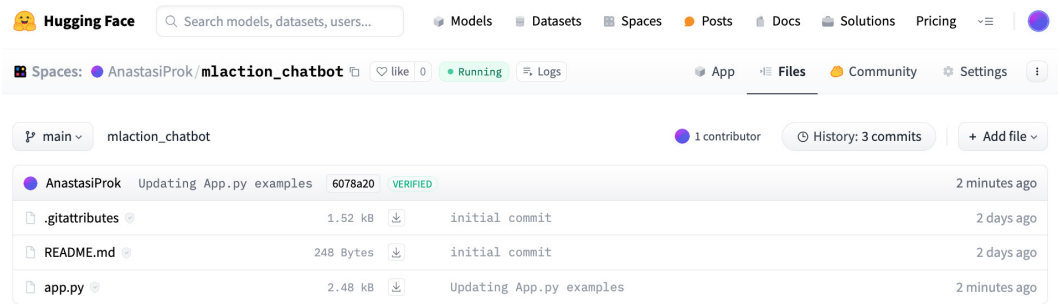


Figure 8.33 – Example of Variables and secrets on HF Spaces

6. When your endpoint is ready, jump back to your HF Space.
7. Go to the **Files** tab under the pre-created Space and click **+Add File**.
8. Now add `CH8-app.py` that was given to you for *Chapter 8* – you can create your own web application. Feel free to experiment with the design according to your business needs.

Let's talk a bit about the `respond` function in the `CH8-app.py` file – see *Figure 8.34*, which is passed to our app's UI chatbot. The `respond` function, in this case, is the caller of your deployed endpoints, where we do not just send and receive responses but also can shape the format of the input or output. In our case, the endpoint expects to receive a request in the format of a JSON with field inputs with the questions within a list, while the output is a JSON with field predictions.

```
import itertools
import gradio as gr
import requests
import os
from gradio.themes.utils import sizes


def respond(message, history):

    if len(message.strip()) == 0:
        return "ERROR the question should not be empty"

    # Set your secrets under HF secrets for your Space
    local_token = os.getenv('API_TOKEN')
    local_endpoint = os.getenv('API_ENDPOINT')

    if local_token is None or local_endpoint is None:
        return "ERROR missing env variables"

    # Add your API token to the headers
    headers = {
        'Content-Type': 'application/json',
        'Authorization': f'Bearer {local_token}'
    }

    # Pay attenction to what your model expects as schema format
    q = {"inputs": [message]}
    try:
        response = requests.post(
            local_endpoint, json=q, headers=headers, timeout=100)
        response_data = response.json()
        response_data=response_data["predictions"][0]

    except Exception as error:
        response_data = f"ERROR status_code: {type(error).__name__}"

    return response_data
```

Figure 8.34 – The respond function written in the Gradio app

To create a chatbot, as was mentioned in the introduction section, we are using a simple example from Gradio where we add options such as the title of our application, a description, and example questions. *Figure 8.35* shows the full code.

```
theme = gr.themes.Soft(text_size=sizes.text_sm,radius_size=sizes.radius_sm, spacing_size=sizes.spacing_sm,)

demo = gr.ChatInterface(
    respond,
    chatbot=gr.Chatbot(show_label=False, container=False, show_copy_button=True, bubble_full_width=True),
    textbox=gr.Textbox(placeholder="Ask me a question",
                       container=False, scale=7),
    title="Ml in Action LLM RAG demo - Chat with Mixtral from Foundational Databricks model serving endpoint",
    description="This chatbot is a demo example for the llm chatbot. <br>This content is provided as a LLM RAG educational example, without support.
    It is using Mixtral, can hallucinate and should not be used as production content.<br>Please do not use this for production.",
    examples=[["Can LLM's impact wages and how ?"],
              ["Will AI impact work forces in the US ? "],
              ["Can LLM's impact wages and how ?"],],
    cache_examples=False,
    theme=theme,
    retry_btn=None,
    undo_btn=None,
    clear_btn="Clear",
)
```

Figure 8.35 – The Gadio app.py interface for your LLM

The chatbot is now available in a more user-friendly interface, as shown in *Figure 8.36*.

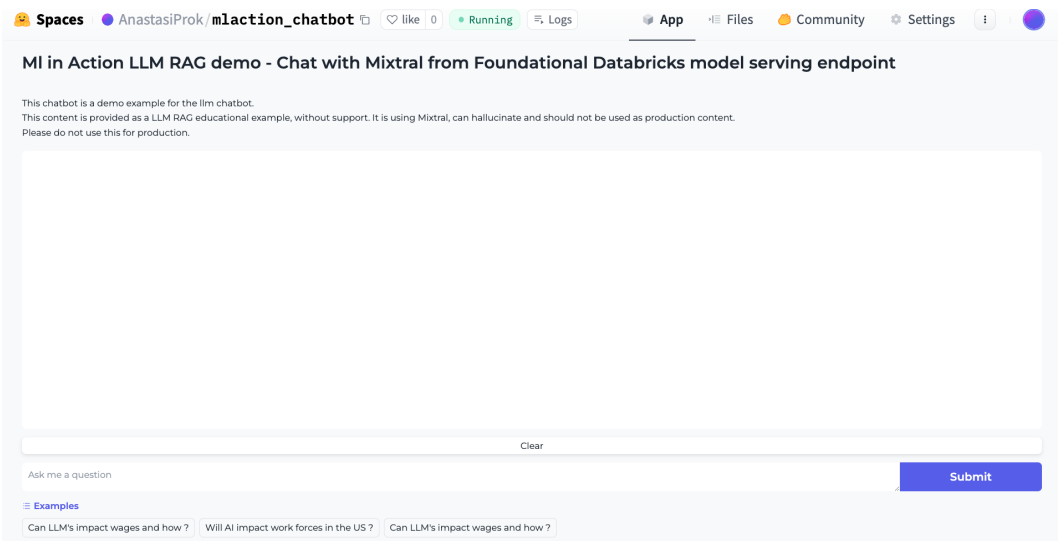


Figure 8.36 – The interface for your chatbot application

Let's ask a few questions to make sure our RAG chatbot is providing correct results.

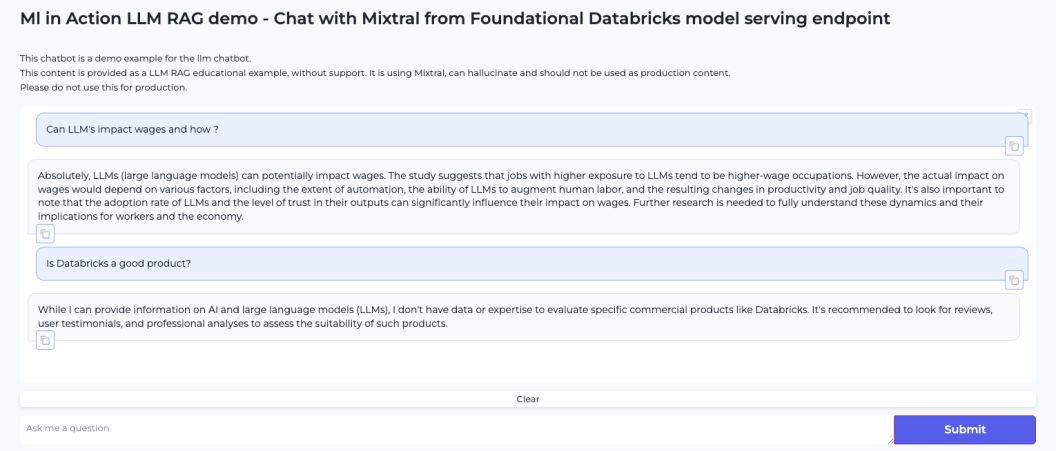


Figure 8.37 – Examples of chatbot answers from our RAG application

If the responses look good, your application is ready to be used!

Summary

Sharing insights from your models is an important way to get value from your machine learning practice. Using dashboards as a medium for sharing your information is an accessible way to communicate insights to business users and teams outside of the data science team. In this chapter, we discussed how to build the gold layer, tips on presenting your data using DBSQL dashboards, taking advantage of innovations such as OpenAI models in DBSQL, and how you can share data and AI artifacts through Databricks Marketplace to get the most value from your enterprise data.

We hope you have had a chance to get hands-on with building your lakehouse. From exploration, cleaning, building pipelines, and building models to finding insights hidden in your data, all the way to sharing insights – it's all doable on the Databricks Platform. We encourage you to take the notebooks and experiment! The authors would love to hear feedback, and whether this has been helpful on your journey with the Databricks Platform.

Questions

Let's test ourselves on what we've learned by going through the following questions:

1. What are some differences between the gold layer and the silver layer?
2. What is one way you could set an alert to identify that a table has an invalid value?

3. Why would you choose to use an external dashboarding tool?
4. If you use a language model through an API such as OpenAI, what are some considerations about the data you send via the API?
5. What are some reasons a company would share data in Databricks Marketplace?

Answers

After putting thought into the questions, compare your answers to ours:

1. The gold layer is more refined and aggregated than the silver layer. The silver layer powers data science and machine learning, and the gold layer powers analytics and dashboarding.
2. You can monitor the values of a field and send an email alert when the value is invalid.
3. Sometimes companies use multiple dashboarding tools. You might need to provide data in a dashboard a team is accustomed to using.
4. If I were a language model through an API, I would be cautious about sending sensitive data, including PII, customer information, or proprietary information.
5. A company might share data in Databricks Marketplace in order to monetize the data or make it available externally for people to use easily and securely.

Further reading

In this chapter, we pointed out specific technologies, technical features, and options. Please look at these resources to get deeper into the areas that interest you most:

- *Databricks SQL Statement Execution API*: `https://www.databricks.com/blog/2023/03/07/databricks-sql-statement-execution-api-announcing-public-preview.html`
- *Power to the SQL People: Introducing Python UDFs in Databricks SQL*: `https://www.databricks.com/blog/2022/07/22/power-to-the-sql-people-introducing-python-udfs-in-databricks-sql.html`
- *Actioning Customer Reviews at Scale with Databricks SQL AI Functions*: `https://www.databricks.com/blog/actioning-customer-reviews-scale-databricks-sql-ai-functions`
- *Databricks sets the official data warehousing performance record*: https://dbricks.co/benchmark
- *Databricks Lakehouse and Data Mesh*: `https://www.databricks.com/blog/2022/10/10/databricks-lakehouse-and-data-mesh-part-1.html`
- *Hugging Face*: `https://huggingface.co/spaces`

- *Gradio*: https://www.gradio.app/
- *Hugging Face Spaces*: https://huggingface.co/docs/hub/en/spaces-overview
- *Databricks Lakehouse monitoring documentation*: https://api-docs.databricks.com/python/lakehouse-monitoring/latest/databricks.lakehouse_monitoring.html#module-databricks.lakehouse_monitoring
- Databricks personal access token authentication https://docs.databricks.com/en/dev-tools/auth/pat.html

Index

A

access control lists (ACLs) 4
AI Functions 231
analysts, with SQL AI Functions
  LLMs, incorporating 231-235
applications
  models, connecting to 231
Auto Loader 49
  schema evolution 51
  used, for transforming data to Delta 50, 51
Auto Loader, with DLT
  used, for continuous data ingestion 60
Auto Loader, with Structured Streaming
  used, for continuous data ingestion 58-60
automated machine learning (AutoML) 79, 141
  baselining with 143, 144
  data profiles, generating with 88-90
  used, for creating baseline model 154-157
  experiments, tracking with MLflow 144-147
Azure Data Lake Storage (ADLS) 23

B

batch feature 112
Bronze data
  with DLT 53
bronze table
  generating, in DLT pipeline 63

C

change data capture (CDC) 9
change data feed (CDF) 9
chatbot responses
  evaluating 211-214
ChatGPT model 151
continuous integration and continuous deployment (CI/CD) 179

D

data
  optimizing 73
  transforming, to Delta with Auto Loader 50, 51
Data and AI Summit (DAIS) 73

Databricks
data and models, monitoring 30
modeling 28, 29
Databricks Asset Bundles (DABs) 179
Databricks Assistant 84
used, for exploring data 84-87
Databricks' Auto Loader
using, benefits 50
Databricks AutoML 7
Databricks Feature Engineering (DFE) 111, 142
batch feature 112
in Unity Catalog 112, 113
on-demand feature 112
on streaming data 114, 115
on streaming pipelines 114, 115
streaming feature 112
Databricks File System (DBFS) 24
Databricks Lakehouse Monitoring
alerting 82
mechanics 82
monitor, creating 83, 84
used, for data monitoring 82
visualization 82
Databricks LLM Playground 148-150
Databricks Machine Learning Runtimes 164
Databricks Marketplace
data, integrating via 147
Databricks Model Serving 15
Databricks SQL dashboards
big data, visualizing with 221-224
Databricks SQL (DBSQL) 4, 221
Databricks Vector Search (VS) 79
benefits 92
data retrieval, enhancing with 92, 93
embedding model support 93
setting up 93
Databricks Workflows 178, 179
Databricks workspace
Kaggle setup 32
projects, starting 40
requisites 31
setting up 32
data integrity
improving, with DLT 80, 81
Data Intelligence (DI) 175
Data Intelligence Engine 3
Data Intelligence Platform, advantages
advantages 4, 5
Databricks AutoML 7
open file formats 7
open source features 5, 6
reusability and reproducibility 7
Data Intelligence Platform (DI)
components 3, 4
data loaders 72, 73
Data Manipulation Language (DML) 54
data preparation 25, 26
Bronze layer 25
features, creating 27
Gold layer 25
Silver layer 25
data profiles
generating, with AutoML 88-90
data retrieval
enhancing, with Databricks VS 92, 93
data science projects
learning 117
streaming transactions 117-119
technical requirements 117
time series forecasting 132-137
data science projects, streaming transactions
on-demand feature, building with Python UDF 130, 131, 132
streaming feature, building with Spark Structured Streaming 119-129

Deep Learning (DL) 164
Delta 176
Delta Kernel connection ecosystem
benefits 9, 10
Databricks' lakehouse architecture speed 9
integration and control 8
time-travel versioning 9
Delta Lake Universal Format (Delta Lake UniForm) 5
Delta Live Tables (DLT) 49, 51, 79
benefits 52
data integrity, improving with 80, 81
features 52
pipeline configuration 61, 62
streaming table created with DLT, querying 63
used, for Bronze data 53
Delta tables 92
Liquid Clustering 54
maintaining 54
OPTIMIZE command 55
optimizing 54
predictive optimization 55
VACUUM command 54
DI Platform
planning 20
Directed Acyclic Graph (DAG) 53
Distributed Data Parallel (DDP) 164
DLT pipeline
bronze table, generating 63
methods 62

E

embeddings 91
using 92
using, for unstructured data 91
evaluation workflow
chatbot responses, evaluating 211-214
setting up 206-211
experiments, with MLflow
Databricks LLM Playground 148-150
data, integrating via Marketplace 147
external data and models, integrating 146
tracking 144-146
experiment tracking 28
exploratory data analysis (EDA) 88

F

Favorita Baseline Model
building 151
creating, with AutoML 154-157
training set, building 151-154
Favorita Store Sales project 13, 64, 65, 94, 95, 183-186
AutoML, for data exploration 96-99
multilabel image classification 105, 106
oil price data, cleaning 99, 100
oil price data, exploring 99, 100
RAG chatbot 101-105
Streaming Transactions 41-43
time series forecasting 40, 41
transactions, streaming 101
feature tables 112
training set, building with 142, 143
Foundation Model APIs (FMAPIs) 69, 231
pay-per-token 69
provisioned throughput 69
Fully Shared Data Parallel (FSDP) 164

G

generative artificial intelligence (GenAI) 4, 45, 65, 85, 141
GitHub 176

glass box 7
gold layer visualizations
building 219
Google Cloud Storage (GCS) 23

H

Hugging Face (HF) 231
HyperOpt 157
hyperparameters 141

I

Infrastructure as Code (IaC) 179

K

Kaggle setup 32, 33
code base, setting up 37
compatibility, checking 38
compute, creating 34-36
Databricks CLI and secrets, setting up 37
default catalog, setting up 39
GitHub repository, setting up 33, 34
permissions, granting 39
project-specific database, setting up 39
variables, passing via widgets 38
volumes, setting up 40

L

Lakehouse Monitoring 12
Lakeview dashboards
leveraging 219-221
Large Language Models (LLMs) 148
incorporating, for analysts with SQL AI Functions 231-235
Light Gradient-Boosting Machine (LightGBM) 12

M

machine learning (ML) projects 81
Machine Learning Operations (MLOps) 175
Medallion architecture pattern 25, 49, 50
metastore
data 23
selecting 22, 23
source control 24, 25
MLflow Tracking 141
MLOps inner loop
collaborative development 176, 177
deploying 176
model, registering 176
MLOps outer loop
DABs 179, 180
deploying 177
REST API 180
workflows 178, 179
model
connecting, to applications 231
deploying 180
inference 180, 181
monitoring 218, 219
serving 181, 182
multi-hop architecture 25
multilabel classification model
fine-tuning 164
scoring 168-172
training 164-168
Multilabel Image Classification (MIC) 14, 45, 71, 72, 197-204
data loaders 72, 73
data, optimizing 73, 74

O

on-demand feature 111-112
building, with Python UDF 130-132
computing 116
online feature store 9
OpenAI 151
OPTIMIZE command 55

P

personal access token (PAT) 31, 235
point-in-time lookups 111
predictive optimization 55
primary tables 30, 82
Production Batch Inference 187
Production Streaming Transactions 187
Python UDF 224-226
used, for building on-demand feature 130-132
tips and tricks 226-230

Q

Query Federation 24

R

real-time data 114
REST API 180
Retrieval Augmented Generation (RAG) chatbot 11, 15, 16, 44, 45, 65-70, 85, 204
ground truth labels, loading 205, 206

S

schema evolution 9
Simple Storage Service (S3) 23
Spark Structured Streaming 51
used, for building streaming feature 119-129
SQL chatbot, with project table metadata
building 235
favorita store sales 236-240
retrieval-augmented generation chatbot 241-247
streaming transactions 240, 241
technical requirements 235
stateful streaming 115
streaming data 114
benefits 114
streaming feature 112
building, with Spark Structured Streaming 119-129
streaming pipelines 114
streaming transactions dataset
data solution 11, 12
Streaming Transactions Project 56, 57, 187-197
continuous data ingestion, using Auto Loader with DLT 60
continuous data ingestion, using Auto Loader with Structured Streaming 58-60
project pipeline for 157-164

T

time series feature tables
point-in-time lookups 115, 116
time series forecasting 132-137
training set
building 151-154
building, with feature tables 142, 143
transactionCount 132

U

Unity Catalog Registry (UC Registry) 176
Unity Catalog (UC) 21, 111, 176
Databricks Feature Engineering (DFE) 112, 113
User-Defined Functions (UDFs) 68, 116, 224

V

VACUUM command 54
vector database 91
requirements 91
uses 92
Vector Search (VS) 160

W

workspace
as folder 21
as instance 21
defining 21

Packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

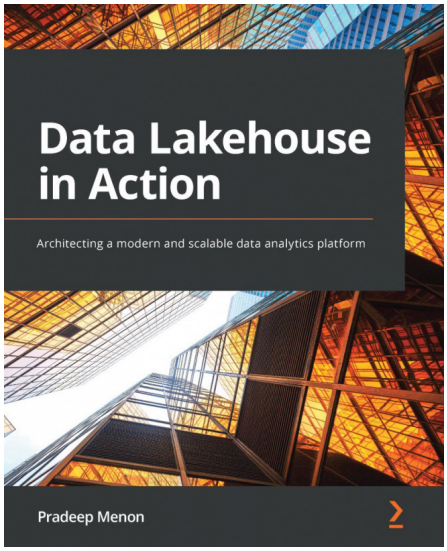
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

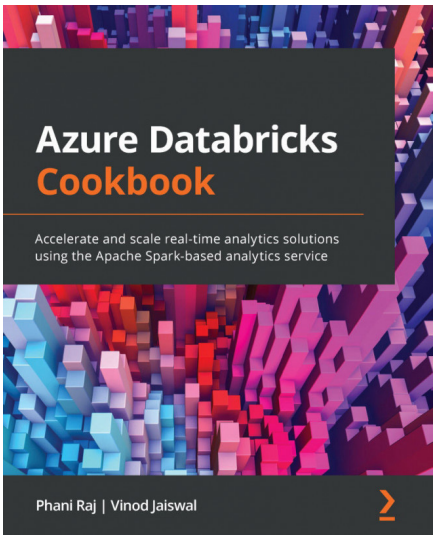


Data Lakehouse in Action

Pradeep Menon

ISBN: 978-1-80181-593-2

- Understand the evolution of the Data Architecture patterns for analytics
- Become well versed in the Data Lakehouse pattern and how it enables data analytics
- Focus on methods to ingest, process, store, and govern data in a Data Lakehouse architecture
- Learn techniques to serve data and perform analytics in a Data Lakehouse architecture
- Cover methods to secure the data in a Data Lakehouse architecture
- Implement Data Lakehouse in a cloud computing platform such as Azure
- Combine Data Lakehouse in a macro-architecture pattern such as Data Mesh



Azure Databricks Cookbook

Phani Raj , Vinod Jaiswal

ISBN: 97-81-78980-971-8

- Build a modern data warehouse with Delta tables and Azure Synapse Analytics
- Create real-time dashboards in Databricks SQL
- Implement data governance with Unity Catalog
- Build end-to-end data processing pipelines for near real-time data analytics
- Integrate Azure DevOps for version control as well as for deploying and productionizing solutions with continuous integration and continuous deployment (CI/CD) pipelines
- Enhance Azure Databricks with OpenAI and Microsoft Fabric integration for cutting-edge data solutions

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Databricks ML in Action*, we'd love to hear your thoughts! If you purchased the book from Amazon, please `click here to go straight to the Amazon review page` for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below

`https://packt.link/free-ebook/978-1-80056-489-3`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly